

Bus Tour / バスの乗り継ぎ

JOI Spring Camp day1
Shogo Murai

問題の概要

- グリッド状のバス専用道路がある
- 道路の上で、長方形の形をしたコースを延々廻り続けるバスが N 台ある
- 同じ点に来るバス同士は、1 分以上の乗り継ぎ時間があれば乗り継げる
- JOI 君の初期位置と目的地、バスの運行情報が与えられる
- **Time to Destination** を求めたい

解法へのアプローチ

- 最短時間とか言っているし最短経路問題？
 - Dijkstra 法が使えるそう
- 何をキーに Dijkstra をする？
 - (位置、乗っているバス)くらい覚えておけばよさそう
- 位置とバスごとに、そのバスに乗っているための最短時間を求めればよさそう

最短路問題に還元

- (位置, 乗っているバス) を頂点にする
- 辺は、次の2通りに対応するものが考えられる
 - 今のバスにそのまま乗って次の交差点
 - 今の交差点でバスを乗り換える
- いちいちバスの次の交差点を求めるのは大変なので最初にバスの通る点を **vector** などに入れておきましょう
- 交差点ごとにも通るバスをすべて **vector** などに入れておきましょう

計算量の見積もり？

- 位置は $O(WH)$ 、バスは $O(N)$ なので頂点は $O(WHN)$ 個
- 辺の数は 2 通りそれぞれについて考える
 - バス移動 → 頂点数と同じで $O(WHN)$ 本
 - バス乗り継ぎ → 各頂点から高々 $O(N)$ 本なので $O(WHN^2)$ 本
- これで Dijkstra すると $O(WHN^2 \log WHN)$
 - 30 点？

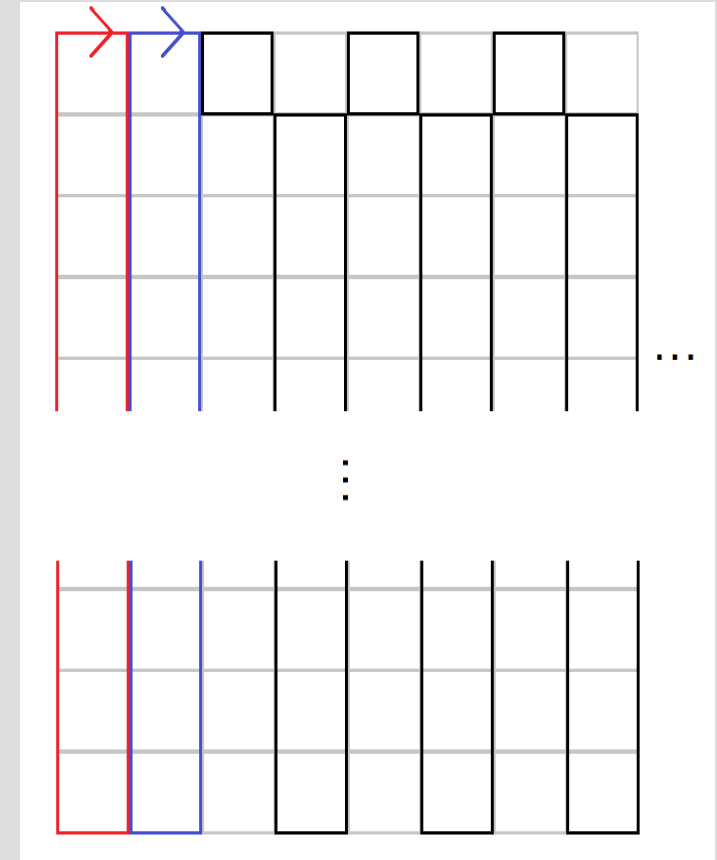
もう少し高速化

- バスは $O(W+H)$ 個の頂点しか通らない
- そのバスが通らない頂点も覚えておくのは無駄
 - 各バスが通る位置だけで考える
- 頂点は $O((W+H)N)$ 個
- 辺の数は
 - バス移動が $O((W+H)N)$
 - バス乗り継ぎは $O((W+H)N^2)$
- よって $O((W+H)N^2 \log((W+H)N))$
 - 80 点??

変なテストケースだと
間に合わない

変なテストケース

- 04-04.txt
- 右図のような形をしています
- 赤線と緑線は **400** 重になっている
- 最短所要時間も長い
- うまくやらないと **TLE** する
- こういう **TLE** 狙いのケースはいつでも存在すると思ったほうがいいです



もっと高速化

- 問題は、考えるべきバス乗り換えが多すぎること
- 同じ点で、2 回以上乗り換え処理は必要？
- 2 回目(遅い方)の乗り換え処理は、1 回目(速い方)の処理をやっておけば十分
- それによってより速い経路が得られることはない
- 乗り換え処理をしたらその点でフラグを立てて、フラグが立っていたら以後は処理をしない

queue も高速化 (1)

- 最短時間の最大値ってどれくらい？
- 大雑把に、待ち時間と移動時間の最大値を求める
 - 待ち時間: どのバスでも、3999 分より多く待たされることはないので 3999000 分
 - 移動時間: すべての点を訪れ、全てのバス経路に乗り換える場合が最大で、1000000+999 分
- よって、高々 4999999 分で目的地に到着

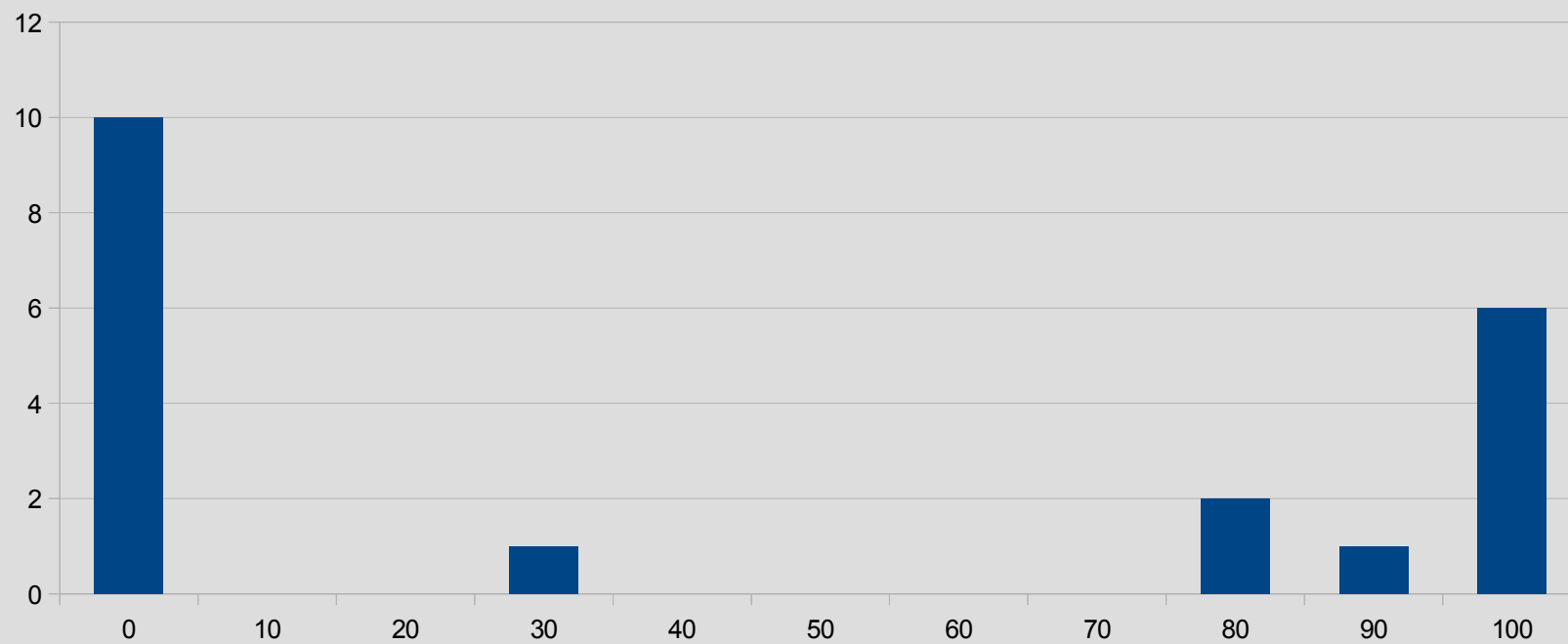
queue も高速化 (2)

- **priority_queue** は、所要時間が小さい順に取り出せればなんでもいい
- 所要時間は **0 ~ 49999999** の整数値
- 所要時間ごとに、リストを持って管理できる
- リストは、**STL** のものよりも自前で連結リストを書いたほうが定数倍で有利です
- この **priority_queue** なら、追加は **$O(1)$** 、取り出しは全部で **$O(\text{要素数} + \text{最大所要時間})$**

まとめ

- 頂点数は $O((W+H)N)$
- 辺の数は $O((W+H)N)$
- **Dijkstra** が(ここでは) **log** なしで解ける
- 乗り換えフラグの初期化に $O(WH)$
- よって、最終的に $O((W+H)N+WH)$
- これで **100** 点
(**log** を取らなくても **100** 点は得られるようです)

得点分布



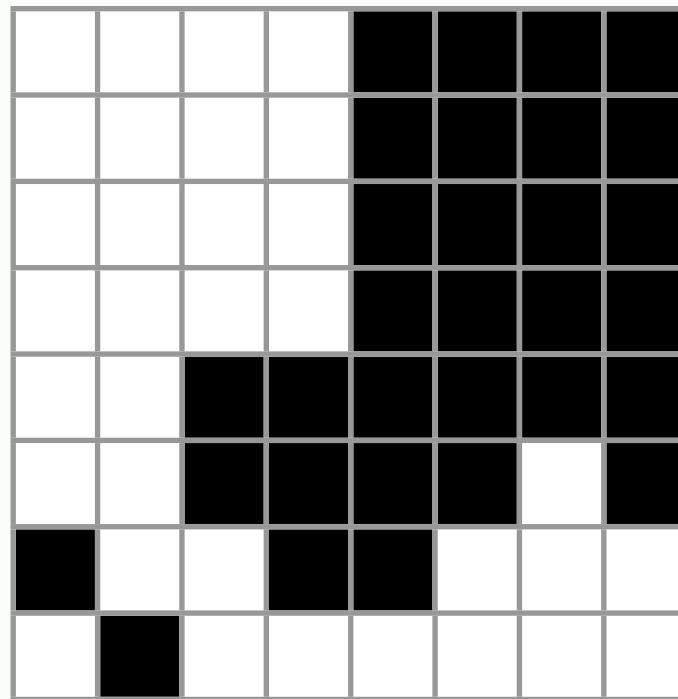
Collecting Images is Fun

今西 健介(@japlj)

問題概要

Collecting
Images is Fun

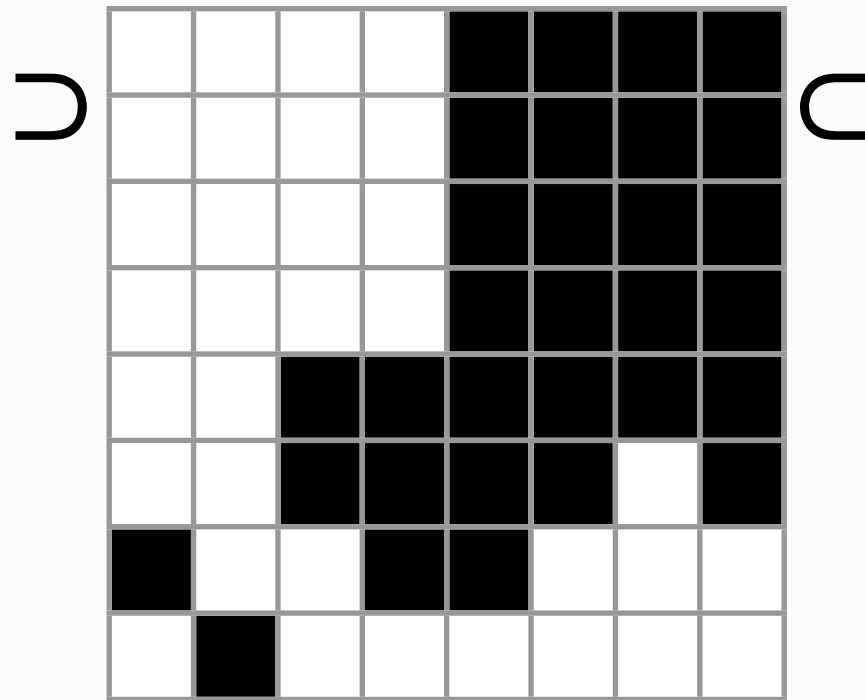
画素数が2のべき乗で正形状の白黒画像が
大好きな JOI 君の悩みを解決しよう！！



問題概要

画素数が2のべき乗で正形状の白黒画像が
大好きな JOI 君の悩みを解決しよう！！

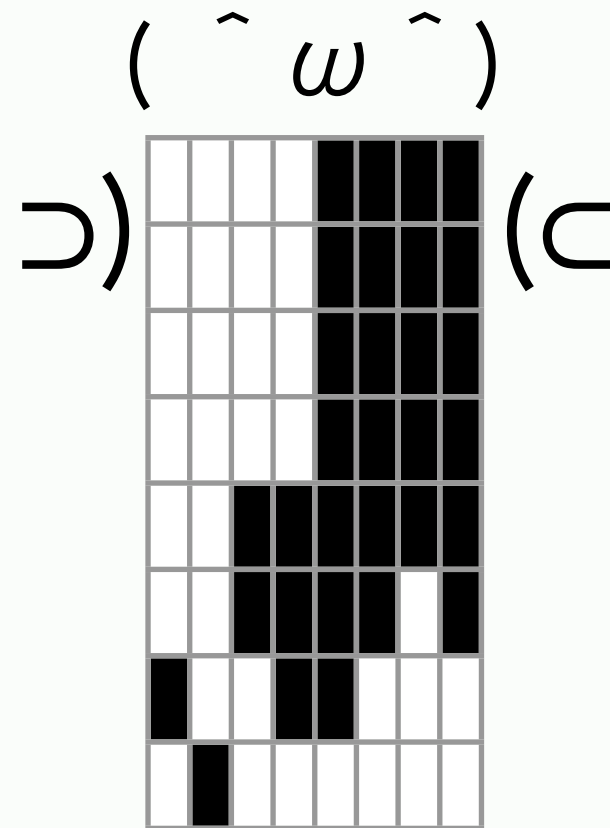
(^ ω ^)



ここに画像が
あるじゃろ

問題概要

画素数が2のべき乗で正方形形状の白黒画像が
大好きな JOI 君の悩みを解決しよう！！



これを

問題概要

画素数が2のべき乗で正形状の白黒画像が
大好きな JOI 君の悩みを解決しよう！！

(^ ω ^)

≡ ⊃ C ≡ こうして



問題概要

画素数が2のべき乗で正形状の白黒画像が
大好きな JOI 君の悩みを解決しよう！！

(^ ω ^) こうじゃ

ㄣ .. ㄣ

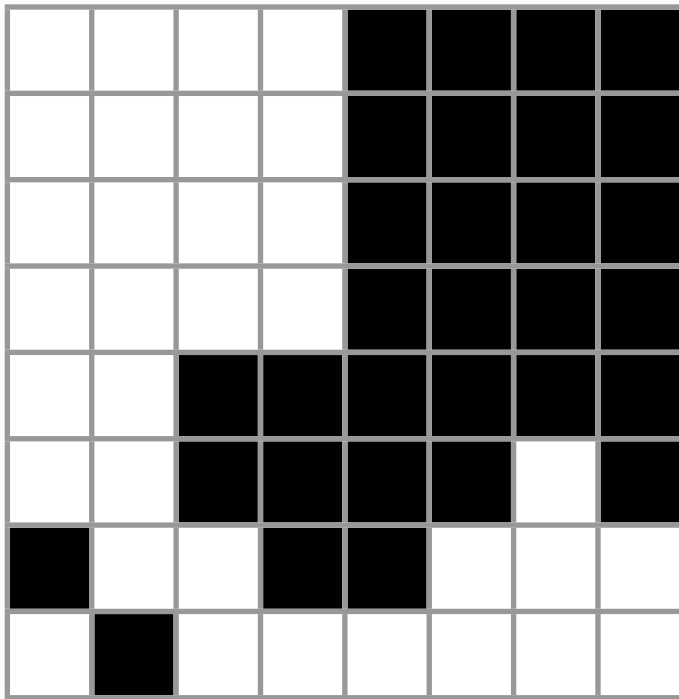
‘ . .
 .

‘ . ’ .
 . ,

実際の圧縮方法

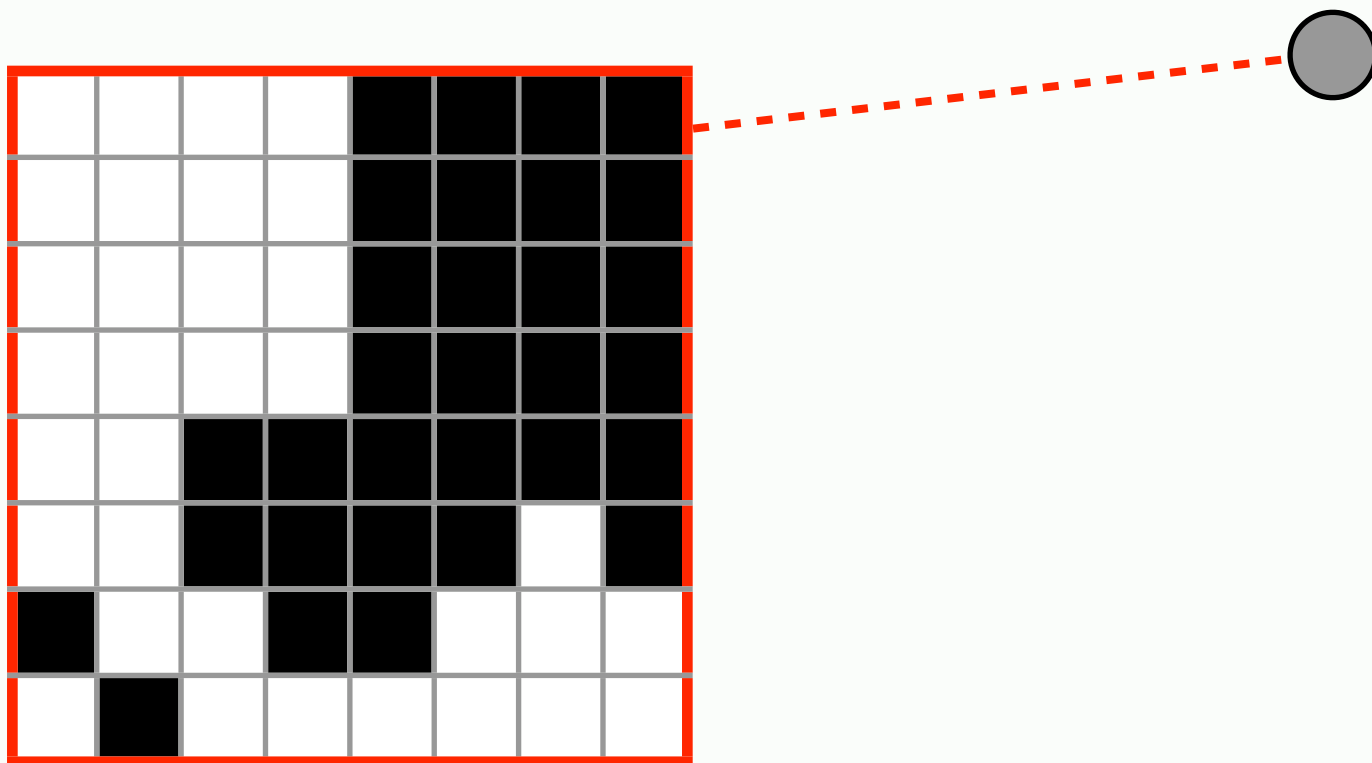
Collecting
Images is Fun

最初の例で説明



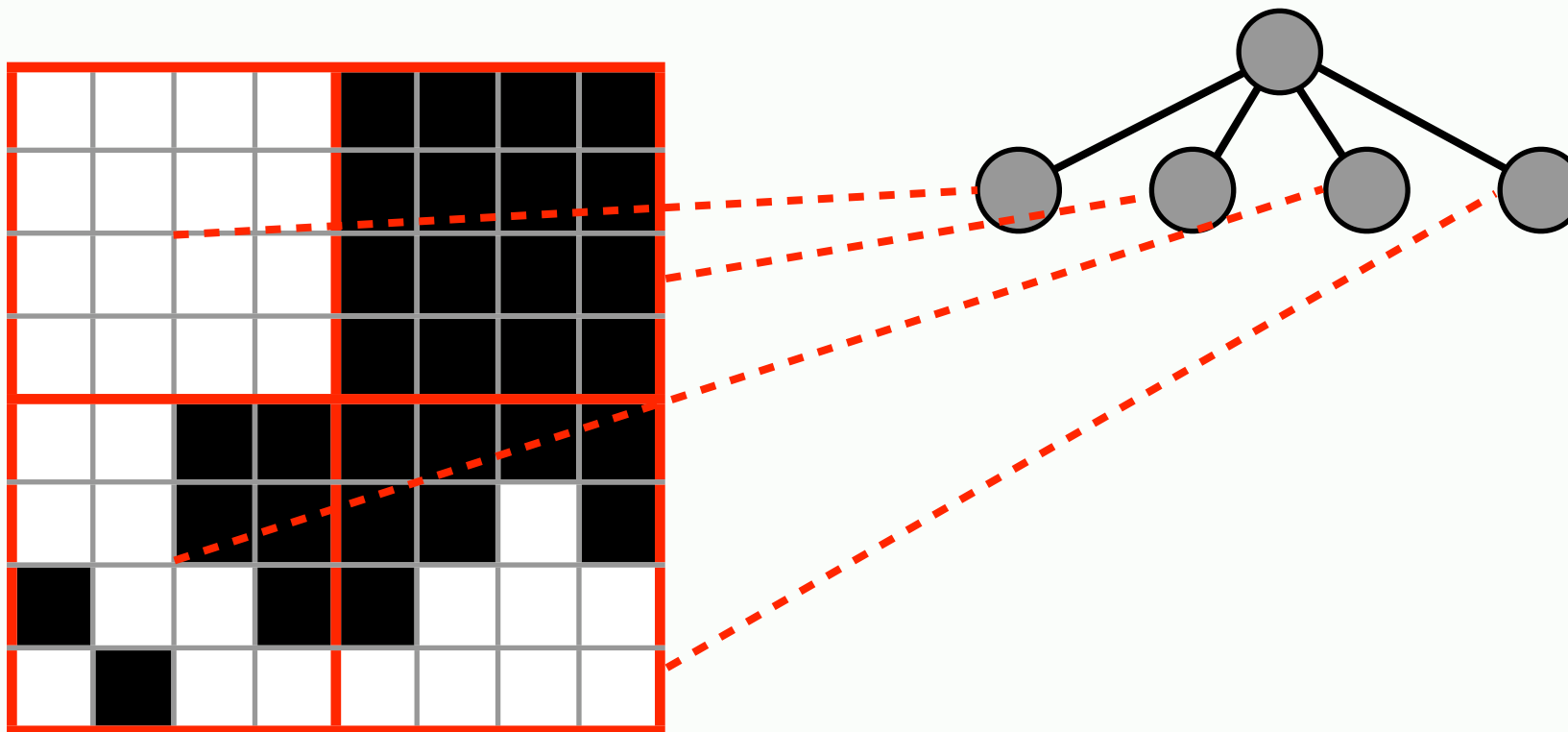
実際の圧縮方法(2)

まず画像全体を見ると，一色ではない



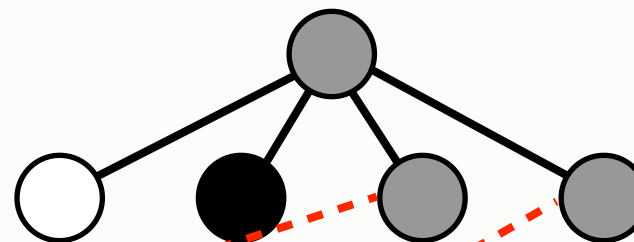
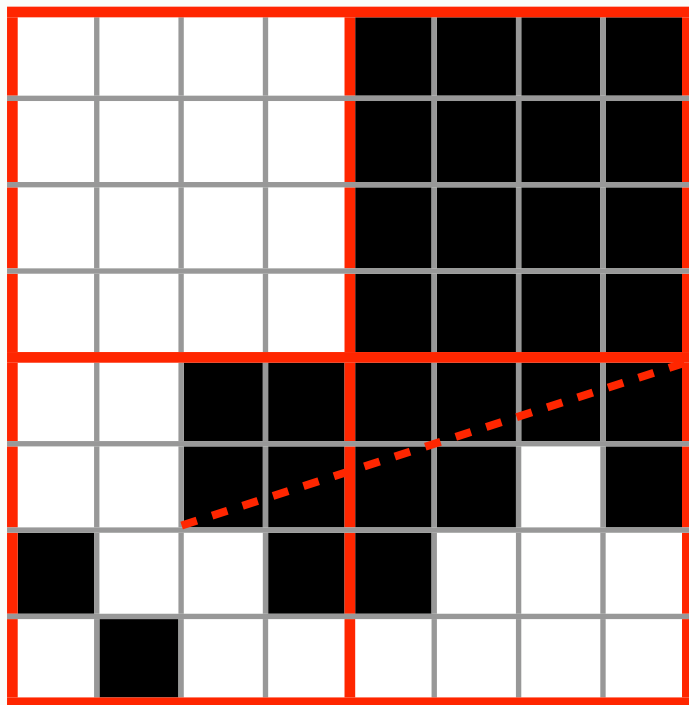
実際の圧縮方法(3)

四分分割して同様に見ていく



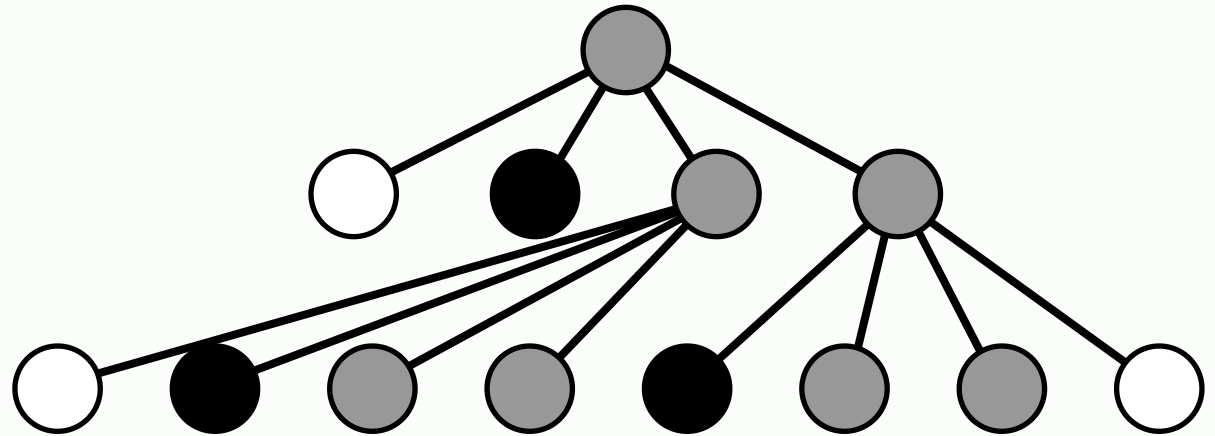
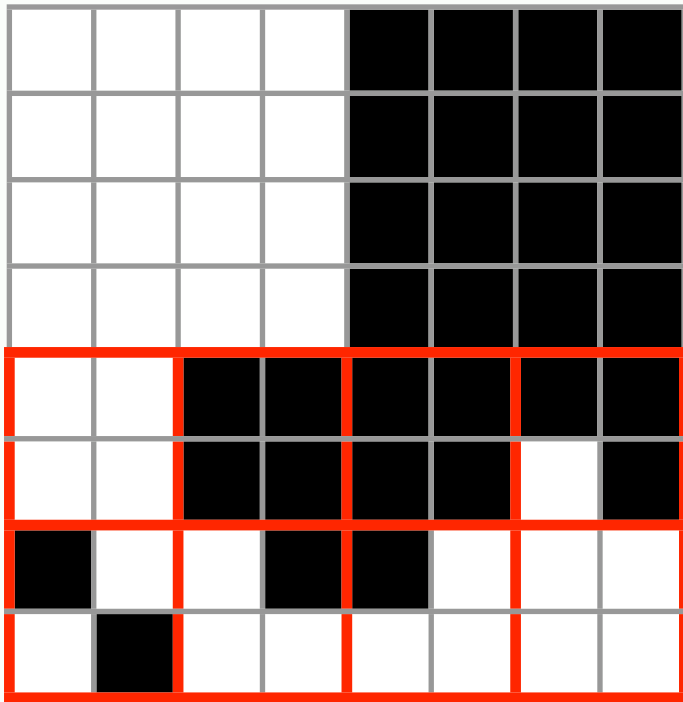
実際の圧縮方法(4)

上側 2 つは一色のみなので終了



実際の圧縮方法(5)

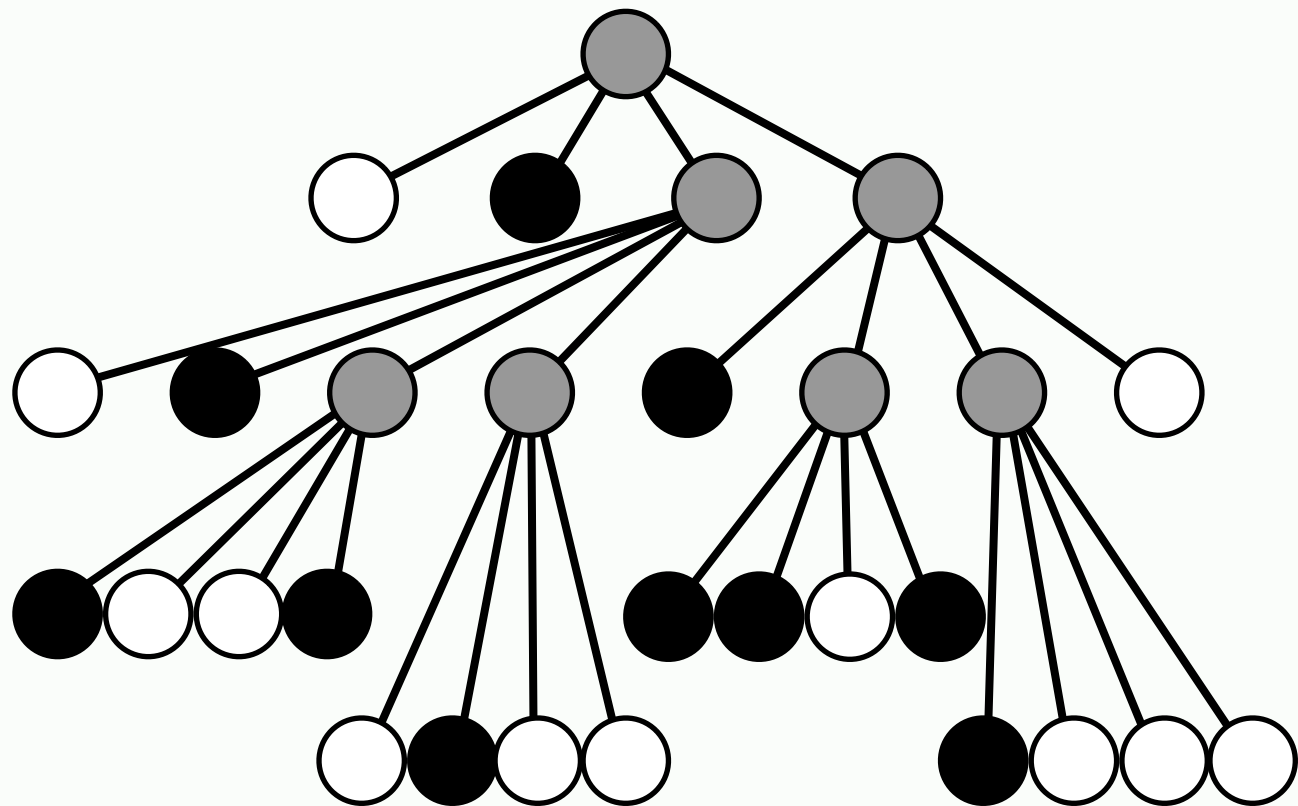
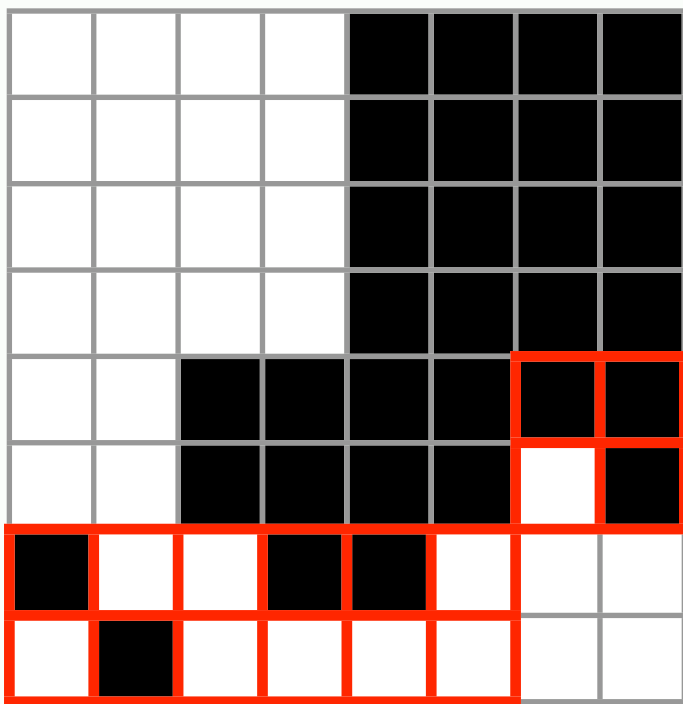
下側はまた四分分割を繰り返していく



実際の圧縮方法(6)

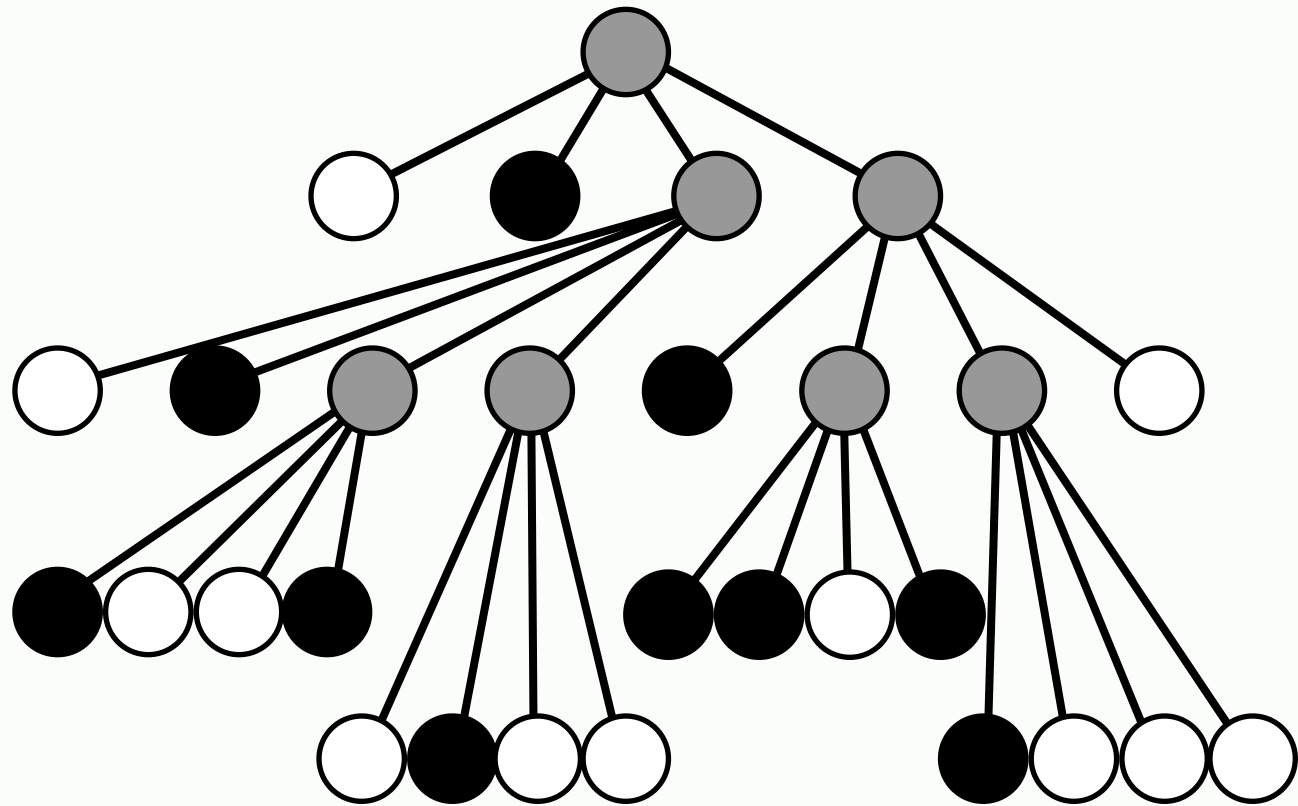
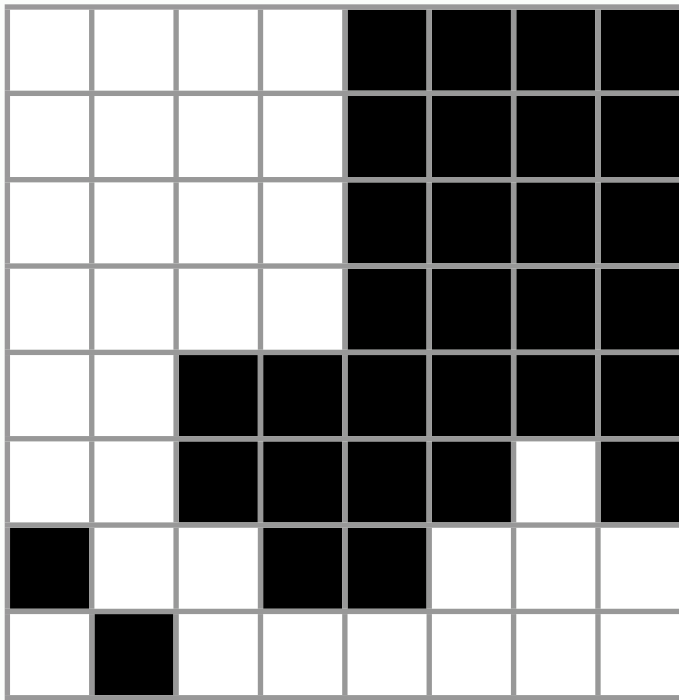
Collecting
Images is Fun

一色でない領域が残ってる間はどんどん分割



実際の圧縮方法(7)

圧縮後のサイズ = 木の頂点数



改めて問題概要

Collecting
Images is Fun

先ほど作った木は四分木という

【問題】

真っ白な画像の行か列の画素をフリップする
クエリが大量に飛んでくるので、
各クエリを処理した後の画像を表す
四分木の頂点数を答えよ

(解くのに四分木の知識は必要ありません)

愚直な解法1

- ・ 画像を $2^N \times 2^N$ の配列で用意
- ・ クエリはきちんと画素をフリップ
- ・ 問題文通りに画像を圧縮してみる

計算量は $O(4^N QN)$ ぐらい

→ 小課題 1 (10点) までなら解ける

愚直な解法2

一回のクエリで、画像を表す四分木はたかだか $2^N N$ 頂点ぐらいしか変更されない(後述)

変更される部分だけを更新することになると
計算量は $O(2^N QN)$ となる

→ 小課題 2 (20点) まで解ける

頂点数に関する考察

四分木の頂点数は簡単には数えにくい

今見ている範囲が一色しかなければ頂点数 1

そうでなければ再帰してサイズの和に 1 を足す

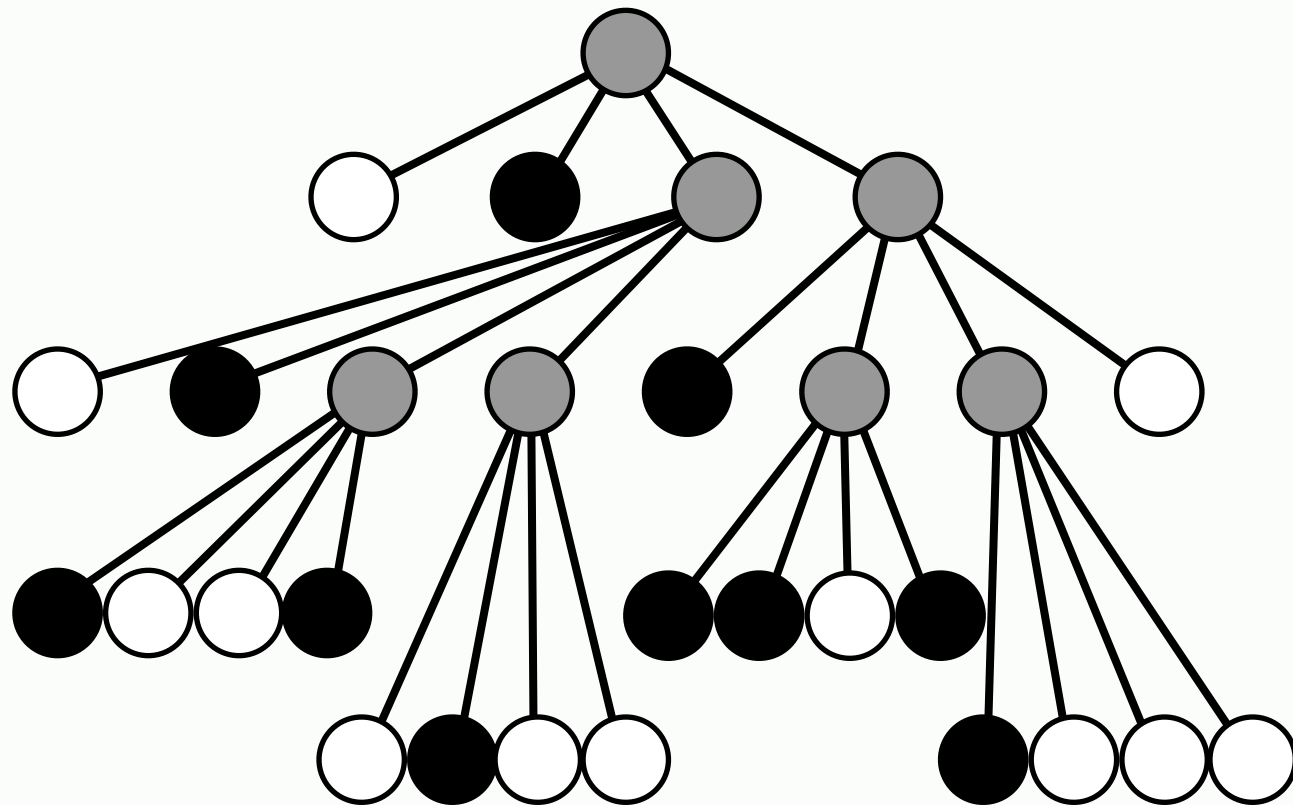
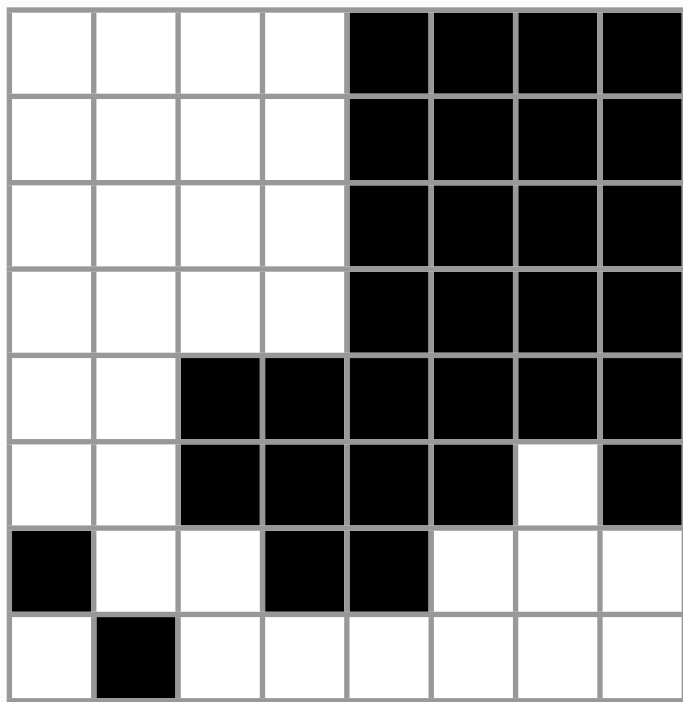
(クッソ複雑な再帰君)

〇〇の数を数えれば、あとはちょっとうまくやれば OK!!! みたいなものがあると嬉しい

再帰より「〇〇を数えるだけ」のほうが簡単っぽい

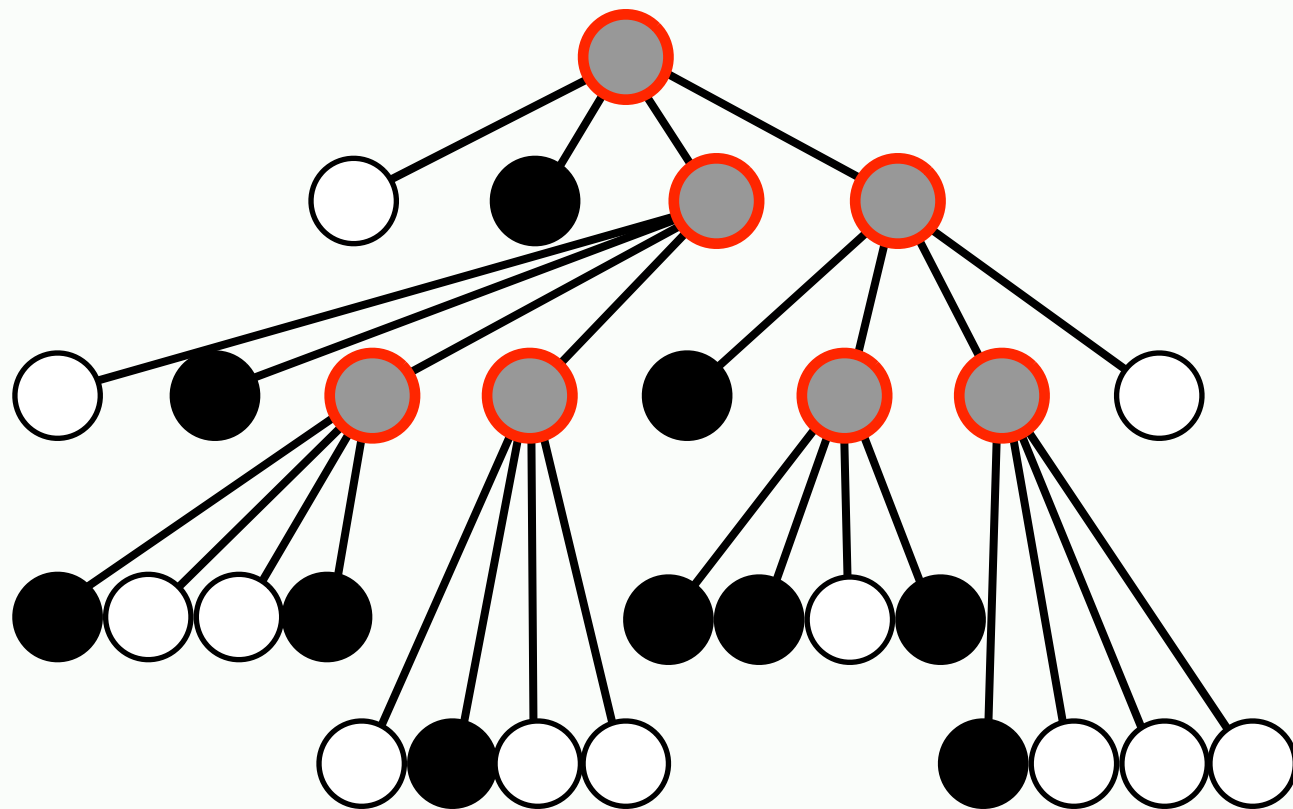
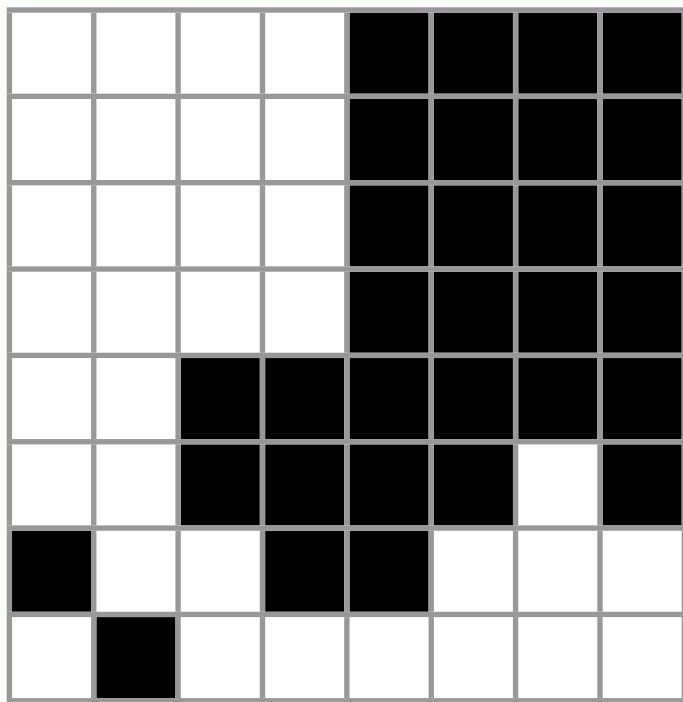
四分木の観察

四分木を観察してみよう(観葉四分木)



四分木の観察(2)

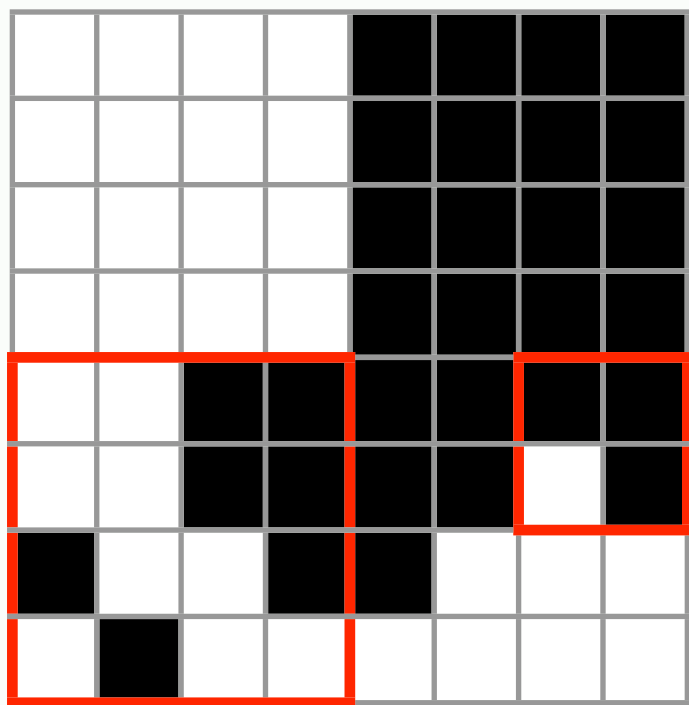
灰色の頂点の数がわかるとよさそう



$$\text{頂点数} = 4 * \text{灰色} + 1$$

四分木の観察(3)

灰色の頂点とはどんな頂点か？



対応する領域が**一色でない**
(四分割する必要がある)
ような頂点が灰色だった

灰色に対応する領域の例

頂点数の数え方

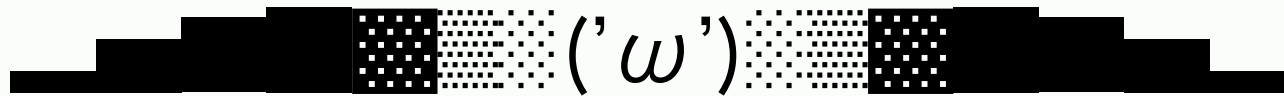
まとめると、画像を 2×2 , 4×4 , 8×8 , ... に
区切った領域のうち一色でない領域の個数を
数えると頂点数が分かる

4ページ前「 $\bigcirc\bigcirc$ の数を数えれば、あとはちょっとうまくやれば
OK!!! みたいなものがあると嬉しい」

→あった

頂点数の数え方(2)

もちろん一個ずつ数えると



なので、賢い数え方が必要

ひとまず

一色でない領域を数えると後々闇が生えるので

一色の領域を数えて全体から引くことにする

クエリの特徴を観察

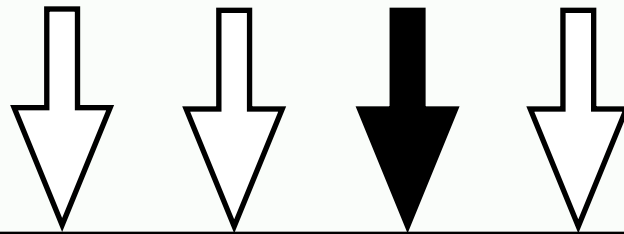
今回のクエリはわりと特殊(観葉クエリ)

行と列のフリップしかこないということを利用して、一色の領域を爆速で数えられないか？

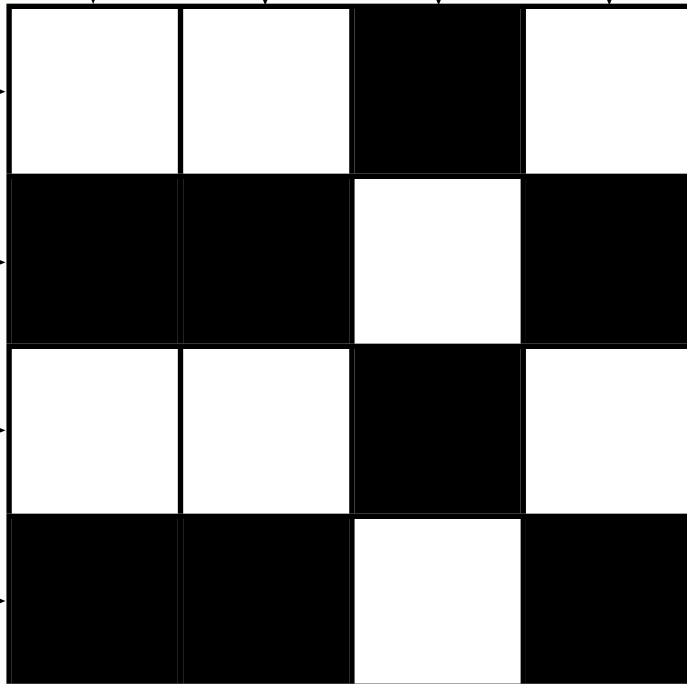
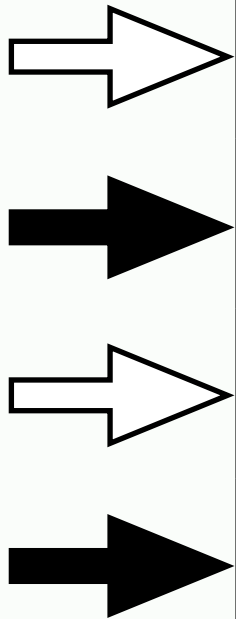
→ある領域が一色である条件を考えよう

一色条件

列の反転され具合



行の反転され具合

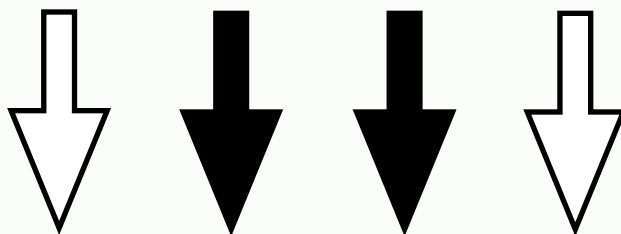


←

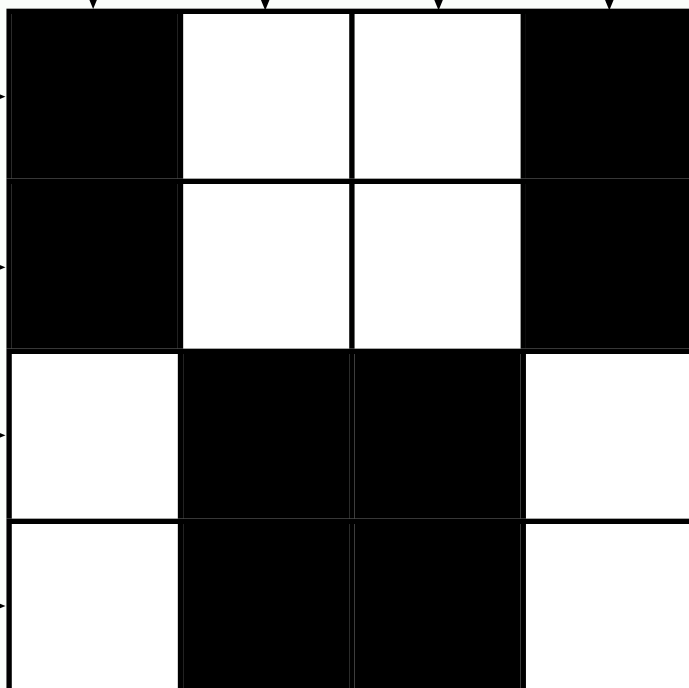
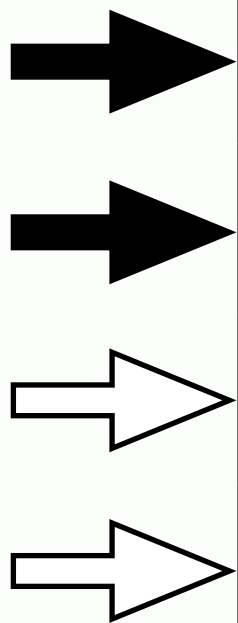
この領域が一色になるためには行と列の反転され具合がどのようなになればよいだろうか??

一色条件(2)

列の反転され具合



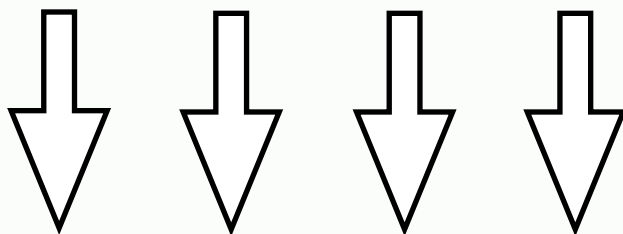
行の反転され具合



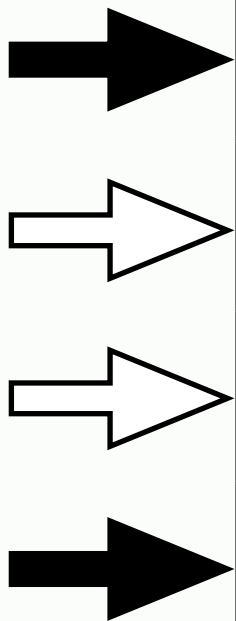
こういう反転され具合
のやつは何をやっても
ダメ

一色条件(3)

列の反転され具合



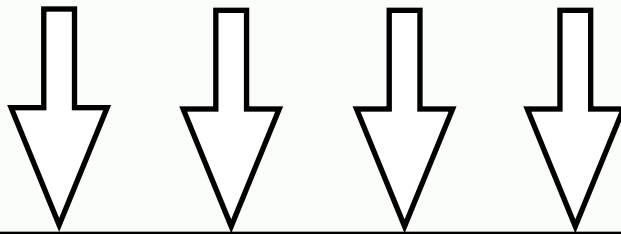
行の反転され具合



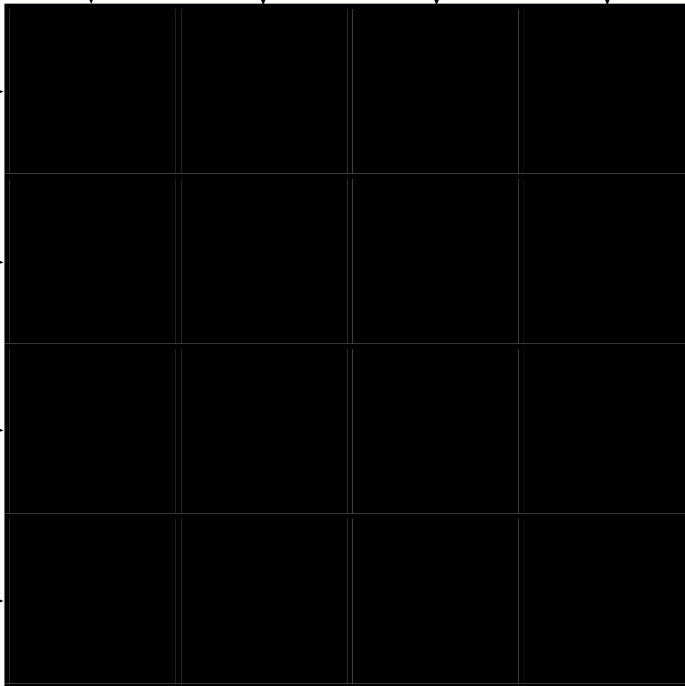
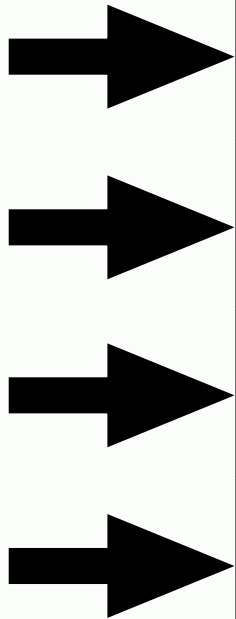
こういう反転され具合
のやつも何をやっても
ダメ

一色条件(4)

列の反転され具合



行の反転され具合

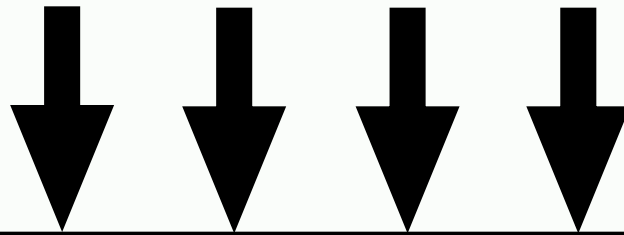


こういう反転され具合
のやつは優秀

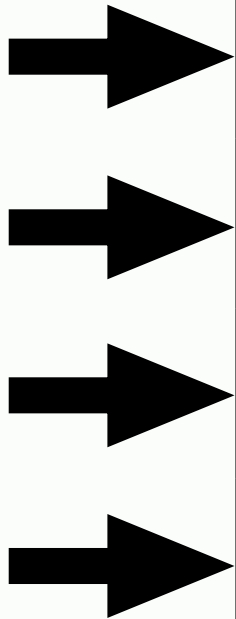
就職にも困らない

一色条件(5)

列の反転され具合



行の反転され具合

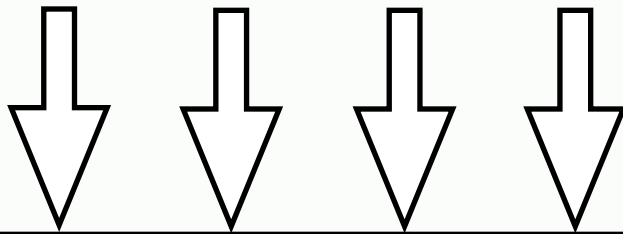


こういう反転され具合
のやつも優秀

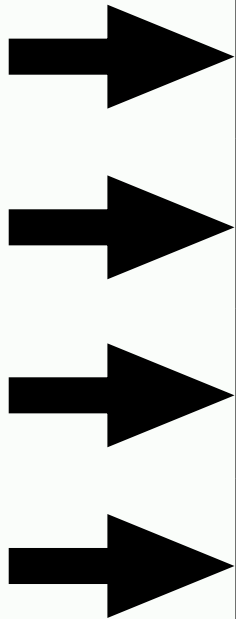
就職にも困らない

一色条件(6)

列の反転され具合



行の反転され具合



行と列の反転され具合
がそれぞれ同じなら神



行と列は別に考えられる

頂点数の数え方(3)

$k = 2, 4, 8, \dots$ (領域の幅) を固定
次の問題を解くことを考えよう

【問題】

画像を $k \times k$ の領域に分割したときに
一色になる領域はいくつあるか？

頂点数の数え方(4)

行と列を別に考えれば, 次の問題でよい

【問題】

反転され具合を k 個ごとに分割したときに
全体が同じになっている箇所はいくつあるか？

頂点数の数え方(5)

もっと問題を分離した形にする

【問題】

n 個の 0 が並んでいる列がある

i 番目を 0,1 フリップするクエリが来るので

各クエリを処理後に, 列を k 個ごとに分割した

ときに同じ数からなる部分がいくつあるか?

頂点数の数え方(6)

これは segment tree っぽいもので解ける

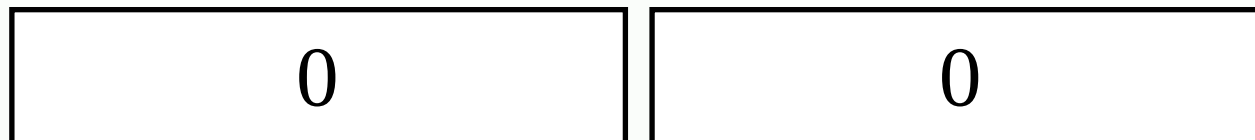
同じ数からなる区間の数

範囲の和を持っている

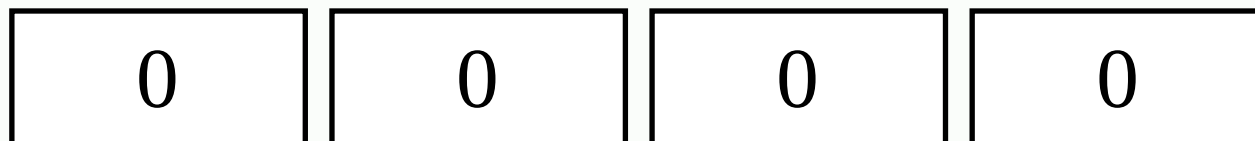
1 →



2 →



4 →



8に決まってるだろ！

いい加減にしろ！！



頂点数の数え方(7)

Collecting
Images is Fun

変更クエリの様子を観察(観葉クエリ Part 2)

同じ数からなる区間の数

範囲の和を持っている

0 →



1 →

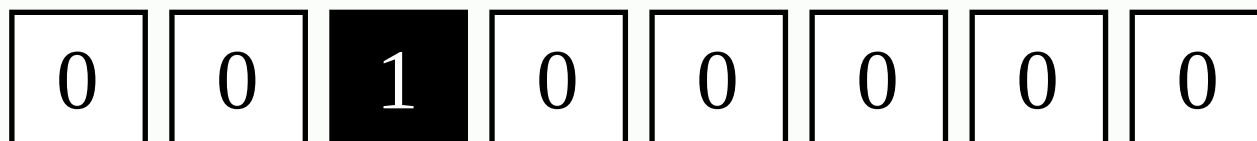


3 →



8に決まってるだろ！

いい加減にしろ！！



頂点数の数え方(8)

Collecting
Images is Fun

変更クエリの様子を観察(観葉クエリ Part 2)

同じ数からなる区間の数

範囲の和を持っている

0 →



0 →

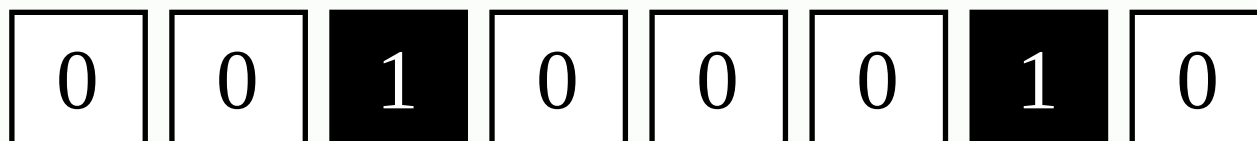


2 →



8に決まってるだろ！

いい加減にしろ！！



頂点数の数え方(9)

Collecting
Images is Fun

変更クエリの様子を観察(観葉クエリ Part 2)

同じ数からなる区間の数

範囲の和を持っている

0 →



0 →



3 →



8に決まってるだろ！

いい加減にしろ！！



頂点数の数え方(10)

あとは仕上げ

各 $k = 2, 4, 8, \dots$ に対し

「行について問題を解いた答」 $\times$

「列について問題を解いた答」

を足せば、一色の領域の個数が求められる。

計算量

Collecting
Images is Fun

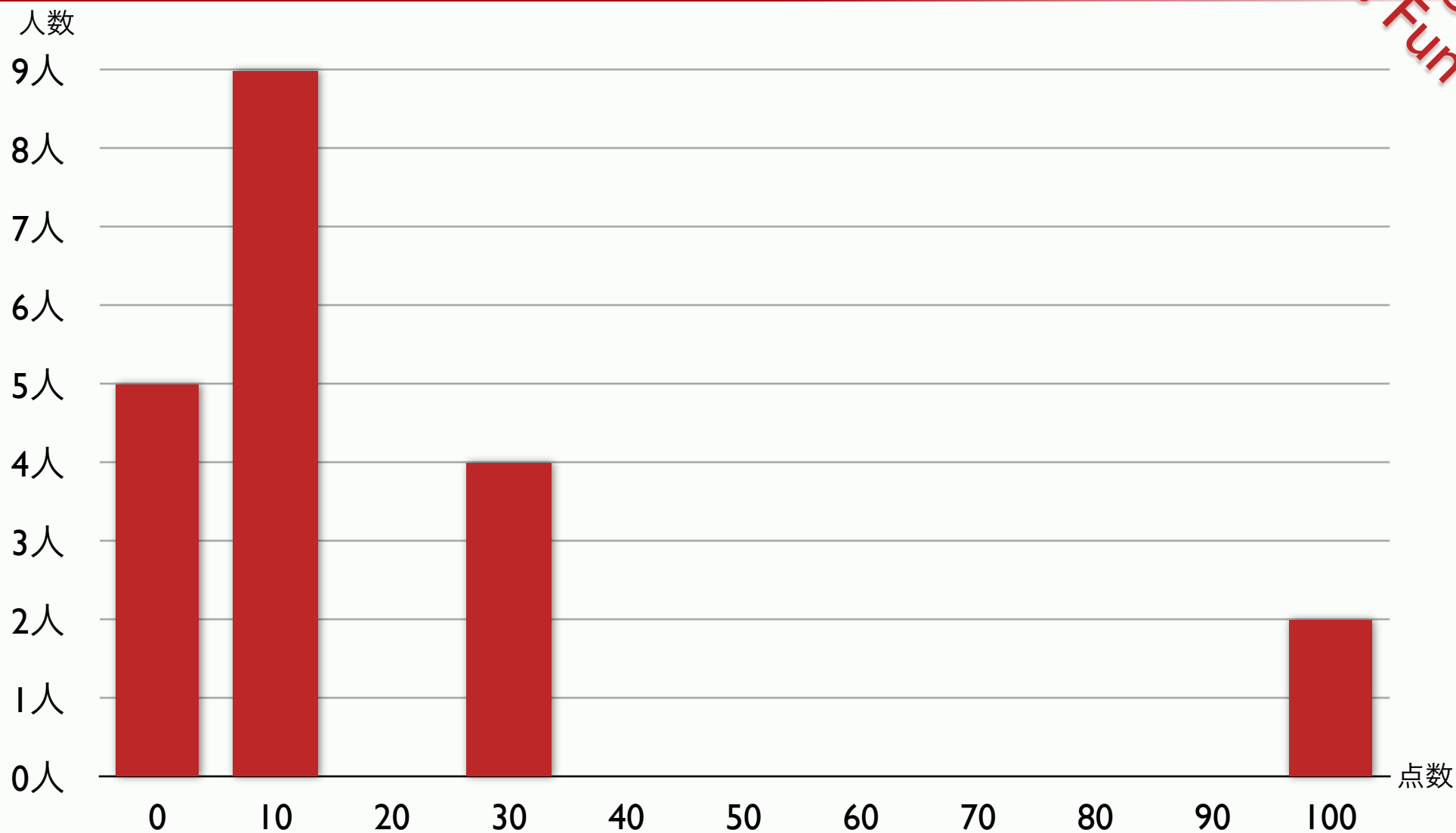
各クエリに大して segment tree っぽいものを
更新する計算量は $O(N)$

→全体では $O(NQ)$

小課題 3 (70点) が解ける

得点分布

Collecting
Images is Fun



JOI 春合宿 2013 Day1

通信妨害

秋葉 拓哉 (@iwiwi)

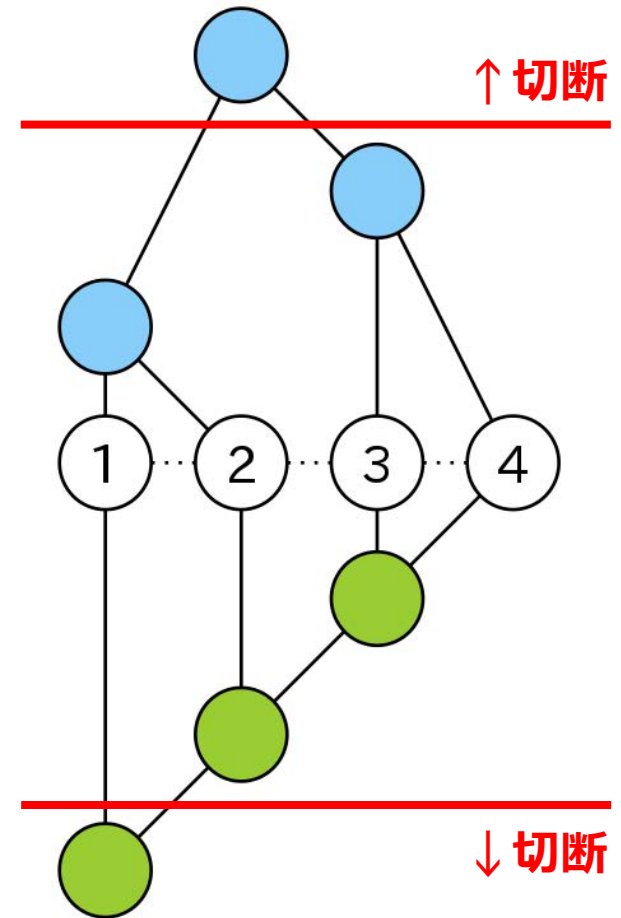
問題

ツリーが 2 つ与えられる

- 上のツリー（青）
- 下のツリー（緑）
- 葉（=村）が共通

クエリいっぱい

- ある高さで上の木を切断
- 下の木はどこの高さで切断？
(連結なままなるべく上で切りたい)



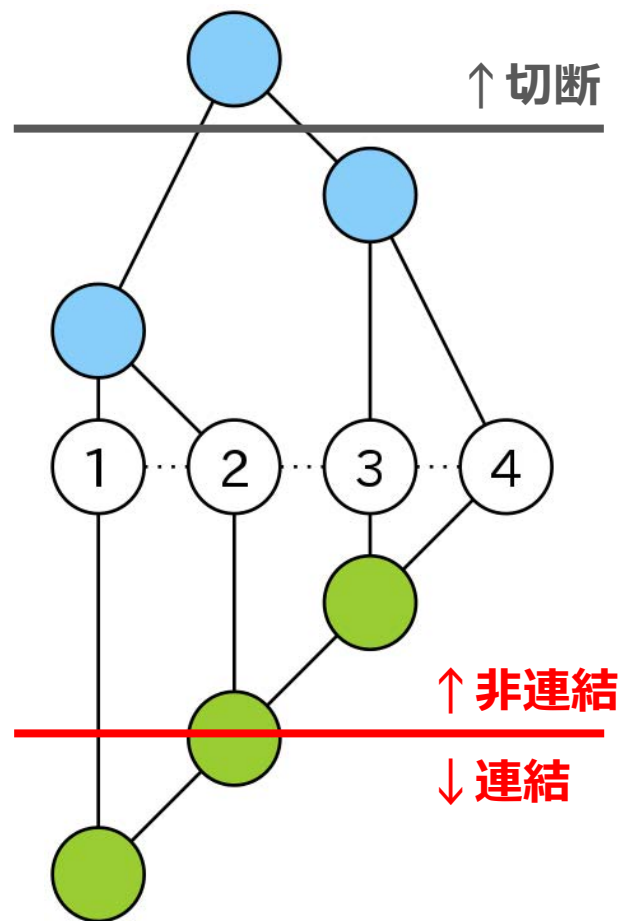
部分点解法 (20 点)

クエリごとに二分探索

- 切る高さを二分探索
- 連結？非連結？

計算量

- 1 回の判定 $O(N)$
- 全体 $O(QN \log N)$



重要な考察

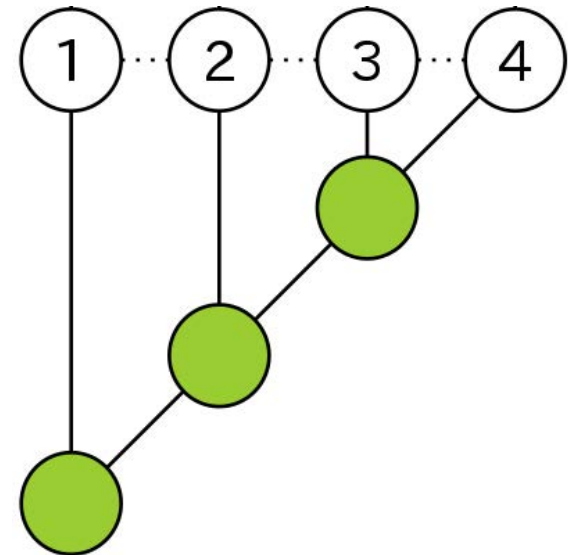
「全ての村が隣と通信可能なら良い」

何故？

- 飛び越えることがない（平面性より）

1-3 を繋ごうとすると、
勝手に 1-2, 2-3 も繋がる

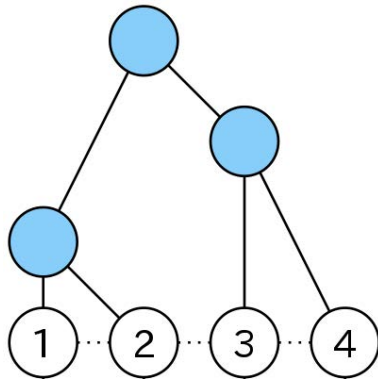
1-2, 2-3, 3-4 を繋げば良い



問題が分裂

問題 1 (前半)

木から，隣同士を繋ぐ
のに必要な高さを計算



1-2	2-3	3-4
1	3	2

問題 2 (後半)

計算した列から，
クエリへの答えを計算

系統 1 :

1	3	2
---	---	---

系統 2 :

3	2	1
---	---	---



$$A_1 = 1 \rightarrow B_1 = -3$$

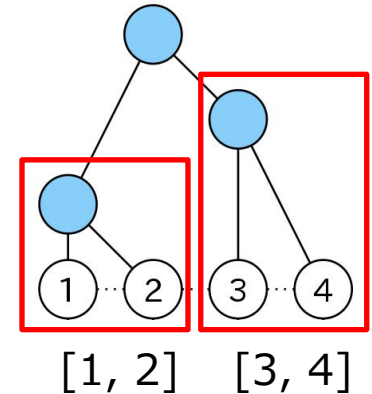
$$A_2 = 2 \rightarrow B_2 = -2$$

$$A_3 = 3 \rightarrow B_3 = 0$$

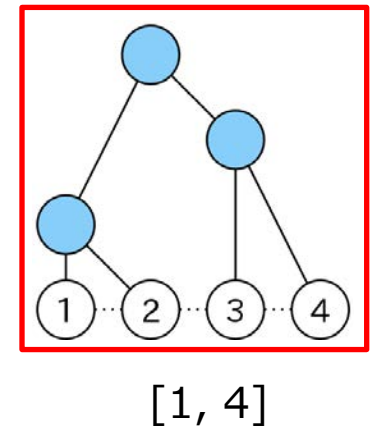
問題 1 : 高さの計算

いろんな方法

- LCA
- DFS, 左手法
- Union-Find ← オススメ



2-3 間の
高さ判明

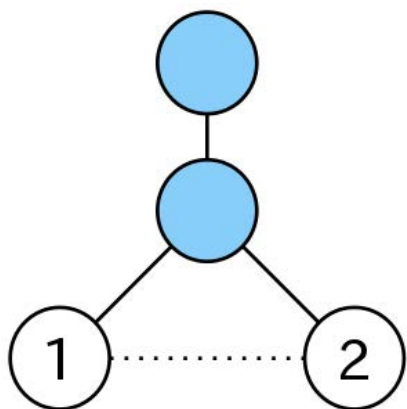


Union-Find による高さの計算

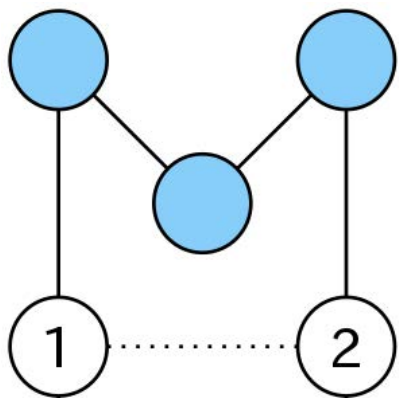
- 低い頂点から順に, マージ
- 区間を管理
- 新たに繋がった場所がその高さ

$O((N + M) \log(N + M))$ 時間

気をつけるテストケース



間の高さが 2 になってしまう？
(DFS 等)



1 と 2 が繋がらない？
(Union-Find 等)

問題 2 : クエリへの答え

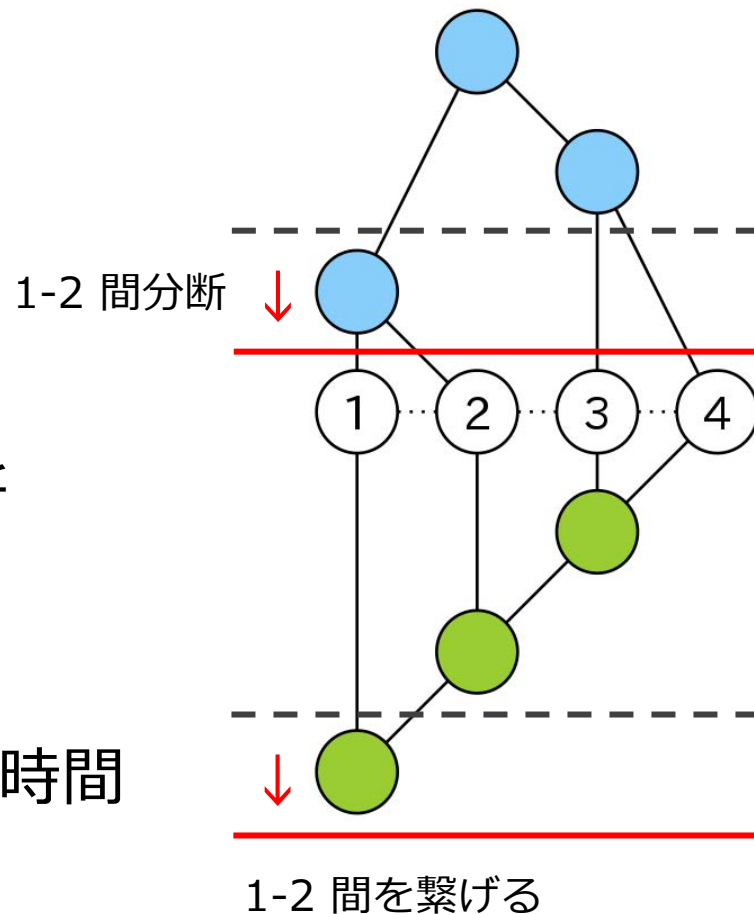
基本アイデア

- 上を切る場所を下げていく
- 下を切る場所も下がる

何処まで下がるか

- 上で新たに分断される箇所
- そこが繋がるまで下げる

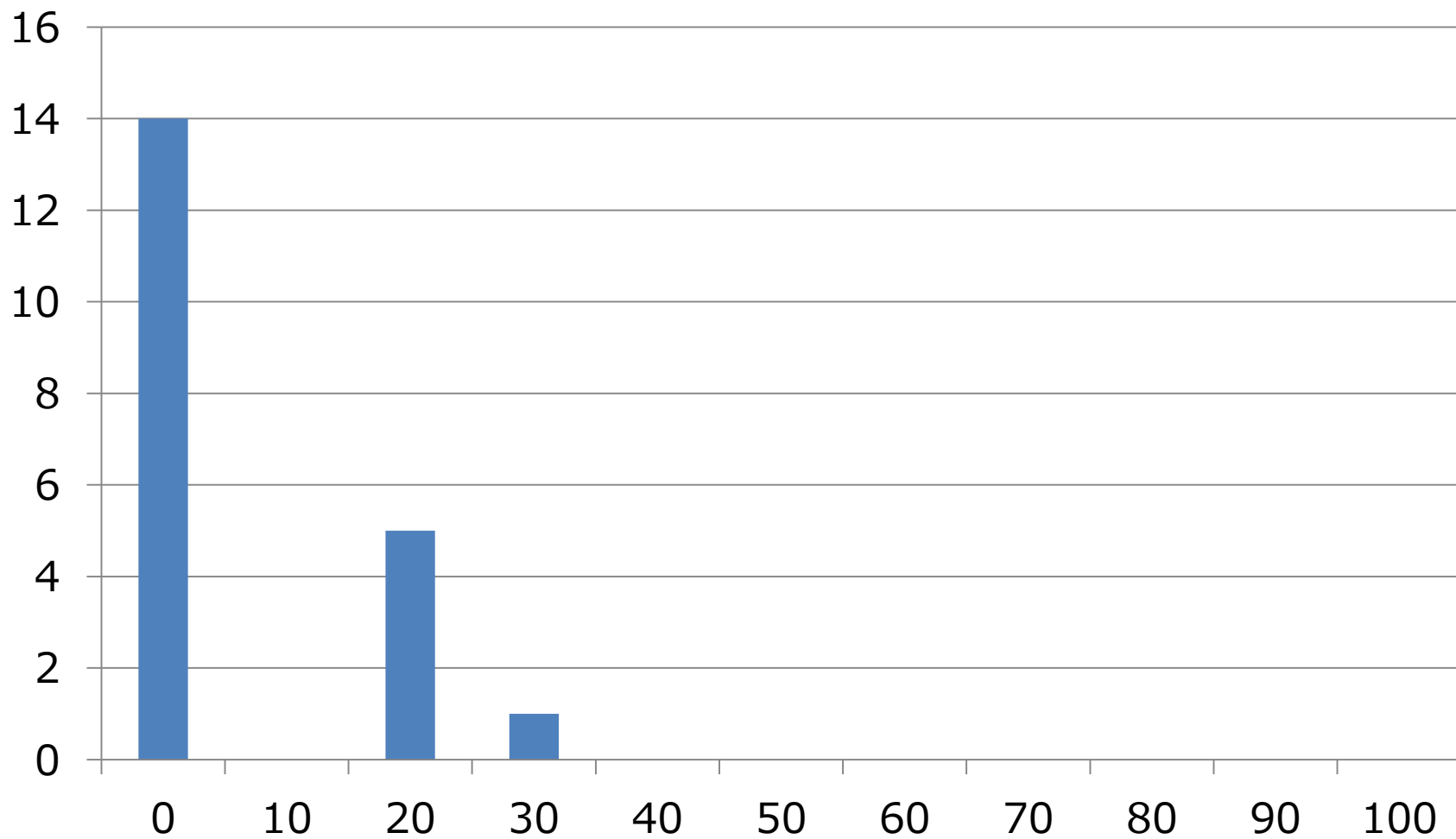
$O((N + M) \log(N + M) + Q \log Q)$ 時間



おまけ

- 別にツリーじゃなくても平面グラフなら全く同じアルゴリズムで解ける

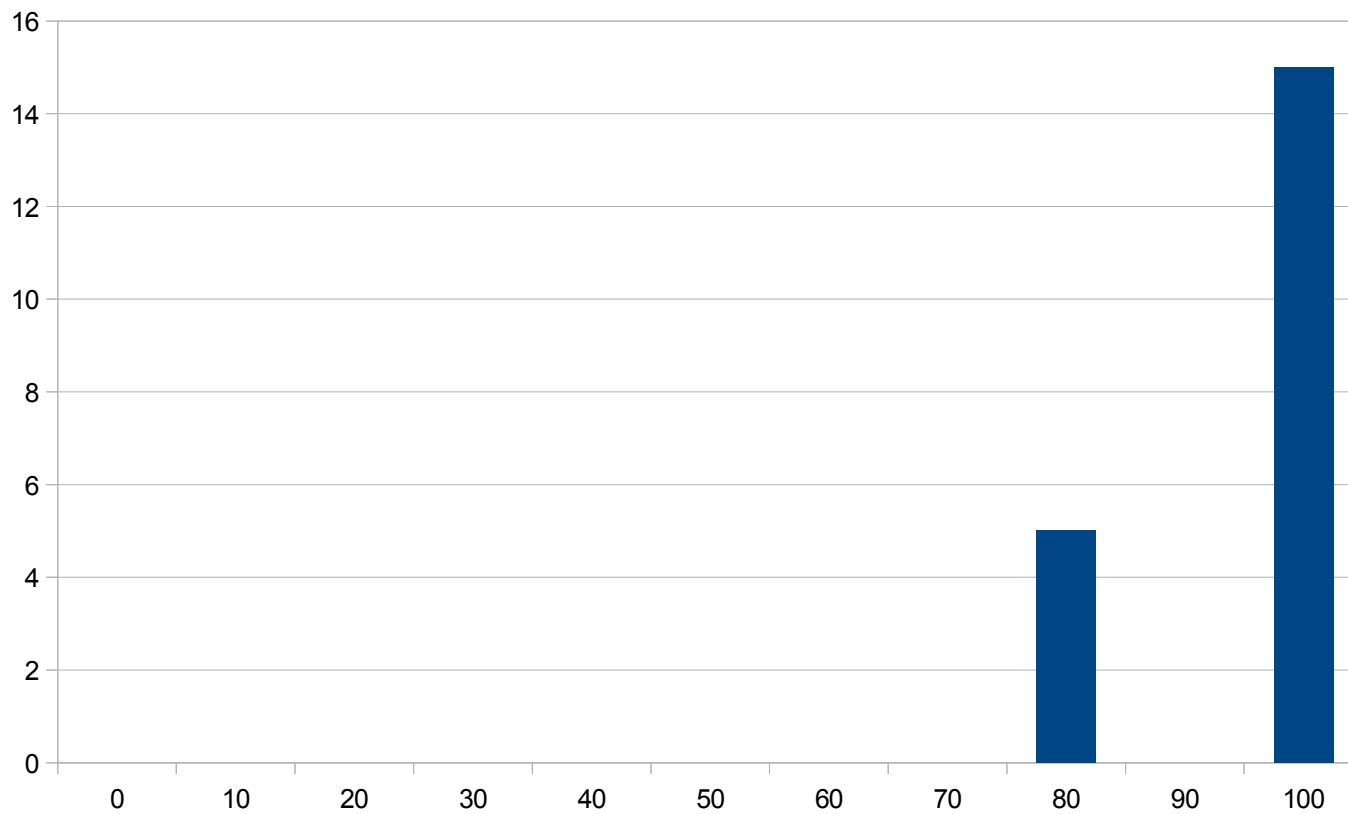
得点分布



JOI Poster

解説：平野湧一郎

得点分布

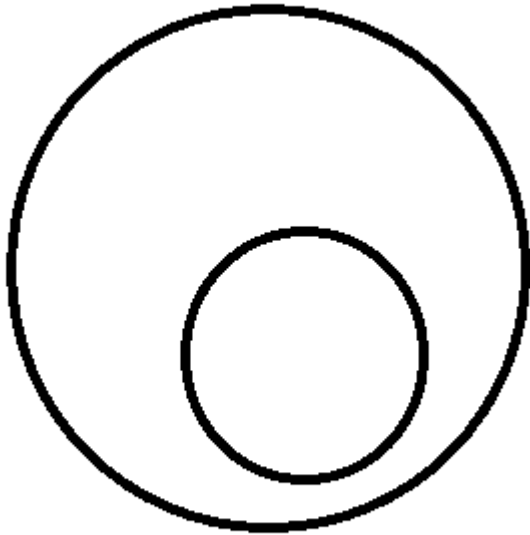


- 平均点 : 95 点
- 標準偏差 : 8.66 点

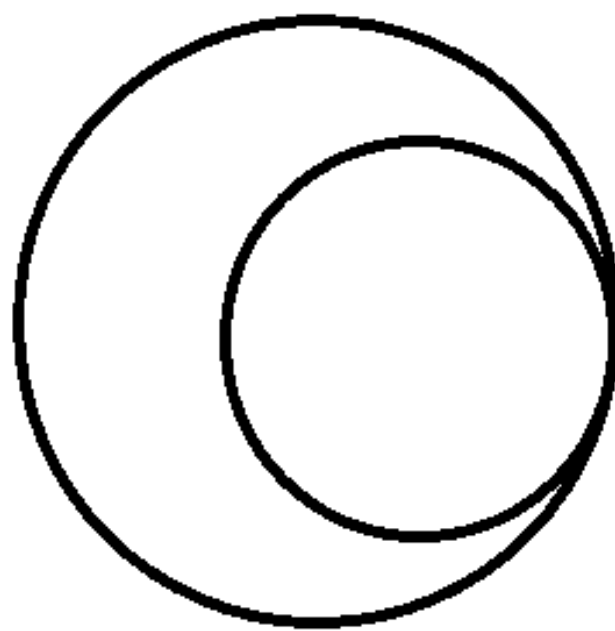
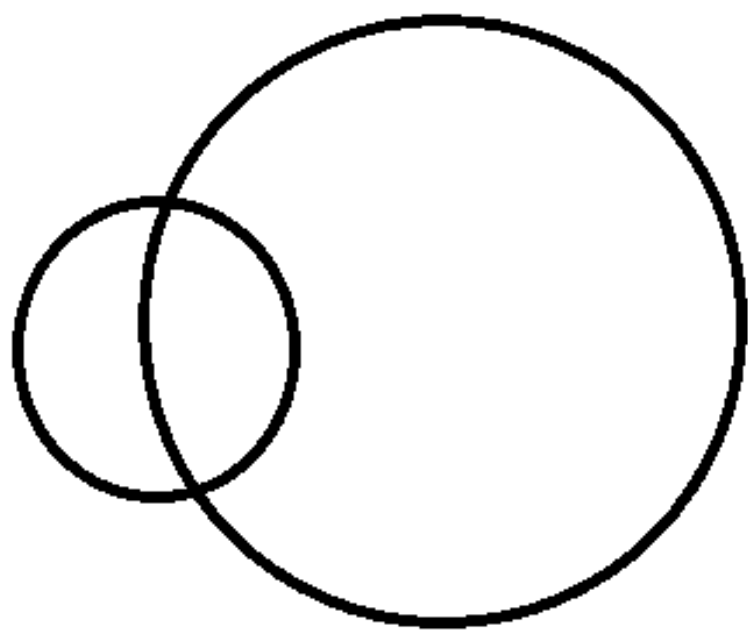
問題概要

- ポスター上に N 個の点が存在
- 4 つの点を選び円 O_1, O_2 の 2 つを描く
- 以下の条件を満たす選び方は何通りあるか
 - 円 O_1 が円 O_2 を内部に含む
 - どちらの円もポスターからはみ出さない

OK な例



NG な例



接する

制限

すべての入力データは以下の条件を満たす.

- $4 \leq N \leq 50$.
- $1 \leq W \leq 1000$.
- $1 \leq H \leq 1000$.
- $0 \leq X_i \leq W$.
- $0 \leq Y_i \leq H$.

$$N \leq 50$$

解法

- 総当りやるだけ.
- 2 点 $(X_1, Y_1), (X_2, Y_2)$ の距離は
$$\sqrt{(X_1 - X_2)^2 + (Y_1 - Y_2)^2}$$
- 円 1 が円 2 を内部に含むための条件は
$$r_2 + d < r_1$$
 - r_1 : 円 1 の半径
 - r_2 : 円 2 の半径
 - d : 円 1 の中心と円 2 の中心の距離

小課題

小課題 1 [80 点]

- 4 つの星 A, B, C, D をどのように選んでも, 円 O1 と円 O2 は接しない.
- 接すると何が困るか？

問題点 – 「距離」は実数である

- コンピュータ上では、実数を正確に扱うことは不可能
- どんなに精度を上げても誤差は不可避
- 詳しくは「丸め誤差」で検索

条件式の確認

- 円 1 が円 2 を内部に含むための条件は

$$r_2 + d < r_1$$

- 上記の 3 つの値はどれも $\sqrt{N}$ (N は整数) という形で表される実数
- 2 つの円が接する場合,
$$r_2 + d = r_1$$
- 誤差によりこの等号 (=) が不等号 (<) として判定されてしまう可能性が存在

解決策(1) – 2 乗して整数で比較

- $\sqrt{a} < b$ であるための必要十分条件は,
 $b \geq 0$ かつ $a < b^2$
 - 整数で比較することが可能
- $\sqrt{a} + \sqrt{b} < \sqrt{c}$ であるための必要十分条件は？
 - まず両辺 2 乗して $a + b + 2\sqrt{ab} < c$
 - 移項して $2\sqrt{ab} < c - a - b$
 - よって $c - a - b \geq 0$ かつ $4ab < (c - a - b)^2$

実装上の注意

- 最大で座標の 4 乗 ($1000^4=1$ 兆) になるため, int に収まらない
 - long long を使いましょう
- $c - a - b > 0$ のチェックを忘れないように

実装

```
bool rrr_lt(long long a, long long b, long long c) {  
    //  $\sqrt{a} + \sqrt{b} < \sqrt{c}$  ?  
    long long n = c - a - b;  
    return n > 0 && 4 * a * b < n * n;  
}
```

```
// 呼び出し
```

```
if (rrr_lt(r2 の 2 乗, d の 2 乗, r1 の 2 乗)) {  
    // ...  
}
```

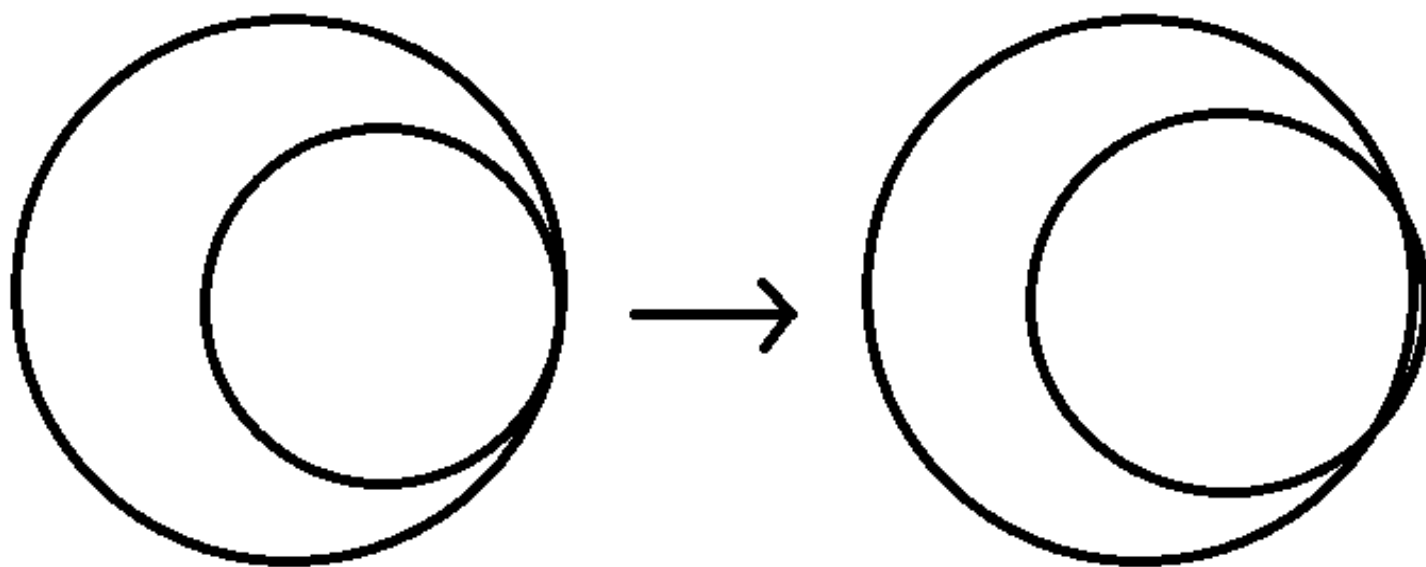
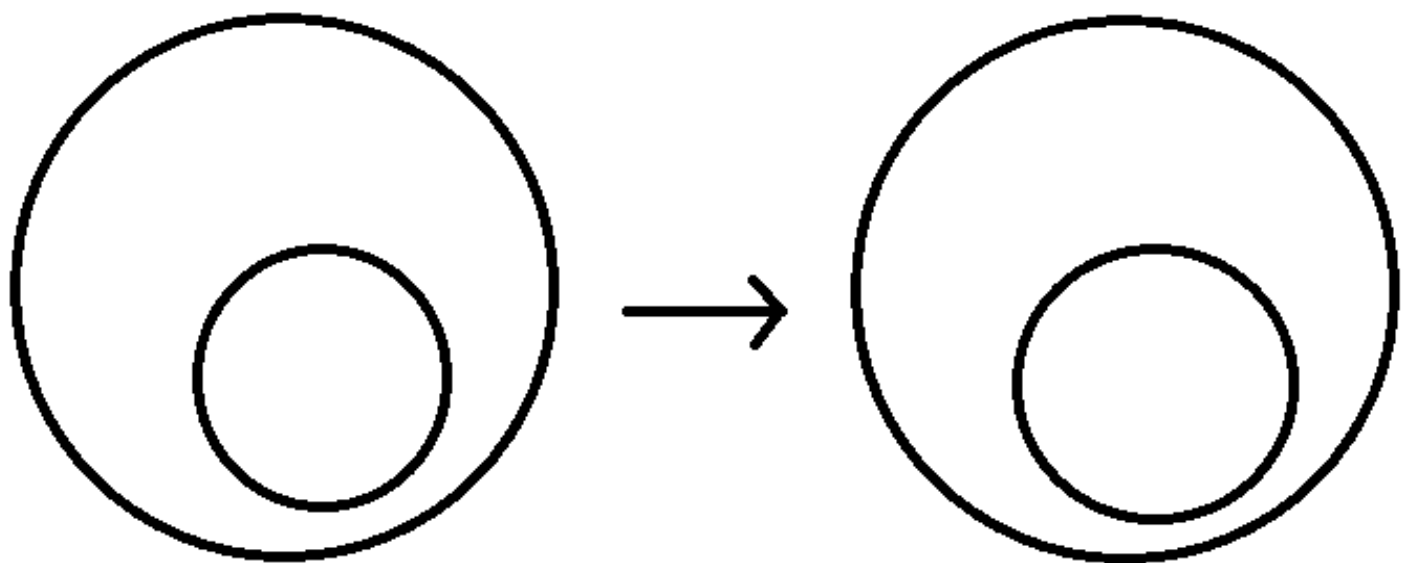
解決策 (2) – EPS の利用

- 判定式に以下の式を用いる

$$r2 + d + \varepsilon < r1$$

(ただし, ε はとても小さい正の数)

- 本当に $r2 + d < r1$ なら, とても小さい ε を足しても不等号の向きは変わらない
- 実際は $r2 + d = r1$ で, 誤差で $r2 + d < r1$ と判定されている場合, ε を足すことで不等号の向きが反転し, 正しく判定が行える
 - 実数の比較に頻出の手法



EPS の設定

- ε が大きすぎると, 実際は $r_2 + d < r_1$ なのに誤って $r_2 + d > r_1$ と判定されてしまう
- ε が小さすぎると, 誤差に負けてしまう

EPS の設定

- $r2 = \sqrt{a}$, $d = \sqrt{b}$, $r1 = \sqrt{c}$ とする
- ε は $\sqrt{c} - \sqrt{a} - \sqrt{b}$ の正の最小値未満にすれば OK
- $\sqrt{a} + \sqrt{b} = \sqrt{c} - e$ ($e > 0$) とおくと,

両辺 2 乗して $a + b + 2\sqrt{ab} = c - 2\sqrt{c}e$ (e^2 は無視)

移項して $2\sqrt{ab} = c - a - b - 2\sqrt{c}e$

両辺 2 乗して $4ab = (c - a - b)^2 - 4(c - a - b)\sqrt{c}e$

よって

$$e = \frac{(c - a - b)^2 - 4ab}{4(c - a - b)\sqrt{c}}$$

EPS の設定

$$e = \frac{(c-a-b)^2 - 4ab}{4(c-a-b)\sqrt{c}}$$

- 分子の絶対値の最小値は 1
- $c \leq 200$ 万なので,
分母の絶対値の最大値は 4×200 万 $\times \sqrt{200}$ 万
($\doteq 10^{10}$) 以下
- $\varepsilon = 10^{-11}$ で OK
 - 実際には, こんな細かいことを考えずに,
最初から $\varepsilon = 10^{-10}$ ぐらいにしておけばだいたい OK
 - ちなみに丸め誤差はだいたい 10^{-16} のオーダー

注意点

- ポスターの境界との大小判定は EPS で行わない
 - 円 2 つが接すると × だが, ポスター境界とは接しても○
 - ポスター境界の座標は整数なので, 普通に大小判定ができる
 - EPS を用いると, 接している場合にはみ出していると判定する可能性

まとめ

- 制限を見て使うアルゴリズムを決定
 - $N=50$ なら $O(N^4)$ でも通る→総当りで十分
 - 実はこの問題には $O(N^3)$ などのアルゴリズムが存在するが、使う必要はない
- 実数を扱うときは丸め誤差に注意
 - 2 乗して整数で比較
 - EPS の利用

講評

- 開始 2 時間半後以降も質問がいくつか来た
 - 質問の回答が返って来ない場合があります
 - 質問すること自体は良いことです
- 特に今合宿は問題の配置はタイトルの辞書順なので、本問のように簡単な問題が後半にあることも
- できればコンテスト開始後早めに全ての問題に目を通しましょう

Construction / 建設計画

2013/03/21

JOI Spring Camp day2

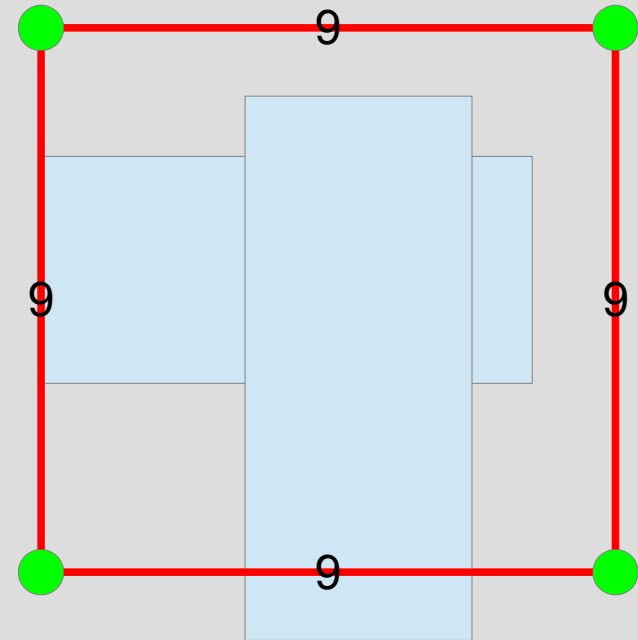
Shogo Murai

問題の概要

- N 個の町があり、国際空港と道路を建設する
 - 道路は、町同士を結ぶ x, y 軸に平行な線分
 - 長さ L の道路を建設するのにはコストが L かかる
 - M 個の長方形の領域には道路を建設できない
 - どの町からも道路を通して国際空港のある町に行けなくてはならない
-
- 建設業者が C 社あり、それぞれ国際空港の建設コストと最大建設数が決まっている
 - 交通網の整備にかかる各社のコストを求める

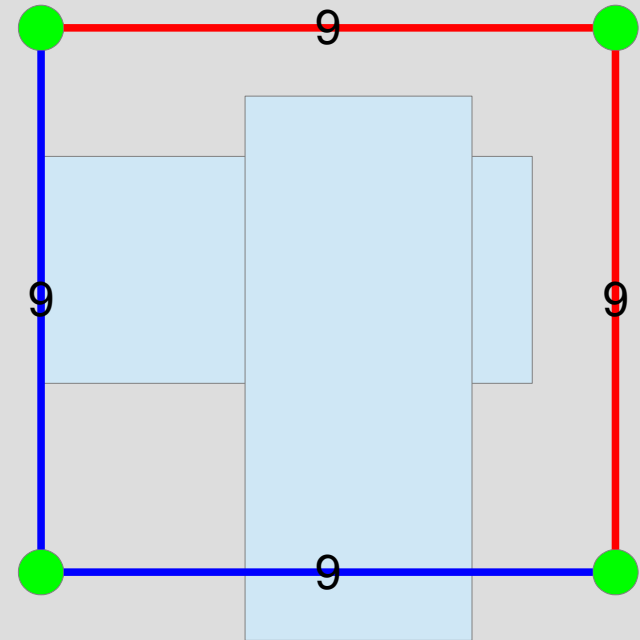
サンプルの解説 (1)

- が町、■が道路を敷設できない場所
- 赤線は引けるかもしれない道路
- 数字は道路の長さ
- 斜めには道路を敷設できない



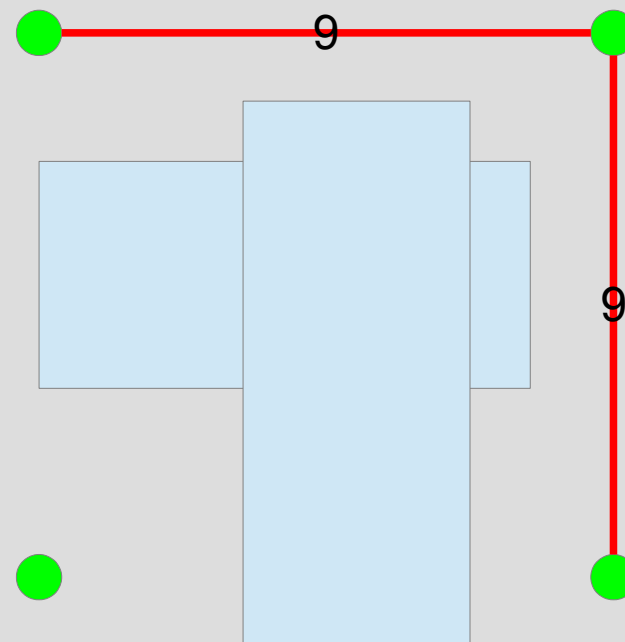
サンプルの解説 (2)

- 青色の道路は□と交わる
→ 建設できない
- 長方形の外周部と交わってもだめ



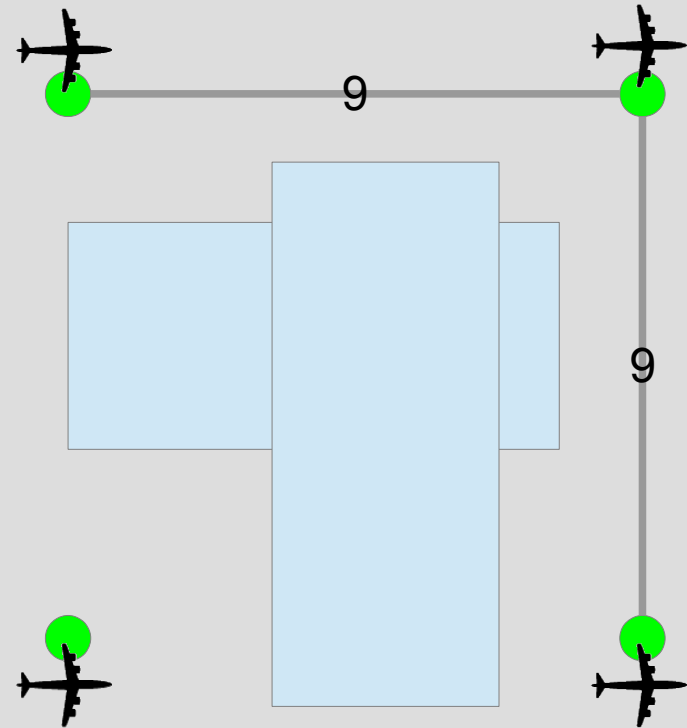
サンプルの解説 (3)

- 使える道路はこんな感じ



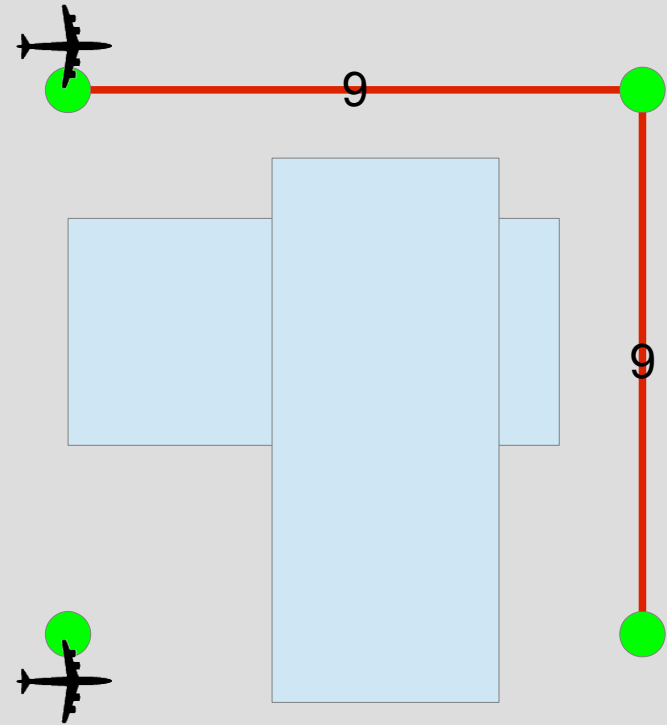
サンプルの解説 (4)

- 建設会社 1
 - 空港のコスト: 7
 - 建設できる数: 4
- 右のように建設するのが最適 (✈: 空港)
- コストは 28



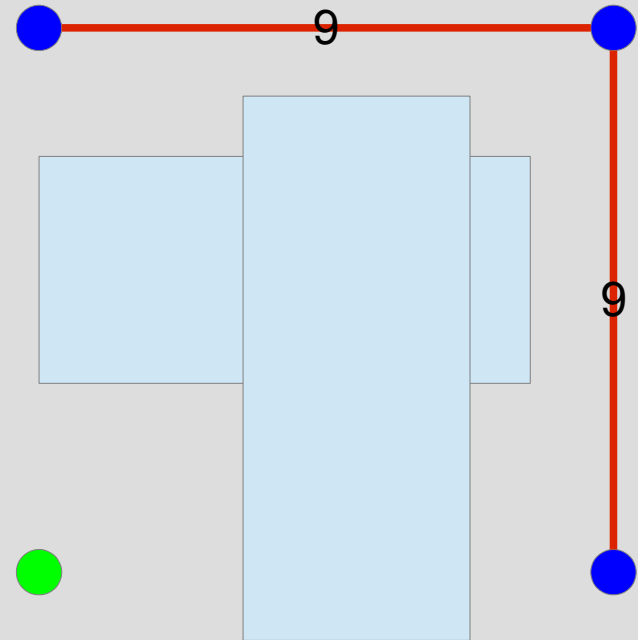
サンプルの解説 (5)

- 建設会社 2
 - 空港のコスト: 10
 - 建設できる数: 3
- 例えば、右のように建設するのが最適
- コストは 38



サンプルの解説 (6)

- 建設会社 3
 - 空港のコスト: 1
 - 建設できる数: 1
- どうやったらって空港 2 つは必要
- 無理、答えは -1

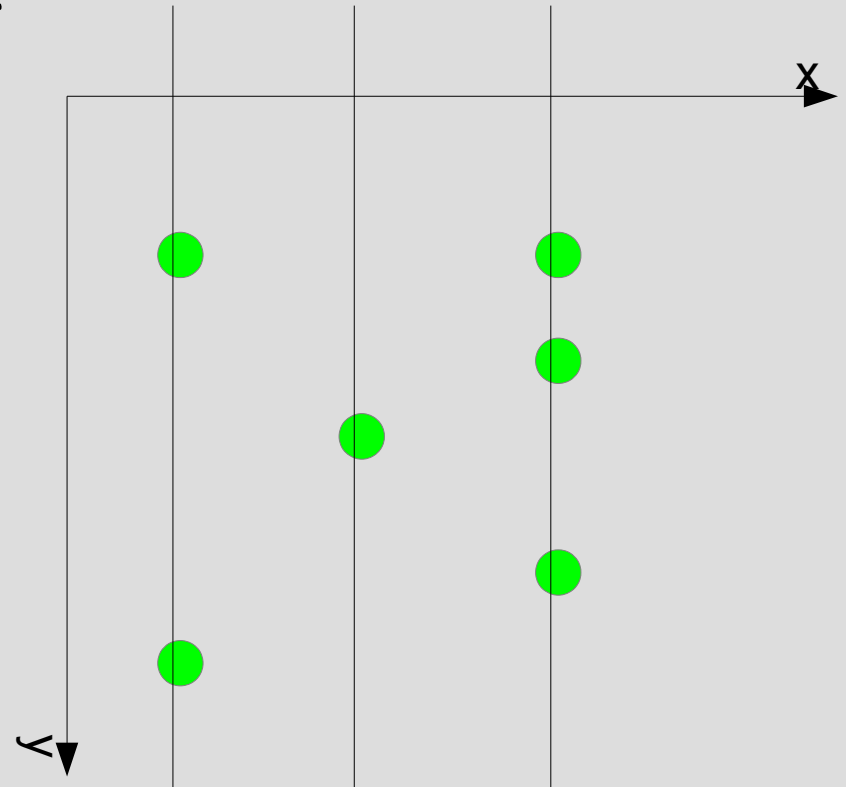


考えること

1. 建設できる道路はどれ？
2. 道路はどの長方形とも交わらない？
3. 使える道路網がわかったとき、道路のよい建設方法は？
4. 各社の建設コストは？

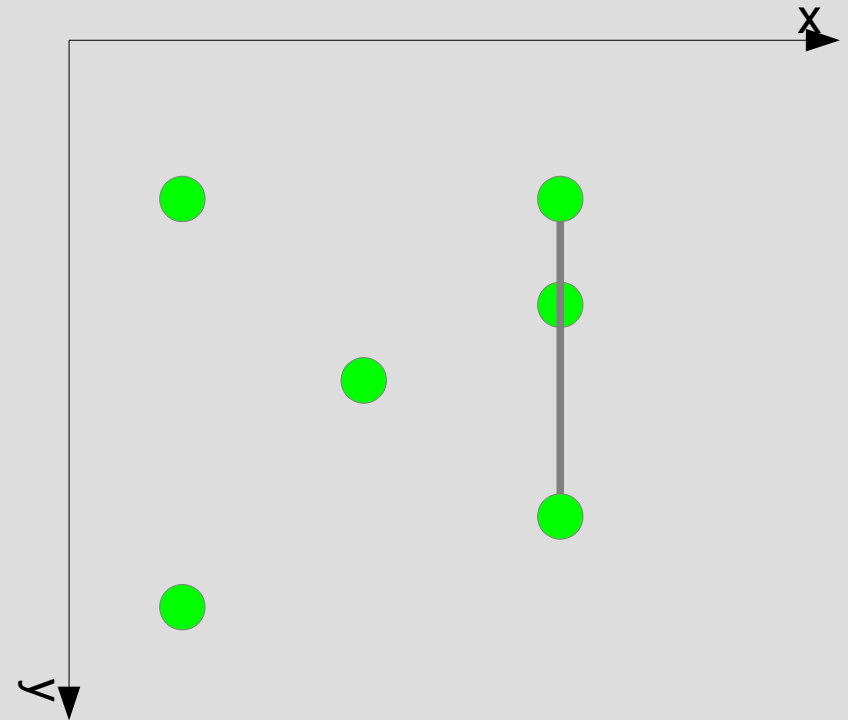
1. 建設できる道路 (1)

- とりあえず y 軸に平行なものだけ考える
- すると、道路が結ぶ町の x 座標は等しい



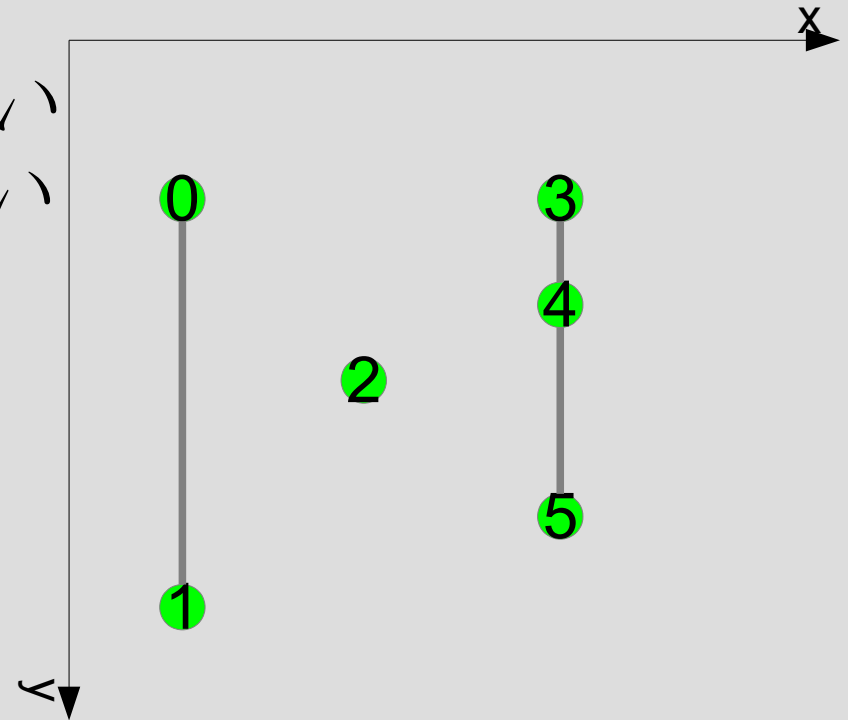
1. 建設できる道路 (2)

- 間に他の町があるのに飛び越えて道路を建設するのは無意味
- 間に町がない道路だけ考えれば良い



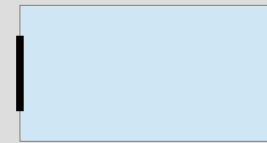
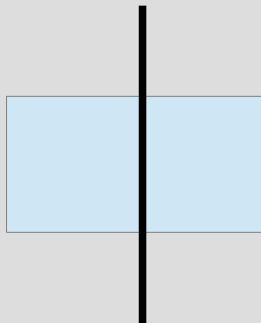
1. 建設できる道路 (3)

- (x, y) を `pair` としてソート
 - ソート列上で隣り合っていない所には道路はない
 - $N-1$ 箇所だけ見ればよい
 - 計算量: $O(N \log N)$
 - 道路は高々 $N-1$ 本
-
- 2. をナイーブにやると $O(NM)$



2. 長方形との交差判定(1)

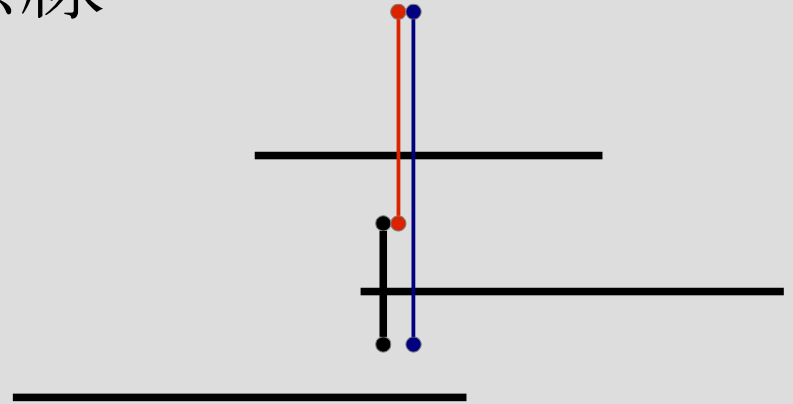
- y 軸に平行な線分と長方形が交わる条件は？
- 長方形の x 軸に平行な 2 辺だけ見ればよい (理由)
- y 軸に平行な辺と共通部分を持つとき、問題文の条件から端点と共通部分を持つ
- よってこのとき x 軸に平行な辺とも交わる



こんなもの
はない！

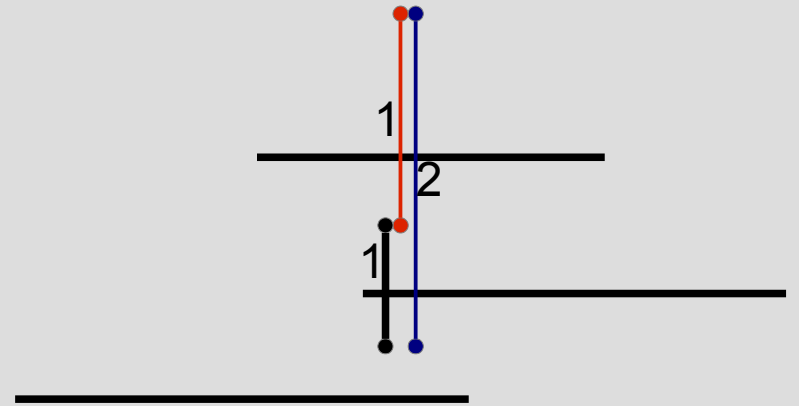
2. 長方形との交差判定 (2)

- 線分と線分の交差判定を考えればよい
- y 軸に平行な線分が x 軸に平行な線分たちとそれぞれ何回交わるかわかればよい
- 図の、(赤線が交わる回数) - (図の青線が交わる回数) で黒線が交わる回数は求まる



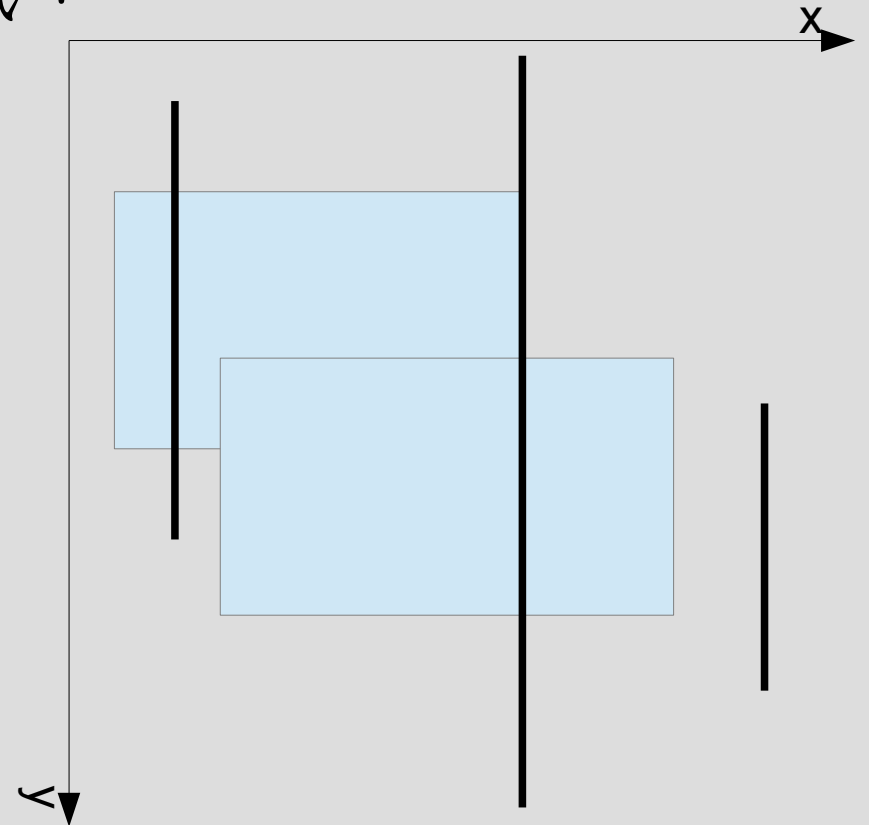
2. 長方形との交差判定 (3)

- y 座標の小さいほうから見ていく (走査)
- 走査の過程で赤線、青線の終端が出てきたら、その線のある x 座標に対し、今までにそこをまたいだ線分を求める
- その方法は後述



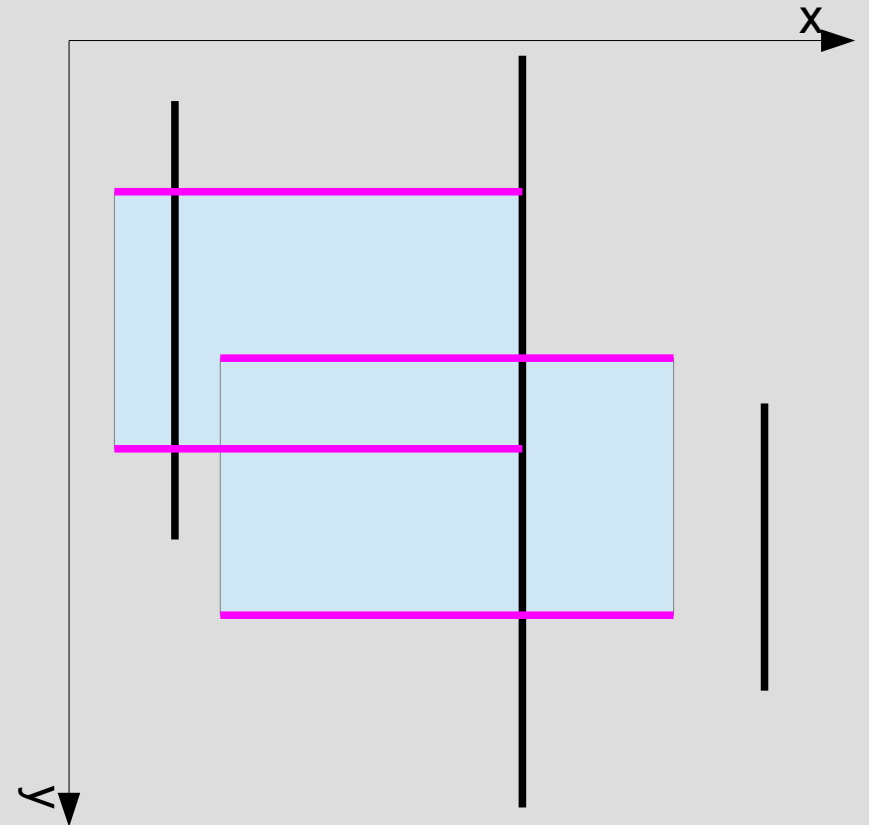
2. 長方形との交差判定 (4)

これだけでは分かりにくい
ので具体例を挙げて
説明します(右図)



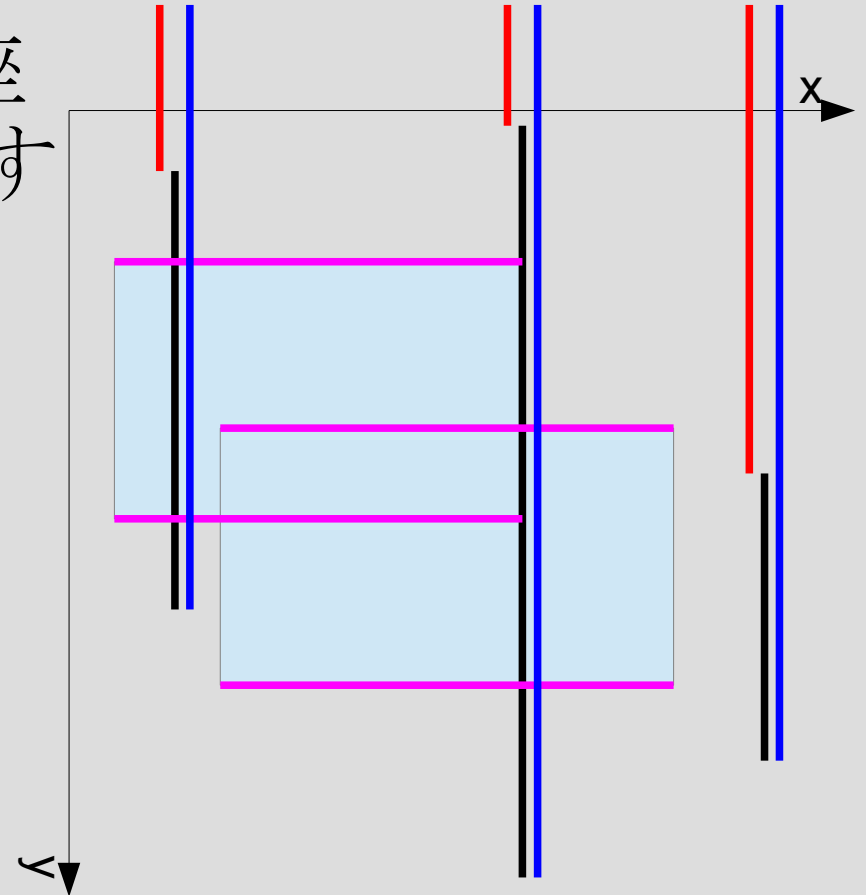
2. 長方形との交差判定 (5)

- 長方形の x 軸に平行な辺だけを取り出す



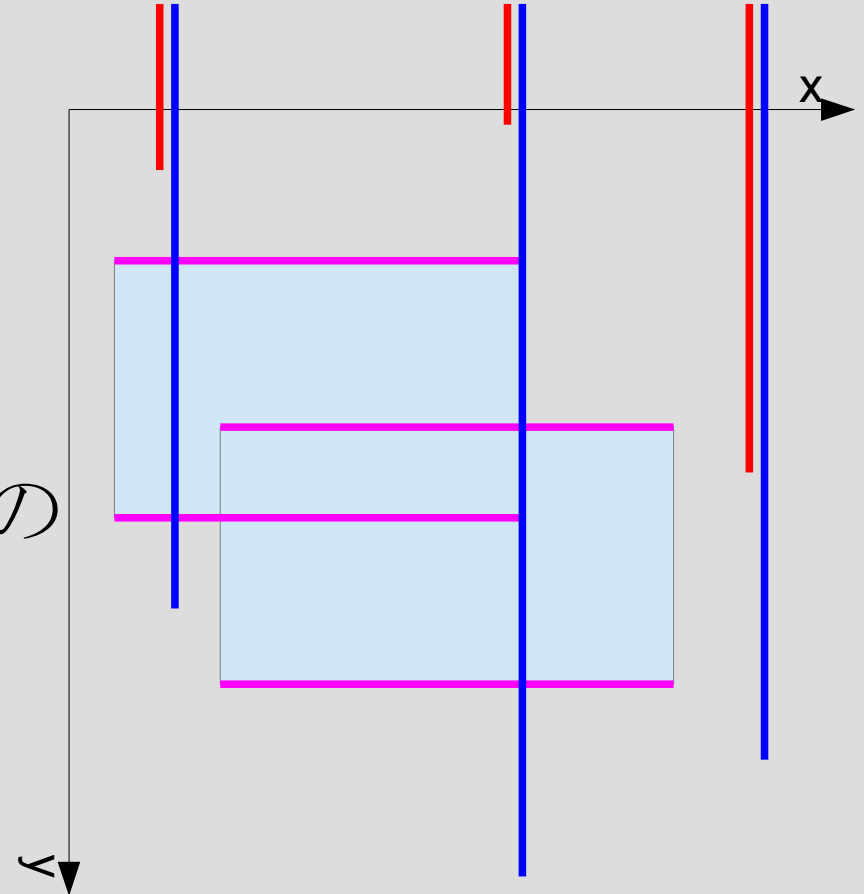
2. 長方形との交差判定 (6)

- 各道路に対し、同じ x 座標に青線と赤線を生やす
- 青線、赤線は $y = -\text{inf}$ から生えている



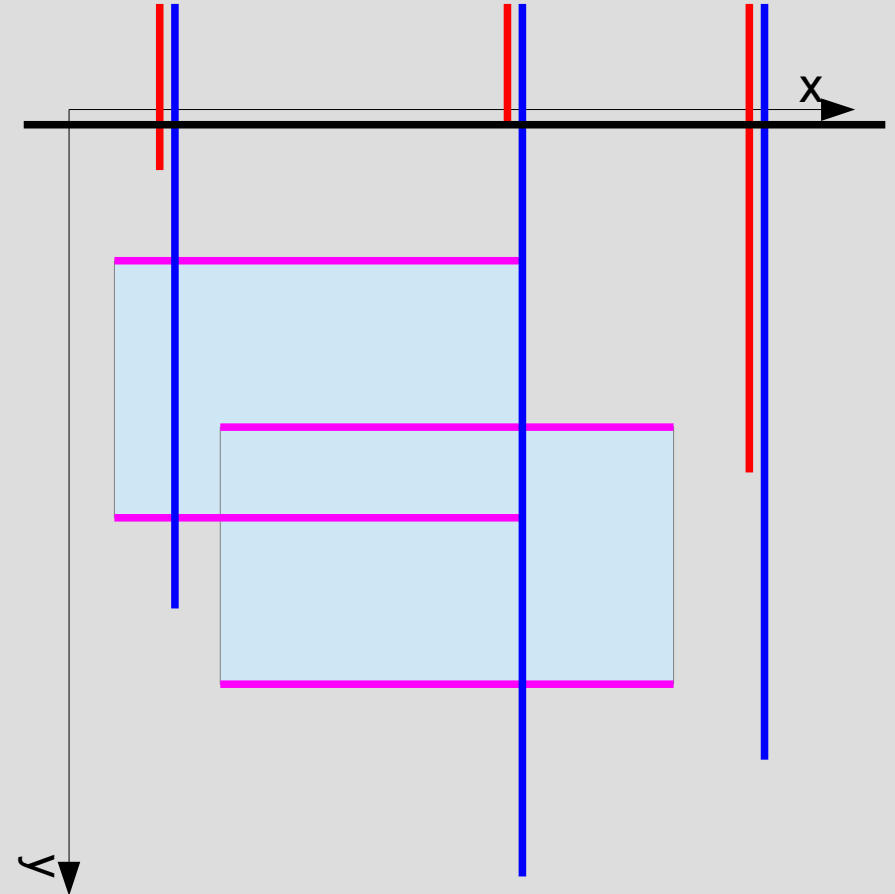
2. 長方形との交差判定 (7)

- y 座標の小さいほうから走査線を動かし、
 - 赤線、青線の下端
 - x 軸に平行な線分が出てくるたびに処理
- 実装上は、下端や線分の y 座標でソートして処理



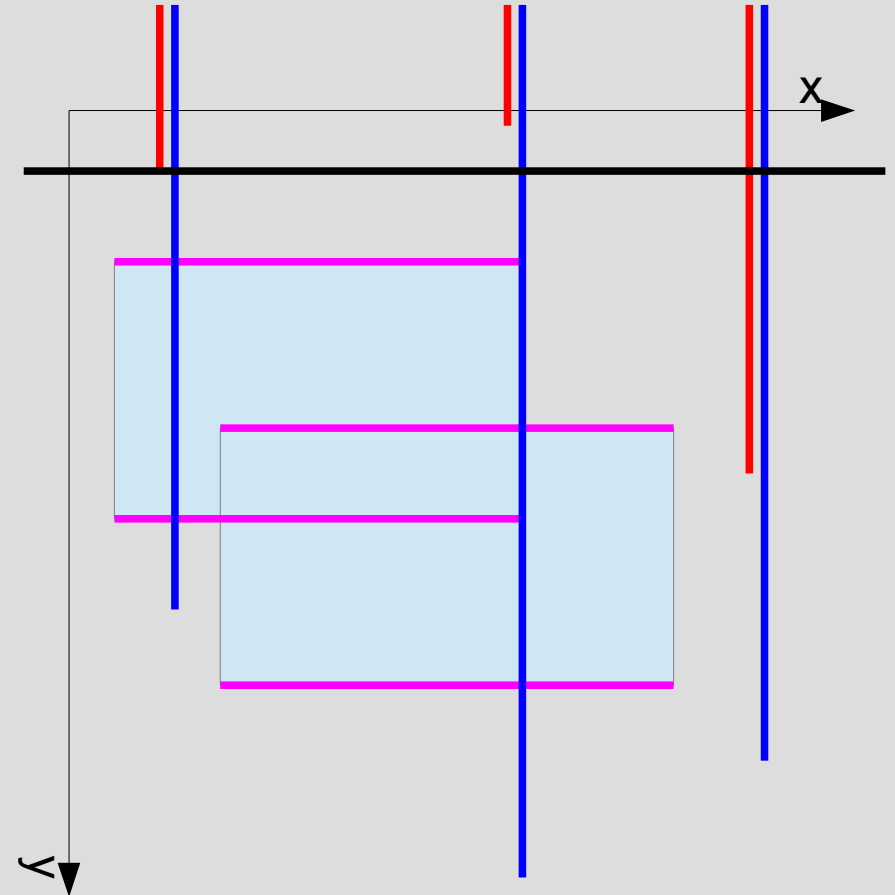
2. 長方形との交差判定 (8)

- 赤線の下端が現れた
- この x 座標で今までに見かけた紫線の数: 0



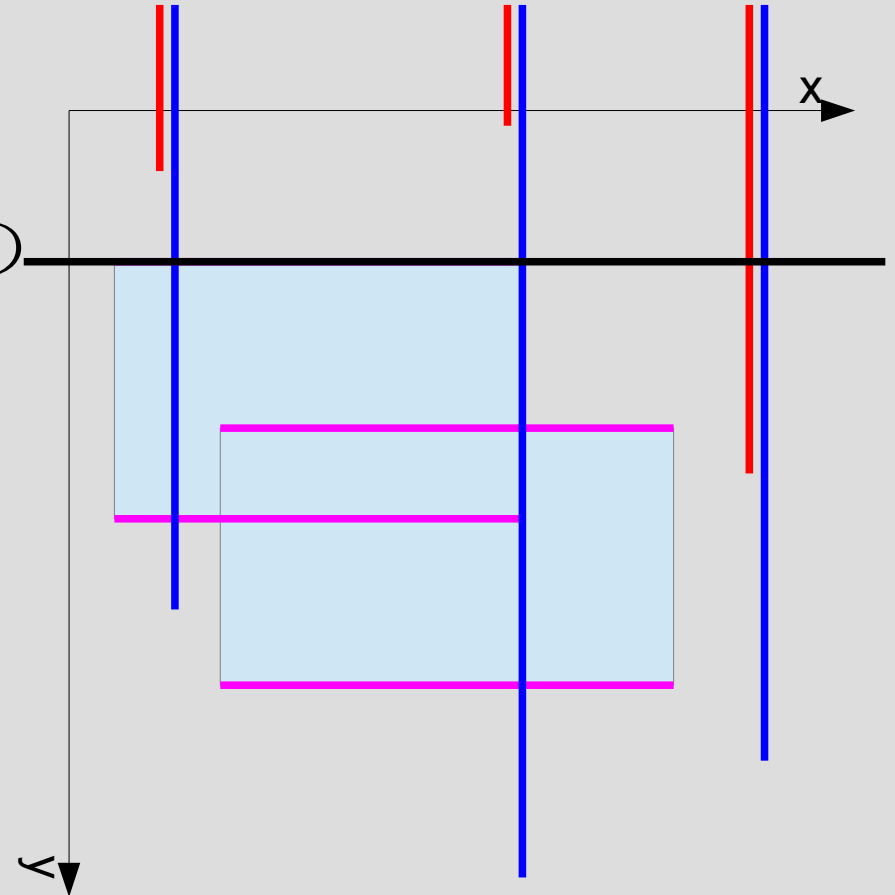
2. 長方形との交差判定 (9)

- 赤線の下端が現れた
- この x 座標で今までに見かけた紫線の数: 0



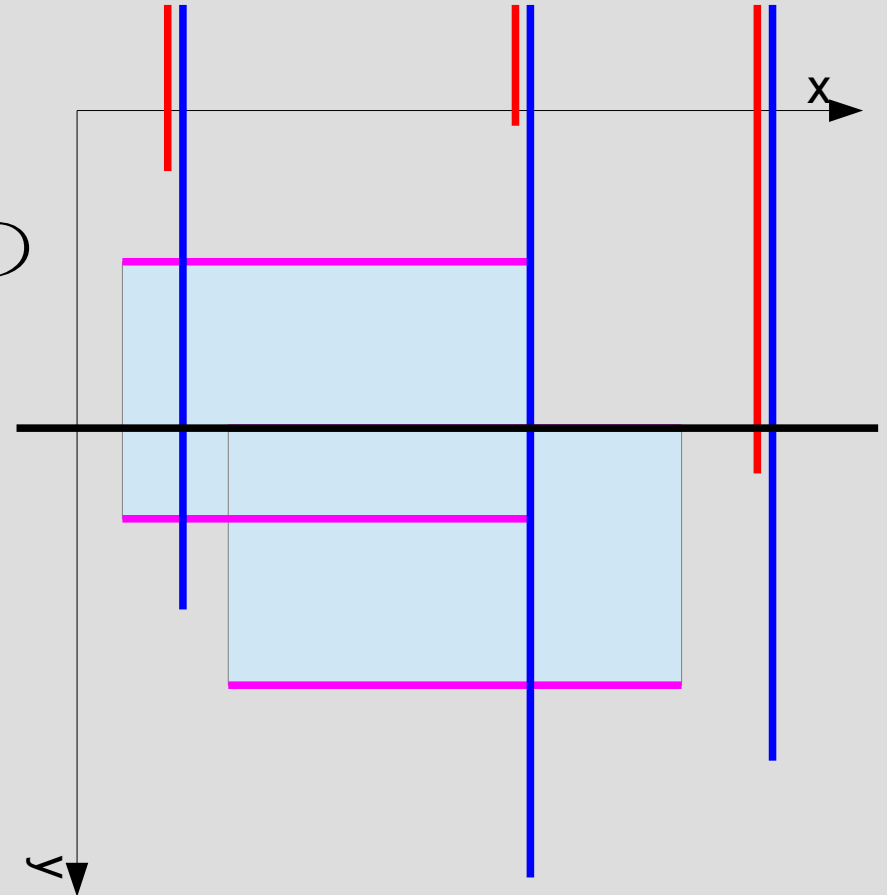
2. 長方形との交差判定 (10)

- 紫線が現れた
- 紫線が占める x 座標の範囲で、出てきた紫線の数に **1** を加算



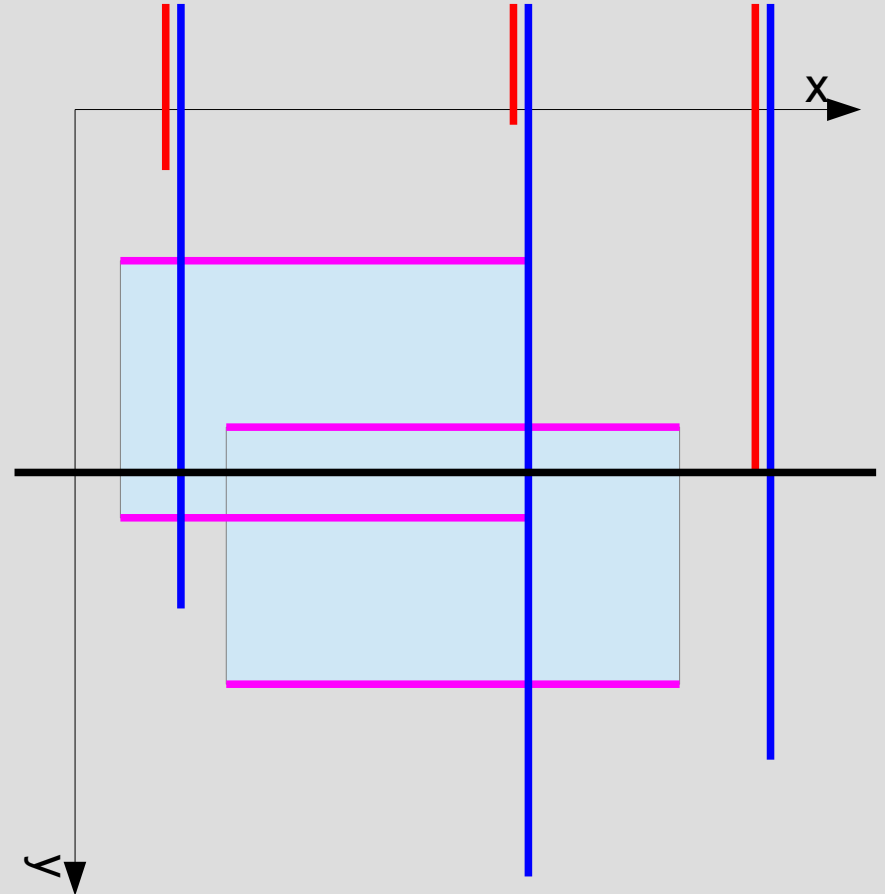
2. 長方形との交差判定 (11)

- 紫線が現れた
- 紫線が占める x 座標の範囲で、出てきた紫線の数に **1** 加算



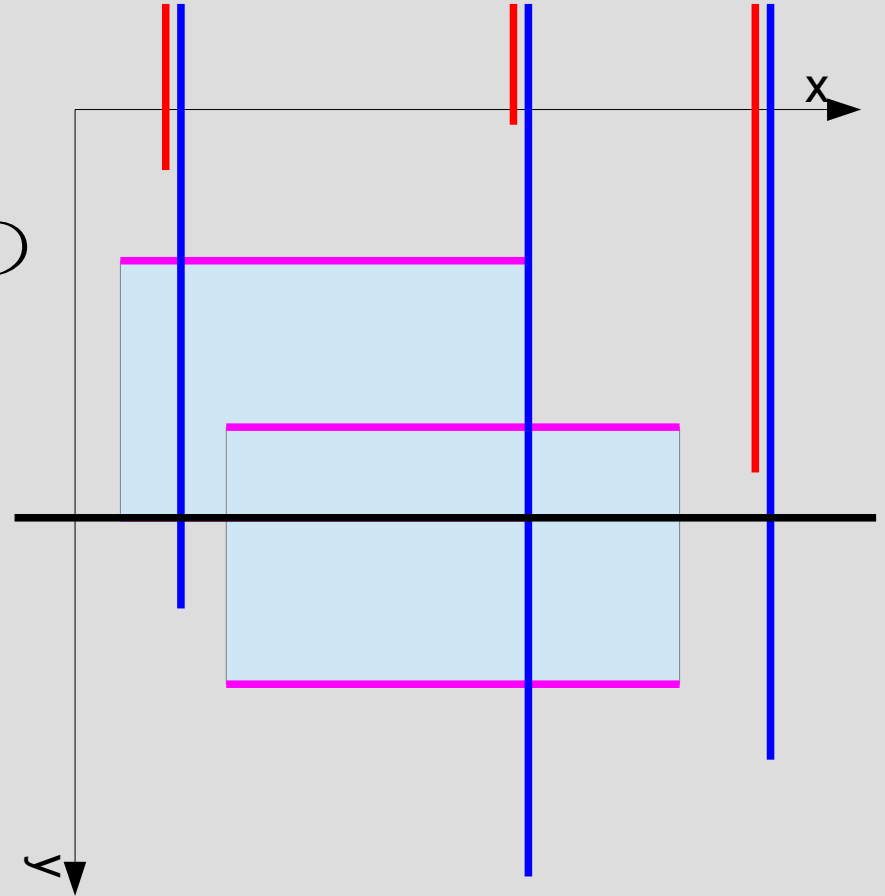
2. 長方形との交差判定 (12)

- 赤線の下端が現れた
- この x 座標で今までに見かけた紫線の数: 0



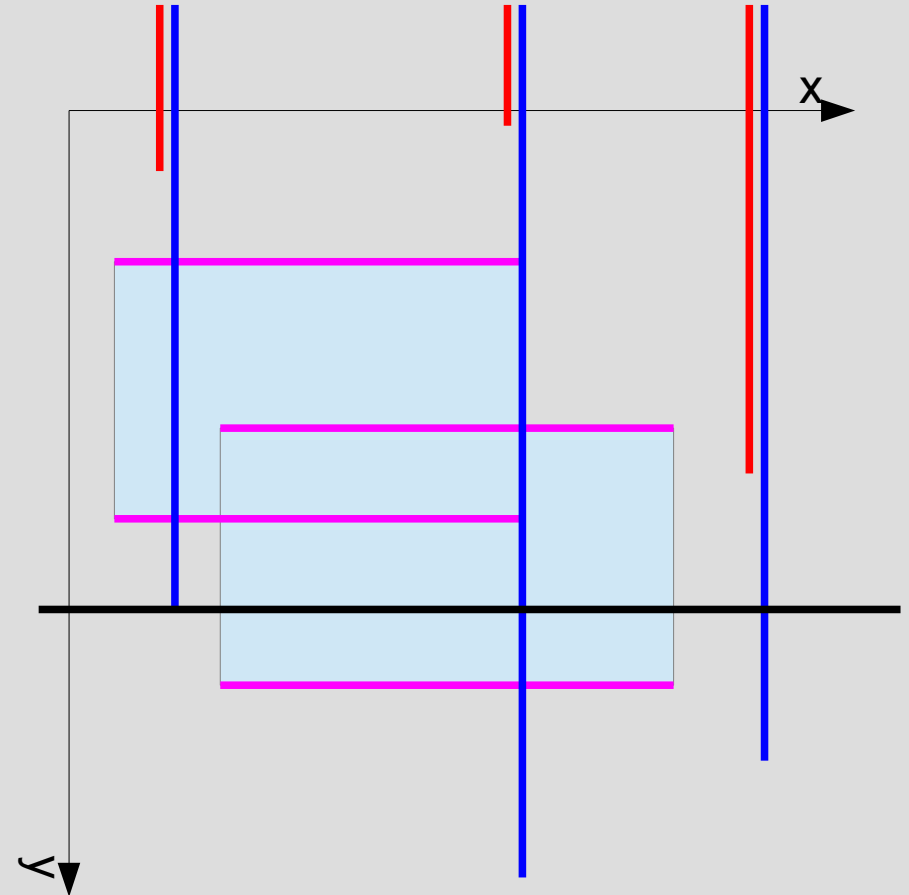
2. 長方形との交差判定 (13)

- 紫線が現れた
- 紫線が占める x 座標の範囲で、出てきた紫線の数に **1** 加算



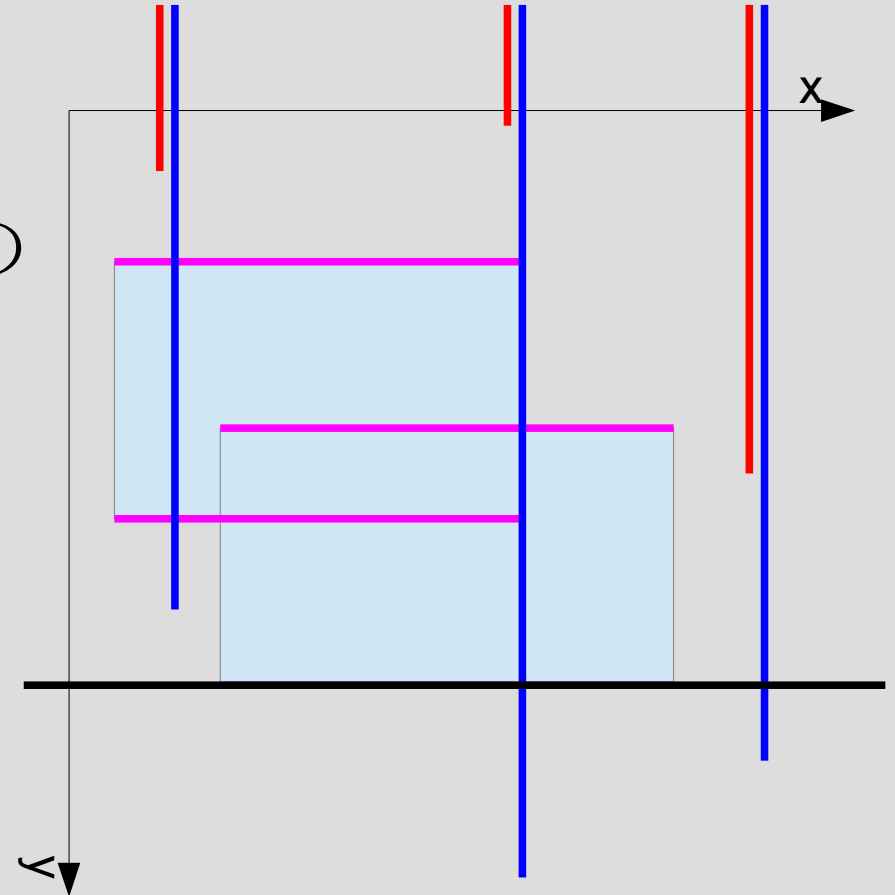
2. 長方形との交差判定 (14)

- 青線の下端が現れた
- この x 座標で今までに見かけた紫線の数: 2



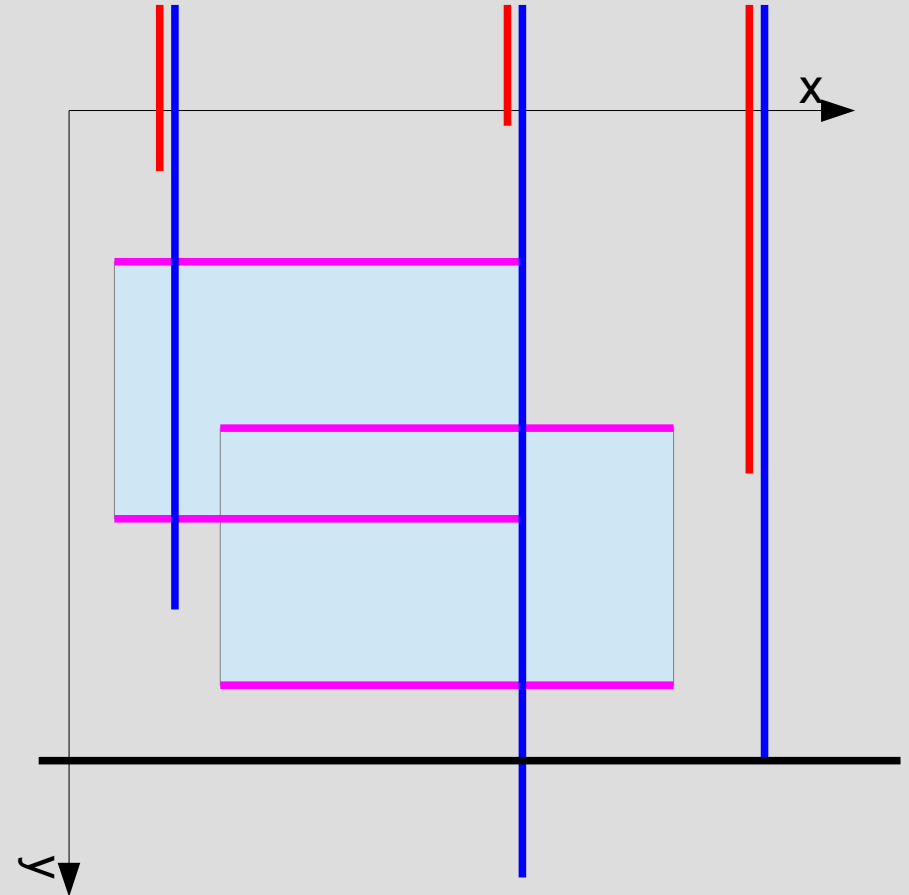
2. 長方形との交差判定 (15)

- 紫線が現れた
- 紫線が占める x 座標の範囲で、出てきた紫線の数に **1** 加算



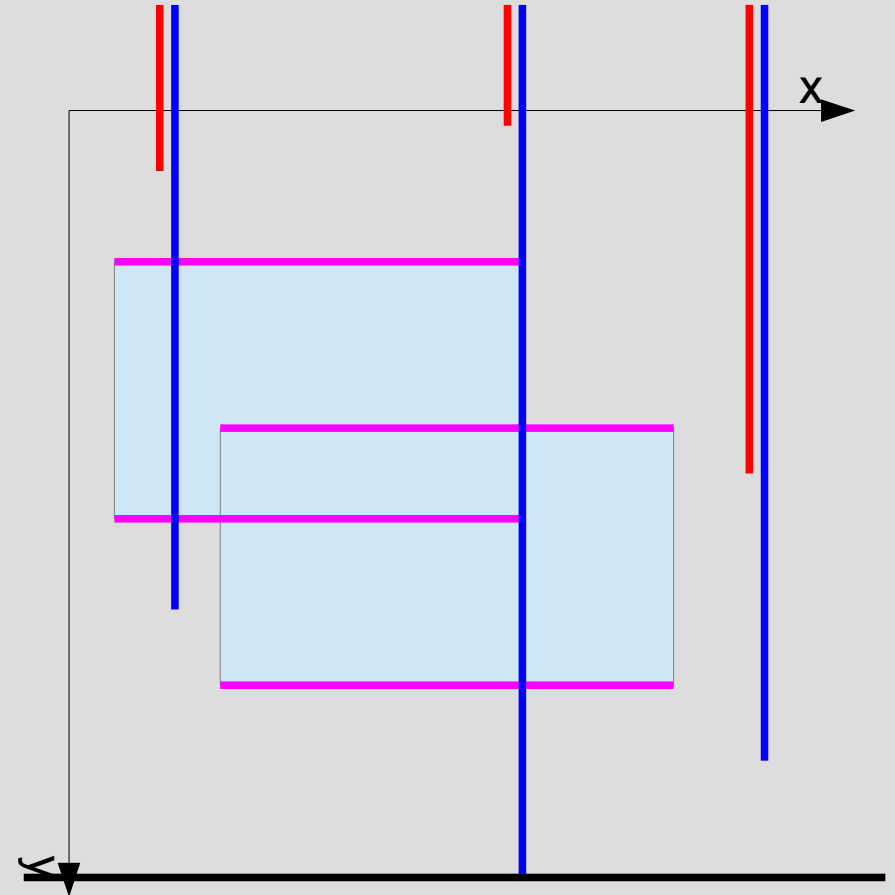
2. 長方形との交差判定 (16)

- 青線の下端が現れた
- この x 座標で今までに見かけた紫線の数: 0



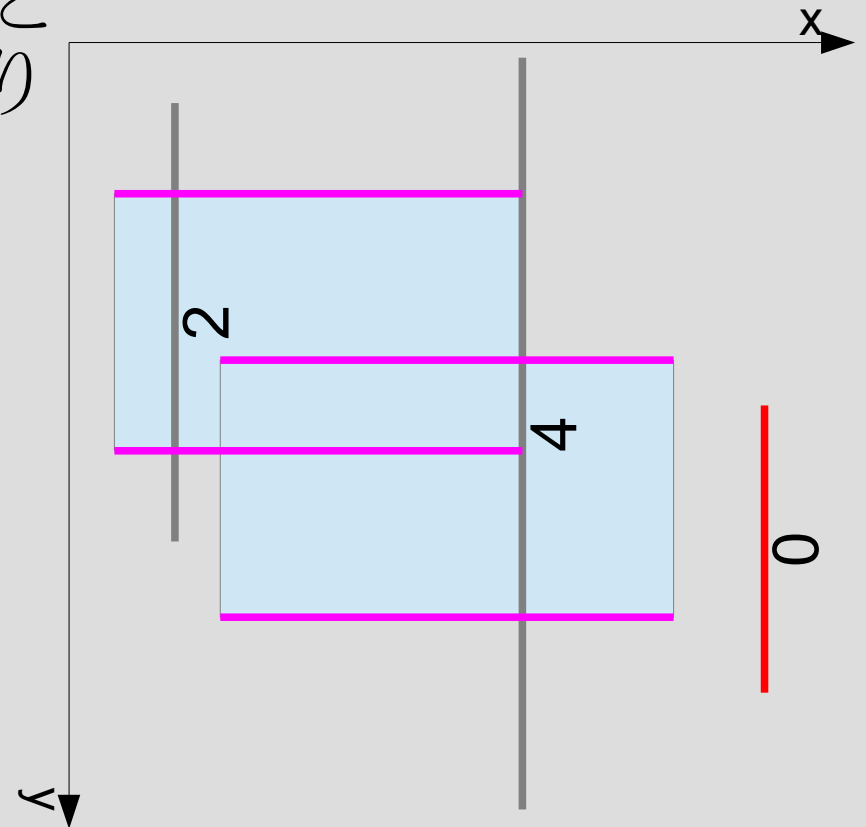
2. 長方形との交差判定 (17)

- 青線の下端が現れた
- この x 座標で今までに見かけた紫線の数: 4



2. 長方形との交差判定 (18)

- それぞれの道路が紫線と交わる回数は右図の通り
- 0 なものだけ建設できる



2. 長方形との交差判定 (19)

- **16** ページ前で「後述」とした、出てきた紫線の数
を数える方法が未解決
- ここでやりたいことは、
 - x 座標の、ある範囲にすべて **1** を加算
 - ある点の値を求めるの **2** つを高速に計算すること
- まさか、愚直に足したりするわけにはいかない！

2. 長方形との交差判定 (20)

- x 座標が $1,000,000,000$ もあって大変
→ 現れる x 座標をソートし、座標圧縮
- 座標圧縮の詳細については省略
 - 多分蟻本に載っています
 - 今年度の予選の 5 の解説も参照
- x 座標は $N+2M$ 個くらいしか現れない
- 高々 $600,000$ 個

2. 長方形との交差判定 (21)

- 要素数高々 $600,000$ 個の配列で、「区間に 1 を足す」「ある場所の値を求める」をしたい
- **!!! Binary Indexed Tree(BIT) !!!**
- BIT の実装は蟻本に書いてあるので省略します
- $a, a+1, \dots, b$ の位置に 1 を加えるときには、
 $v[a] += 1, v[b+1] -= 1;$
- すると、位置 x の値は $v[0] + \dots + v[x]$ で求まる

2. 長方形との交差判定 (22)

- 以上の計算による計算量は？
 - 走査線がぶつかる座標 ($N+2M$ 個) をソート
 $O((N+M)\log(N+M))$
 - $N+2M$ 回のそれぞれで、BIT に $O(1)$ 回アクセス
 $O((N+M)\log(N+M))$
- よって $O((N+M)\log(N+M))$
- この問題のような、「平面走査＋データ構造」は
JOI でよく見る手法です
ex. Starry Sky(09sp), Fortune Telling(12sp)

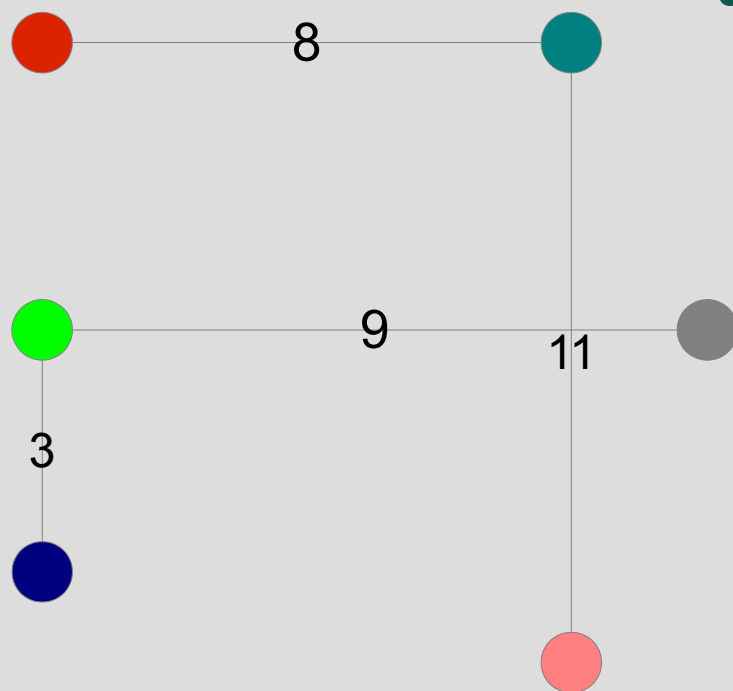
3. 道路のよい建設方法 (1)

- i 個の空港を建設するときの道路コストは？
- $N-i+1$ 個、互いに道路で結ばれた部分が必要
- 各部分に 1 つずつ空港を置く

3. 道路のよい建設方法 (2)

- 最小全域木問題に似ている？
- Kruskal 法 (の少し応用)
- Kruskal 法の過程で記録すると、 $N-i+1$ 個の連結成分にするための最小コストが簡単にわかる
- 辺の数は $O(N)$ なので、Kruskal の計算量は $O(N \log N)$ (辺のソート)

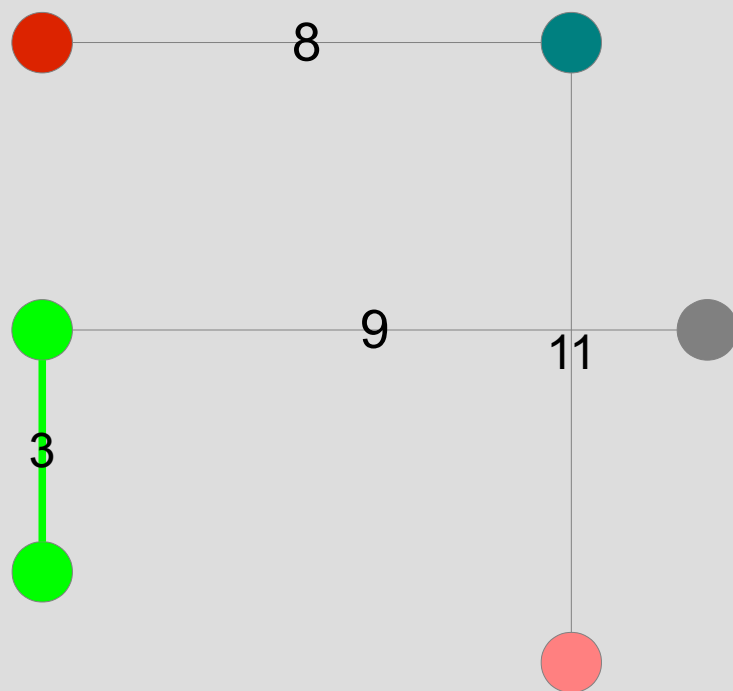
3. 道路のよい建設方法 (3)



- 短い辺から順に使う

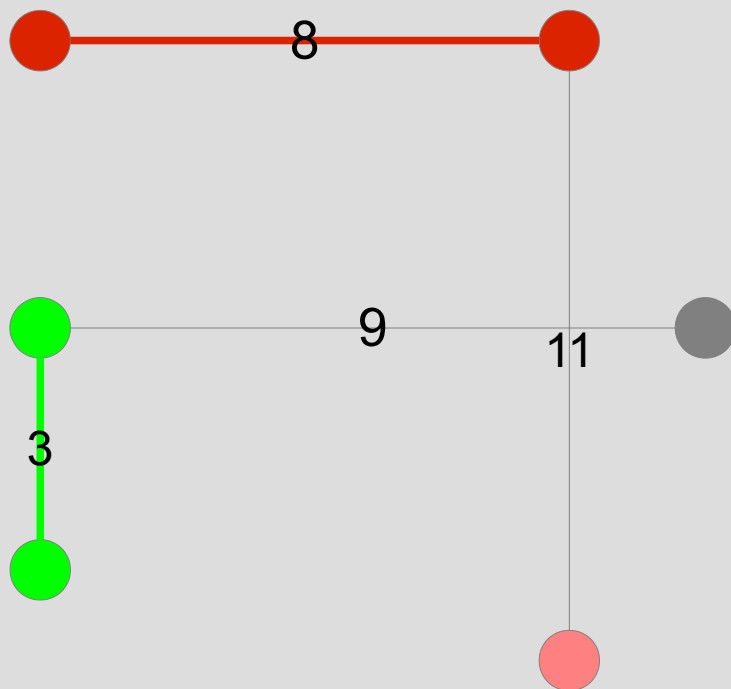
連結成分数	最小コスト
1	?
2	?
3	?
4	?
5	?
6	0

3. 道路のよい建設方法 (4)



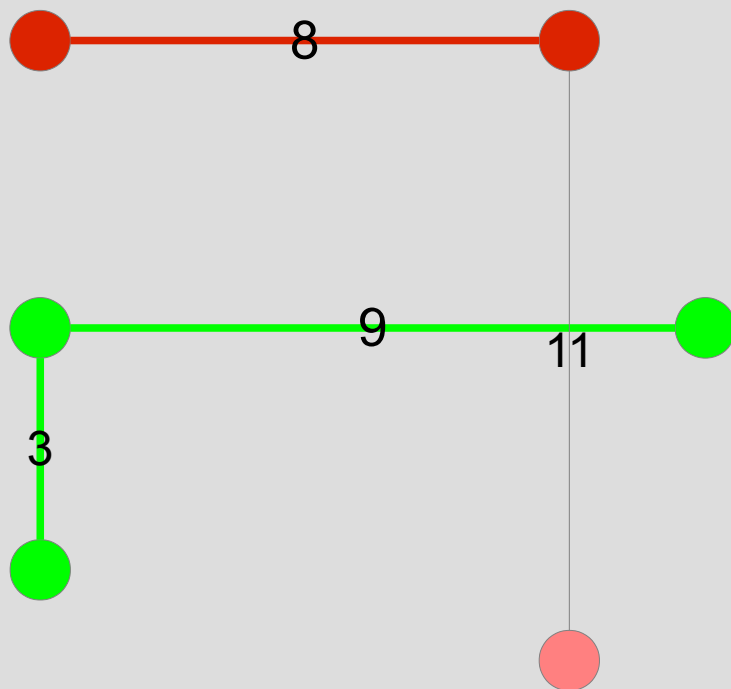
連結成分数	最小コスト
1	?
2	?
3	?
4	?
5	3
6	0

3. 道路のよい建設方法 (5)



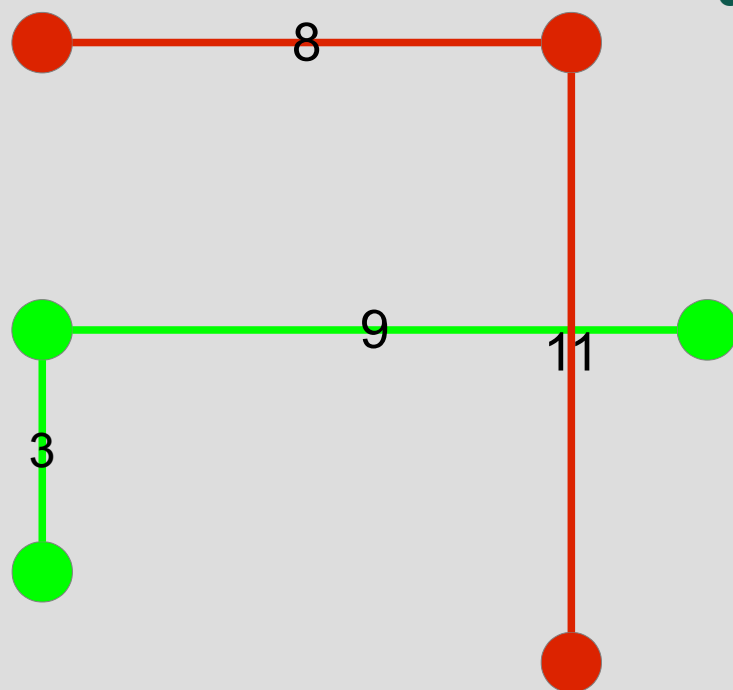
連結成分数	最小コスト
1	?
2	?
3	?
4	11
5	3
6	0

3. 道路のよい建設方法 (6)



連結成分数	最小コスト
1	?
2	?
3	20
4	11
5	3
6	0

3. 道路のよい建設方法 (7)



- これ以上連結できない

連結成分数	最小コスト
1	∞
2	31
3	20
4	11
5	3
6	0

4. 各社の建設コスト(1)

- 普通にやると $O(NC)$ だけでもっと速くしたい
- i 個の空港を建設するときの道路コストを $\text{cost}[i]$ とする
- $C[i] = B * i + \text{cost}[i]$ を考えると、
 $1 \leq i \leq H$ での $C[i]$ の最小値が答え

4. 各社の建設コスト (2)

- $C[i+1] - C[i] = B - (\text{cost}[i] - \text{cost}[i+1])$
- $\text{cost}[i]$ は、 i 個の連結成分を作る場合のコスト
- $\text{cost}[i] - \text{cost}[i+1]$ は、連結成分の個数を $i+1 \rightarrow i$ にするとき増やす辺のコスト
- Kruskal のアルゴリズムの動作から、辺を増やせば増やすほど新たに必要なコストも増していく

4. 各社の建設コスト (3)

- つまり、 $\text{cost}[i] - \text{cost}[i+1]$ は単調減少
- これが B 以上である限り i を大きくしてよい
- 二分探索を使うと適切な i を求められる
- 建設できる空港が少なすぎる場合に注意
- これで計算量は $O(C \log N)$

4. 各社の建設コスト (4)

- また例 (さっきのグラフと同じ) を挙げます

i	cost[i]	cost[i]-cost[i+1]
1	∞	∞
2	31	11
3	20	9
4	11	8
5	3	3
6	0	—

4. 各社の建設コスト (5)

- $B=10$ とすると、 $\text{cost}[i] - \text{cost}[i+1]$ が 10 未満になるのは $i=3$
- H を無視すると $i=3$ が最適 ?

i	cost[i]	cost[i]-cost[i+1]
1	∞	∞
2	31	11
3	20	9
4	11	8
5	3	3
6	0	—

4. 各社の建設コスト (6)

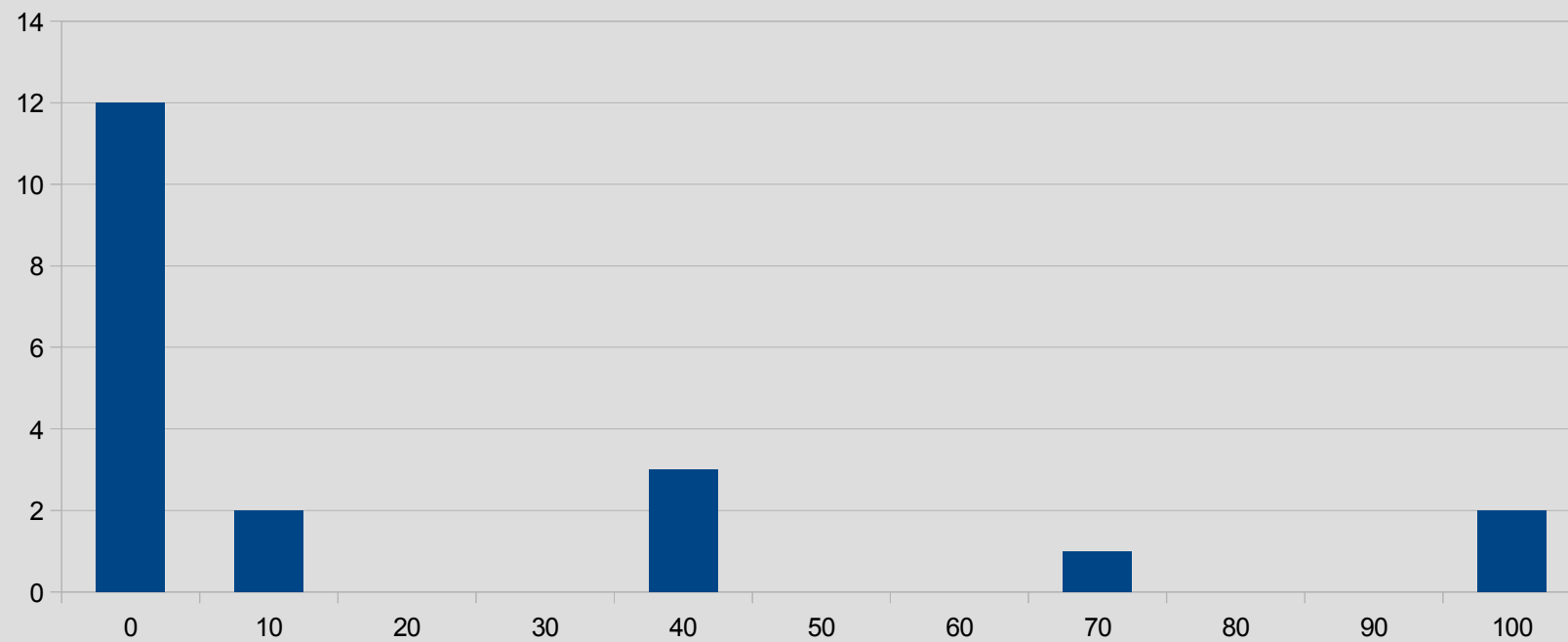
- 実際に $C[i]$ を計算してみます
- やっぱり $i=3$ が最善

i	cost[i]	cost[i]-cost[i+1]	C[i]
1	∞	∞	∞
2	31	11	51
3	20	9	50
4	11	8	51
5	3	3	53
6	0	—	60

まとめ

- ナイーブにやると $O(NM+CN)$
 - 10 点 (Subtask 1)
- 2. を頑張ると $O((N+M)\log(N+M)+CN)$
 - 40 点 (Subtask 1, 2)
- 4. を頑張ると $O(NM+C\log N)$
 - 40 点 (Subtask 1, 3)
- 両方頑張ると $O((N+M)\log(N+M)+C\log N)$
 - 100 点 (Subtask 1, 2, 3, 4)

得点分布





JOI 2013 春合宿 Day2

Mascots 解説

解説：城下慎也
(phidnight)

問題概要

- マスコットを縦 R 行，横 C 列の長方形に整列する.
- 途中で全体が長方形になる回数（少し幸せになる回数）が最大となるようにする置き方は全部で何通りあるか？
- マスコットの種類の違いは考えず，置く順番のみ考える.

問題概要

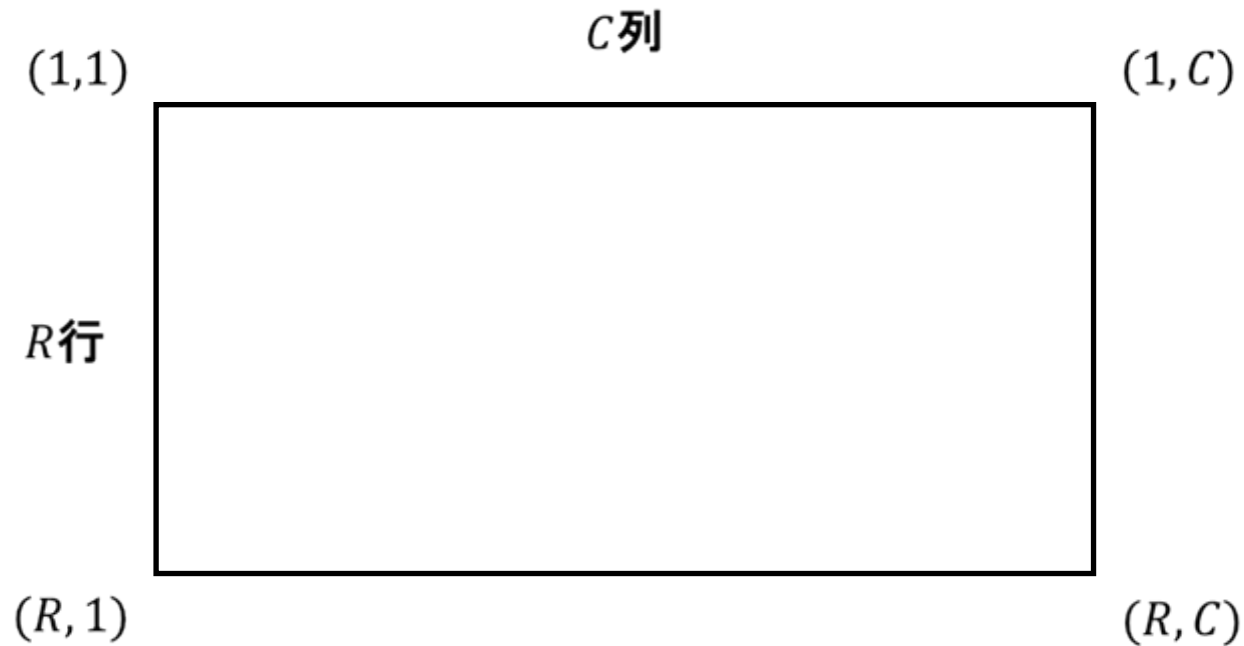
- マスコットを縦 R 行，横 C 列の長方形に整列する.
- 途中で全体が長方形になる回数（少し幸せになる回数）が最大となるようにする置き方は全部で何通りあるか？
- マスコットの種類の違いは考えず，置く順番のみ考える.
- 初日の講義でやったやつだ!!!!

全探索の解法

- 全ての順番を試し，置くたびに長方形になっているか判定する.
- 空きマスは $O(RC)$ なので長方形の判定を $O(RC)$ として全体で $O(RC(RC)!)$
- $R, C \leq 3$ となる10点のケースはとれる.
- $R, C \leq 50$ や $R, C \leq 3,000 \rightarrow$ ヤバい

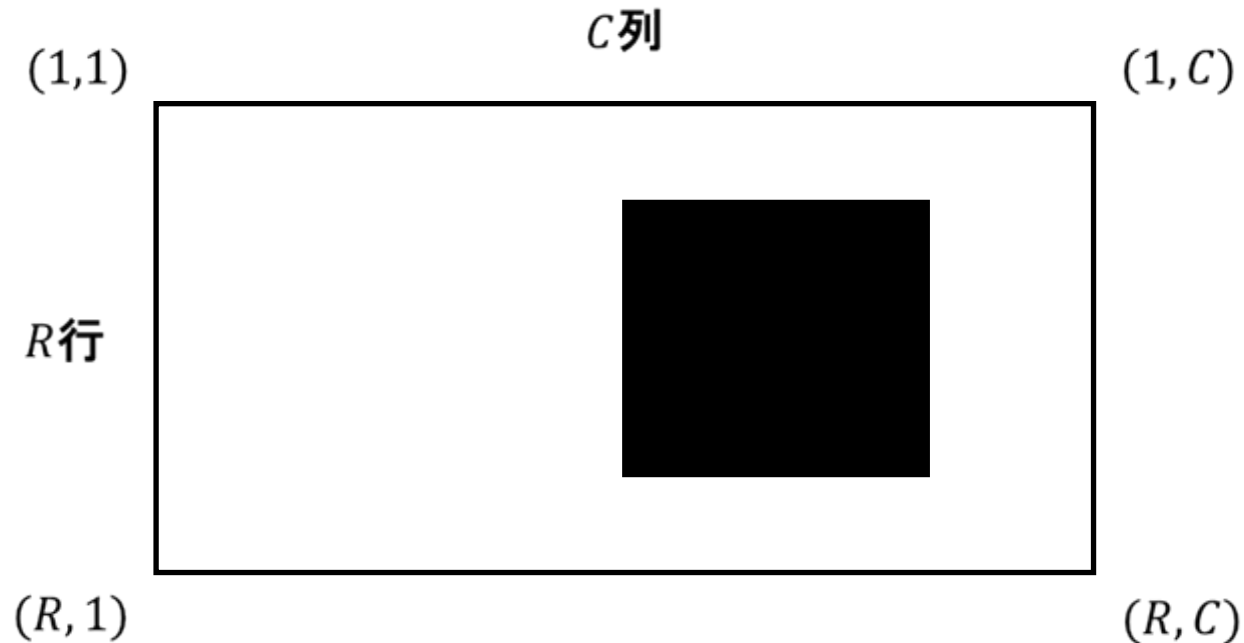
最大値とは?

- そもそも最大値はいくらになるか?



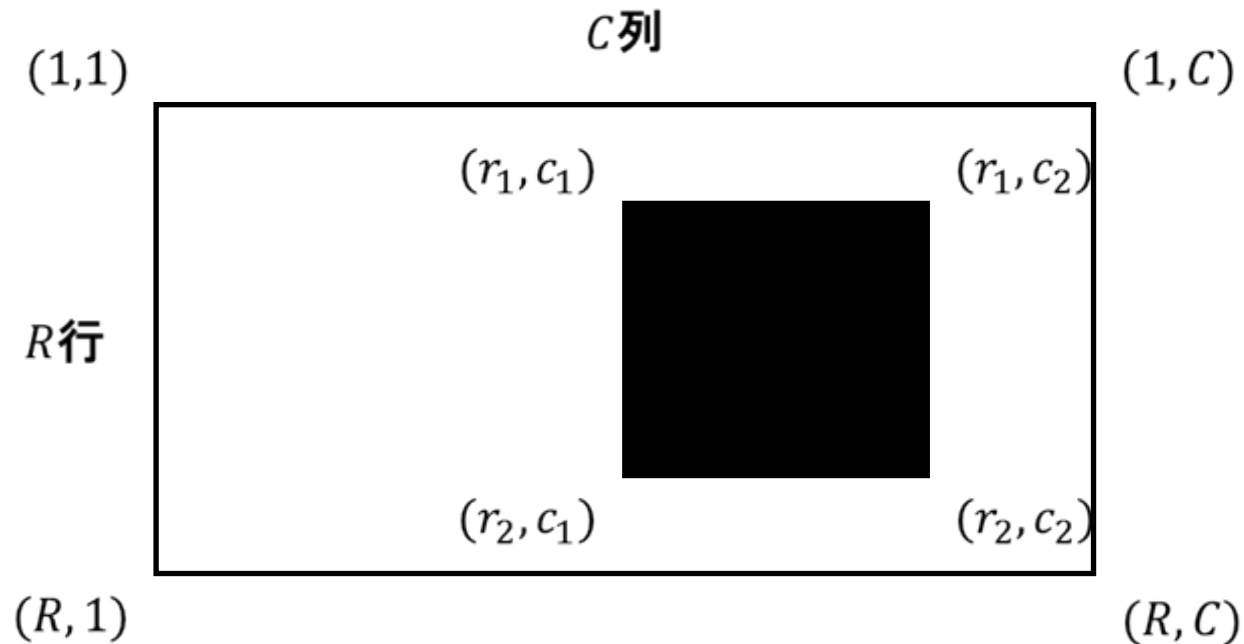
最大値とは?

- そもそも最大値はいくらになるか?
- 一度長方形を作った後について考える.



最大値とは?

- マスコットが長方形の形となったときの四隅を r_1, r_2, c_1, c_2 と置く.



最大値とは?

- マスコットが長方形の形となったときの四隅を r_1, r_2, c_1, c_2 と置く.
- 新しく長方形を作るとき, その長方形は元の長方形を含む.

最大値とは?

- マスコットが長方形の形となったときの四隅を r_1, r_2, c_1, c_2 と置く.
- 新しく長方形を作るとき, その長方形は元の長方形を含む.
- つまり, 新しい長方形の四隅を r_1', r_2', c_1', c_2' と置くと,

$$r_1' \leq r_1$$

$$c_1' \leq c_1$$

$$r_2' \geq r_2$$

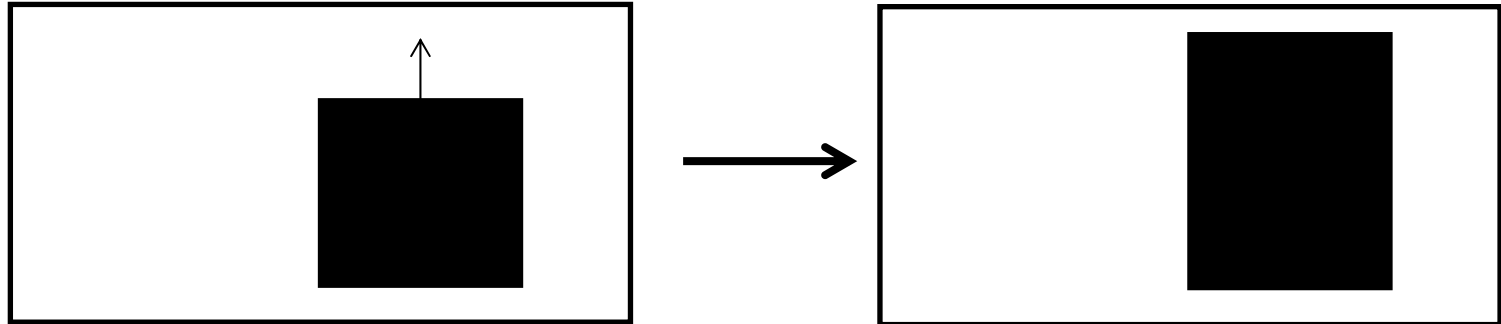
$$c_2' \geq c_2$$

r_1', r_2', c_1', c_2' のうち少なくとも1つは異なる.

最大値とは?

- よって前の長方形と新しい長方形で $t = r_2 - r_1 + c_2 - c_1$ の値は大きくなる.
- 最終的に t の値は $R + C - 2$ となるので、最初の長方形の四隅を r_1, r_2, c_1, c_2 と置くと最大値は
$$(R + C - 2) - (r_2 - r_1 + c_2 - c_1)$$
以下.
- 実はこれを満たす置き方が存在する.

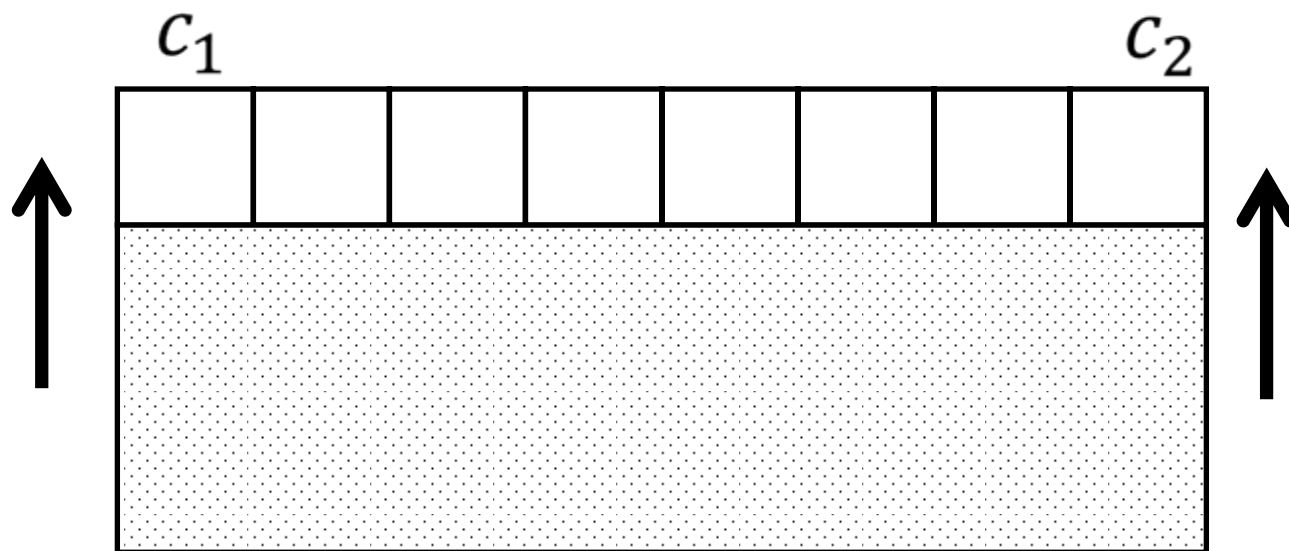
最適となる置き方



- 元の長方形から縦, あるいは横に1だけ大きい長方形を作り続ける.
- t は常に1ずつ大きくなるので前に述べた最大値の上限となる.

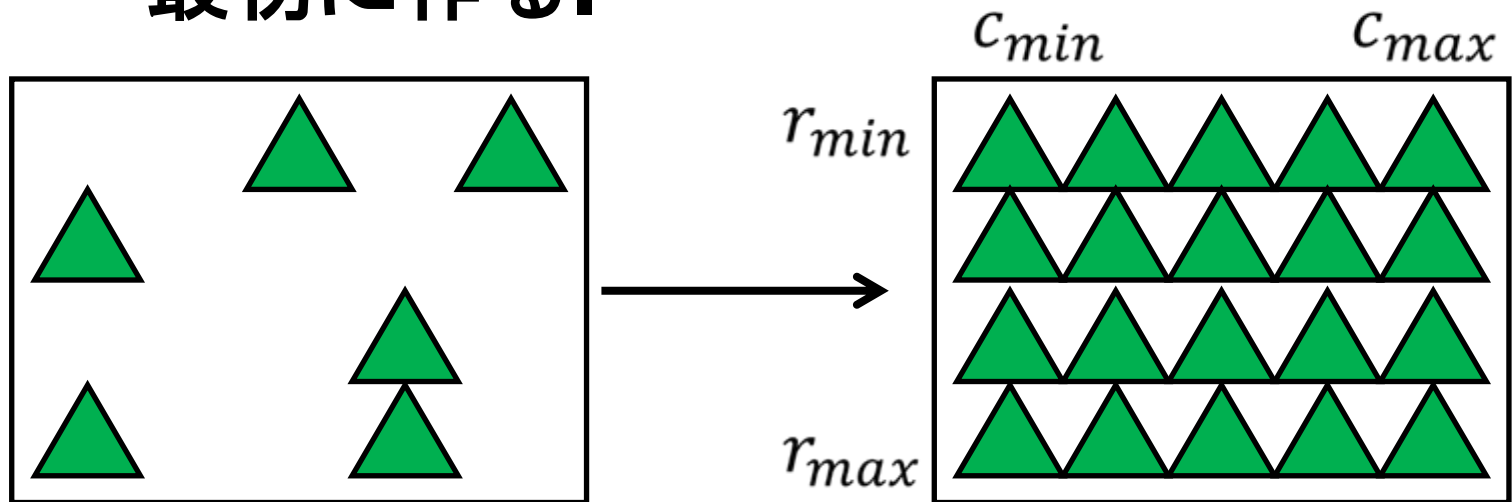
最適となる置き方

- 幅，または高さを1つ増やすときの置き方は，どの順番で置いてもいいので，
- 幅を1増やす $\rightarrow (r_2 - r_1 + 1)!$ 通り
- 高さを1増やす $\rightarrow (c_2 - c_1 + 1)!$ 通り



最適となる置き方

- 元が長方形でない場合でも，初期状態のマスコットの集合を含む最小の長方形は必ず含まれるので，この長方形を最初に作る．



- 組み合わせは(空きマスの数)!

DPによる解法

- $dp[rx][ry][cx][cy]$ を長方形の四隅がこの条件となる置き方と置くと,

初期値 $dp[rmin][rmax][cmin][cmax]$
= (空白マスの数)!

$$\begin{aligned} dp[r1][r2][c1][c2] &= dp[r1+1][r2][c1][c2] \times (c_2 - c_1 + 1)! \\ &+ dp[r1][r2-1][c1][c2] \times (c_2 - c_1 + 1)! \\ &+ dp[r1][r2][c1+1][c2] \times (r_2 - r_1 + 1)! \\ &+ dp[r1][r2][c1][c2-1] \times (r_2 - r_1 + 1)! \end{aligned}$$

DPによる解法

- $dp[rx][ry][cx][cy]$ を長方形の四隅がこの条件となる置き方と置くと,

初期値 $dp[rmin][rmax][cmin][cmax]$
=(空白マスの数)!

$dp[r1][r2][c1][c2]$

$$\begin{aligned} &= dp[r1+1][r2][c1][c2] \times (c_2 - c_1 + 1)! \\ &+ dp[r1][r2-1][c1][c2] \times (c_2 - c_1 + 1)! \\ &+ dp[r1][r2][c1+1][c2] \times (r_2 - r_1 + 1)! \\ &+ dp[r1][r2][c1][c2-1] \times (r_2 - r_1 + 1)! \end{aligned}$$

- 階乗を前計算すれば $O(R^2C^2)$ で40点

DPによる解法（補足）

- マスコットが長方形の領域からはみ出す場合はdpの値が0となる.
 - 全体からメモ化再帰でも解ける. ただしこの場合は最適解を満たさない手順と区別するため, dpの値とともに長方形になる回数の合計も保存しておく.
- 途中で長方形の回数の最大値が更新されたら調整する.

DPによる解法の発展

- $dp[rx][ry][cx][cy]$ を長方形の四隅がこの条件となる置き方と置くと,

初期値 $dp[rmin][rmax][cmin][cmax]$
= (空白マスの数)!

$dp[r1][r2][c1][c2]$

$$\begin{aligned} &= dp[r1+1][r2][c1][c2] \times (c_2 - c_1 + 1)! \\ &+ dp[r1][r2-1][c1][c2] \times (c_2 - c_1 + 1)! \\ &+ dp[r1][r2][c1+1][c2] \times (r_2 - r_1 + 1)! \\ &+ dp[r1][r2][c1][c2-1] \times (r_2 - r_1 + 1)! \end{aligned}$$

ここに注目

- 階乗を前計算すれば $O(R^2C^2)$ で40点

DPによる解法の発展

- 階乗の式では $c_2 - c_1 + 1$ および $r_2 - r_1 + 1$ がわかれば良い.
- 右に伸ばすか左に伸ばすかの違いは上下に伸ばすときの 階乗の式に影響しない.
(必要なのは個数のみ)
- 逆も同様.

DPによる解法の発展

- 階乗の式では $c_2 - c_1 + 1$ および $r_2 - r_1 + 1$ がわかれば良い.
- 右に伸ばすか左に伸ばすかの違いは上下に伸ばすときの 階乗の式に影響しない.
(必要なのは個数のみ)
- 逆も同様.
- よって左右, 上下を統合できれば $O(RC)$ で計算できるようになる.

DPによる解法の発展

- 上下方向，左右方向の割り当てを決めた後について考える．
- 例えば左が l 個，右が r 個あるとすれば，左右方向 $l + r$ 個のうち l 個を左にする組み合わせを求める．

- $l = 2, r = 3$ での例

左左右右右

左右左右右

⋮

などなど10通り

DPによる解法の発展

- 上下方向，左右方向の割り当てを決めた後について考える.
- 例えば左が l 個，右が r 個あるとすれば，左右方向 $l+r$ 個のうち l 個を左にする組み合わせを求める.

→ ${}_{l+r}C_l$ で計算できる（講義参照）

- $l=2, r=3$ での例

左左右右右

などなど10通り

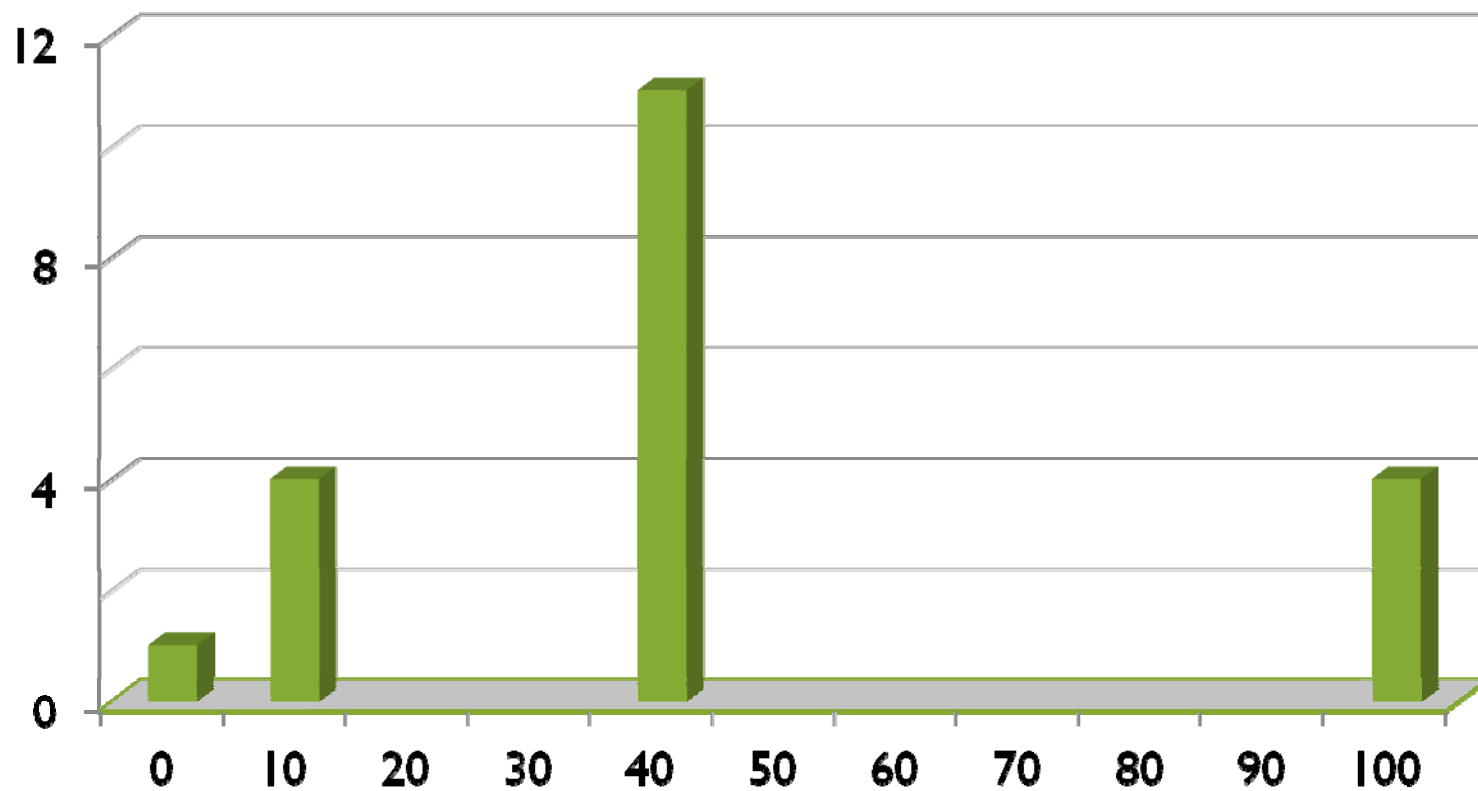
左右左右右

⋮

DPによる解法の発展

- $l+r$ C_l はあらかじめパスカルの三角形を使って求めておけば $O(1)$ で計算できる.
- パスカルの三角形は $O(\max(R, C)^2)$ で生成できる.
- DP部分はさっきのものを小さくすれば $O(RC)$ で計算できるので, 全体で $O(\max(R, C)^2)$ で解ける.
- 満点 (100点) が得られる.

得点分布





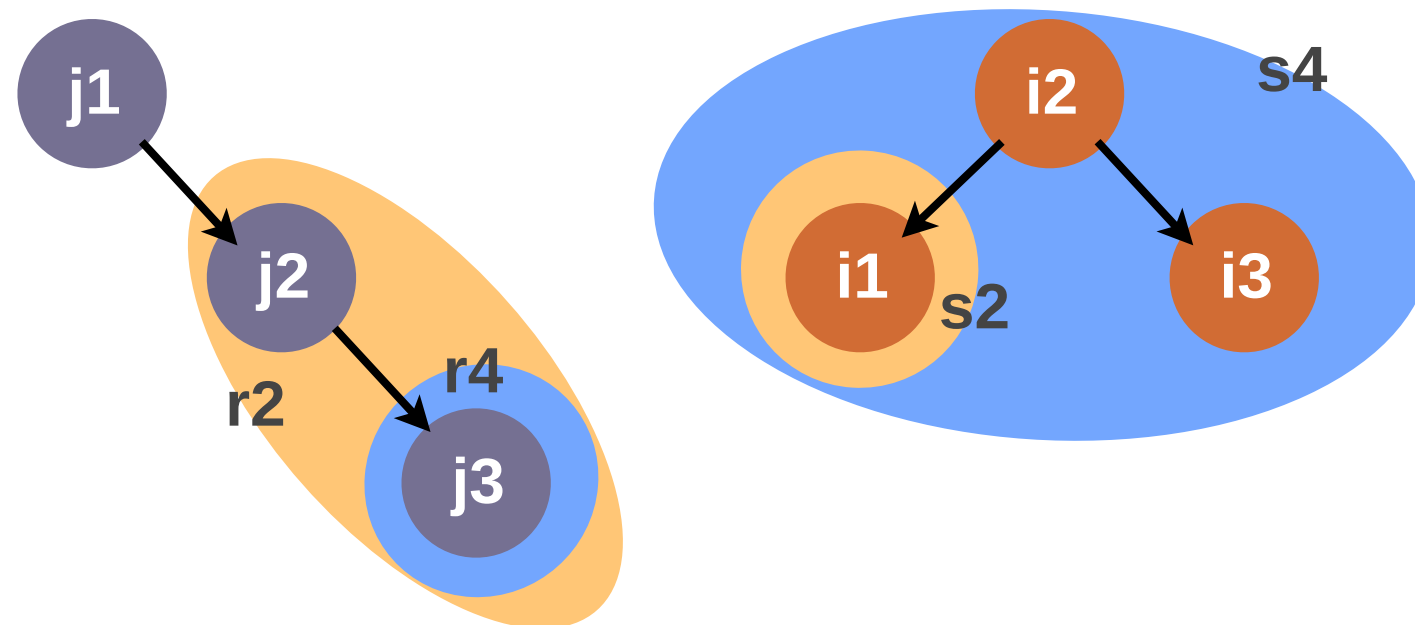
2013 JOI春合宿 Day2

Spy(スパイ) 解説

2013/03/21
山下 洋史

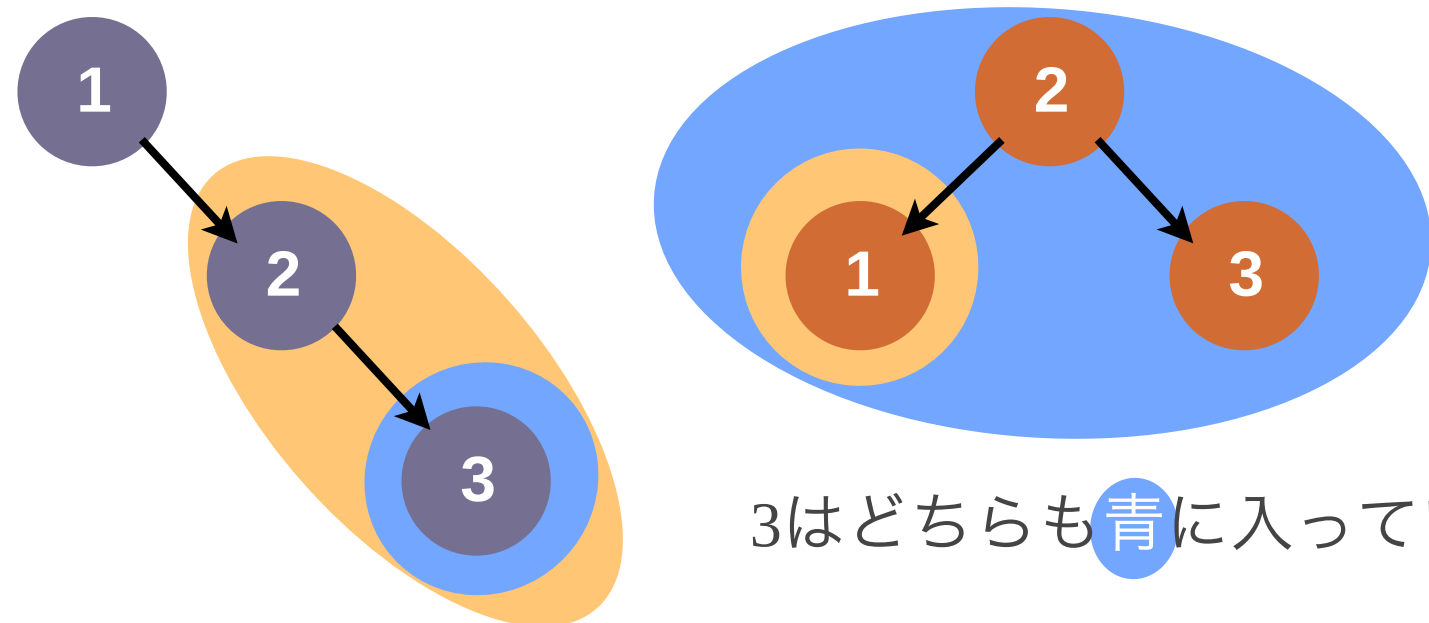
おさらい

- ・ JOI社とIOI社の社員の木構造が与えられる
- ・ リーダーの配下にあるメンバー全員からなるプロジェクト達と与えられる



おさらい

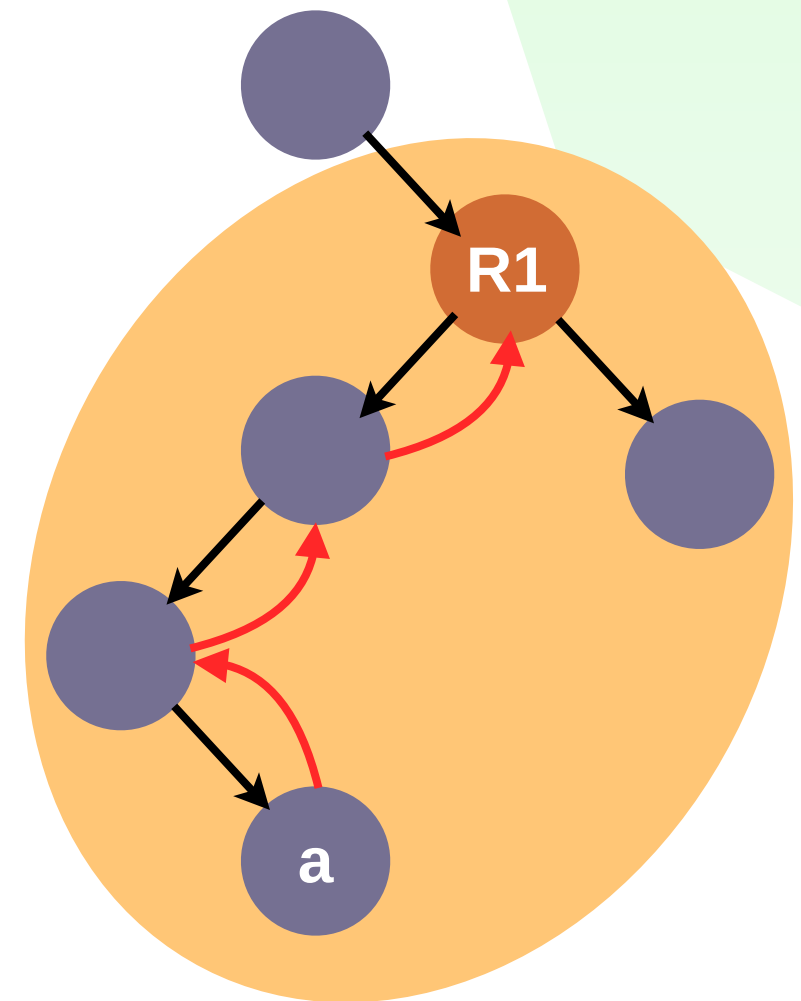
- ・ 2つの木の対応する頂点について, いくつ同じグループに入っているかを数える
- ・ 社員数: $N \leq 2000$, プロジェクト数: $M \leq 500000$



小課題 1:

社員aはプロジェクトbにいる？

- ・「社員aがプロジェクトbに属する」がすべてのa,bについて分かれば $O(NM)$ でできる
- ・木を上に向かってリーダーに辿り着くか調べる
- ・1人1プロジェクトにつき $O(N)$, 合計で $O(N^2M)$

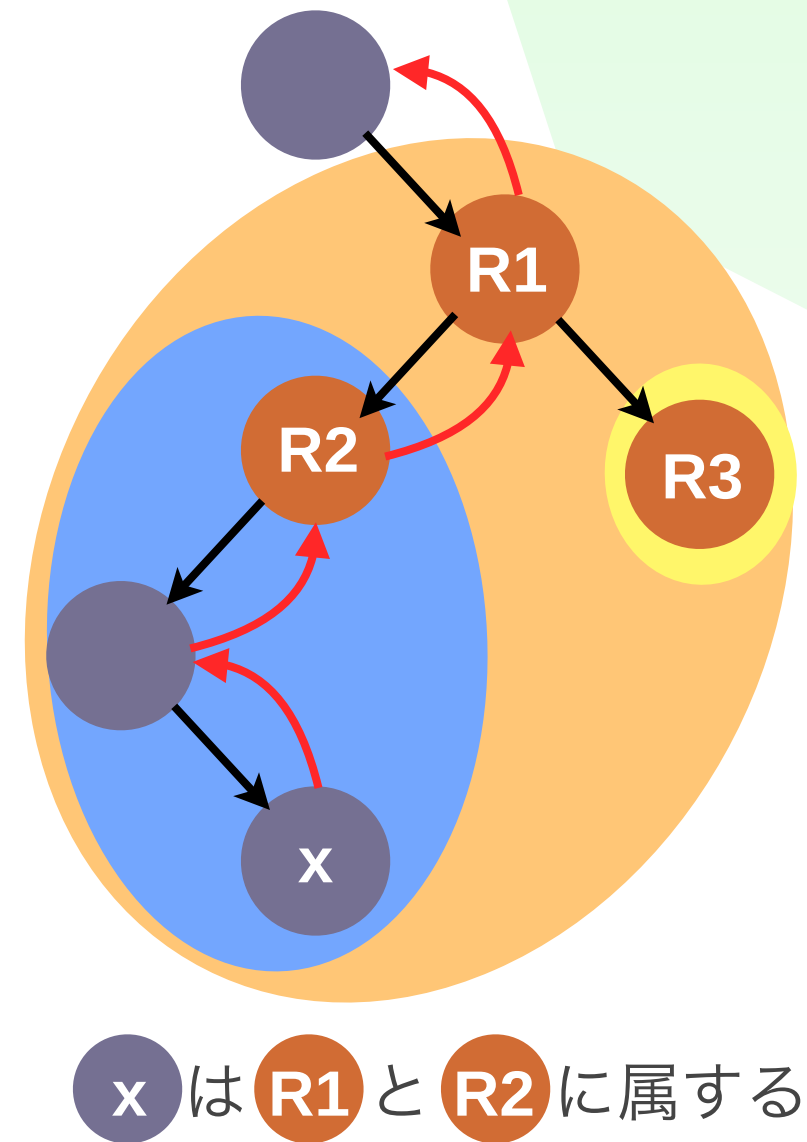


小課題 2 (a):

社員 x はどのプロジェクトにいる？

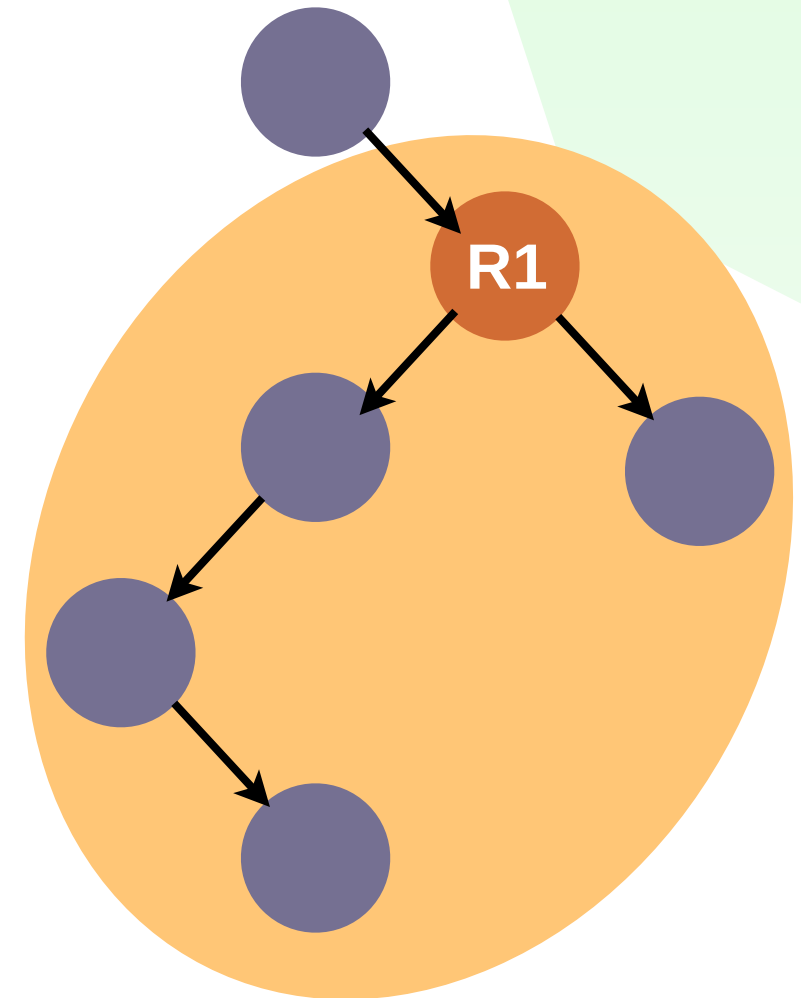
- ・ 1人の社員については, 1回木を辿っておけば全てのプロジェクトについて属するかどうか分かる

- ・ 1回木を辿る : $O(N+M)$
- ・ それが N 回で $O(N(N+M))$
- ・ その後, $O(NM)$



小課題 2 (b):
プロジェクト_xにはどの社員がいる？

- ・リーダーから深さ優先探索すればメンバーが列挙できる
- ・プロジェクトの回数回探索： $O(NM)$
- ・その後， $O(NM)$



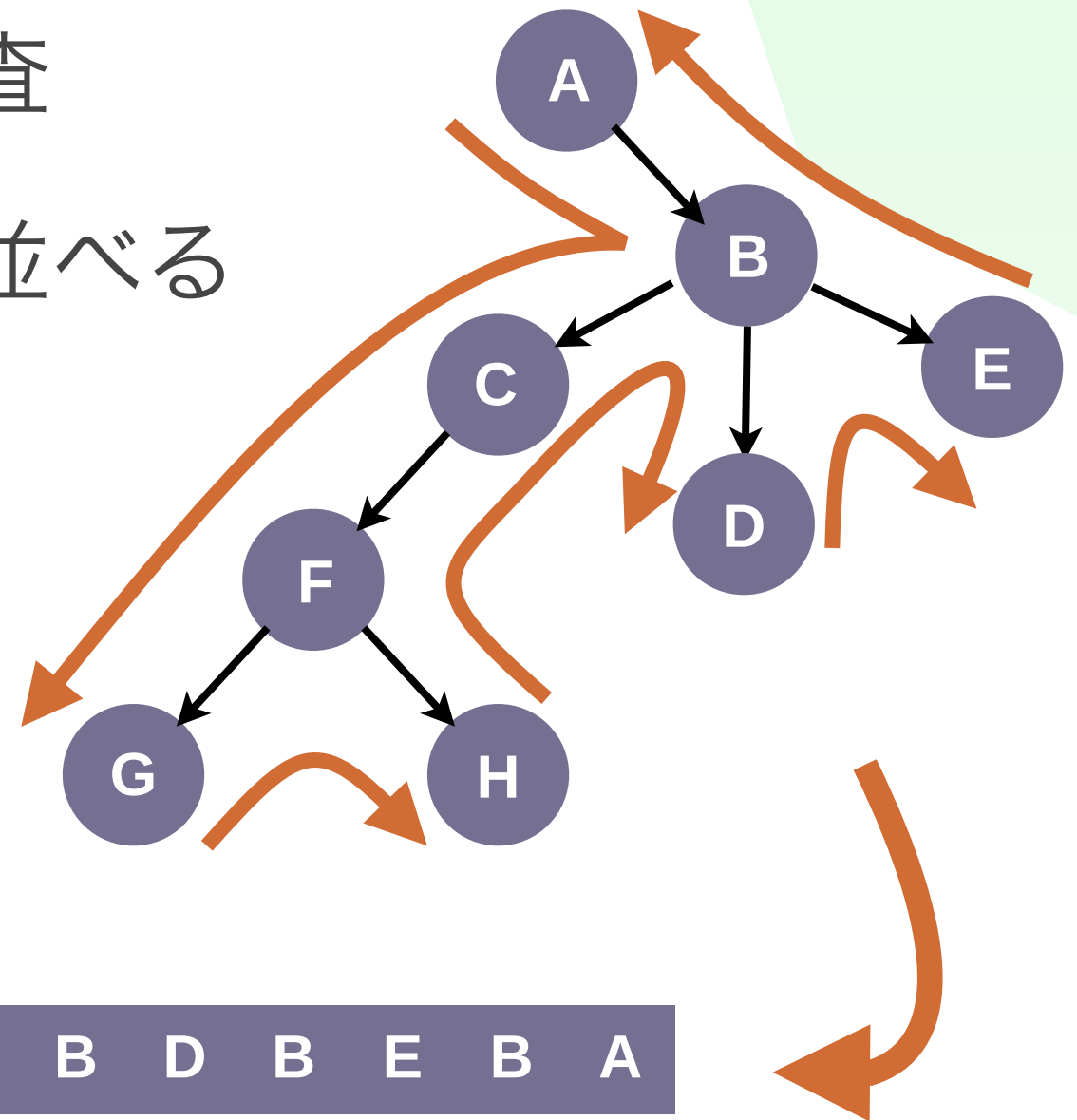


問題点

- ・ プロジェクトが $M=500000$ もある
- ・ 1つあたり $O(1)$ 時間しかかけたくない！

木の嬉しい順番

- ・ 木を深さ優先探索で走査
→ ノードを訪れた順番に並べる
- ・ 木が列のデータになる

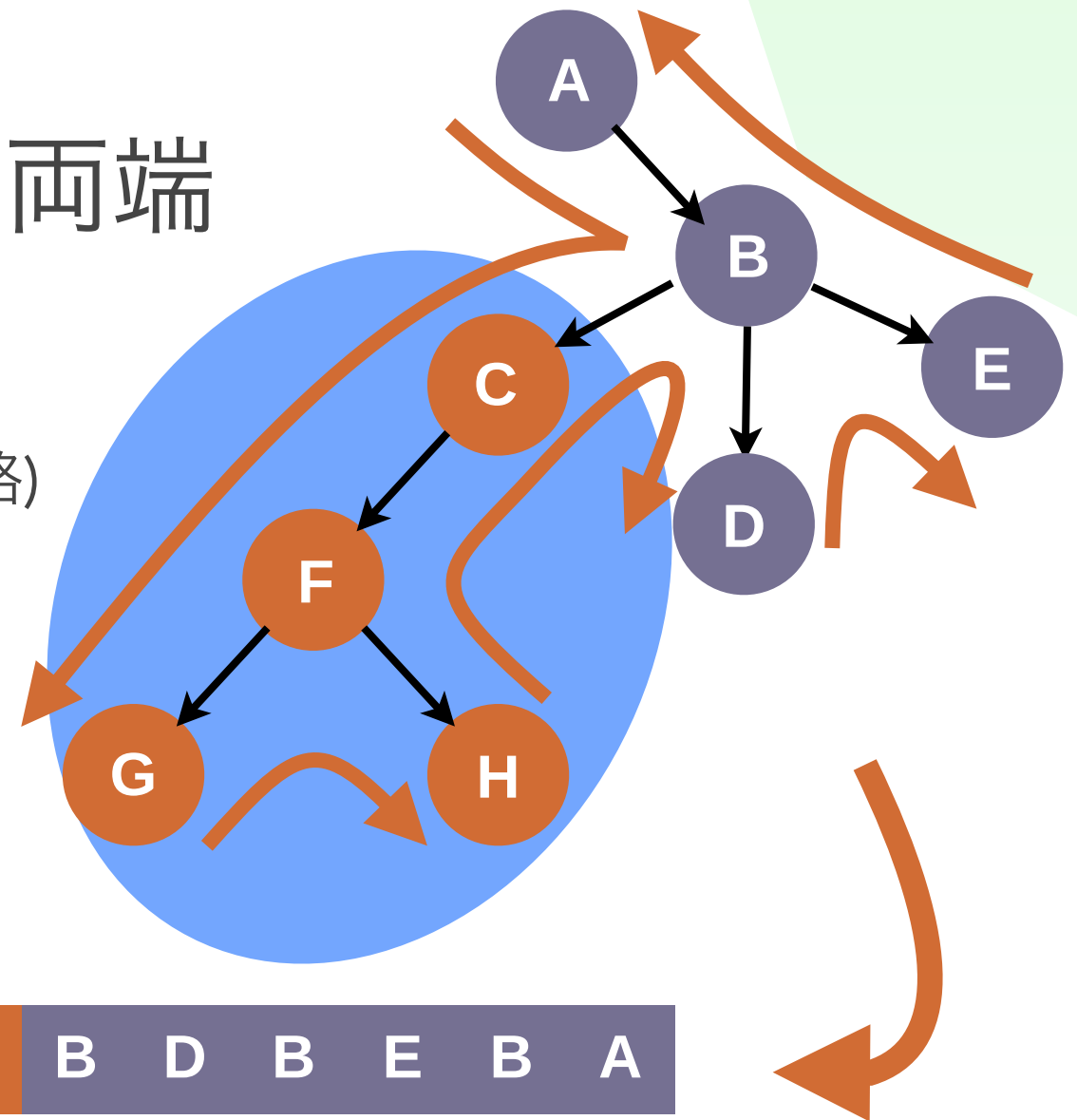


A B C F G F H F C B D B E B A

何が嬉しい？

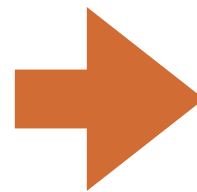
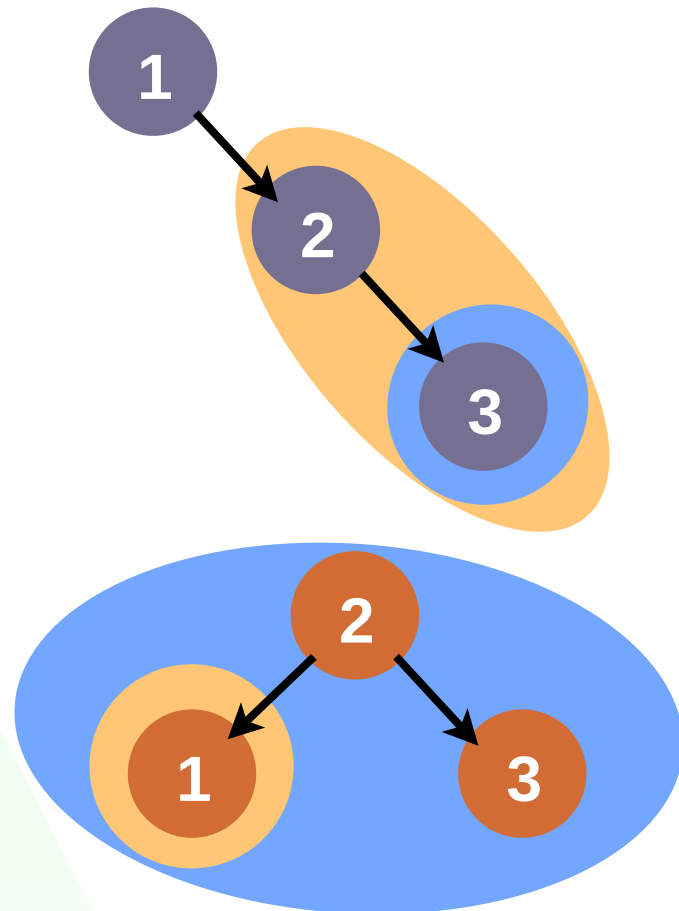
・ **部分木**が根ノードを両端にする **区間**に対応する

- ・ 最小共通祖先(LCA)問題への応用(省略)
- ・ Euler Tour Technique
と呼ばれることがあります



A B C F G F H F C B D B E B A

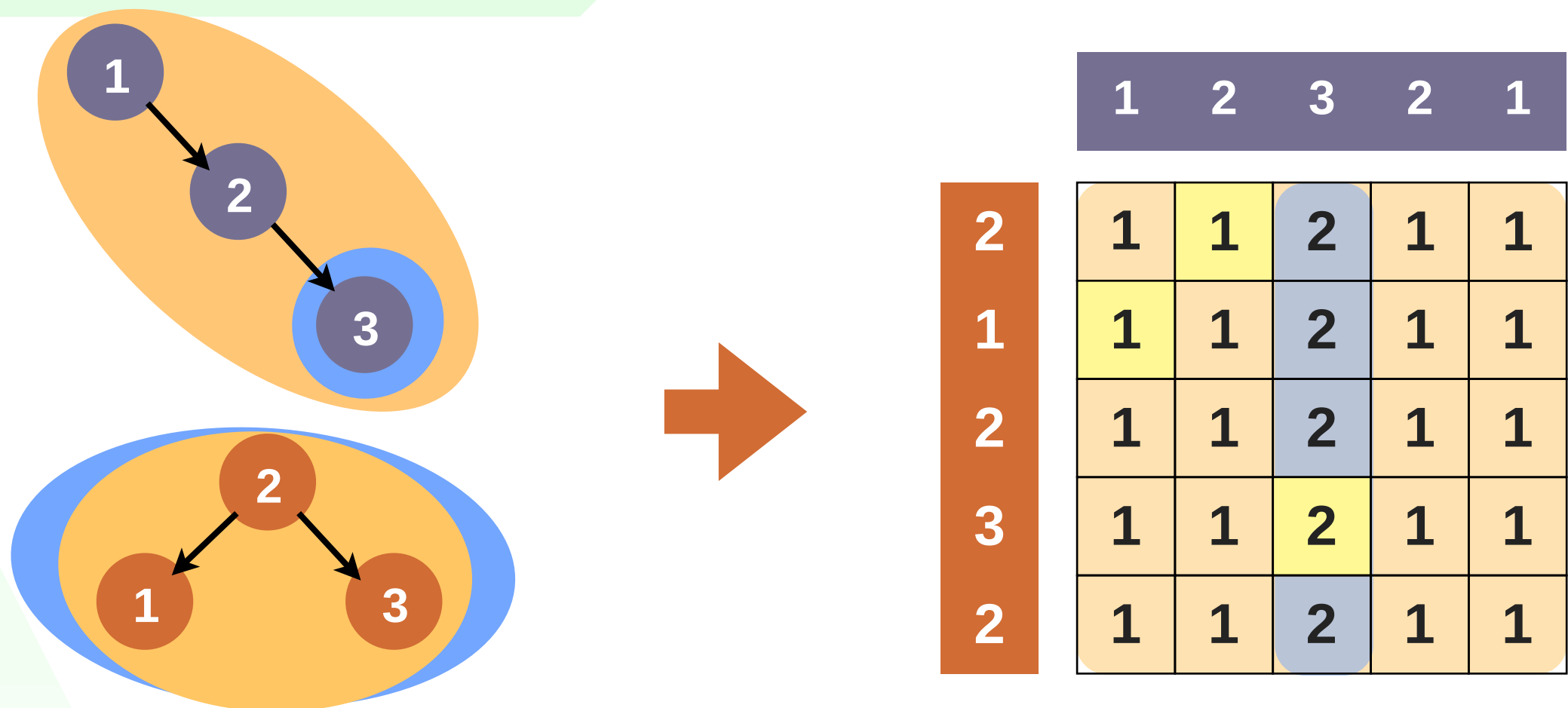
やりたいこと



	1	2	3	2	1
2	0	0	1	0	0
1	0	1	2	1	0
2	0	0	1	0	0
3	0	0	1	0	0
2	0	0	1	0	0

- ・ projectの組が長方形になり，同じ番号の社員の組がマス目になる
→ マス目の上に長方形がいくつ重なっているか を数える

長方形を足すのは大変



- 表は $O(N) \times O(N)$ の大きさ 長方形も同じ
- 愚直にマス目を塗ると $O(N^2M)$ かかる

領域に値を足す: 1次元の場合

- ・ 配列サイズ: N ・ 区間の個数: Q ・ 各区間に 1 を足す

0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

$O(Q)$ ↓ 始点に 1 を足し, 終点の後ろに -1 を足す

0	0	1	0	1	0	0	-1	0	-1
---	---	---	---	---	---	---	----	---	----

$O(N)$ ↓ 累積和を取る

0	0	1	1	2	2	2	1	1	0
---	---	---	---	---	---	---	---	---	---

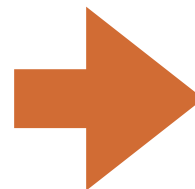


領域に値を足す: 2次元の場合(1)

こうなってほしい

	1	2	3	2	1
2	0	0	1	-1	0
1	0	1	1	-1	-1
2	0	0	1	-1	0
3	0	0	1	-1	0
2	0	0	1	-1	0

横に累積和をとる



こうするために

	1	2	3	2	1
2	0	0	1	0	0
1	0	1	2	1	0
2	0	0	1	0	0
3	0	0	1	0	0
2	0	0	1	0	0

領域に値を足す: 2次元の場合(2)

こうするために

	1	2	3	2	1
2	0	0	1	-1	0
1	0	1	1	-1	-1
2	0	0	1	-1	0
3	0	0	1	-1	0
2	0	0	1	-1	0

縦に累積和をとる

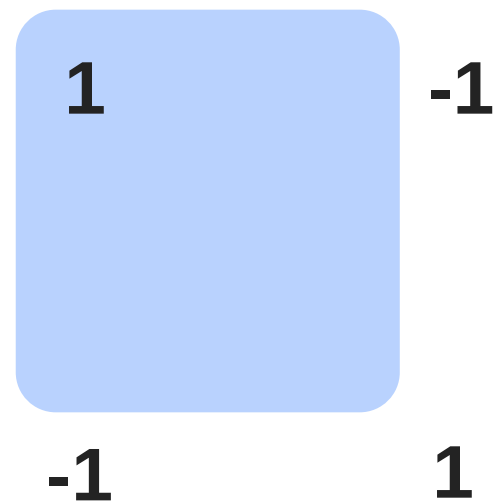


こうなってほしい

	1	2	3	2	1
2	0	0	1	-1	0
1	0	1	0	0	-1
2	0	-1	0	0	1
3	0	0	0	0	0
2	0	0	0	0	0
			-1	1	

領域に値を足す: 2次元の場合(3)

長方形について



を足せばよい.

(長方形1つあたり定数時間)

(こうした手法はJOI界隈では
imos法と呼ばれています)

	1	2	3	2	1
2	0	0	1	-1	0
1	0	1	0	0	-1
2	0	-1	0	0	1
3	0	0	0	0	0
2	0	0	0	0	0
			-1	1	

解法のまとめ

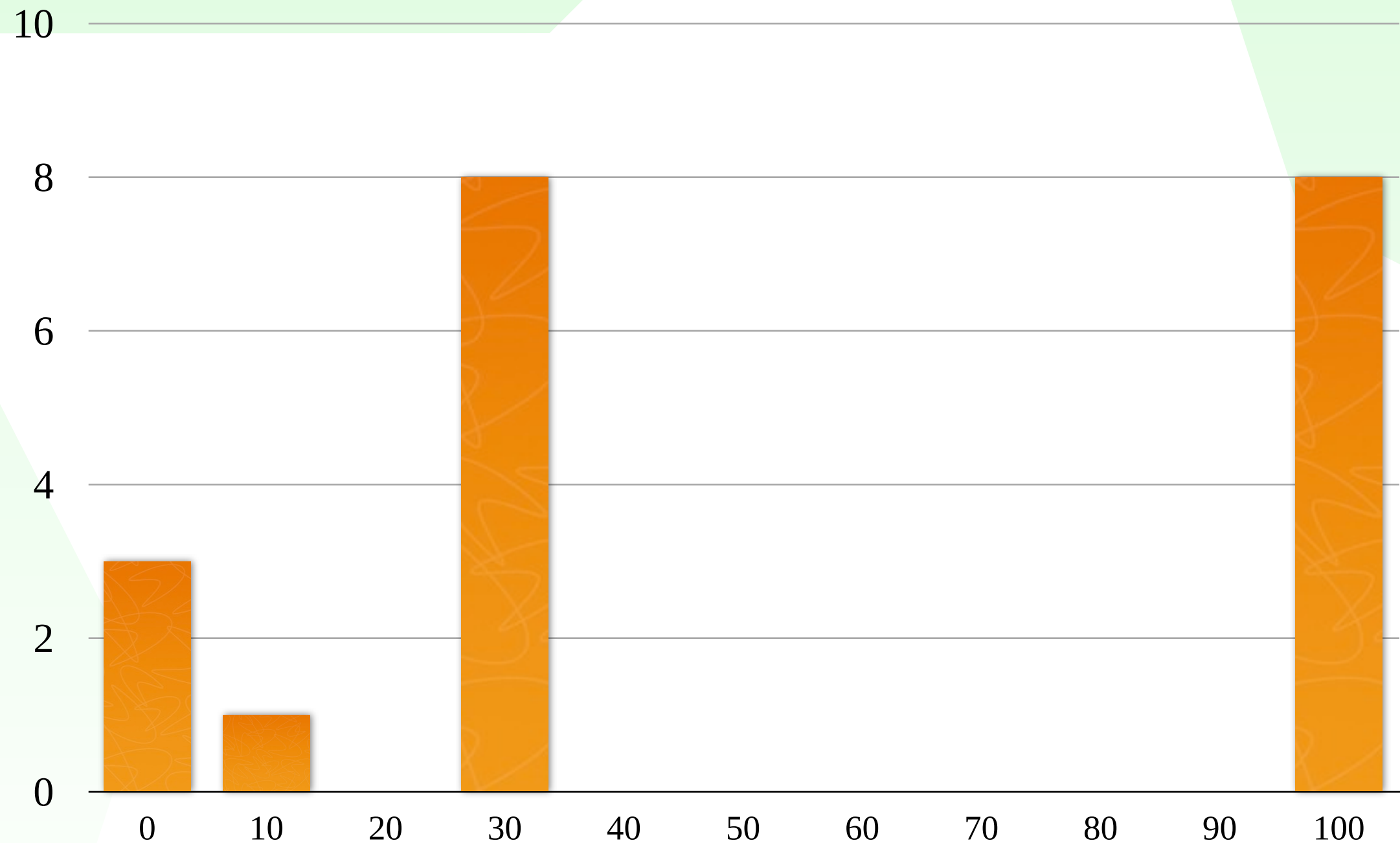
- Euler Tour Techniqueを用いてJOI木, IOI木をそれぞれ1列化
- 各社員の現れる位置を1つ覚えておく
- 各頂点を根とした部分木の区間も覚えておく
- $2N \times 2N$ の配列をとり, 各プロジェクトの組について
[JOI木での区間] $\times$ [IOI木での区間]の長方形を”塗る”
(1,-1を適切なところに足す)
- 縦横に累積和をとる
- 各aについて, 配列の[jaが現れる位置][iaが現れる位置]を出力する

計算量？

- 2つの木のDFS $O(N)$
- 長方形の個数×定数時間 $O(M)$
- 2回累積和をとる $O((2N)^2)$ (実は行き掛け順でよいのでもっと少なく出来る)
- $M \leq 500000, N \leq 2000$
- 間に合う



得点分布



DPによる解法

- ・ 詳しくは説明しません
- ・ キーとなる式がこちら

$$\begin{aligned} dp[j][i] = & \\ & project[j][i] \\ + & dp[j.parent][i] \\ + & dp[j][i.parent] \\ - & dp[j.parent][i.parent] \end{aligned}$$

- ・ スライドをアップロードするのでこれを見て考えてみてください

関連するJOIの問題

- ・ 少し幸せ になる木の順番
- ・ 2010-day4 高速道路(Highway) ・ 2011-day3 報告(Report)
- ・ 領域を $O(1)$ で塗る
- ・ 2012 本選4 釘(Nails) ・ 2010-day4 プラグ(Plugs)

JOISP Day3 解説 ケーキ (Cake)

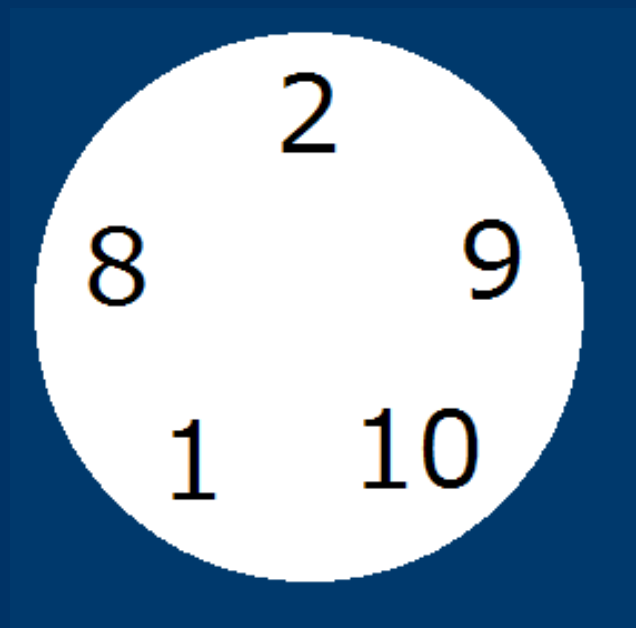
滝間 太基

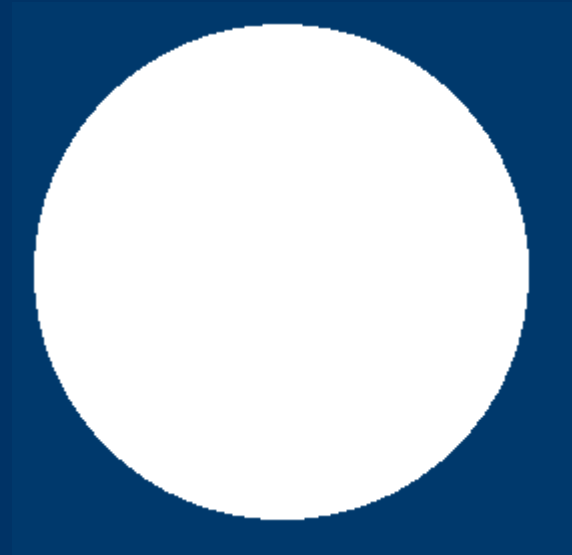
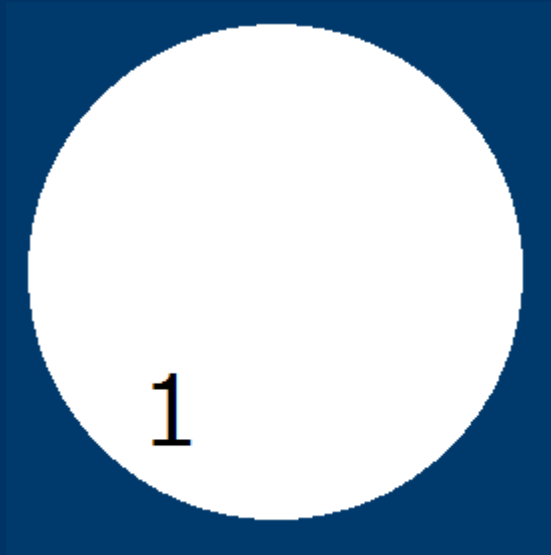
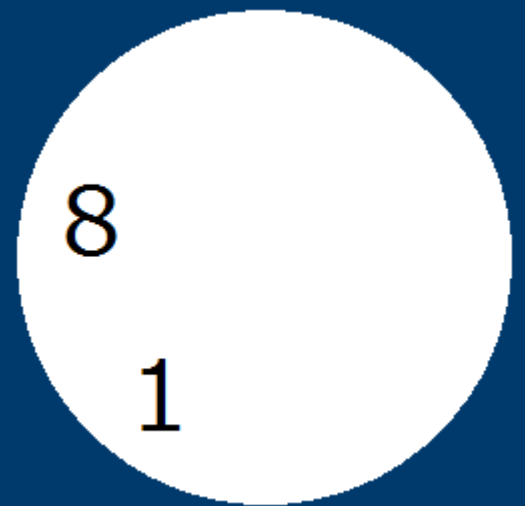
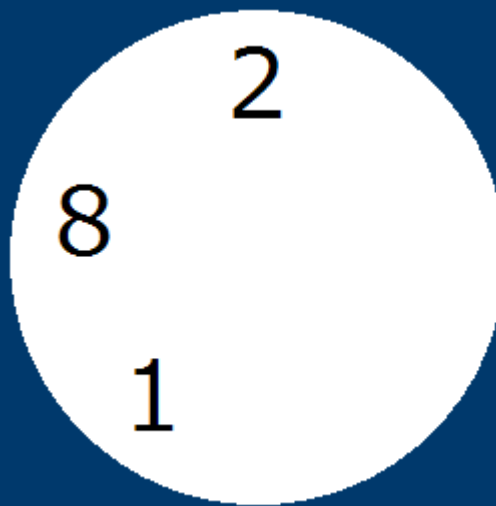
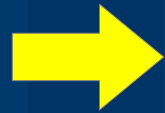
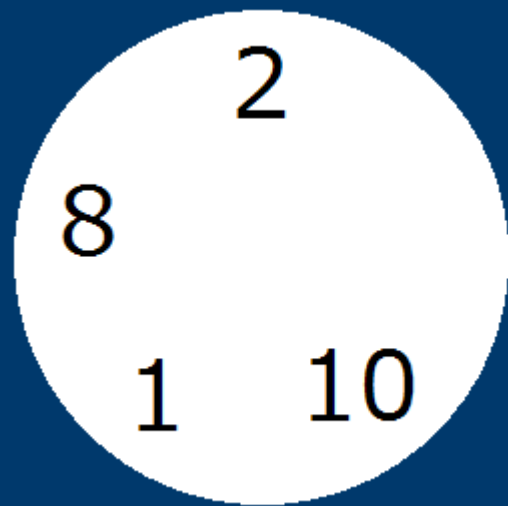


問題の概要

円周上に数が並んでいる

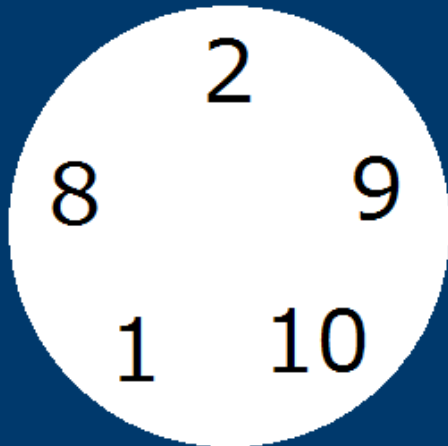
- 最初に 1 個取る
- 残りの両端のうち大きいほうを取る、を繰り返す





最小のピース 1 は明らかに最後まで残る

→ 区間上に並んでいる場合を解けばよい
(初手で 1 を取る場合は別処理)



8 2 9 10



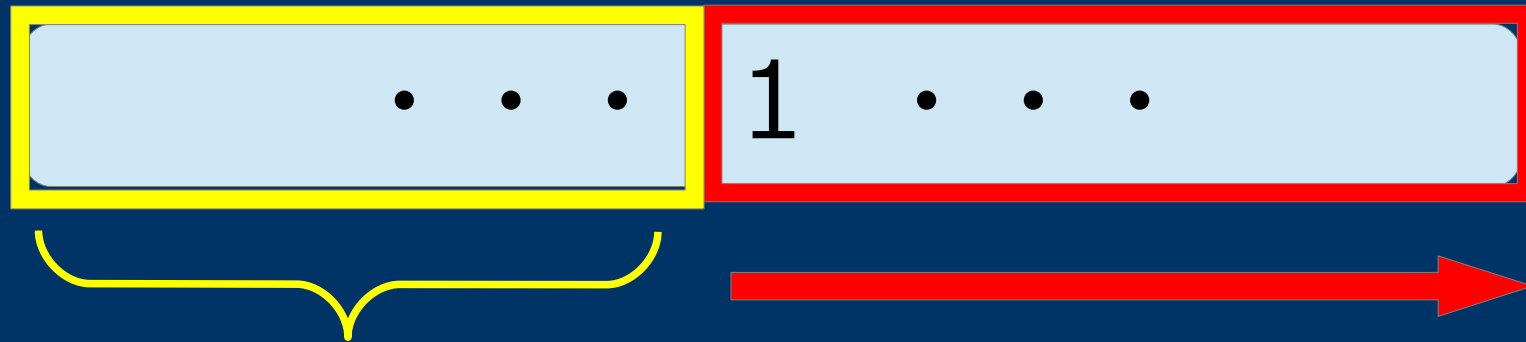
最小の要素に注目



このどこから取っても、

(最小の要素を見つけるのは RMQ を使う)

最小の要素に注目



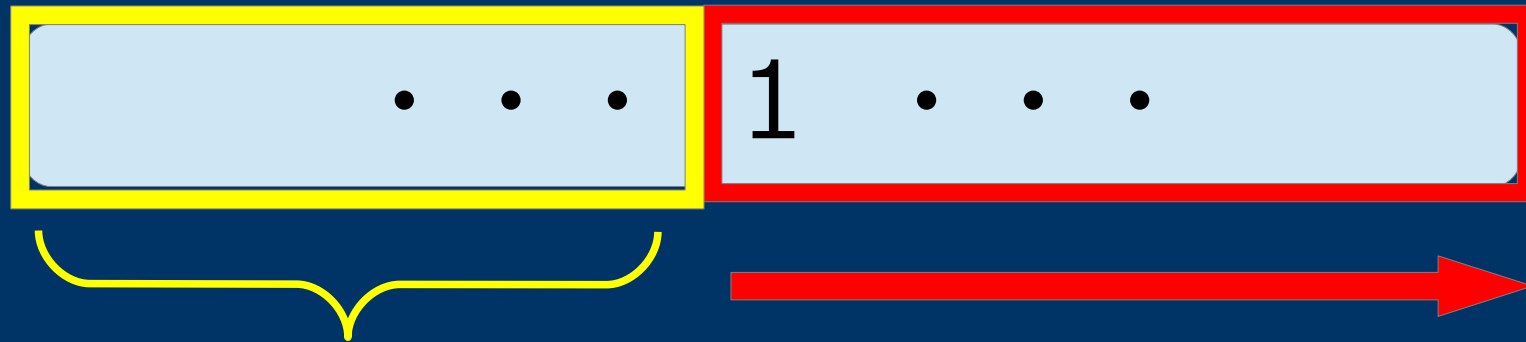
このどこから取っても、

- 黄部分を同じルールで取り尽くした後
- 赤部分を左から順に取っていく

ことは共通

(最小の要素を見つけるのは RMQ を使う)

最小の要素に注目



∴ 最初に取るのが黄部分のどれかのときは

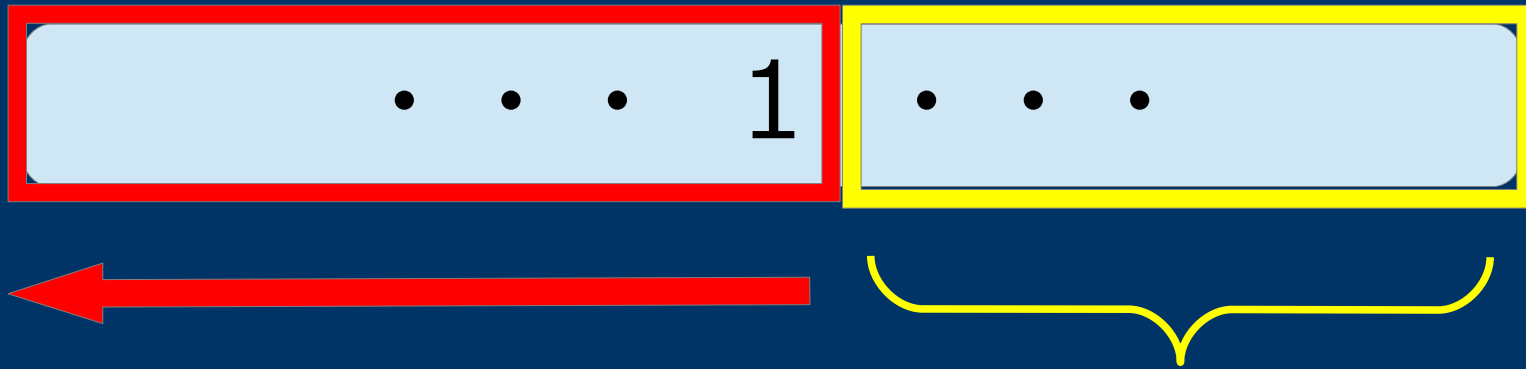
黄部分で食える量 $\leftarrow$ 再帰すればおk

+

赤部分で食える量 $\leftarrow$ 定数だし、爆速で求まる

が答え (↑ 区間に定数を足すのでいもす法を使う)

最小の要素に注目



最初にここのだれか
を取るときも同様

最小の要素に注目



あとは、最初に 1 を取るときに食える量を
 $O(N^2)$
求めればよい



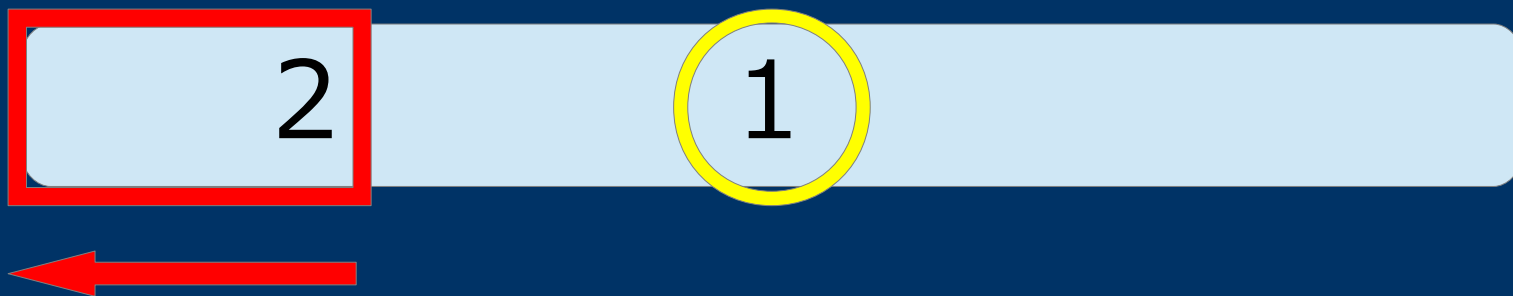
愚直にシミュレーションすると区間の長さ
だけ時間がかかる



全体で

ランダムなケース : $O(N \log N)$

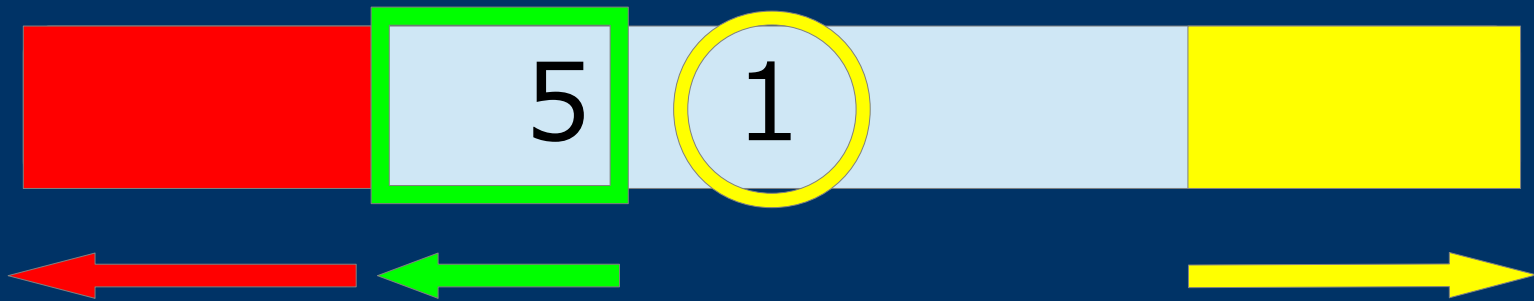
最悪なケース : $O(N^2)$ ← 死



- 2 番目に小さい要素に注目する (RMQ 使う)
→ そこから先は最後に取りられる
(食える量は爆速で求まる)



- 2 番目に小さい要素に注目する (RMQ 使う)
→ そこから先は最後に取りられる
(食える量は爆速で求まる)
- 残りのうち最も小さい要素に注目する (RMQ)
→ そこから先は (残りのうち) 最後に取りられる (食える量は (r y)



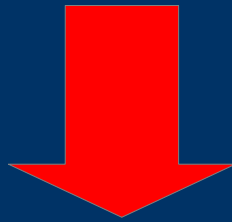
- 2 番目に小さい要素に注目する (RMQ 使う)
→ そこから先は最後に取りられる
(食える量は爆速で求まる)
- 残りのうち最も小さい要素に注目する (RMQ)
→ そこから先は (残りのうち) 最後に取りられる (食える量は (ry))
- 繰り返す ($R(ry)$)

2 4 6 8 1 9 7 5 3

こういう区間だと $O(\text{区間の長さ})$ かかる
→ 最悪だと全体で $O(N^2)$?

2 4 6 8 1 9 7 5 3

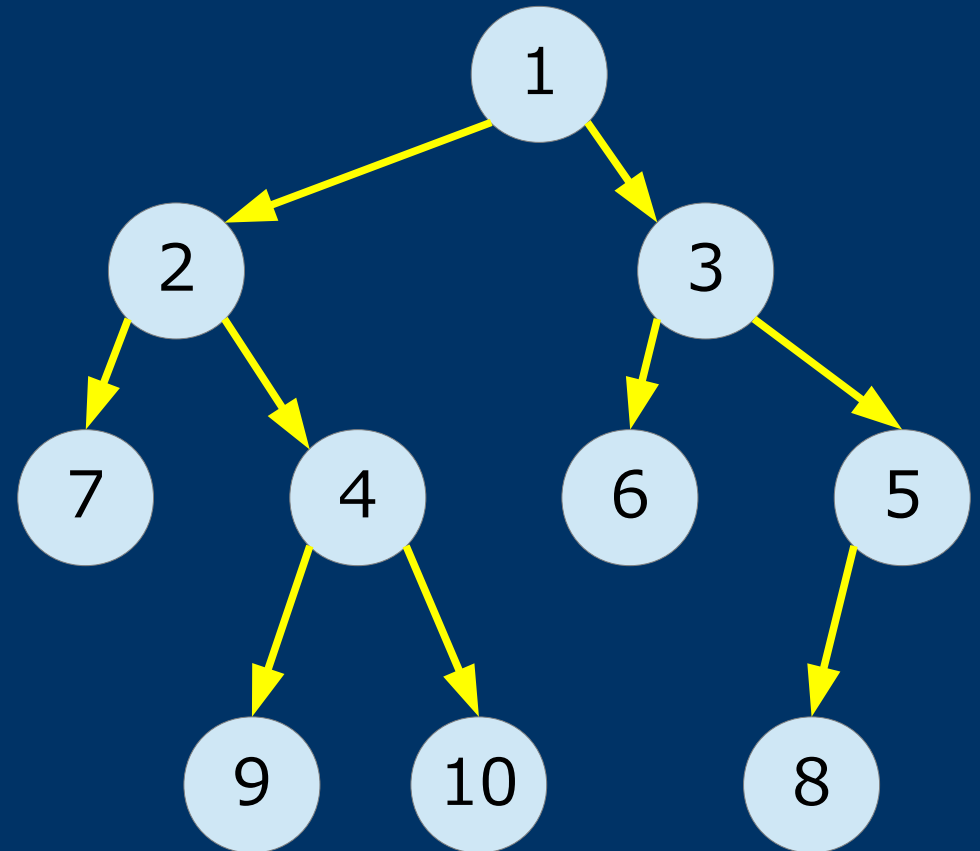
こういう区間だと $O(\text{区間の長さ})$ かかる
→ 最悪だと全体で $O(N^2)$?

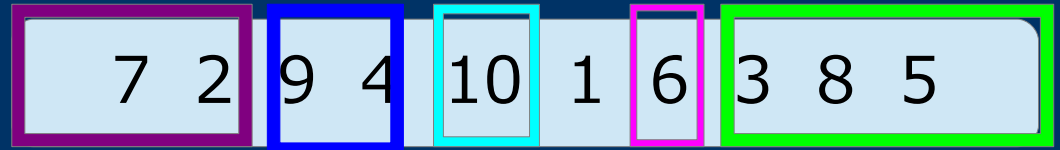


実は全体で $O(N)$! ! !

7 2 9 4 10 1 6 3 8 5

こういう木を考える→
・部分木は区間に対応
していて、その根は区
間の最小値





さっきの方法で区間を切る回数は

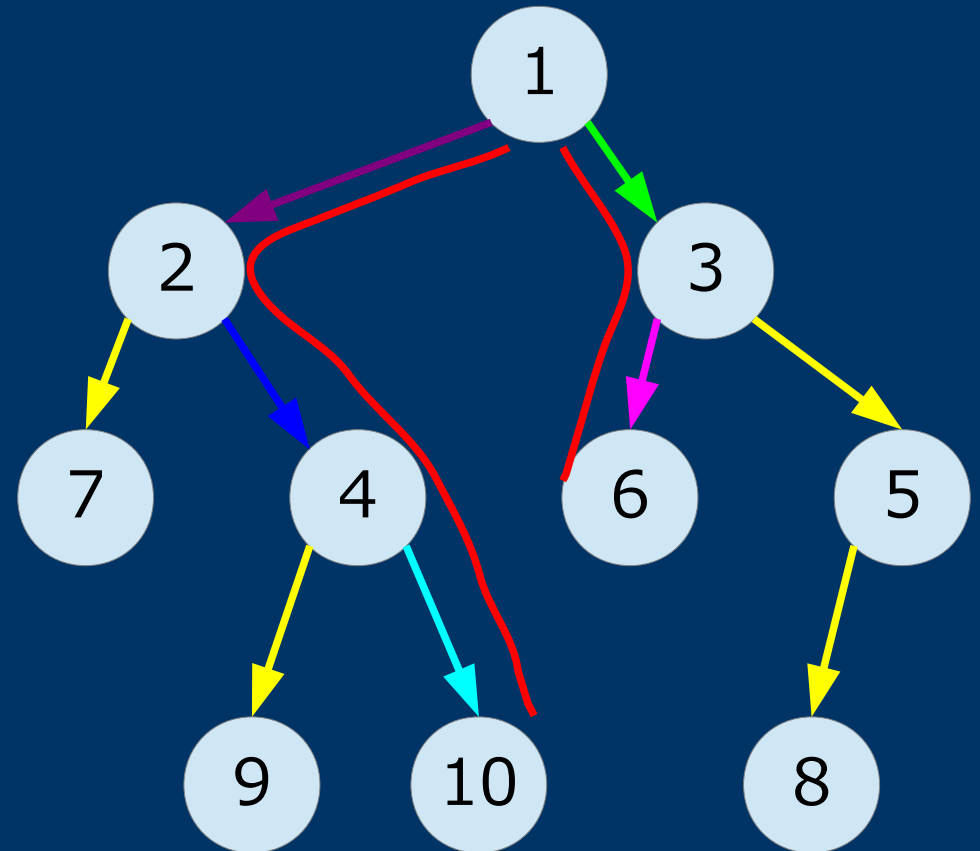
- ・ 左の子から始めて
右の子を辿れる回数

+

- ・ 右の子から始めて
左の子を辿れる回数

||

赤い線の長さ



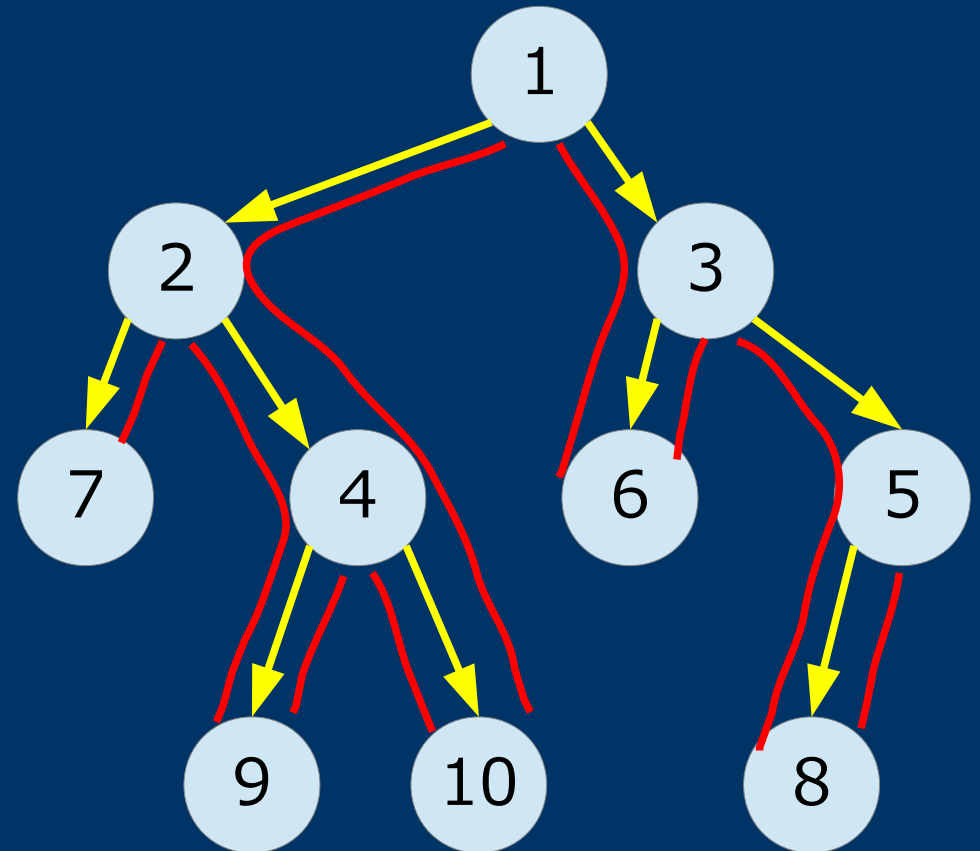
7 2 9 4 10 1 6 3 8 5

赤い線を全部描いた図→

観察

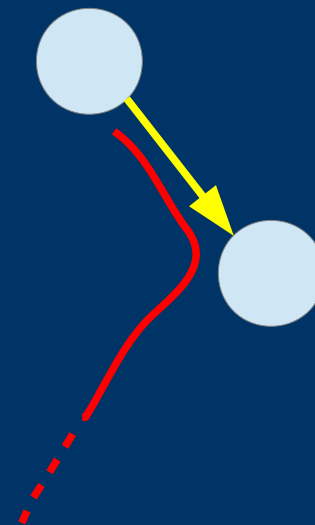
どの  も  に
2 回以下しか覆われてい
ない

⇒  の長さの合計
は高々 $2(N-1)$!!!

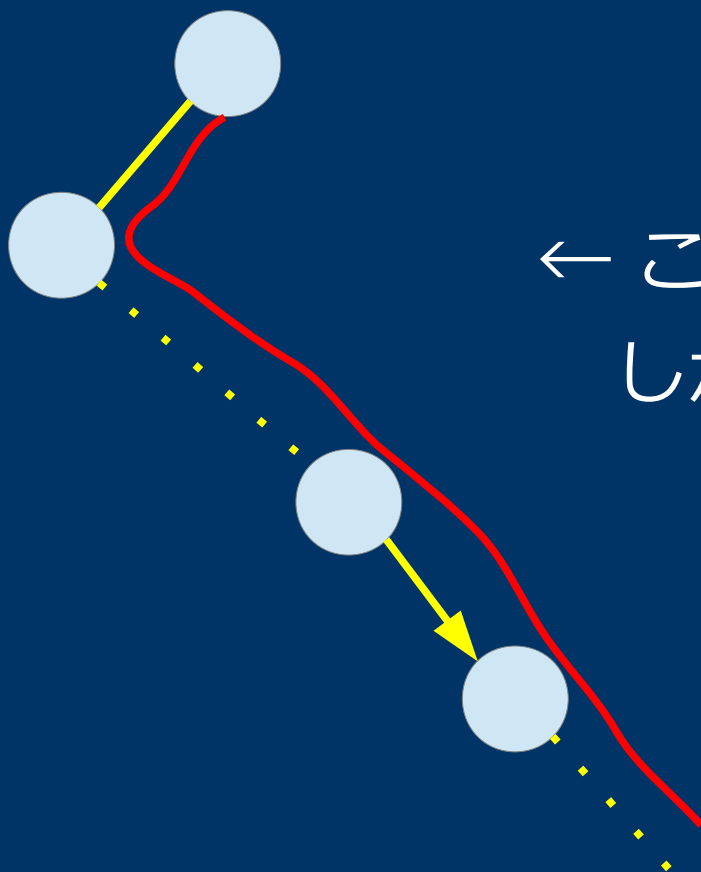


観察の証明（ほぼ明らか）

こういうの（一意に定まる）と→



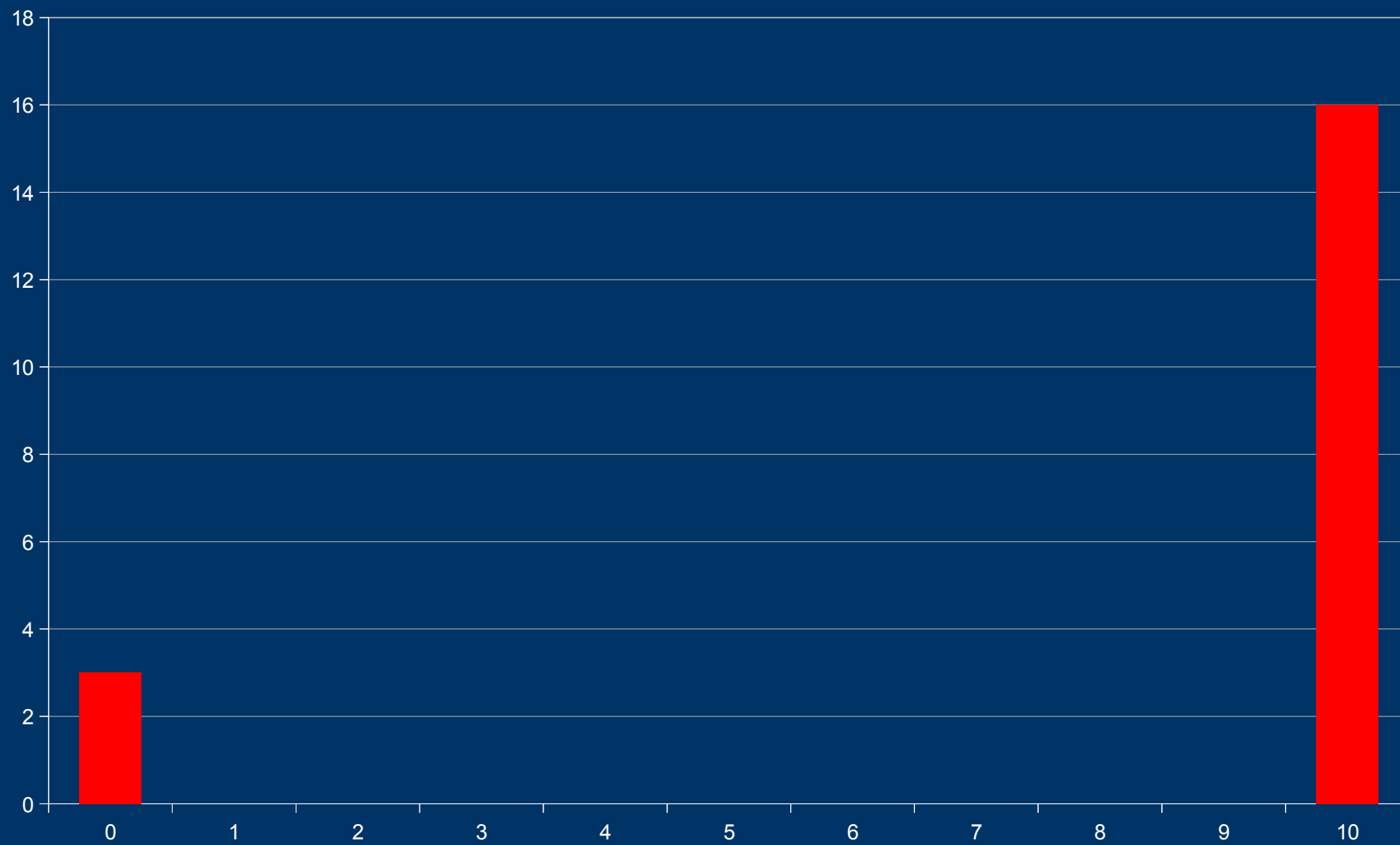
← こういうの（一意に定まる）
しかない



以上 !!!!

全部で $O(N \log N)$!!! 常勝 !!!

得点分布



コ

ア

ラ

解説者:笠浦一海
(ガチクズ勢)

問題

- コアラが座標 K の点から座標 M の点までジャンプして進む。
- 一回のジャンプで距離 D まで移動できる。

問題

- 一回のジャンプで体力をA消費。
- 座標 T_i の点に行くと体力が B_i 回復。

問題

- 体力の初期値は0。
- 最終的な体力の最大値は？

考察

- 後戻りしない。
- 「座標 x にいるときの体力の最大値」をDPで順番に求めればいけそう。
- 愚直にやると $O(M \cdot D)$ 。
→明示的な部分点はない

考察

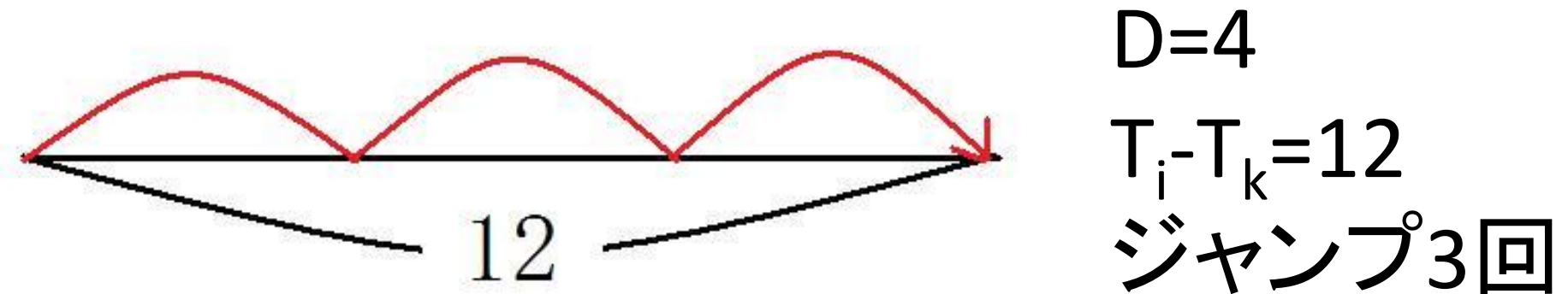
- 家があるところだけ考える。

動的計画法(20点解法)

- $T_0=K, T_{N+1}=M, B_0=B_{N+1}=0$ とする。
- $DP[i]=(T_i$ まで来たときの体力の最大値)。
- $DP[i]$ を $DP[0] \sim DP[i-1]$ までの値を使って求めればよい。

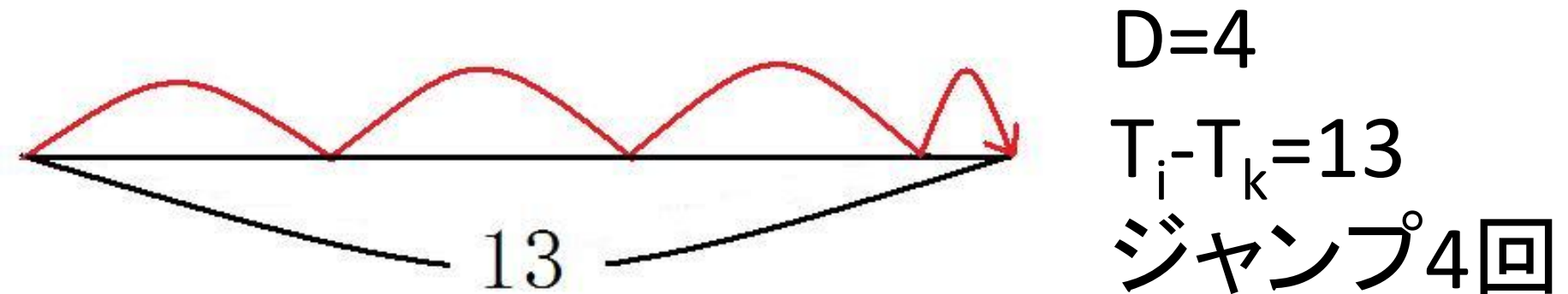
動的計画法(20点解法)

- T_k から T_i まで寄り道せずに移動するとき体力を $\text{ceil}((T_i - T_k)/D) * A$ だけ消費。



動的計画法(20点解法)

- T_k から T_i まで寄り道せずに移動するとき体力を $\text{ceil}((T_i - T_k)/D) * A$ だけ消費。



動的計画法(20点解法)

- $DP[i] = \max\{DP[k] - \text{ceil}((T_i - T_k)/D) * A : k < i\} + B[i]$
- $O(N^2) \rightarrow 20\text{点}$

さらに考察

- それまでの値を全部調べなくても DP[i]を求められるようにしたい。
- DPの式を眺めてみる。
- $\text{ceil}((T_i - T_k)/D)$ が大事。

さらに考察

- $D=1$ のとき。
- $DP[i] = \max\{DP[k] + T_k * A - T_i * A : k < i\} + B[i]$
- k に関する式と i に関する式に分離できる。
- $DP[k] + T_k * A$ の最大値を記録しておけばオツケー

さらに考察

- D が一般の値の時も同じようにできないだろうか？

さらに考察

- $\text{ceil}((T_i - T_k)/D)$ は.....。

さらに考察

- $T_i \% D > T_k \% D$ のとき

$$\text{floor}(T_i/D) - \text{floor}(T_k/D) + 1$$

- $T_i \% D \leq T_k \% D$ のとき

$$\text{floor}(T_i/D) - \text{floor}(T_k/D)$$

- これからは $\text{ceil}()$ はつかわないので $\text{floor}()$ は省略します。

代入して

- $DP[i] = \max(\begin{aligned} &\max\{DP[k] - (T_i/D - T_k/D + 1) * A \\ &\quad : k < i, T_i \% D < T_k \% D\}, \\ &\max\{DP[k] - (T_i/D - T_k \% D) * A \\ &\quad : k < i, T_i \% D \geq T_k \% D\}) \\ &+ B[i] \end{aligned}$

整理

- $P[i] = DP[i] + (T_i/D) * A$ とおくと

$$P[i] = \max(\max\{P[k] - A : k < i, T_i \% D < T_k \% D\}, \\ \max\{P[k] : k < i, T_i \% D \geq T_k \% D\}) + B[i]$$

- きれいな式になった。
- T_i を D で割った余りだけが重要。

30点解法

- $D \leq 100$
- $T_i \% D$ の値ごとに $P[i]$ の最大値を記録。

30点解法

- $S[x] = \max\{P[i] : T_i \% D == x\}$ として
- $P[i] = \max(\max\{S[x] : 0 \leq x < T_i \% D\} - A, \max\{S[x] : T_i \% D \leq x < D\}) + B[i]$
- $S[T_i \% D] = \max(S[T_i \% D], P[i])$
- $O(N * D) \rightarrow 30$ 点

満点解法

- 数列Sに対する操作。
 - ①区間の最大値をとる。②値の更新。
- RMQ(Range Minimum Query)
- segment tree で解ける。
- Dが大きいので愚直に実装すると
MLE

満点解法

- 座標圧縮。
- $T_0 \% D, T_1 \% D, \dots, T_{N+1} \% D$ をソートして座標圧縮。
- ①も②も $O(\log N)$ でできる。
- 時間計算量 $O(N \log N)$
空間計算量 $O(N)$ → 満点！

数式を使わない考え方

- 道を距離Dごとに区切る。



数式を使わない考え方

- 一回のジャンプで次の区間に行けることもあれば、いけないこともある。



次の区間に行く例



次の区間に行かない例

数式を使わない考え方

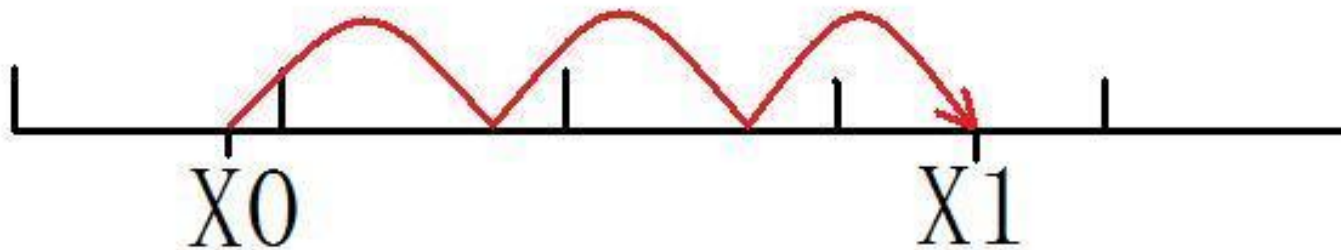
- 次の区間に行くジャンプを良いジャンプ、行けないジャンプを悪いジャンプと呼ぶ。
- ある地点まで行くときの良いジャンプの数は決まってる。
- 悪いジャンプの数だけ考えればよい。

数式を使わない考え方

- 座標 X_0 の地点から座標 X_1 の地点に行くとき.....

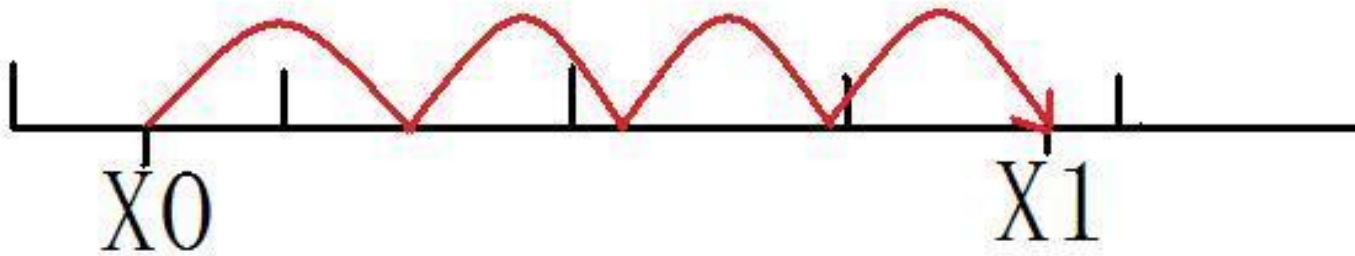
数式を使わない考え方

- $X0 \% D \geq X1 \% D$ ならば良いジャンプのみで行ける。



数式を使わない考え方

- $X0 \% D < X1 \% D$ のときは悪いジャンプが一回必要。



数式を使わない考え方

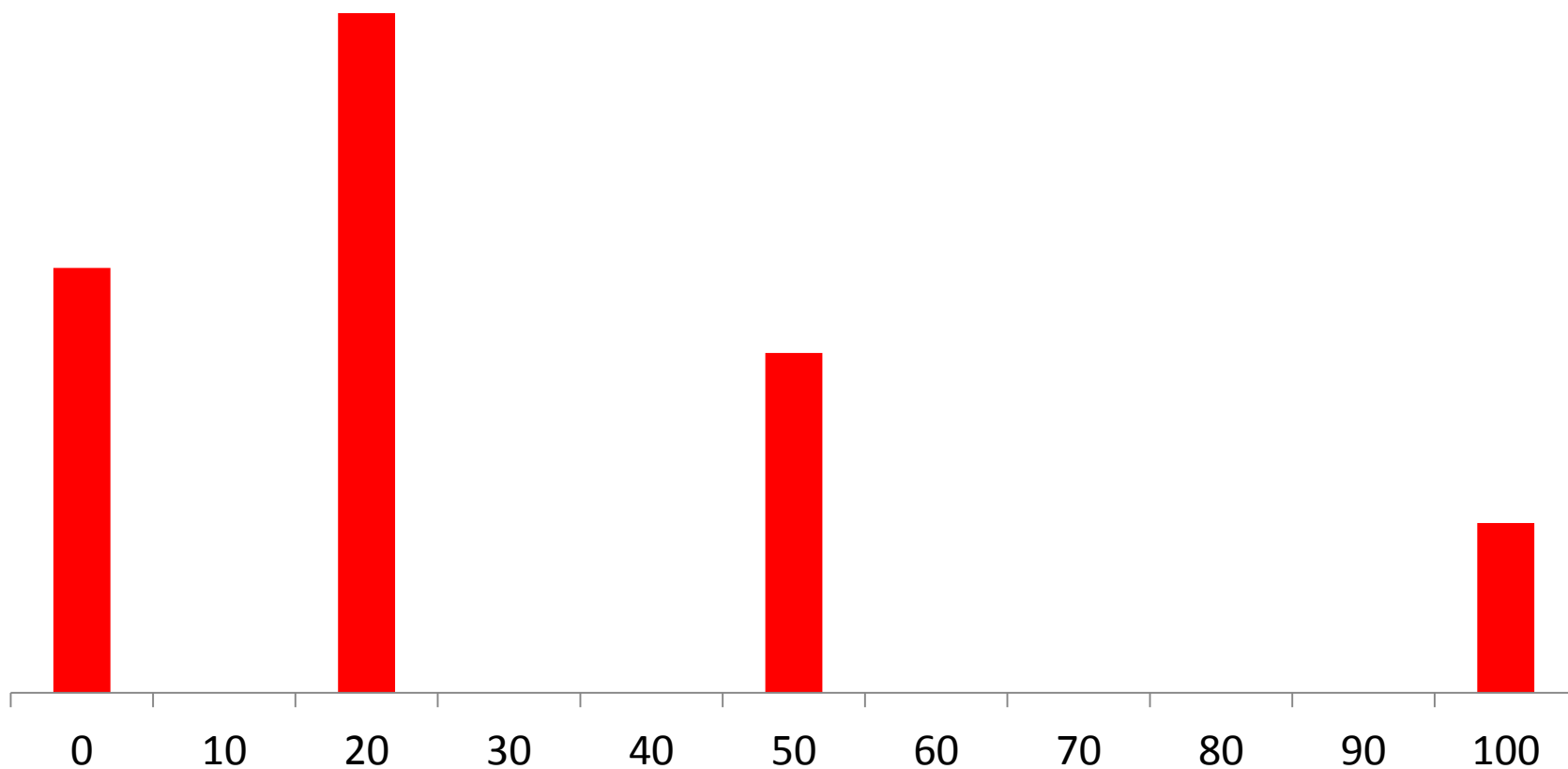
- 悪いジャンプによる疲労のみを考慮してDPを書き直すと、

数式を使わない考え方

- $P[i] = \max(\max\{P[k] - A : k < i, T_i \% D < T_k \% D\}, \max\{P[k] : k < i, T_i \% D \geq T_k \% D\}) + B[i]$
- 同じ式が得られる。
- 良いジャンプによる疲労は最後に引けばよい。

得点分布

得点分布



Mountain Rescue Team

今西 健介(@japlj)

問題概要

Mountain
Rescue Team

まずはこちらをご覧ください↓

<http://japl.pl/play/mountain/mountain.html>

問題概要

Mountain
Rescue Team

遭難者が高所から落下したため、
測定機器を正確な位置を特定したい

いまここ

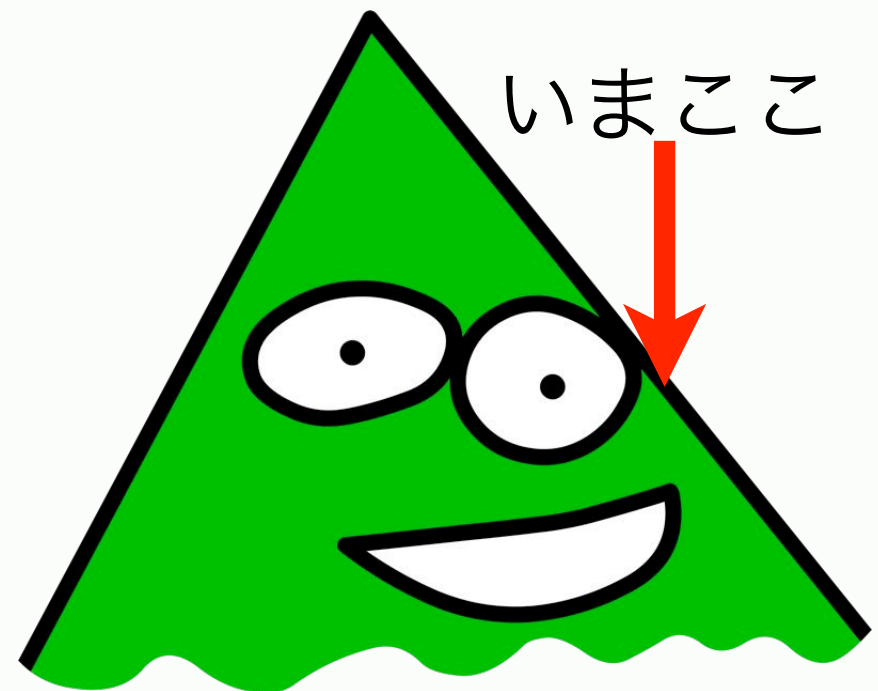
問題概要

Mountain
Rescue Team

遭難者が高度 X の地点にいるので
測定機器を 1,000 回以内使い位置を特定したい

おいしい山形のマス目

- $R \leq 200$
- $C \leq 200$



0点解法

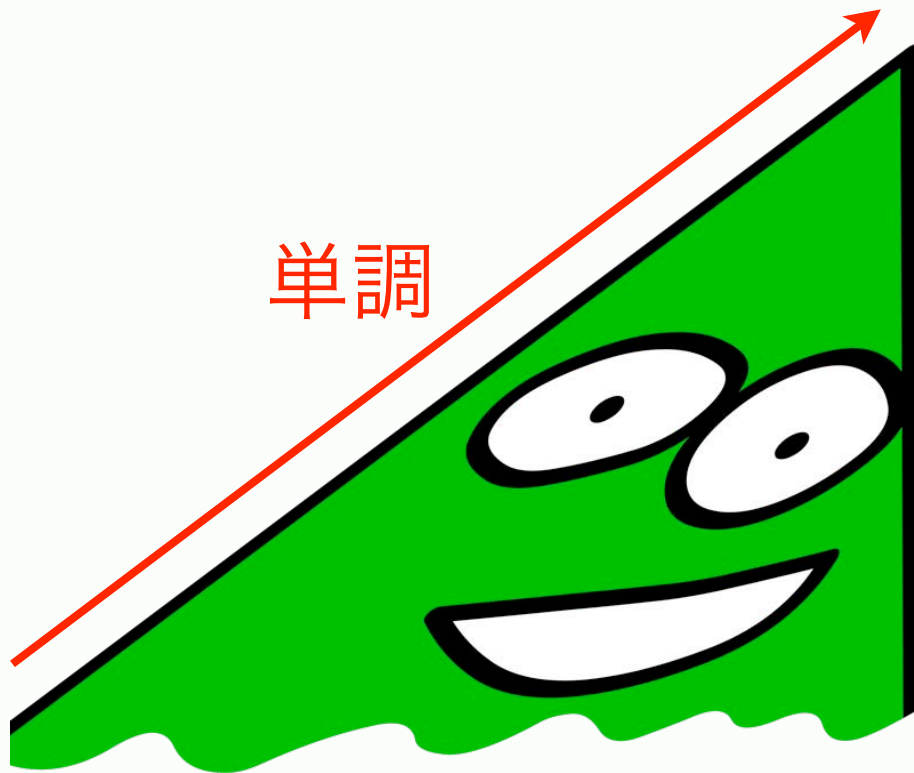
全部のマスを調べて高度 X のマスを見つける

Measure 呼び出し回数

- ・ 最大で $RC \leq 40,000$ 回
- ・ これを実装すると小課題 1 が解けない
- ・ 小課題 1 の制約でも $RC \leq 2,500$ 回ある
- ・ 0 点が得られる

基本的な考察

頂点が端っこにあると山の形が良い感じになる

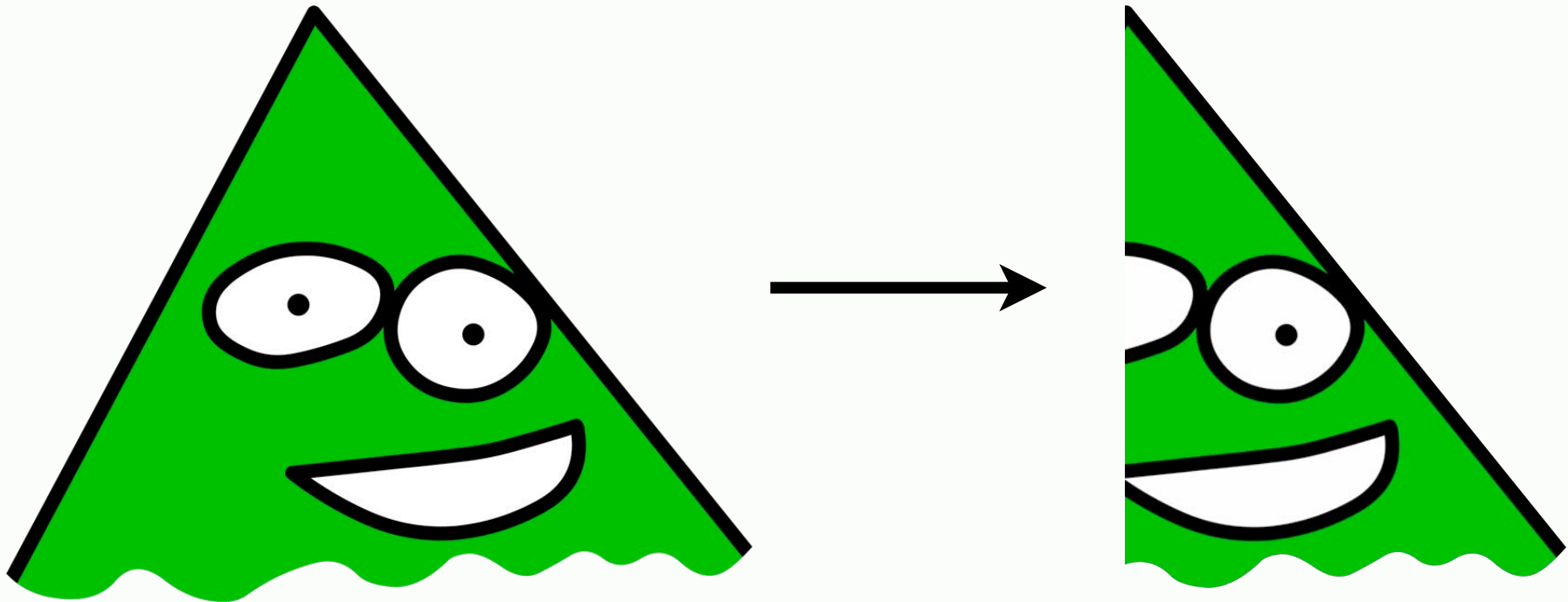


画像はイメージです

5	8	19	29	42
16	24	31	50	55
25	36	68	69	71
33	51	78	88	94
37	66	82	91	100

基本的な考察(2)

頂点を基準に 4 つの部分に分けるとよい
(観葉四分木ゆうり)



基本的な考察(3)

Mountain
Rescue Team

この場合が解ければよいことがわかった



解法

横に見るとただの
ソートされた列である
(縦に見ても同じ)

5	8	19	29	42
16	24	31	50	55
25	36	68	69	71
33	51	78	88	94
37	66	82	91	100

解法(2)

5	8	19	29	42
---	---	----	----	----

単調なので二分探索すればOKであった(完)

解法(3)

Measure 呼び出し回数

- ・ 一行の二分探索には $\log C$ 回ぐらい
- ・ 全部で R 行あるので合計 $R \log C$ 回ぐらい

具体的には？

- ・ 何回ぐらい呼び出すことになるんだろう？
- ・ 何点ぐらいとれるんだろう？

20点解法(7)

実際の呼び出し回数

- 小課題 1 は $R, C \leq 50$
- $50 \log 50 \doteq 282 < 1000$
- 小課題 2 は $R, C \leq 200$
- $200 \log 200 \doteq 1529 > 1000$

→小課題 1 だけが解けて 20 点

100点解法

→
再びこの例で log を
消す方法を説明
 $X = 50$ としよう

5	8	19	29	42
16	24	31	50	55
25	36	68	69	71
33	51	78	88	94
37	66	82	91	100

100点解法(2)

左下からはじめる

5	8	19	29	42
16	24	31	50	55
25	36	68	69	71
33	51	78	88	94
37	66	82	91	100

$$X = 50$$

100点解法(3)

$37 < 50$ なので
37のマスของすぐ上方向
には X は存在しない

次は右方向に進む

5	8	19	29	42
16	24	31	50	55
25	36	68	69	71
33	51	78	88	94
37	66	82	91	100

$$X = 50$$

100点解法(4)

66 > 50 なので
66のマスのすぐ右方向
には X は存在しない

次は上方向に進む

5	8	19	29	42
16	24	31	50	55
25	36	68	69	71
33	51	78	88	94
37	66	82	91	100

$$X = 50$$

100点解法(5)

51 > 50 なので
51のマスのおすぐ右方向
には X は存在しない

次は上方向に進む

5	8	19	29	42
16	24	31	50	55
25	36	68	69	71
33	51	78	88	94
37	66	82	91	100

$$X = 50$$

100点解法(6)

$36 < 50$ なので
36のマスของすぐ上方向
には X は存在しない

次は右方向に進む

5	8	19	29	42
16	24	31	50	55
25	36	68	69	71
33	51	78	88	94
37	66	82	91	100

$$X = 50$$

100点解法(7)

68 > 50 なので
68のマスのすぐ右方向
には X は存在しない

次は上方向に進む

5	8	19	29	42
16	24	31	50	55
25	36	68	69	71
33	51	78	88	94
37	66	82	91	100

$$X = 50$$

100点解法(8)

31 < 50 なので
31のマスのおすぐ上方向
には X は存在しない

次は右方向に進む

5	8	19	29	42
16	24	31	50	55
25	36	68	69	71
33	51	78	88	94
37	66	82	91	100

$$X = 50$$

100点解法(9)

$$50 = 50$$

5	8	19	29	42
16	24	31	50	55
25	36	68	69	71
33	51	78	88	94
37	66	82	91	100

$$X = 50$$

100点解法(10)

Measure 呼び出し回数

- ・ 頂点が端にあれば $R + C - 1$ 回
- ・ そうでなくても高々 $2(R + C)$ 回ぐらい

具体的には？

- ・ $R, C \leq 200$ でも 800 回以下
- ・ 小課題 2 が解ける(100点)

100点別解

色々な解法がありうる
そのうちの一つを紹介

5	8	19	29	42
16	24	31	50	55
25	36	68	69	71
33	51	78	88	94
37	66	82	91	100

$$X = 50$$

100点別解(2)

中心を選んで
高さを調べる

5	8	19	29	42
16	24	31	50	55
25	36	68	69	71
33	51	78	88	94
37	66	82	91	100

$$X = 50$$

100点別解(3)

68 > 50 なので
68より右下には
X は存在しない

5	8	19	29	42
16	24	31	50	55
25	36	68	69	71
33	51	78	88	94
37	66	82	91	100

$$X = 50$$

100点別解(4)

同じ形(右下が頂点)
の部分3つに分かれる



再帰的に
同じ調査方法をすればよい

5	8	19	29	42
16	24	31	50	55
25	36	68	69	71
33	51	78	88	94
37	66	82	91	100

$$X = 50$$

100点別解(5)

Measure 呼び出し回数

- ・ 解析が少しむずかしい
- ・ 最悪で約 $N^{\log_2 3}$ 回になることがわかる

ただし $N = \max(R, C)$

具体的には？

- ・ $200^{\log_2 3} \doteq 4436$ と超でかい
- ・ しかし小課題 2 が解ける(100点)

実際に最悪になることはほとんどない(乱数を使うと)

下界の証明

この問題では実は必要な Measure の呼び出し
回数の下界が証明できる

下界とは

- ・簡単に言うと, 少なくとも何回は呼び出す必要があるかというのがわかる
- ・ $R = C$, 頂点が右下として考えることにする

下界の証明(概略)

山のデータを意地悪に作っていく

- ・ データを作る人の立場になって考える
- ・ クエリが飛んできてから山の高度を考える
- ・ このときできるだけ意地悪なものをつくる

すると？

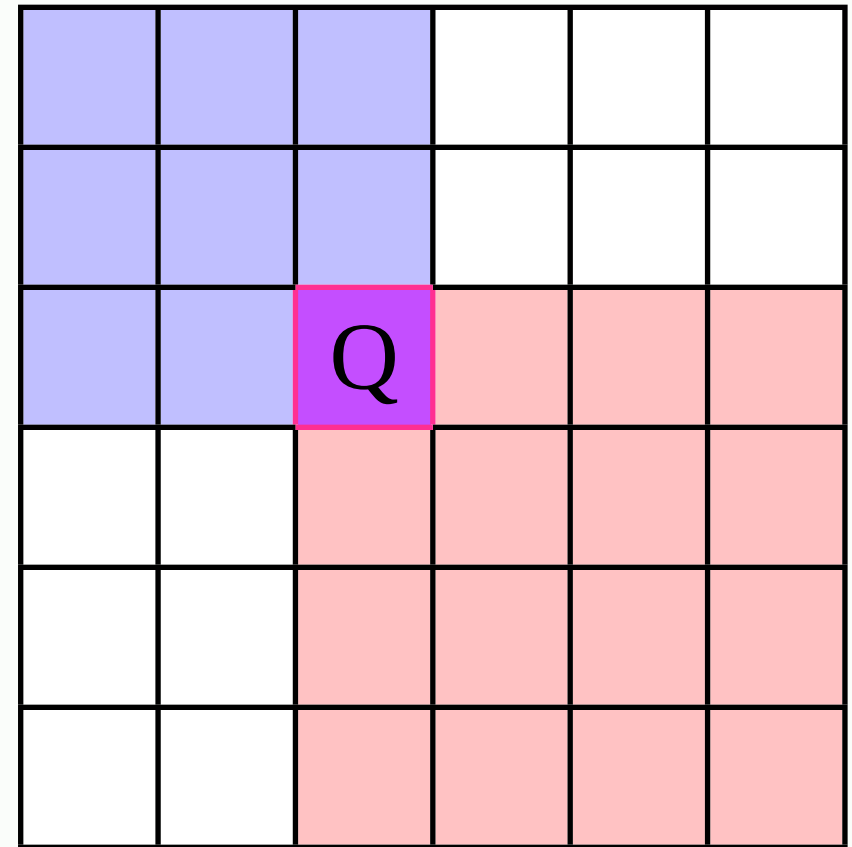
- ・ 下界を考えるには次のような問題を考えればよいことになる

下界の証明(概略2)

Q というマスにクエリ
が飛んできた場合

- ・ 赤のマスが考慮外になる
 - ・ 青のマスが考慮外になる
- のうち好きな方を選ぶ

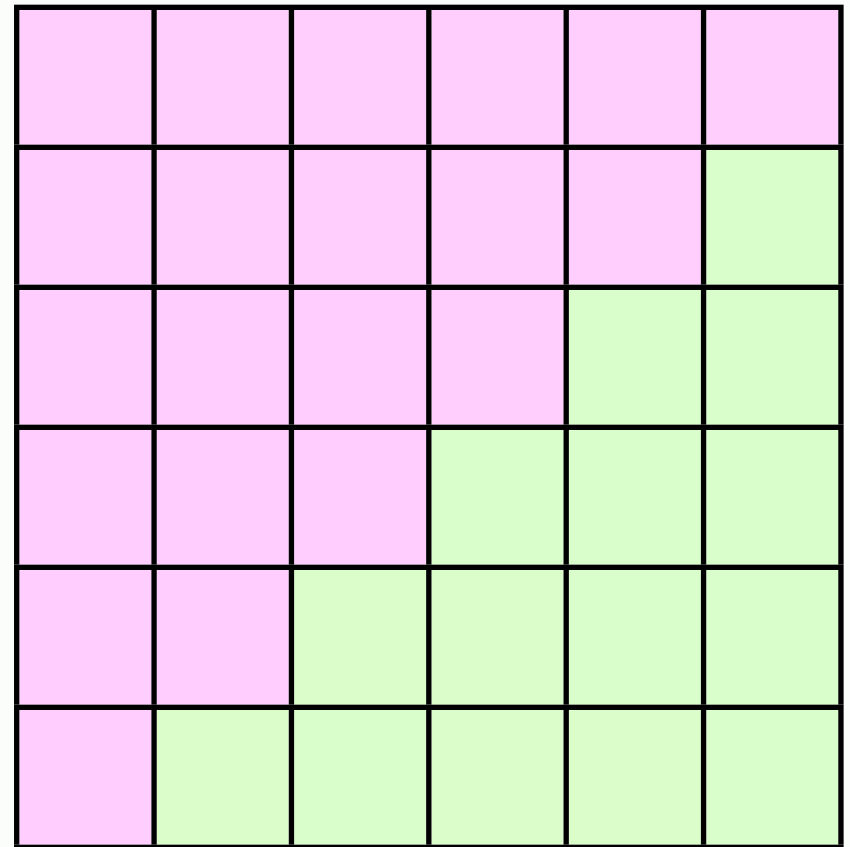
残りが1マスになると終了



下界の証明(概略3)

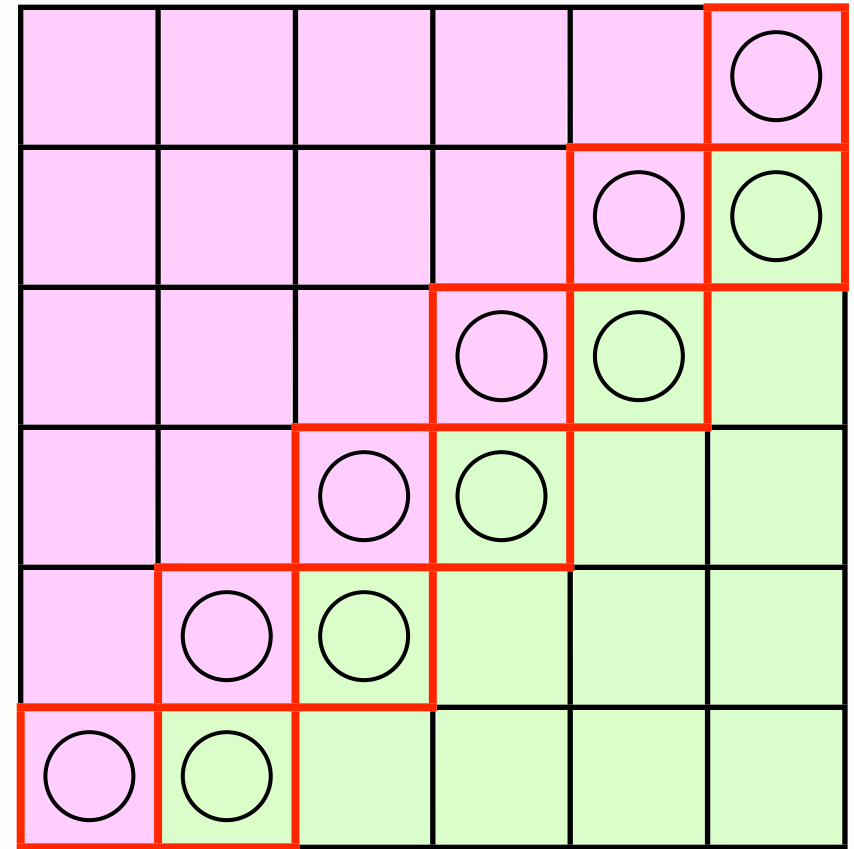
次の戦略を考えよう

- ・ 緑のマスにクエリ
→右下を消す
- ・ 桃のマスにクエリ
→左上を消す



下界の証明(概略4)

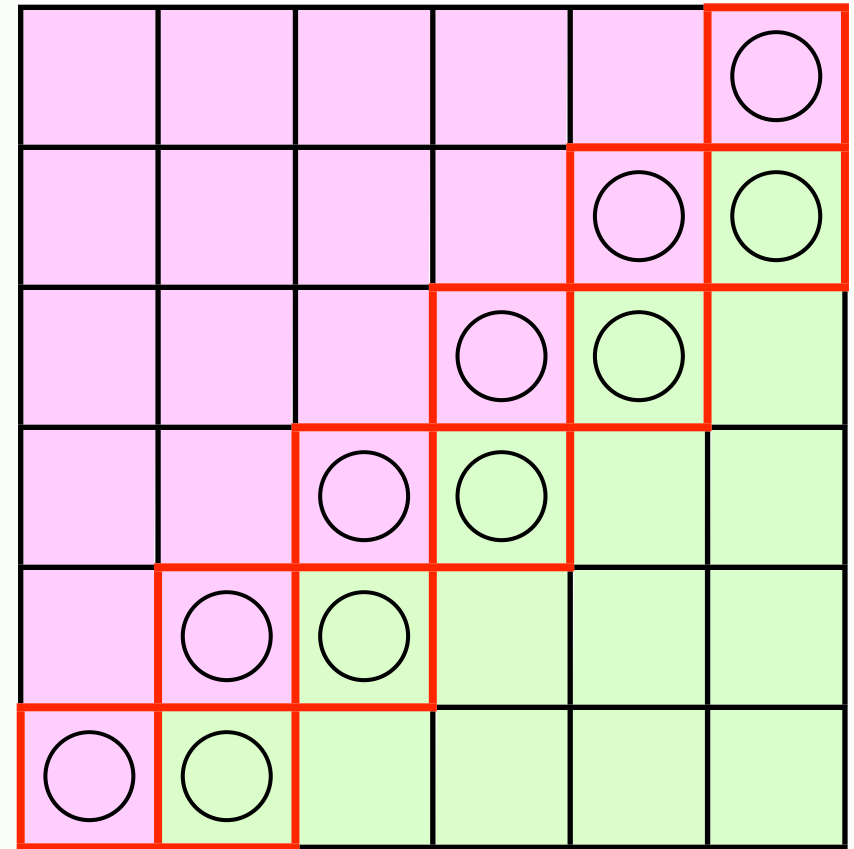
このようなデータでは
○の書かれたマスには
必ずクエリを飛ばさないと
残りが 1 マスにならない



下界の証明(概略5)

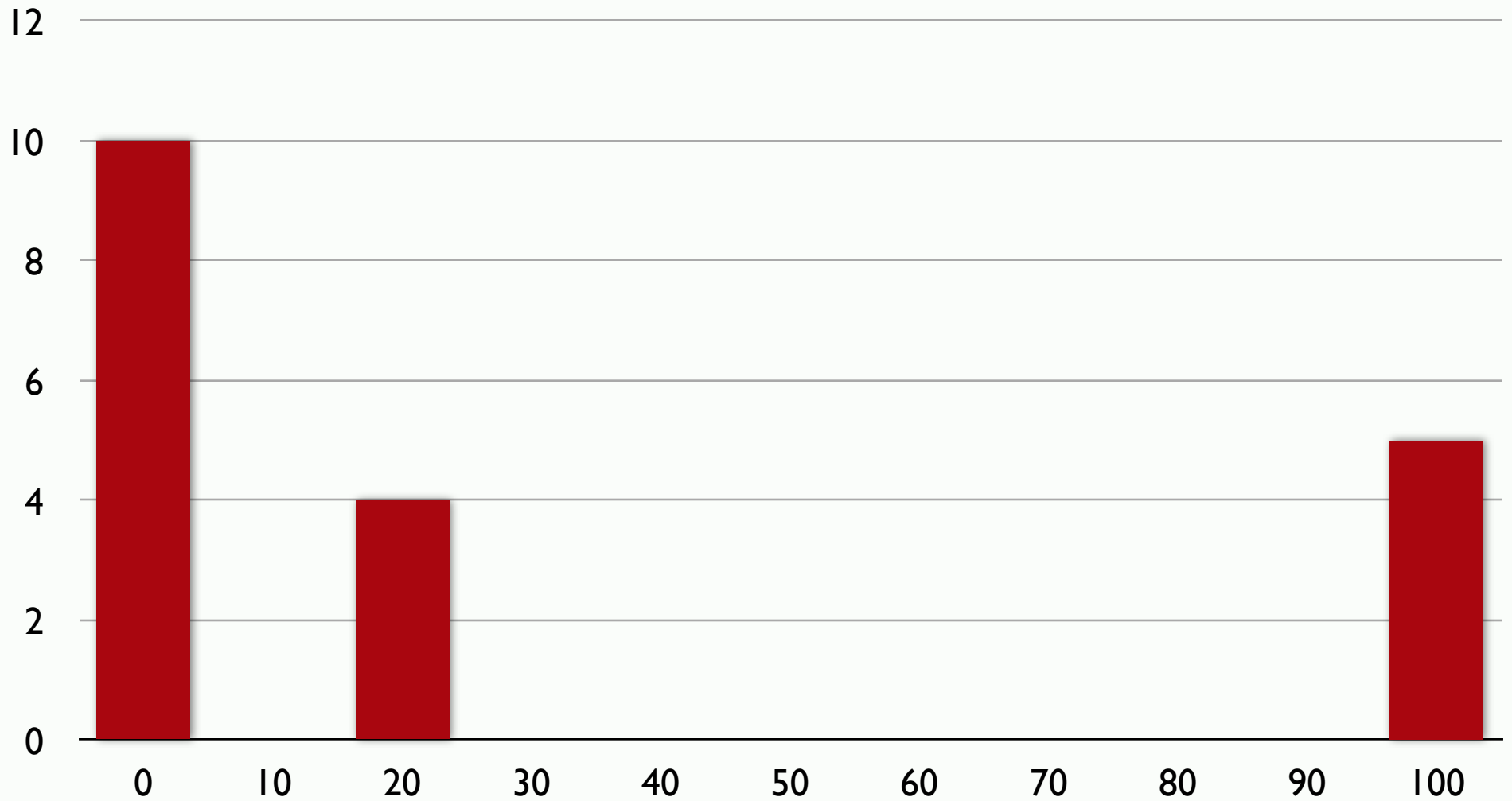
すなわち少なくとも
 $R + C - 1$ 回はクエリを
飛ばす必要があるが
最初に解説した解法は
ちょうど $R + C - 1$ 回の
クエリで X を特定した

→下界!!!



得点分布

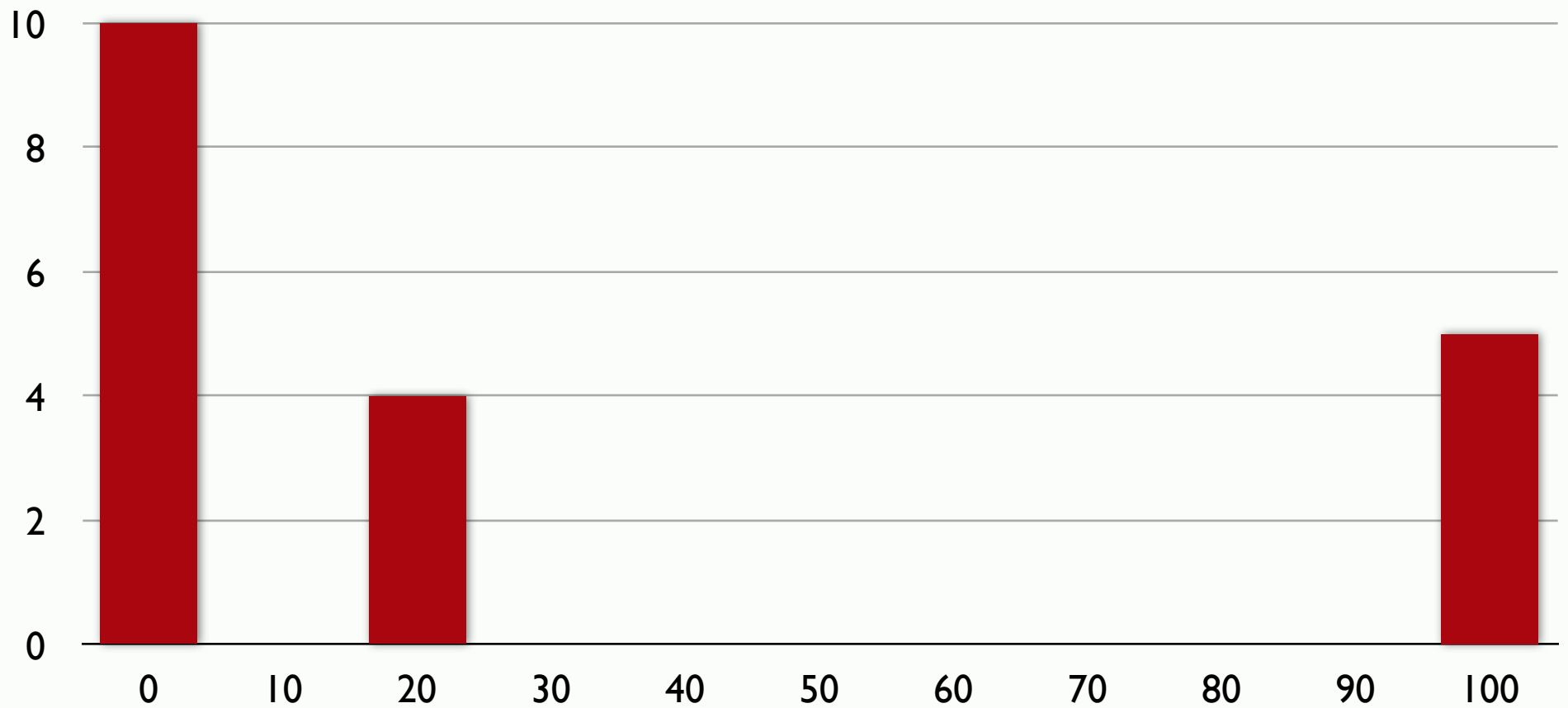
Mountain
Rescue Team



得点分布

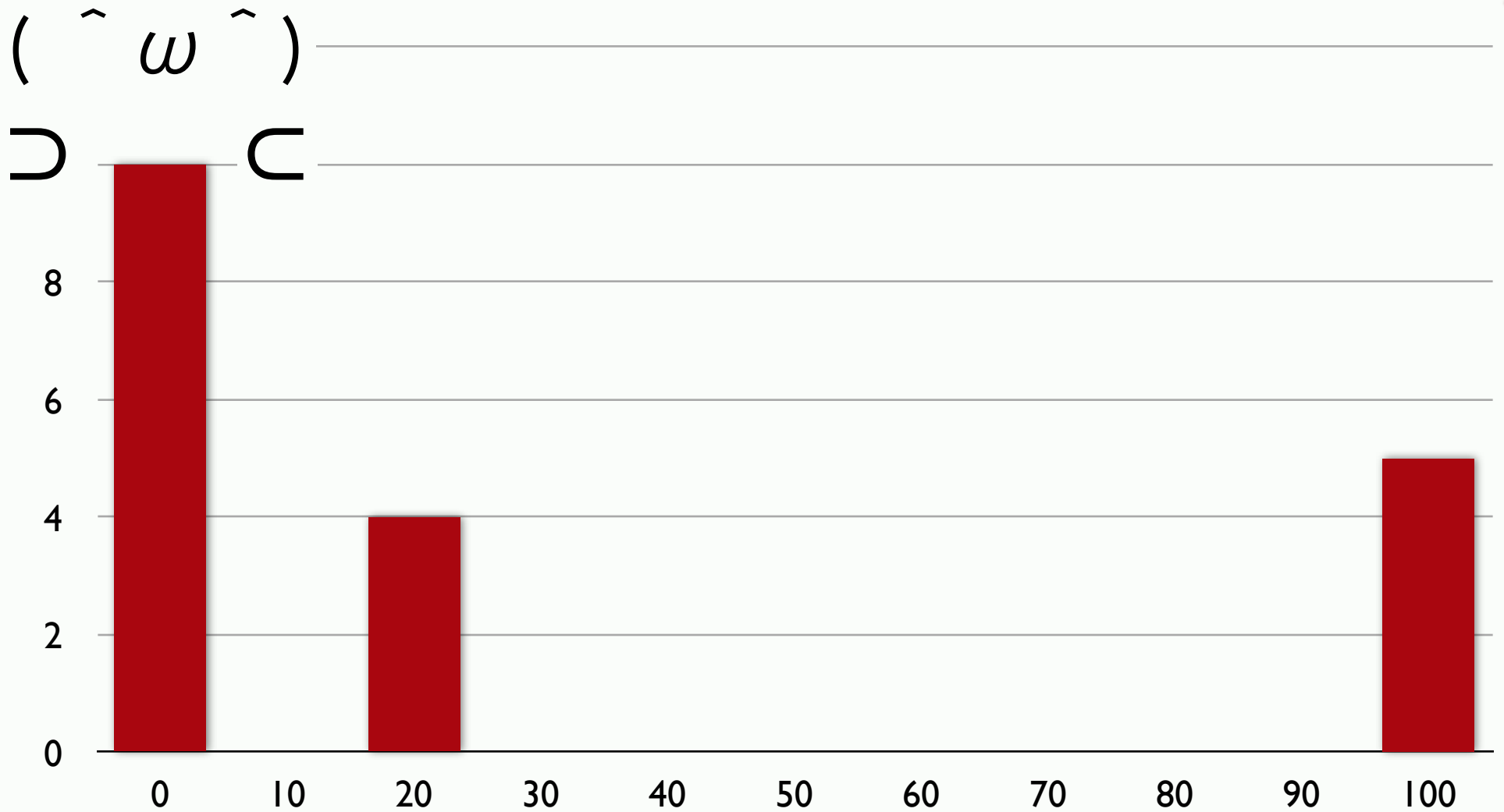
Mountain
Rescue Team

$(\hat{\omega})$



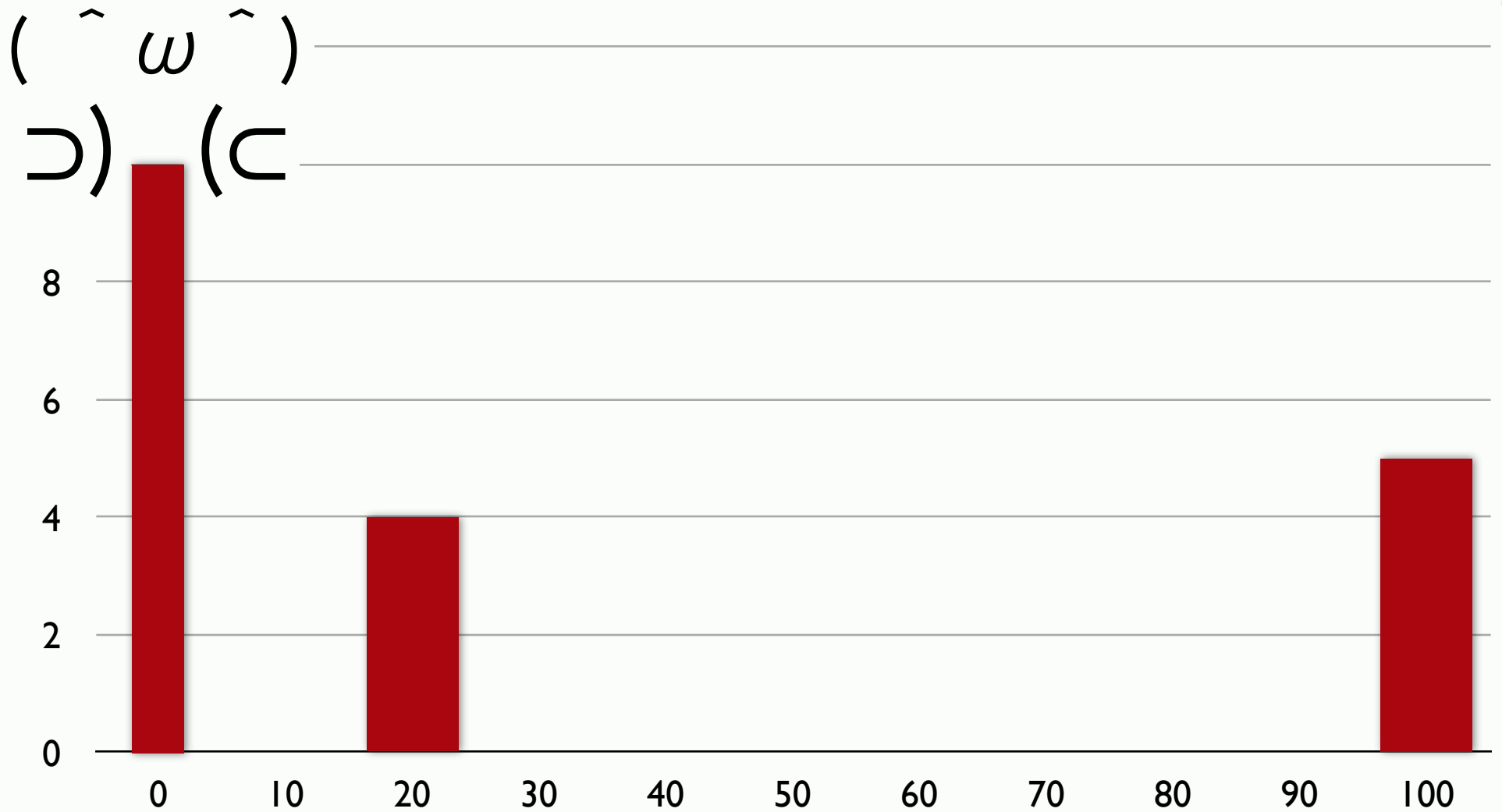
得点分布

Mountain
Rescue Team



得点分布

Mountain
Rescue Team

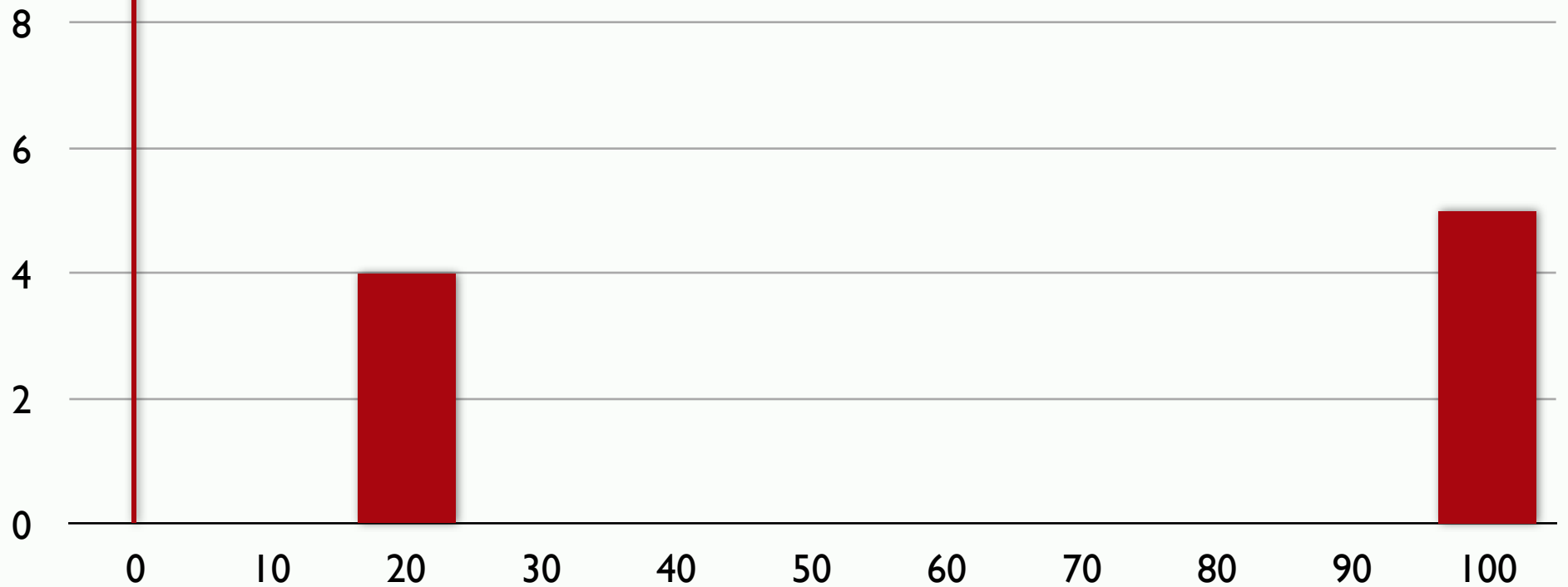


得点分布

Mountain
Rescue Team

$(\hat{\omega})$

$\equiv \sup C \equiv$



得点分布

Mountain
Rescue Team

$(\hat{\omega})$

$\supset \dots \subset$

$\begin{matrix} \cdot & \cdot \\ & \cdot \end{matrix}$

$\begin{matrix} \cdot & \cdot \\ \cdot & \cdot \end{matrix}$

4

2

0

0

10

20

30

40

50

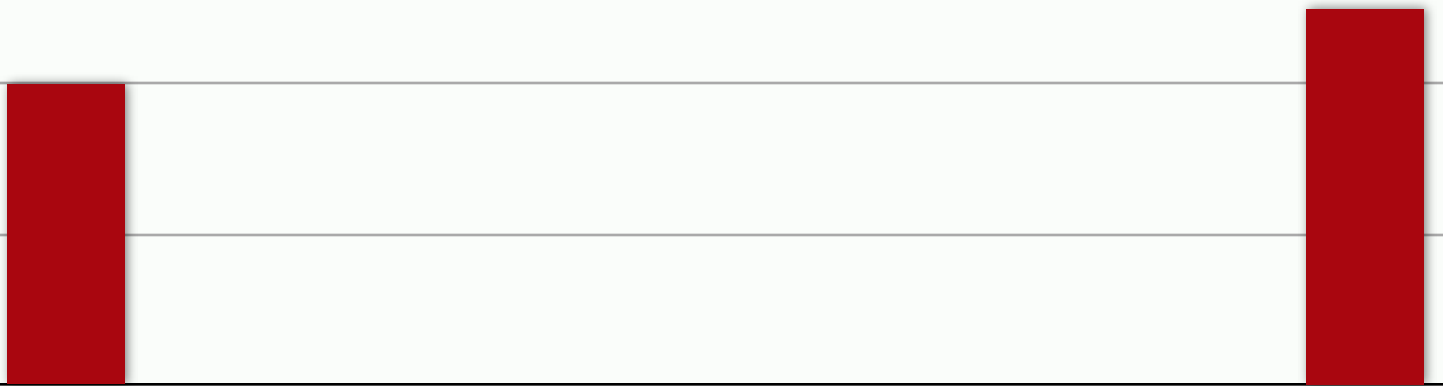
60

70

80

90

100



第12回 日本情報オリンピック春合宿 day4

Messenger

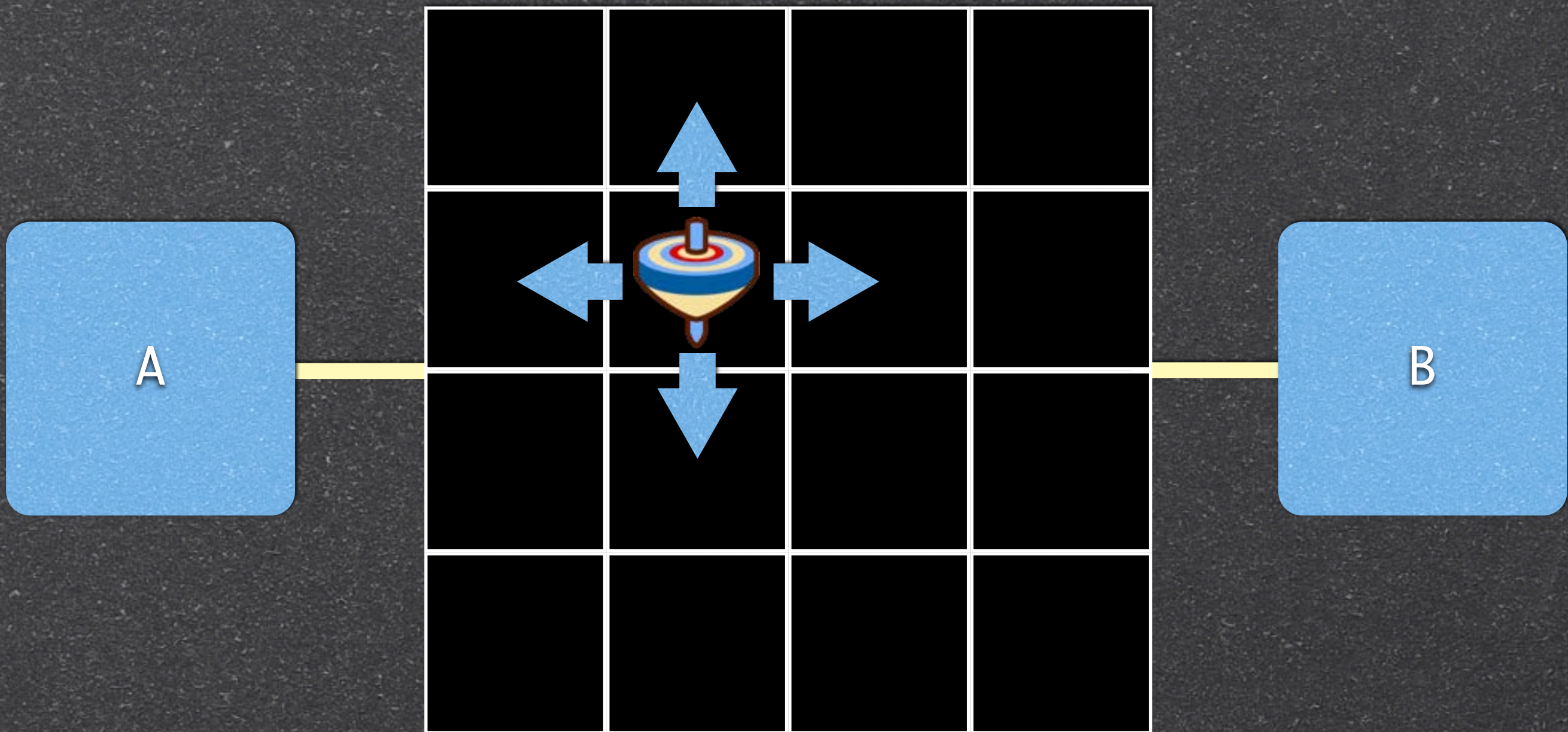
二階堂 建人

(@the_nikaidoes/snuke)

問題概要

- 📌 Aさんにある数 X が伝えられる。 ($1 \leq X \leq 10^9$)
- 📌 AさんとBさんは 4×4 のマス目と駒がある部屋にランダムな順番で呼び出される。
- 📌 駒を動かすことのみによってAさんはBさんに X の値を伝えたい。

K理事長の部屋



基本的な方針

- 📌 X の情報量は 1 度に伝えるには大きすぎる。
- 📌 X の情報を最小単位まで区切る。
- 📌 「情報を最小単位」 = 「yes/no」 = 1bit
- 📌 つまり、X を 2 進数にして 1 桁ずつ伝える。

基本的な方針

- 📌 1bitを伝えるにはどうすれば良いか？



- 📌 0,1に対応する操作を決めておき、Aがその操作を実行して、Bがその操作から1bitの情報を読み取る。

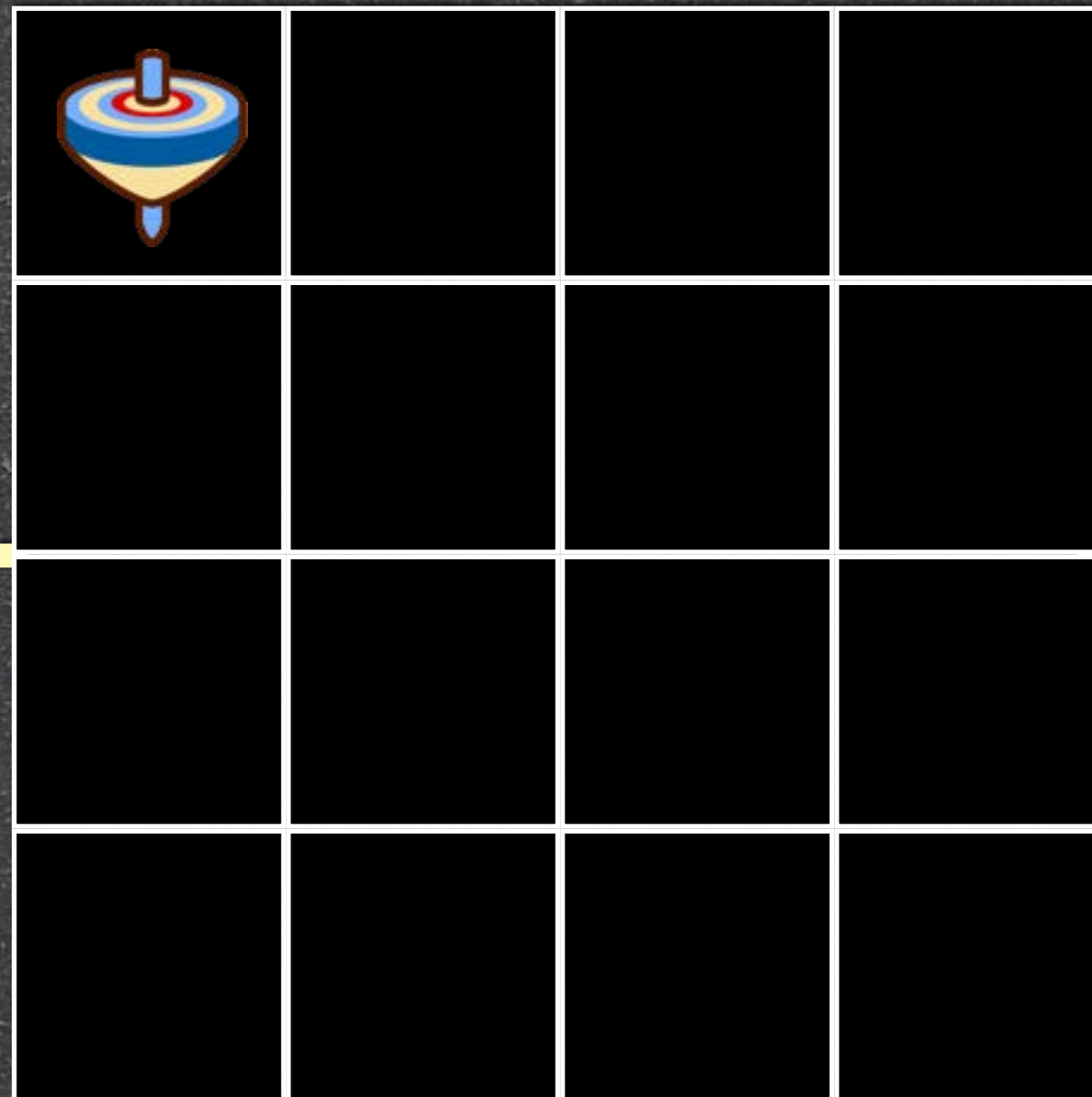
subtask1 (20pts)

- 📌 AさんとBさんがABABABABA...の順で交互に呼び出される。

subtask1 (20pts)

- 📌 0,1に対応する操作の割り当ての例
- 📌 「0なら縦、1なら横に移動させる。」

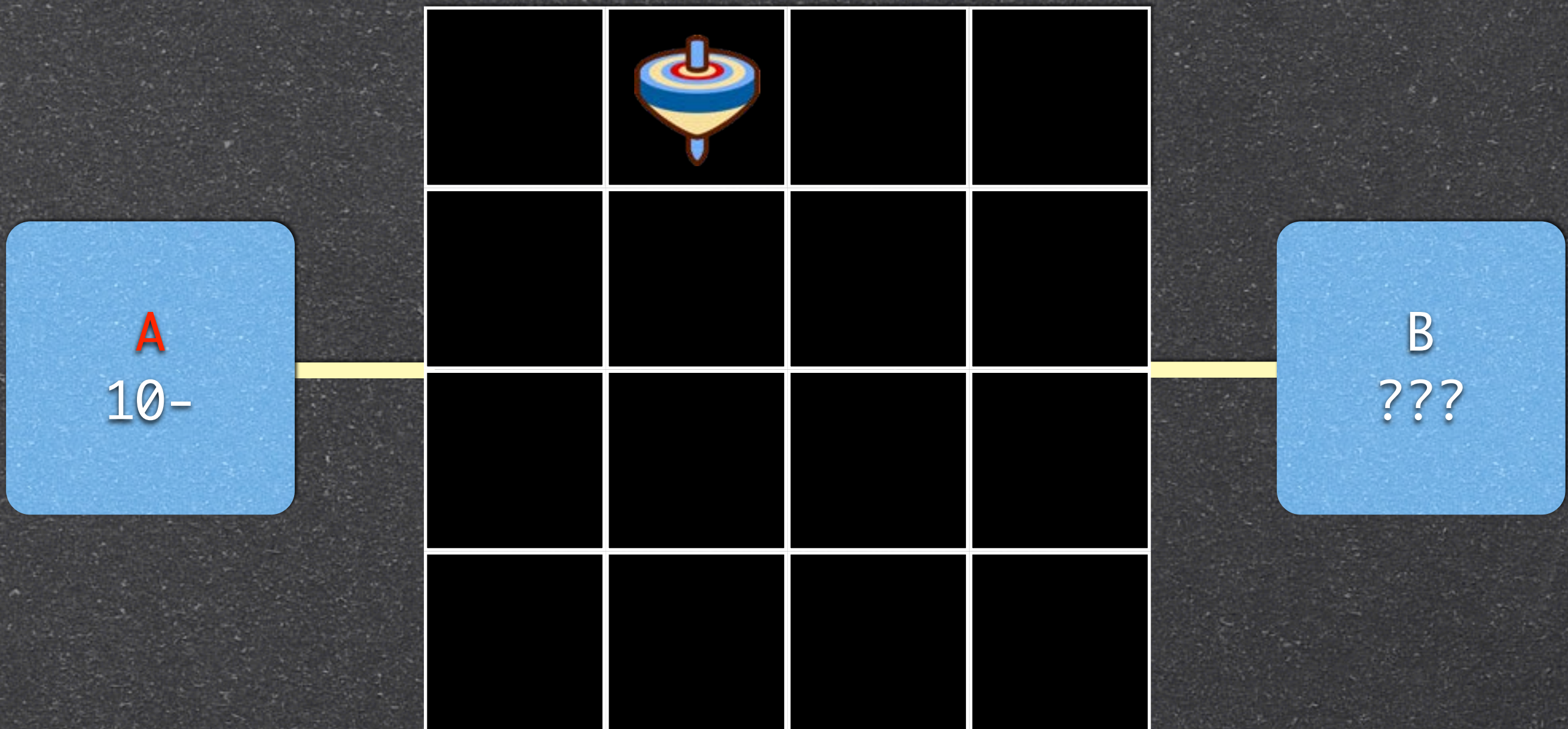
K理事長の部屋



A
101

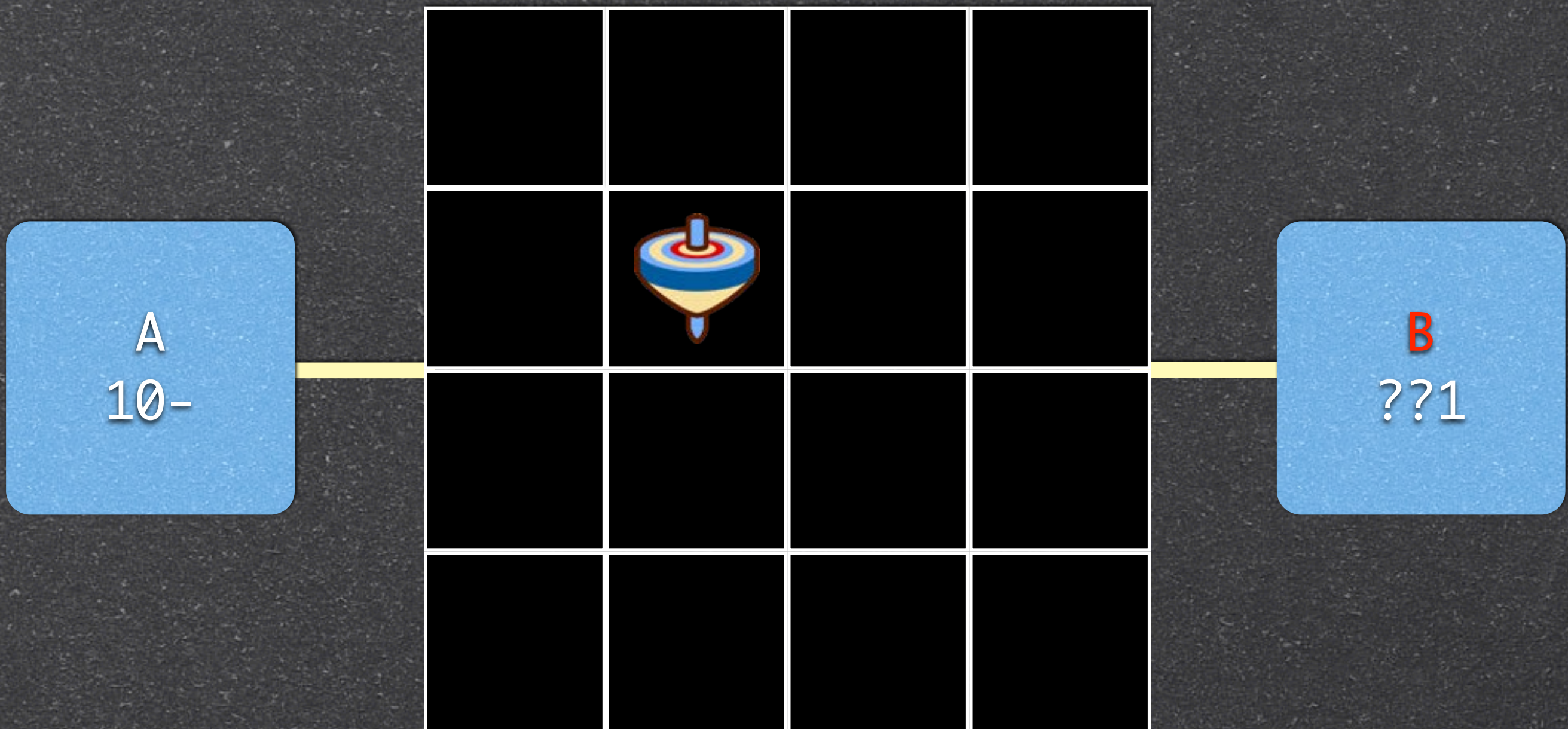
B
???

K理事長の部屋



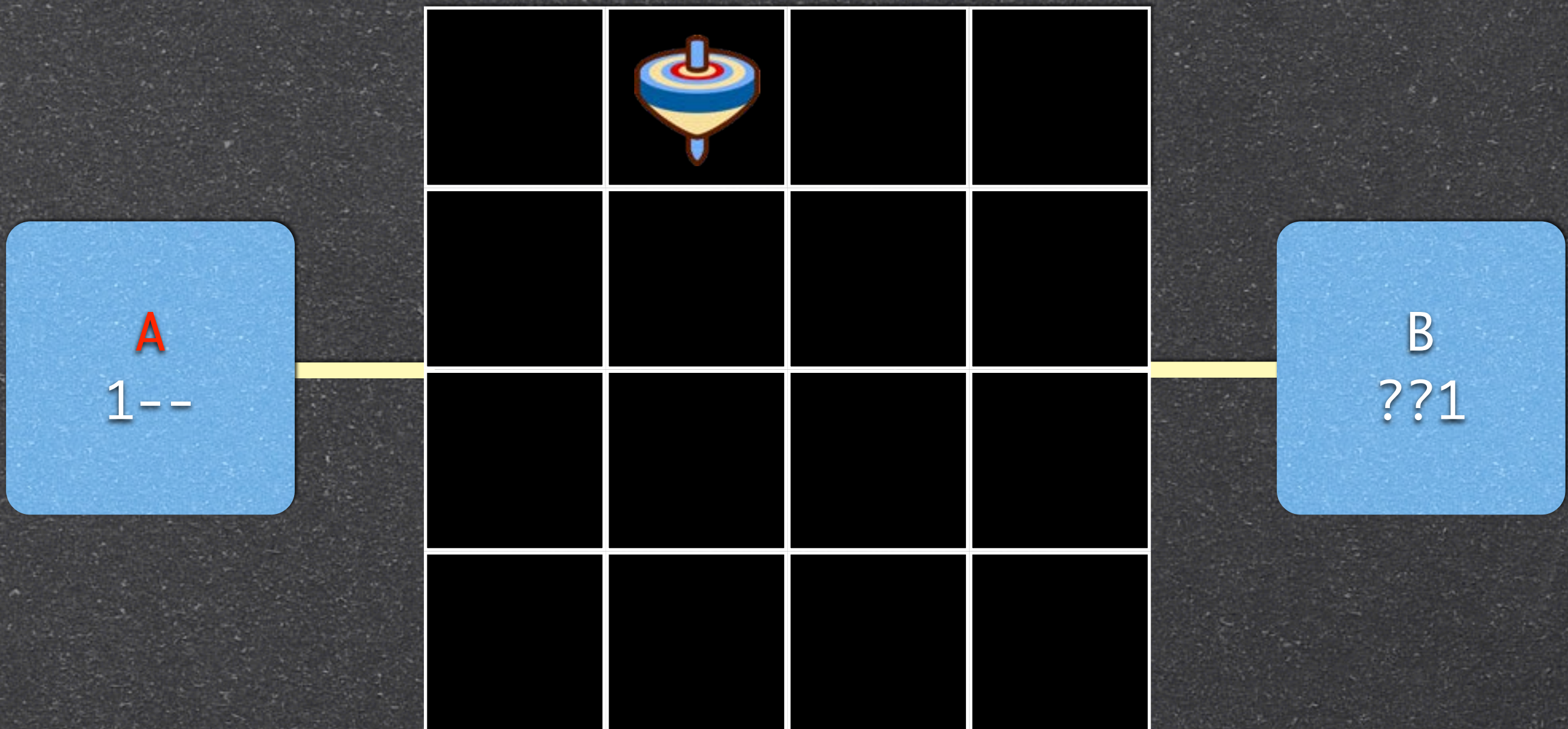
Aが1を伝えるために、駒を横に動かした

K理事長の部屋



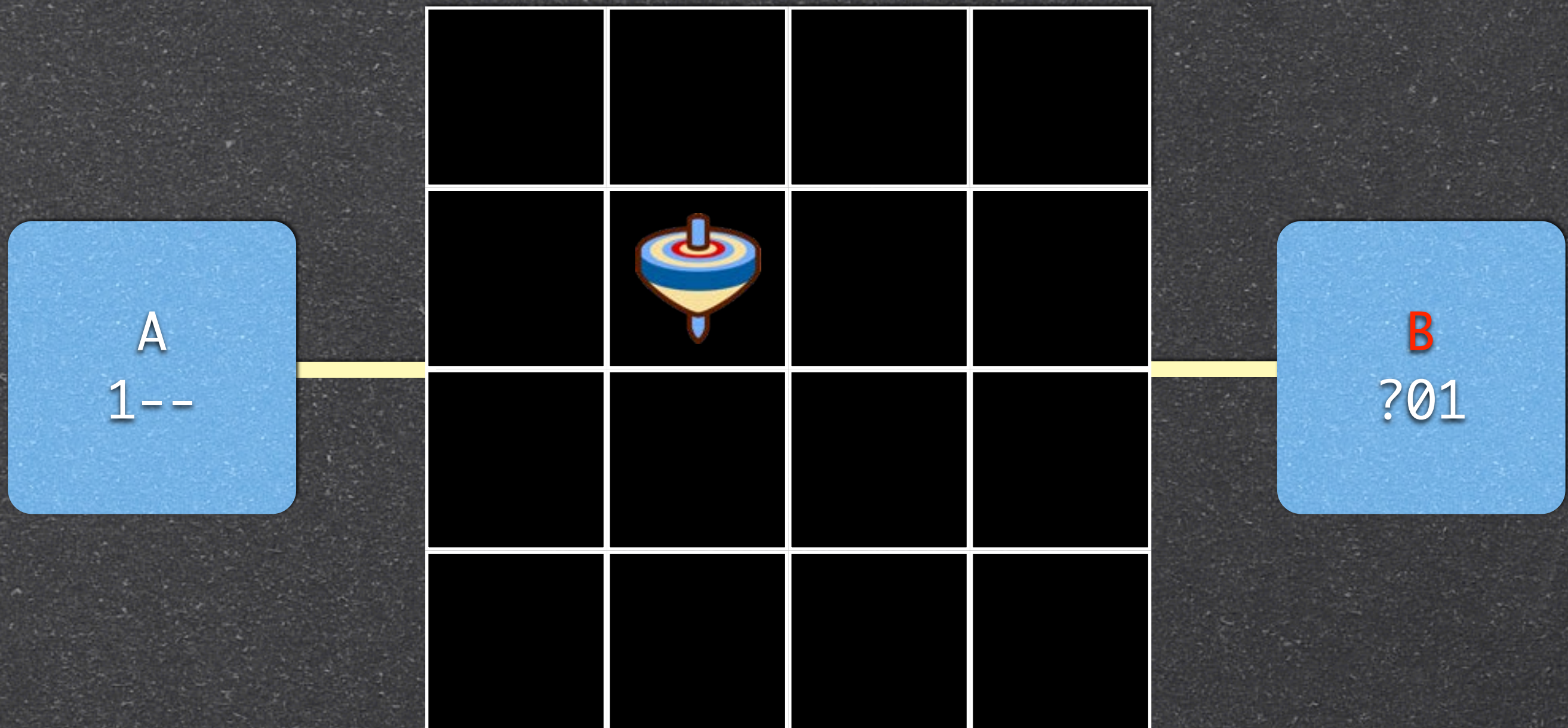
Bが1を読み取り、駒を適当に移動させた

K理事長の部屋



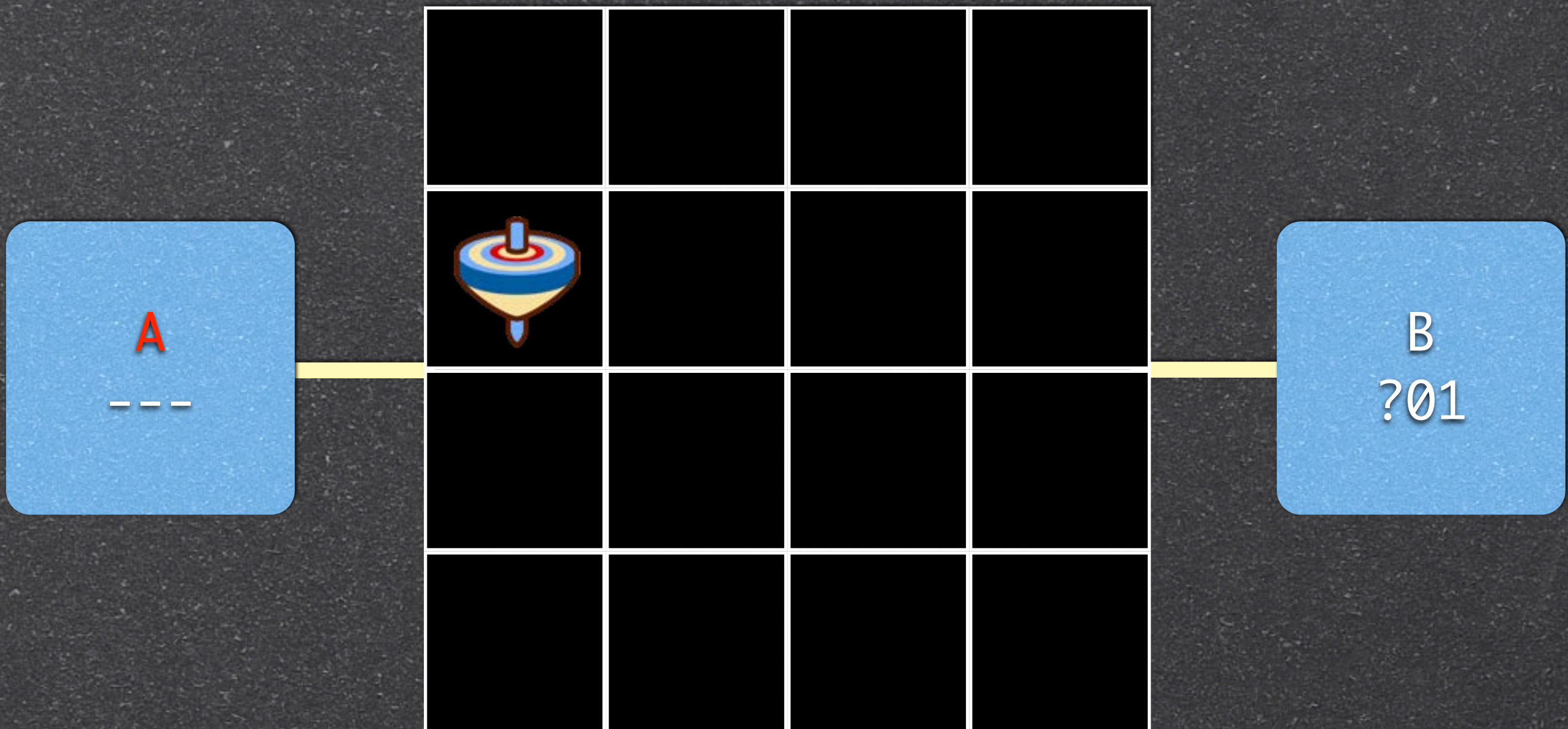
Aが0を伝えるために、駒を縦に動かした

K理事長の部屋



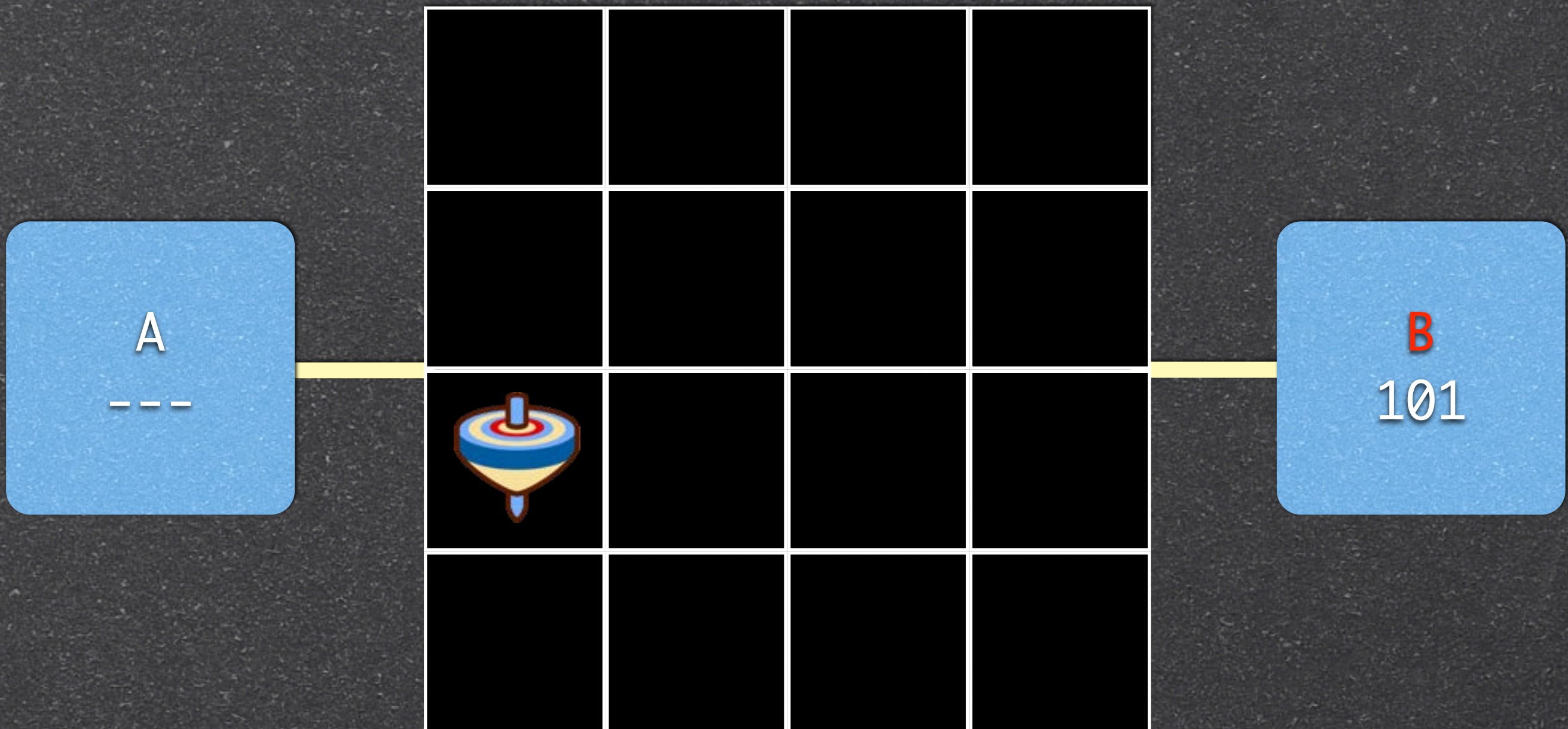
Bが0を読み取り、駒を適当に移動させた

K理事長の部屋



Aが1を伝えるために、駒を横に動かした

K理事長の部屋



Bが1を読み取り、駒を適当に移動させた

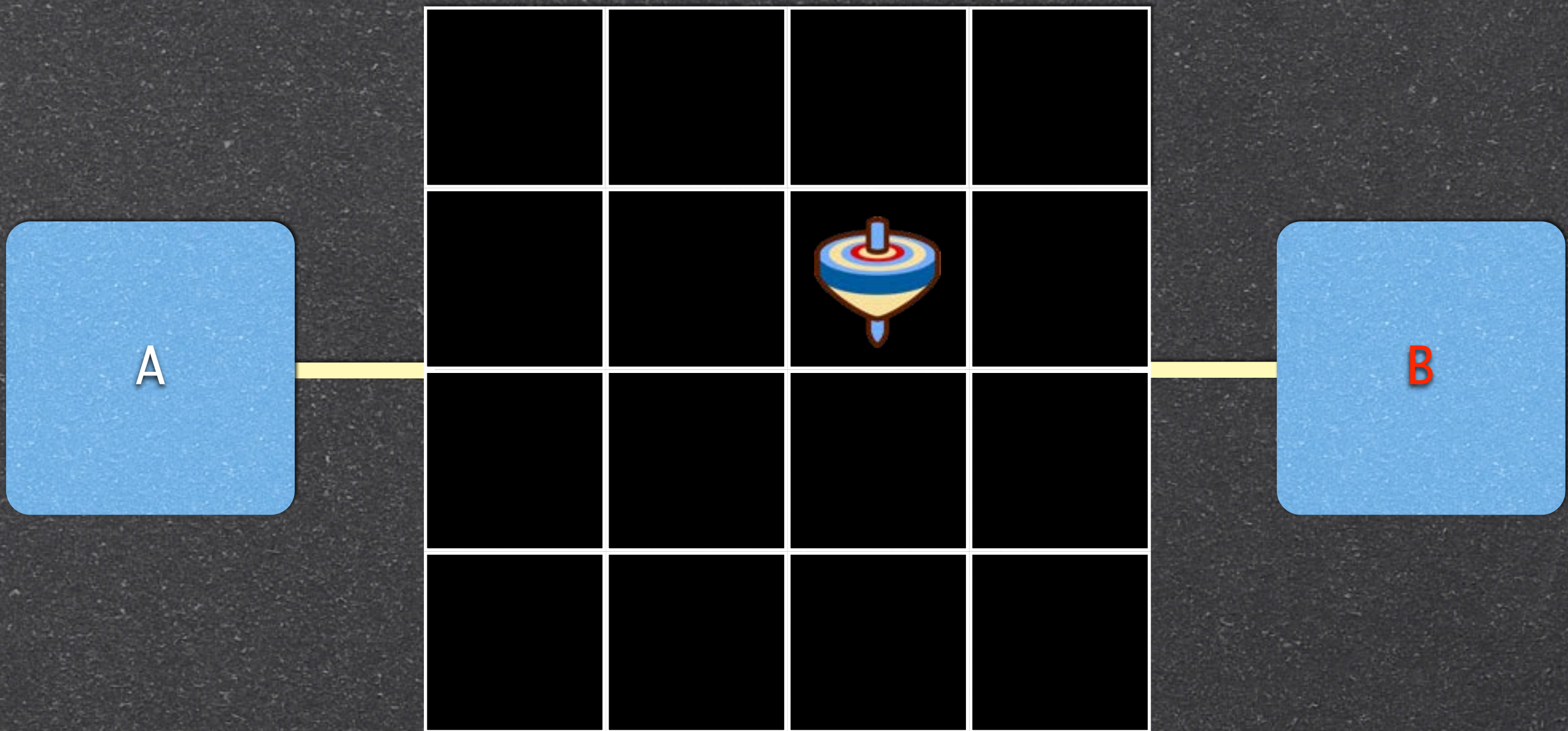
subtask1 (20pts)

- 📌 $2^{29} \leq 10^9 \leq 2^{30}$ なので、30回のやり取りをすれば X の値を伝えることが出来る。
- 📌 Aが縦横どちらに動かしたのかは、Bが動かした後の駒の位置を記憶しておけば良い。
- 📌 ただし1ターン目は、Aがどう動かしたのかがBには分からないので、捨てる。

基本的な方針その2

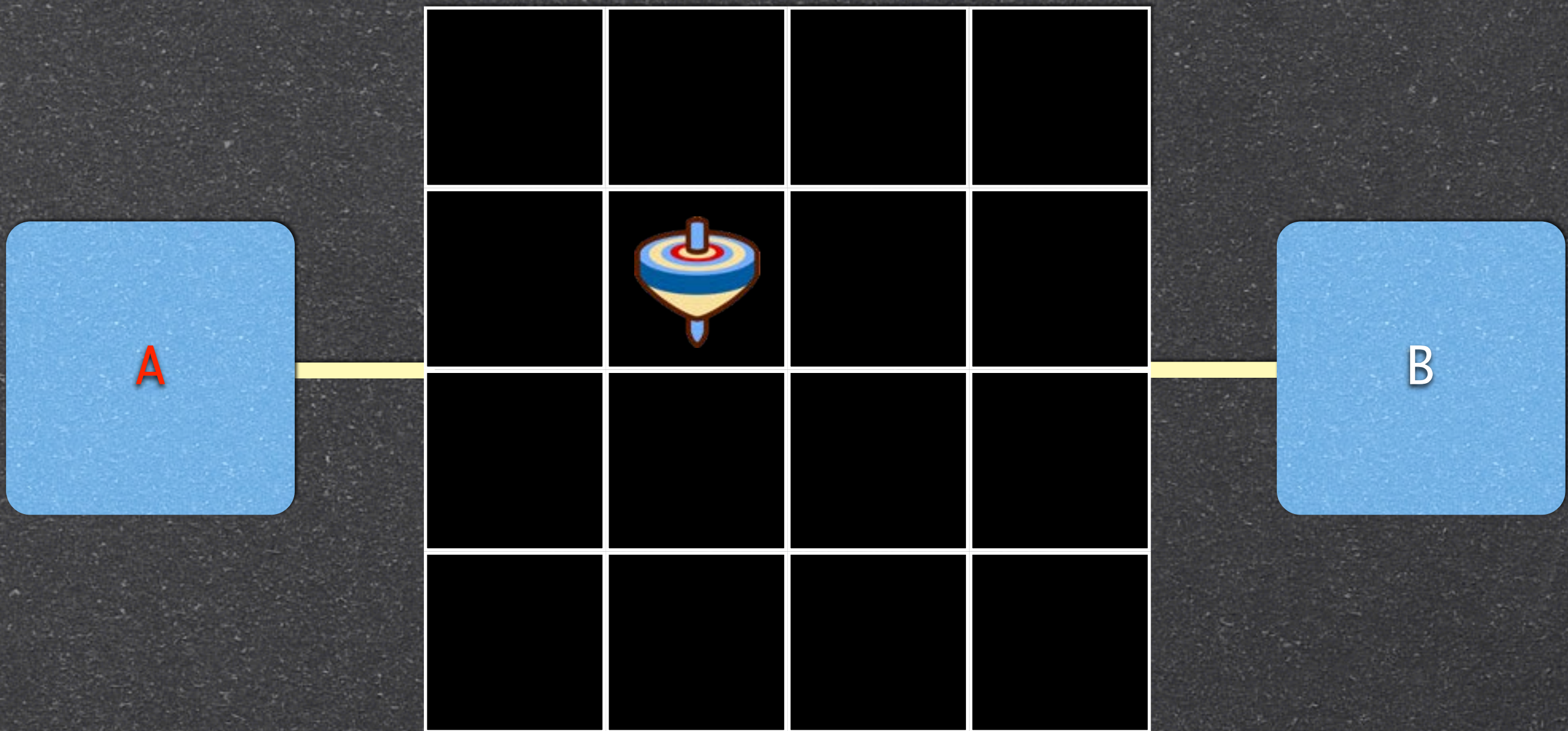
- 📌 これ以降のsubtaskでは、AとBが交互に呼び出されるとは限らない。
- 📌 AAABAABBBBBBABAAA...みたいなものもありうる
- 📌 「止まったまま」が可能なら同じなのだが・・・

K理事長の部屋



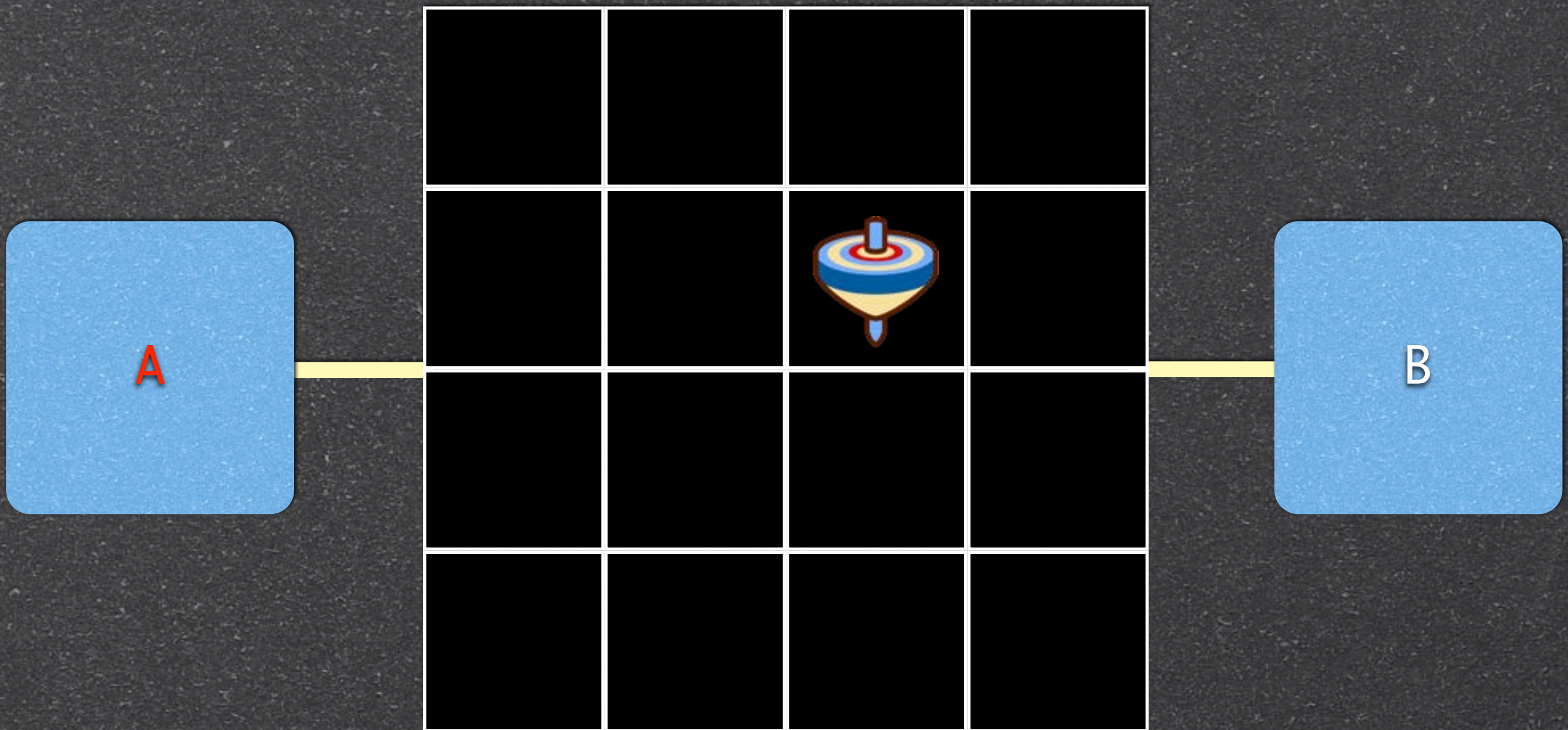
Bが駒を(2,3)に動かした

K理事長の部屋



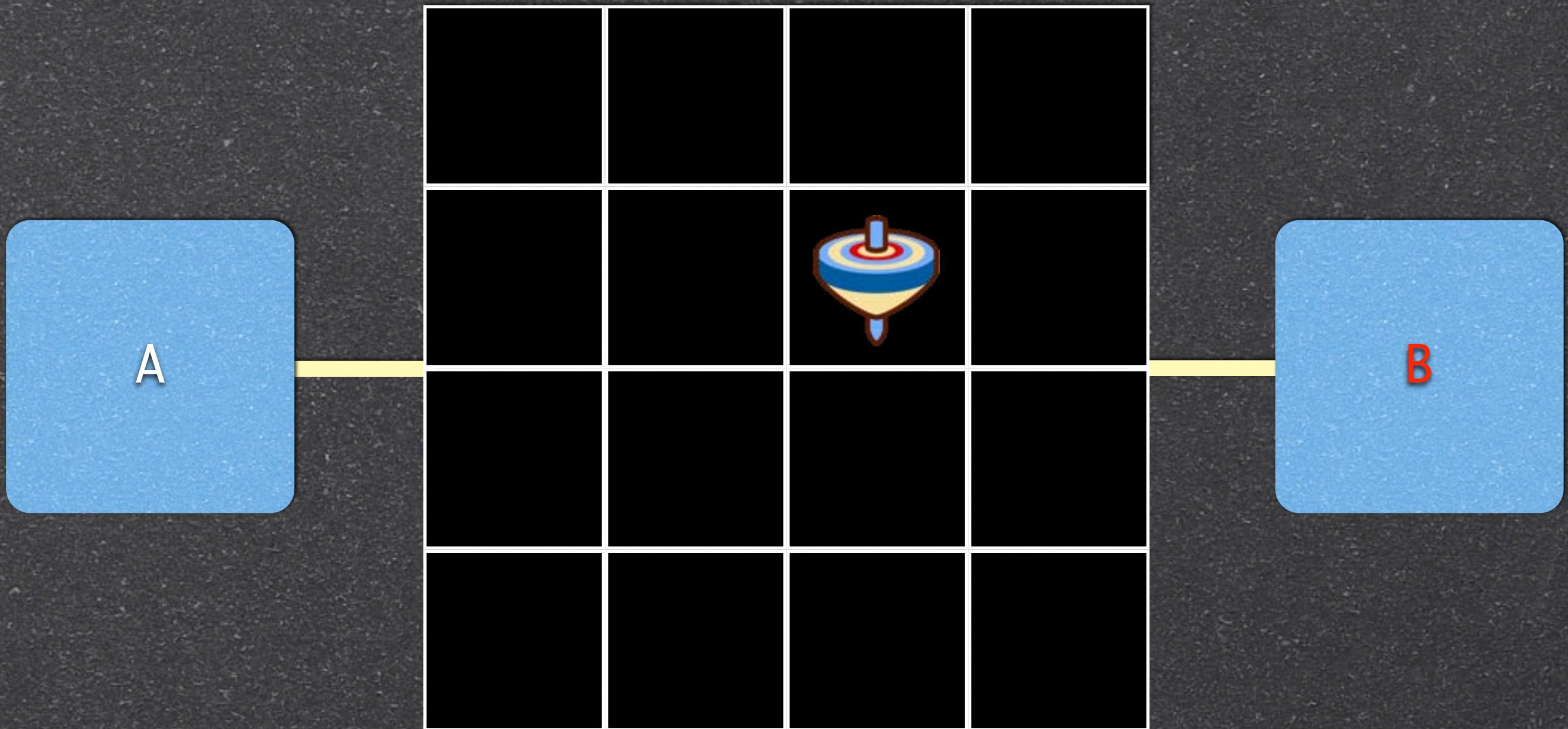
Aが駒を(2,2)に動かした

K理事長の部屋



Aが駒を(2,3)に動かした

K理事長の部屋



Bは駒が(2,3)にあることを知らされる

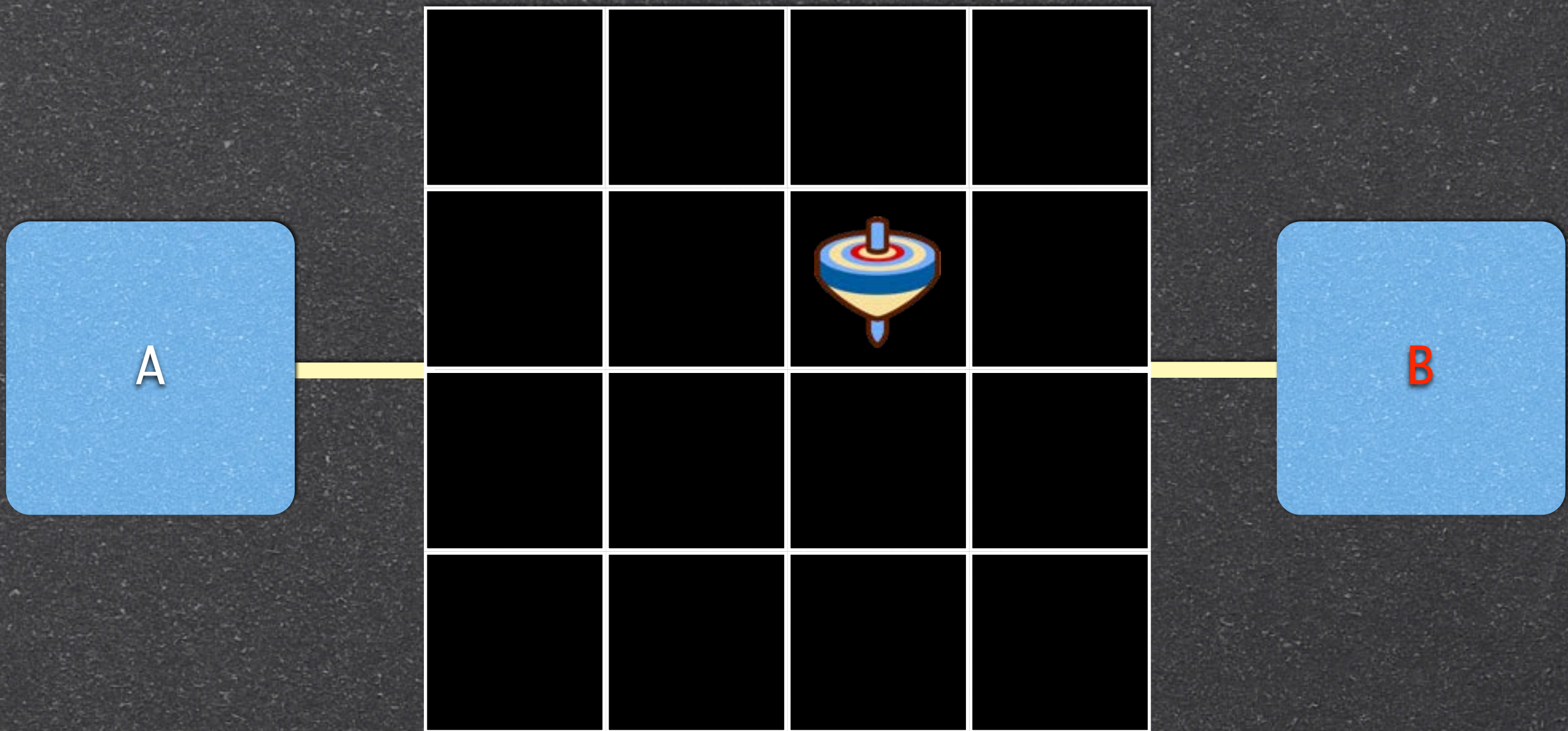
基本的な方針その2

- 📌 このような操作をしてしまった場合、BAABと呼び出された時と、BBと呼び出された時の区別がつかない。



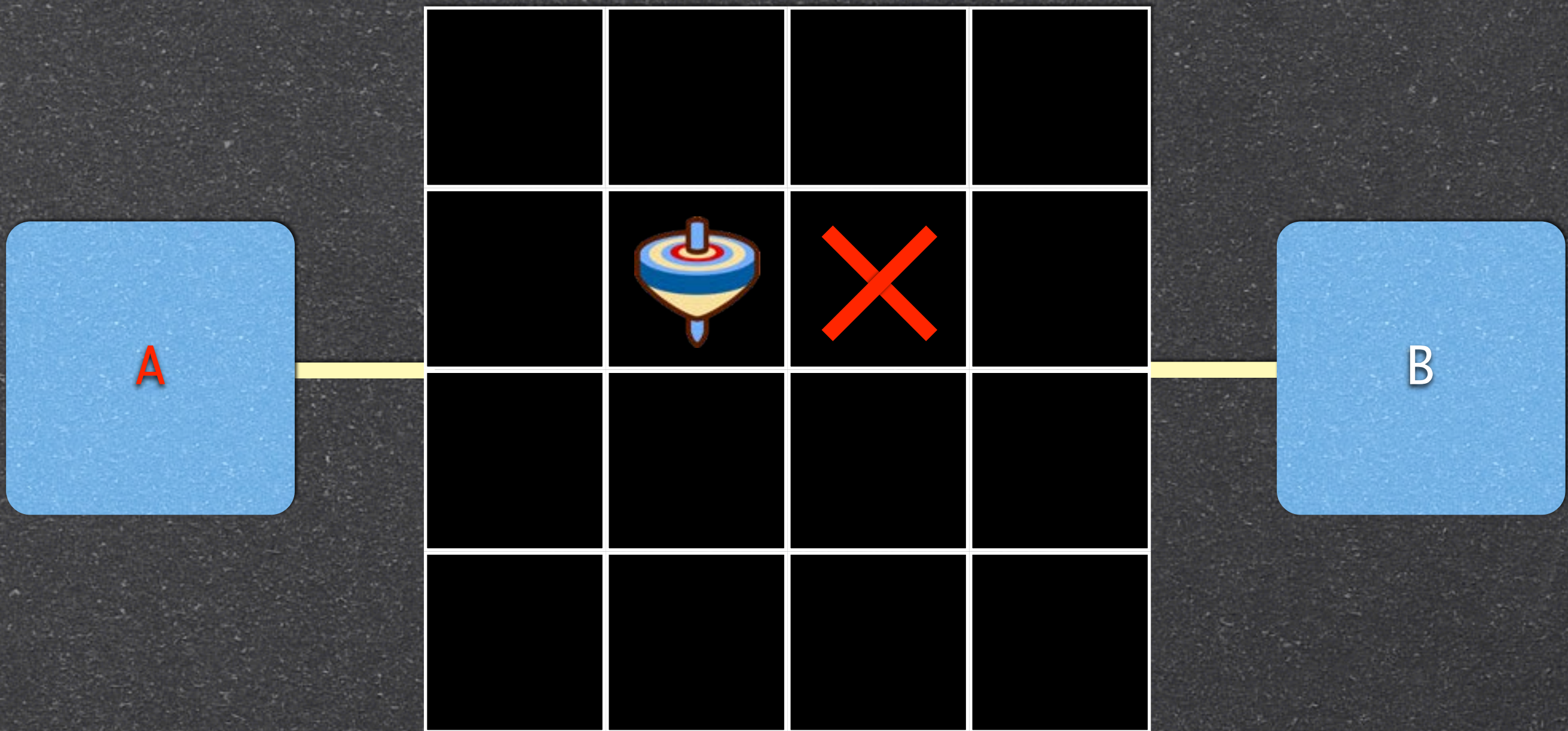
- 📌 相手が動かした場所に動かす操作をすると罠る。

K理事長の部屋



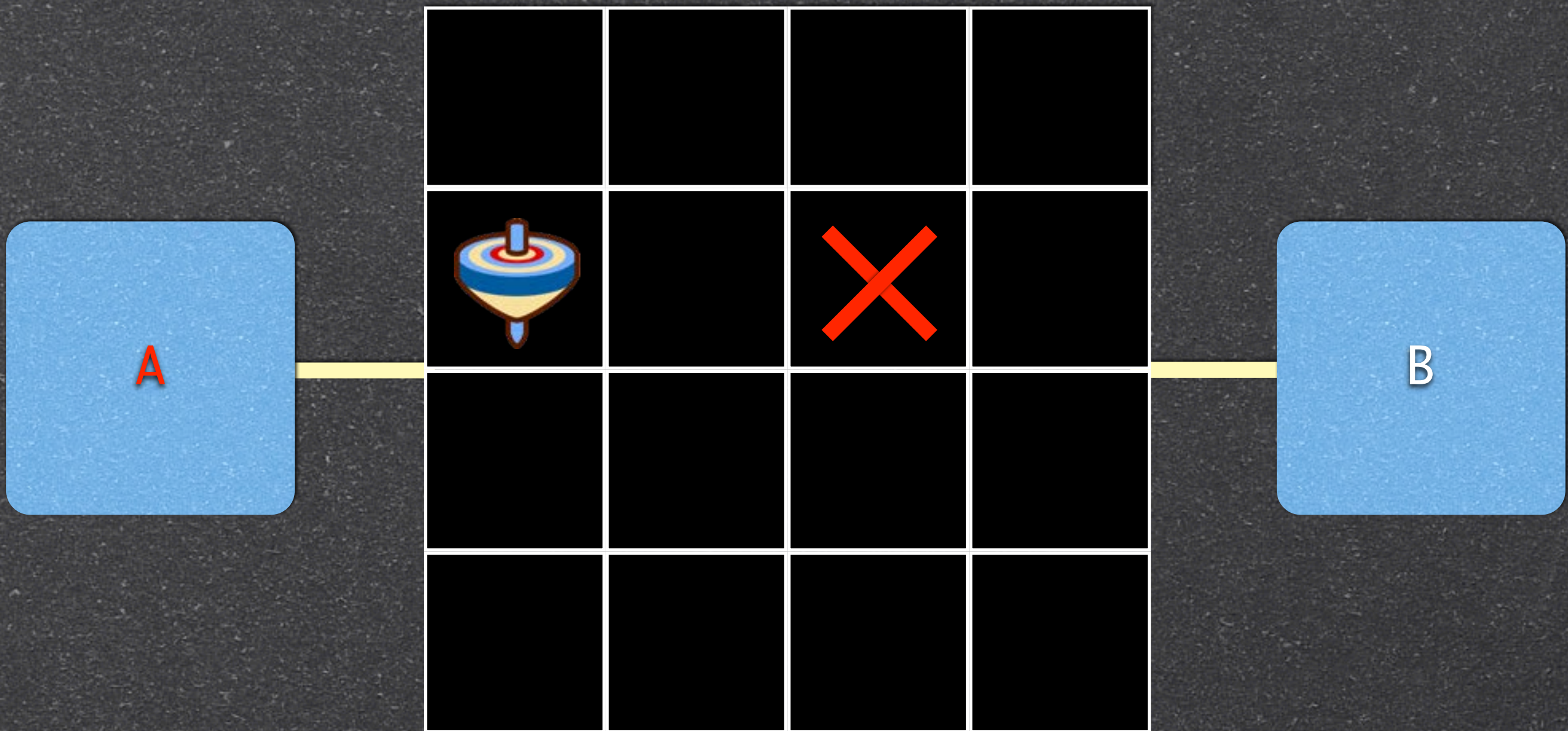
Bが駒を(2,3)に動かした

K理事長の部屋



Aが駒を(2,2)に動かした

K理事長の部屋



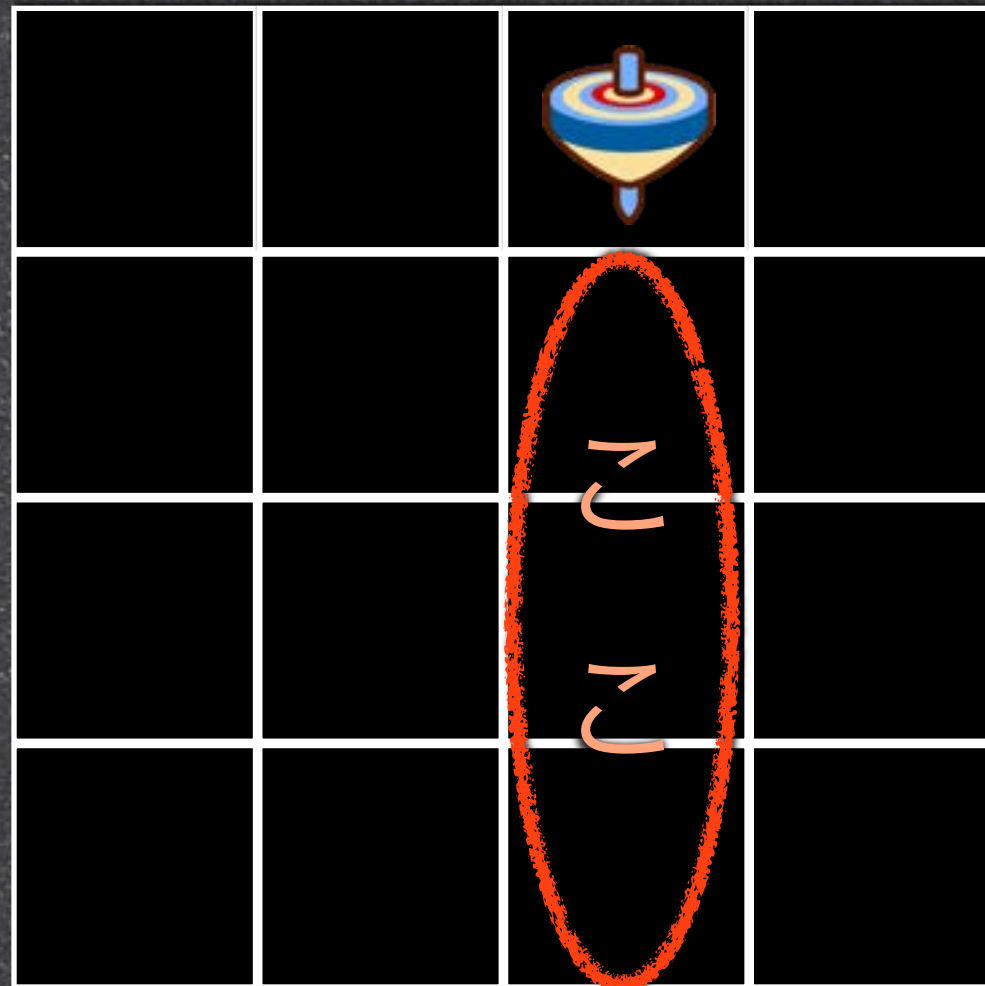
Aは(2,3)以外に駒を動かす

基本的な方針その2

- 📌 連続で呼ばれると「0なら縦、1なら横に移動させる。」という方針に支障が出るか？
- 📌 → 実はそんなに出ない。

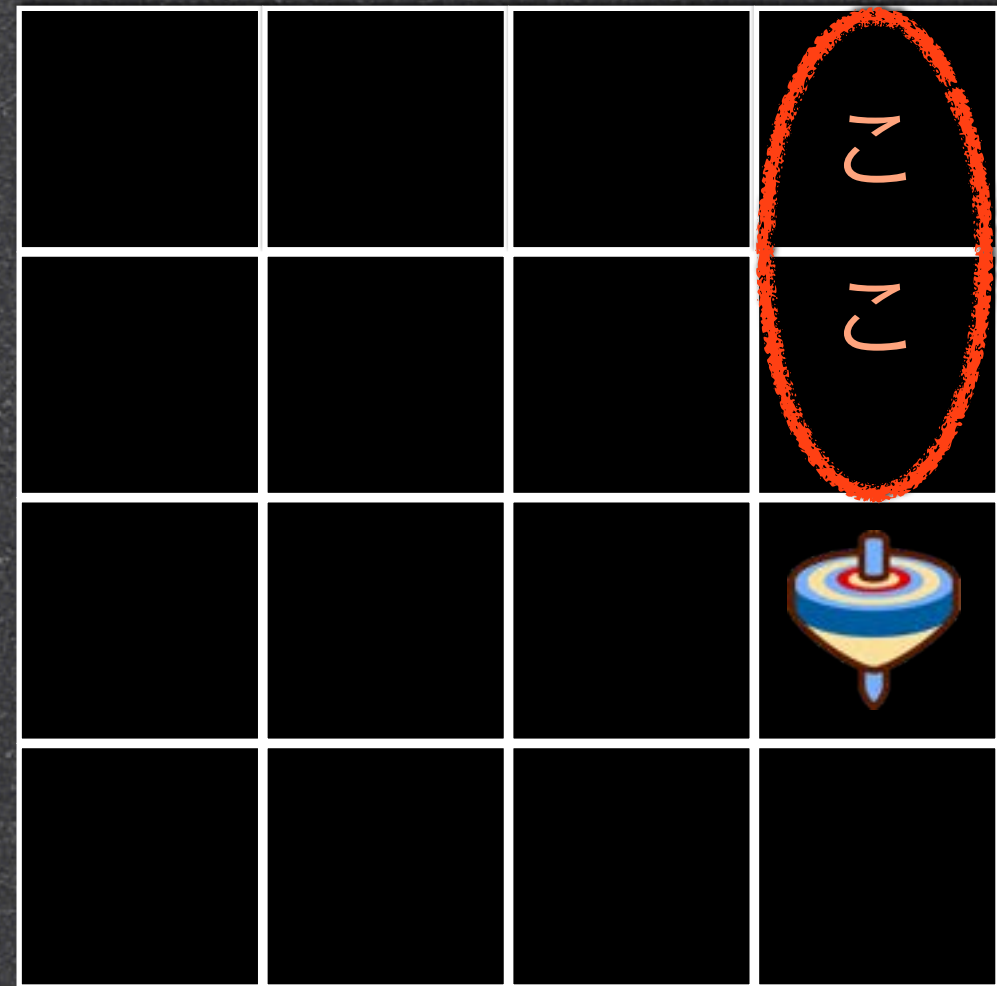
0(縦)を伝える場合の例

K理事長の部屋



駒が上端・下端にある場合

K理事長の部屋



駒が中央の段にある場合

「ここ」をうろうろしていればいい。

1(横)を伝える場合も同様

基本的な方針その3

- 📌 AさんとBさんのどちらが最初に呼び出されるかが分からない。
- 📌 subtask1では「1ターン目は捨てる」という戦略を取ることができたが、今度はうまくいかない。

基本的な方針その3

📌 ABABAB...

📌 BABABA...

📌 緑で示したところは値の伝達をしている部分になるはずだが、下のパターンだとBから始まっており、伝達がうまくいかない。

📌 どちらから始まったのかをうまく伝えたい。

subtask2 (20pts)

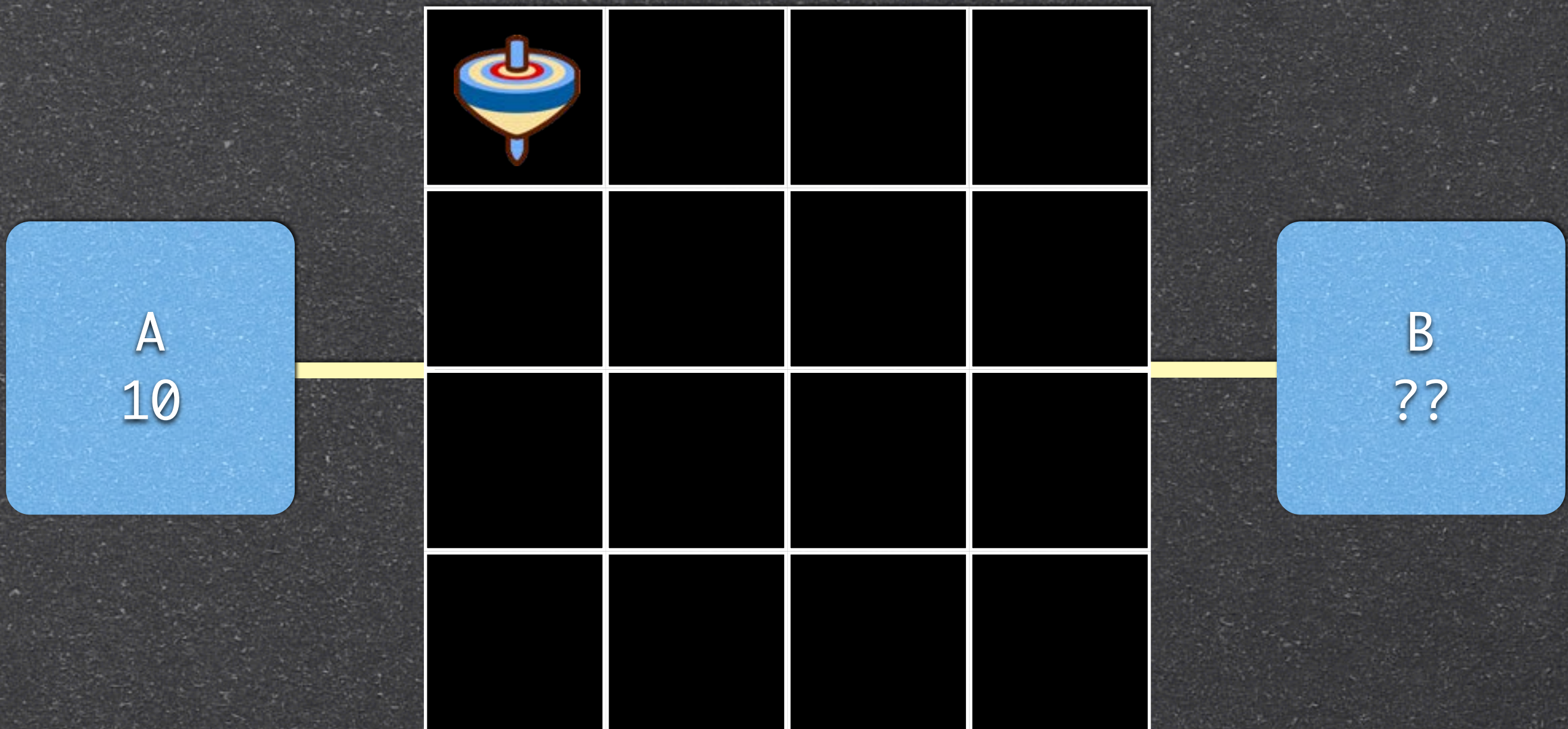
- 📌 初期状態では必ず駒は(1,1)にある。

ここから スタート			

subtask2 (20pts)

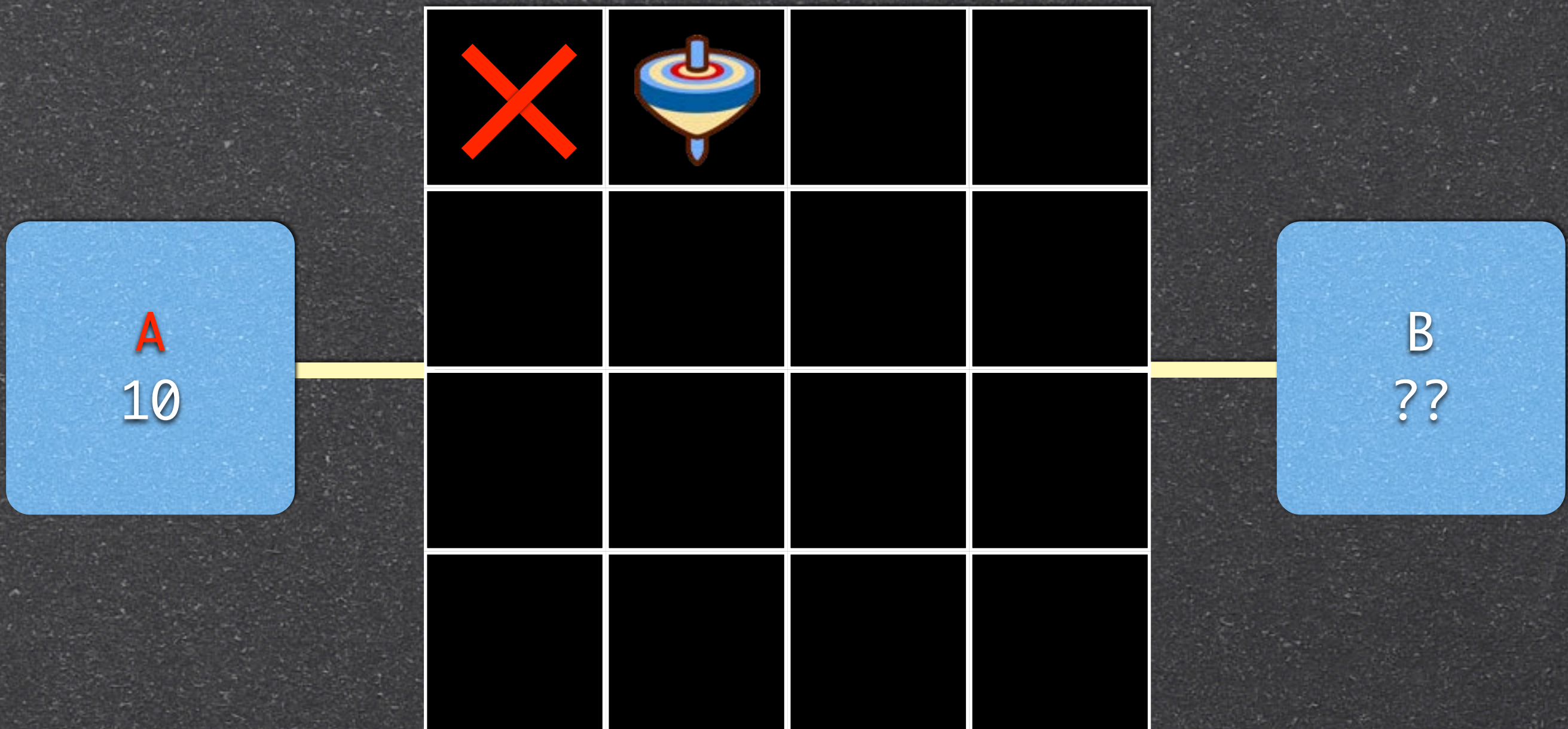
- 📌 $(1,1)$ に駒を動かすことを禁止する。
- 📌 すると各プレイヤーは、自分が最初に呼ばれたのかどうかを判別することが可能になる。
 - $(1,1)$ にある場合：自分が最初に呼ばれた
 - $(1,1)$ にない場合：自分は最初に呼ばれていない

K理事長の部屋



Aから呼び出す場合

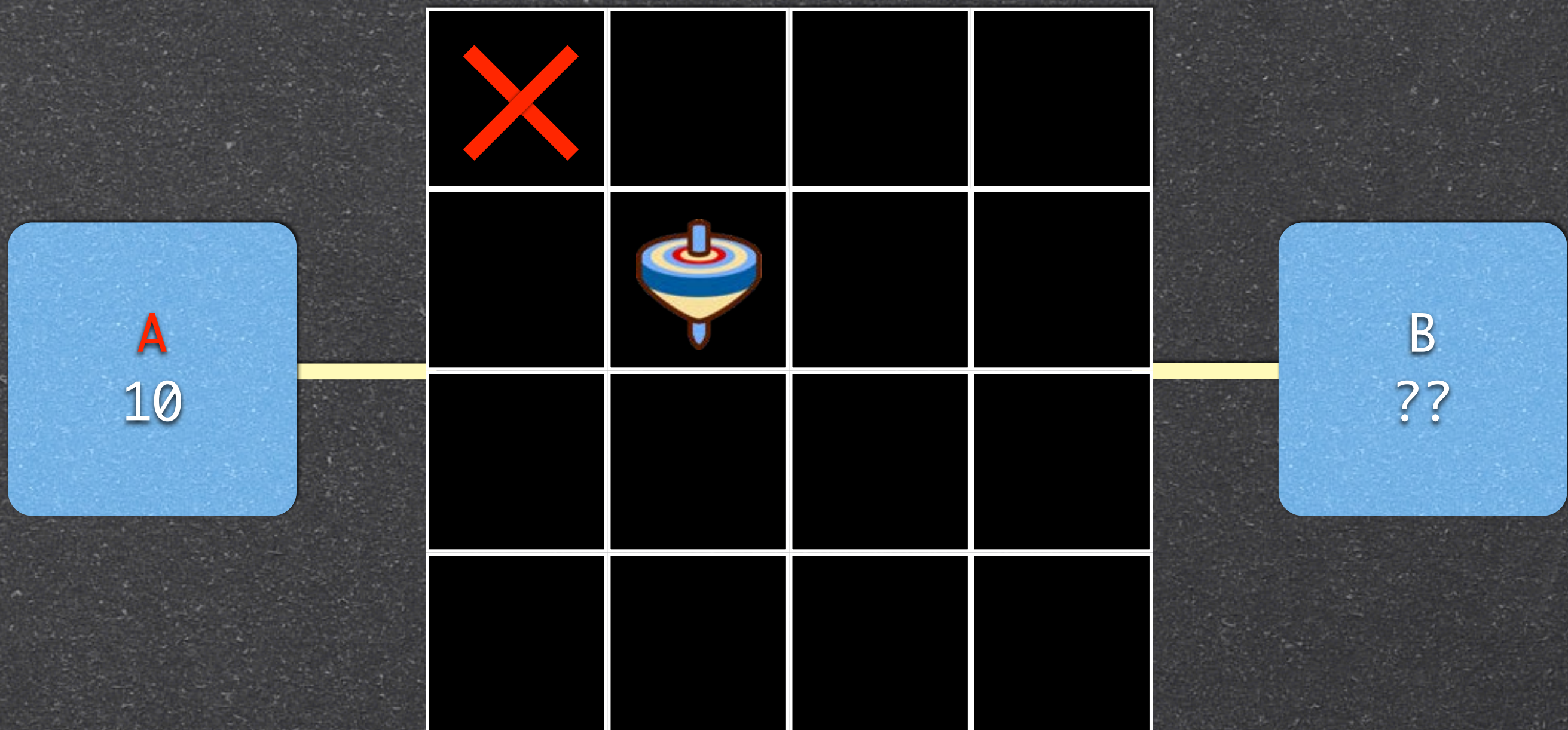
K理事長の部屋



(1,1)に駒があったので、Aは自分が最初に呼ばれたことが分かる

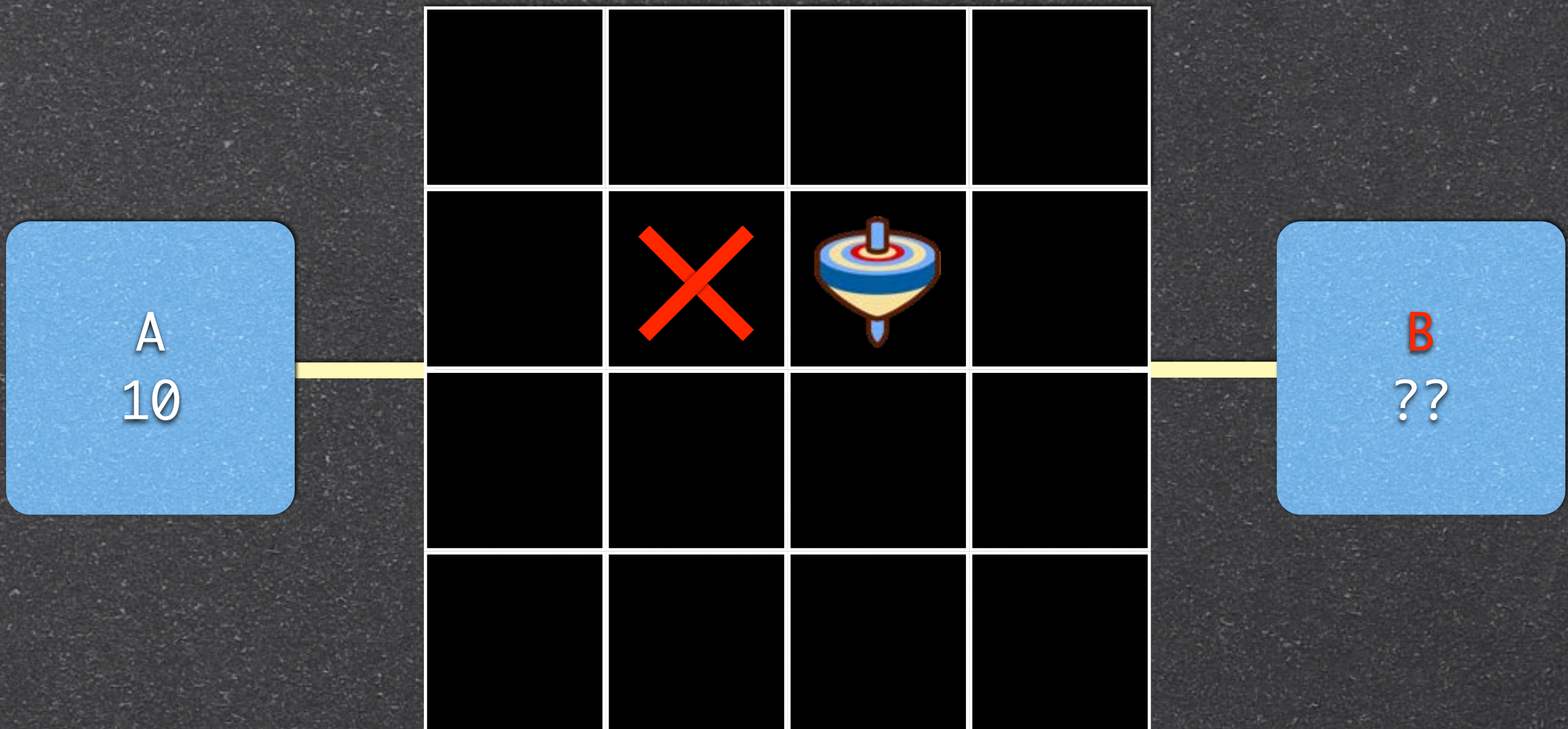
Aは駒を適当に動かした

K理事長の部屋



Aは(1,1)以外のマスに駒を動かした

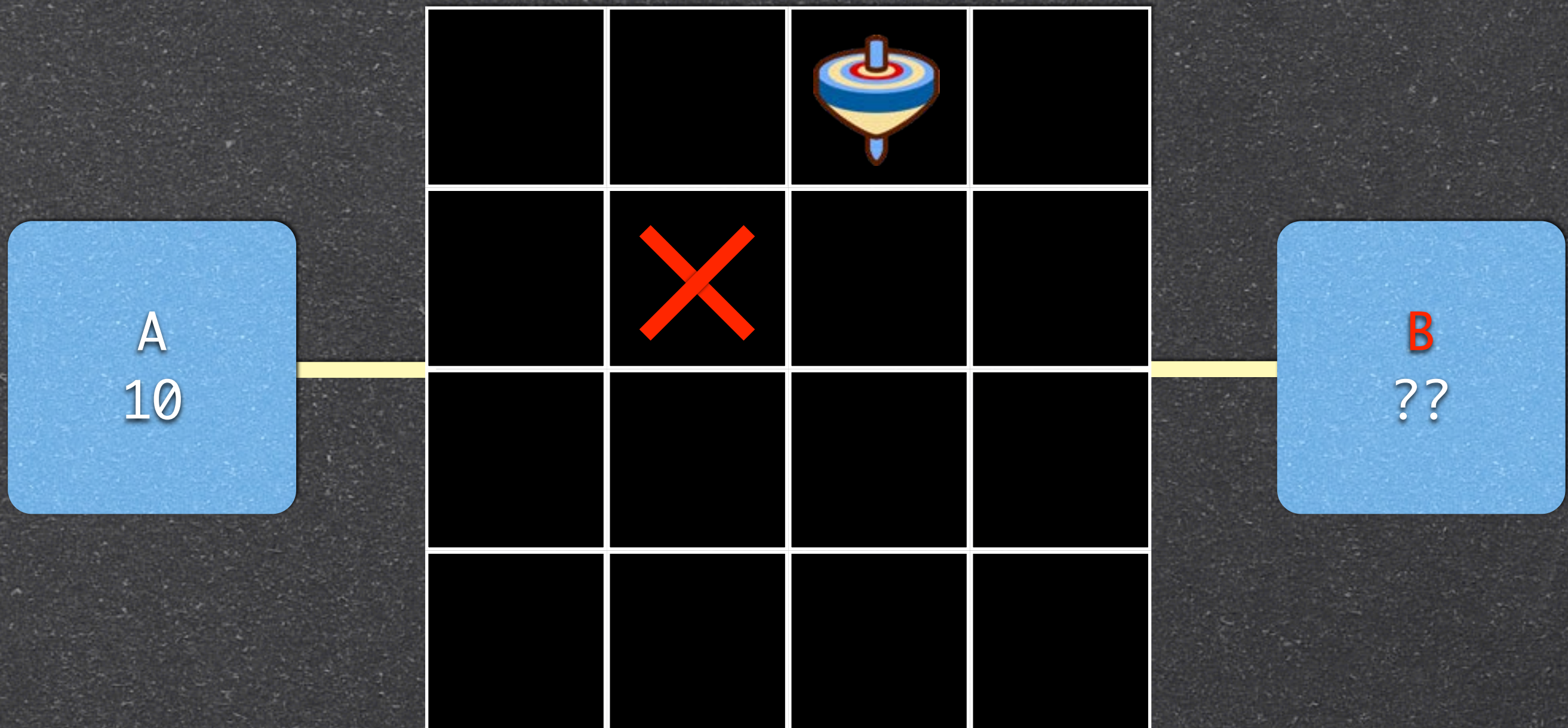
K理事長の部屋



(1,1)に駒がなかったなので、BはAが最初に呼ばれたことが分かる

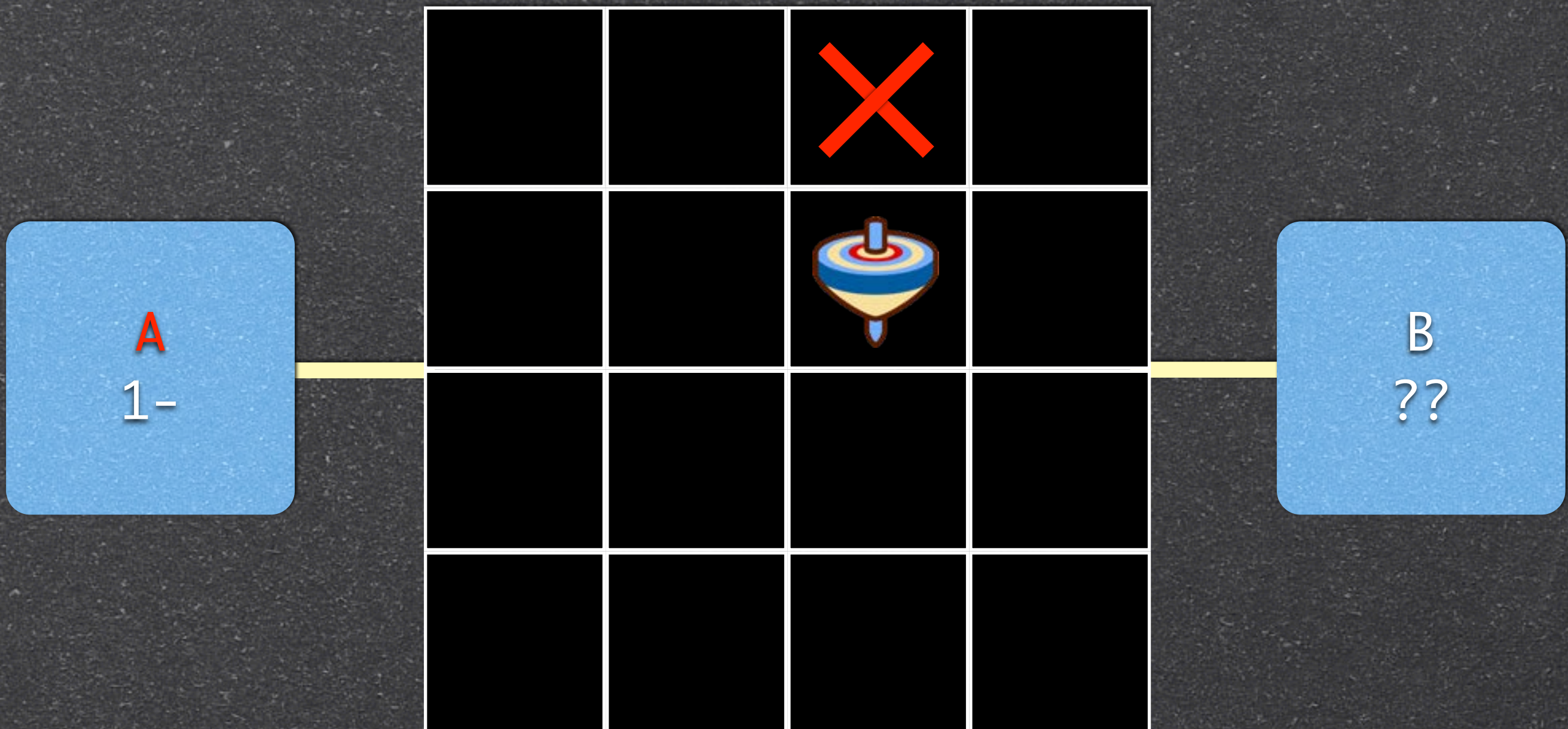
Bは駒を適当に動かした

K理事長の部屋



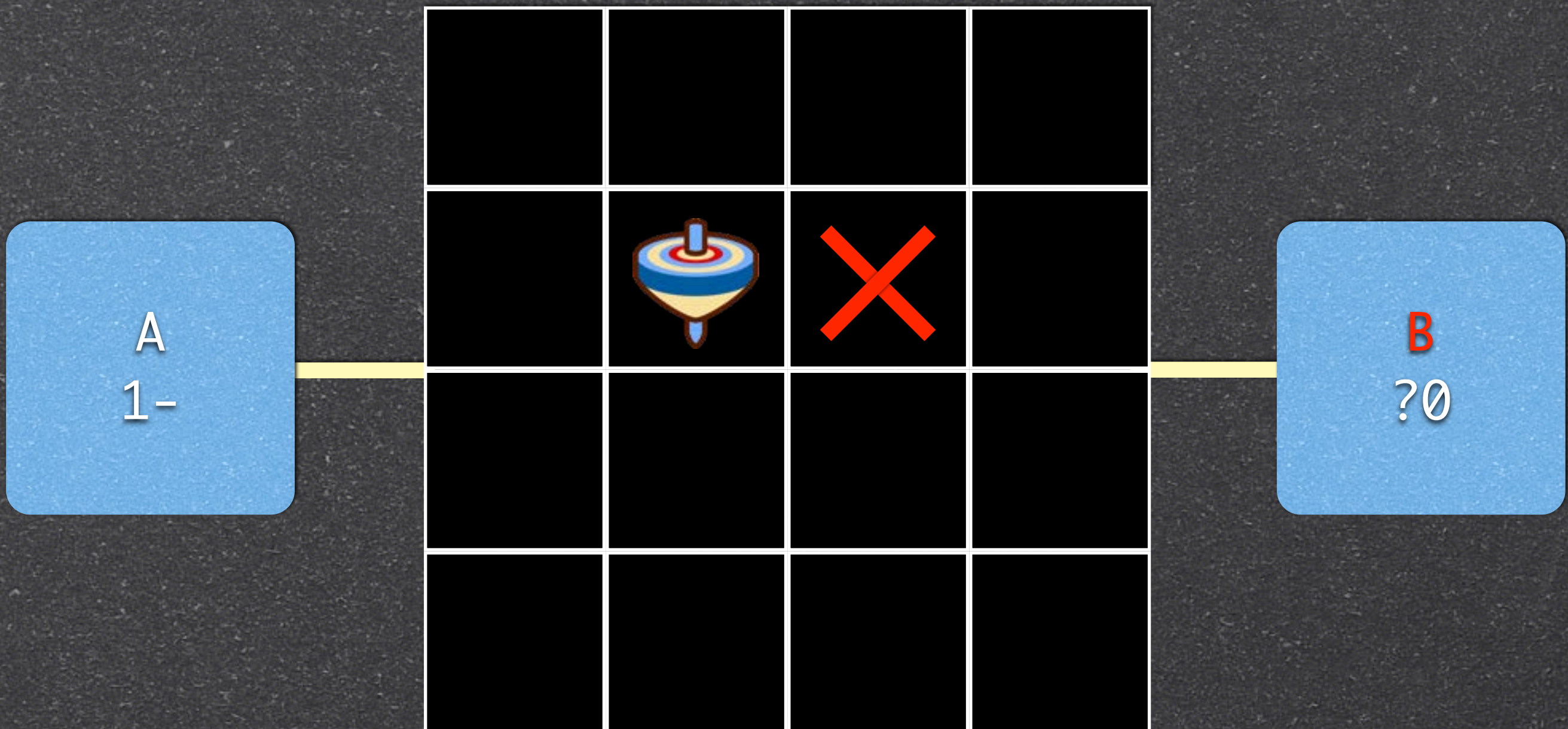
Bは(2,2)以外のマスに駒を動かした

K理事長の部屋



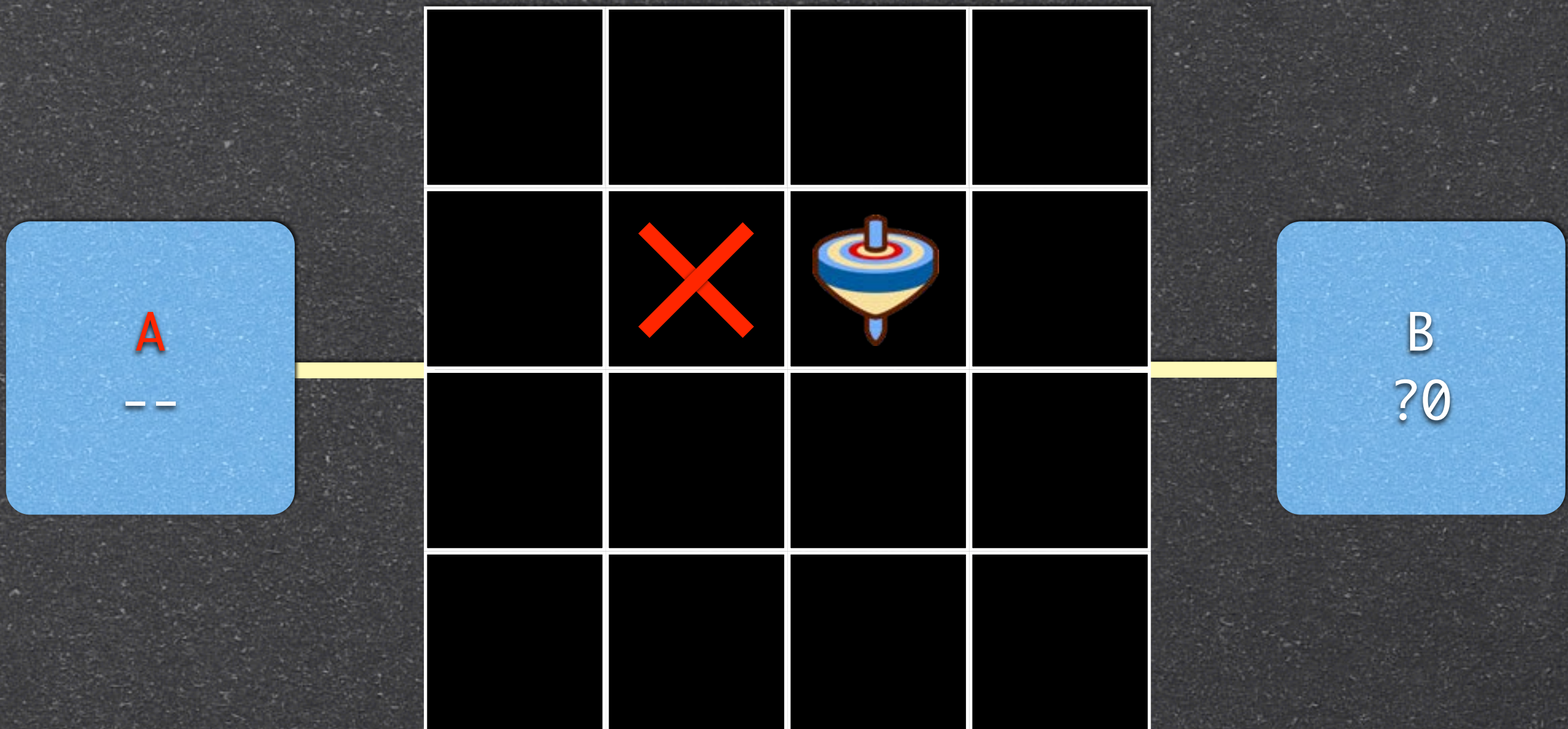
Aが0を伝えるために、駒を縦に動かした

K理事長の部屋



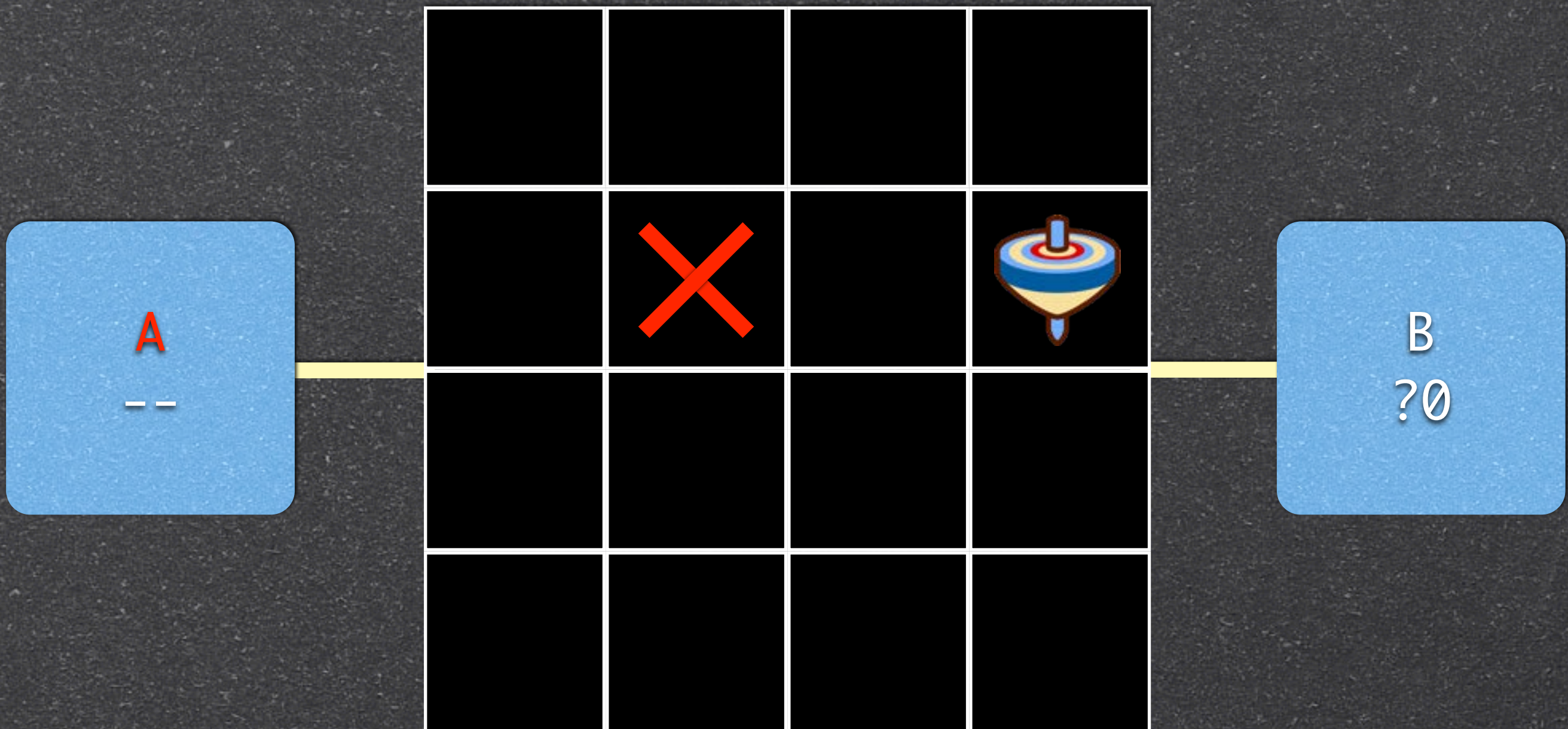
Bが0を読み取り、駒を適当に移動させた

K理事長の部屋



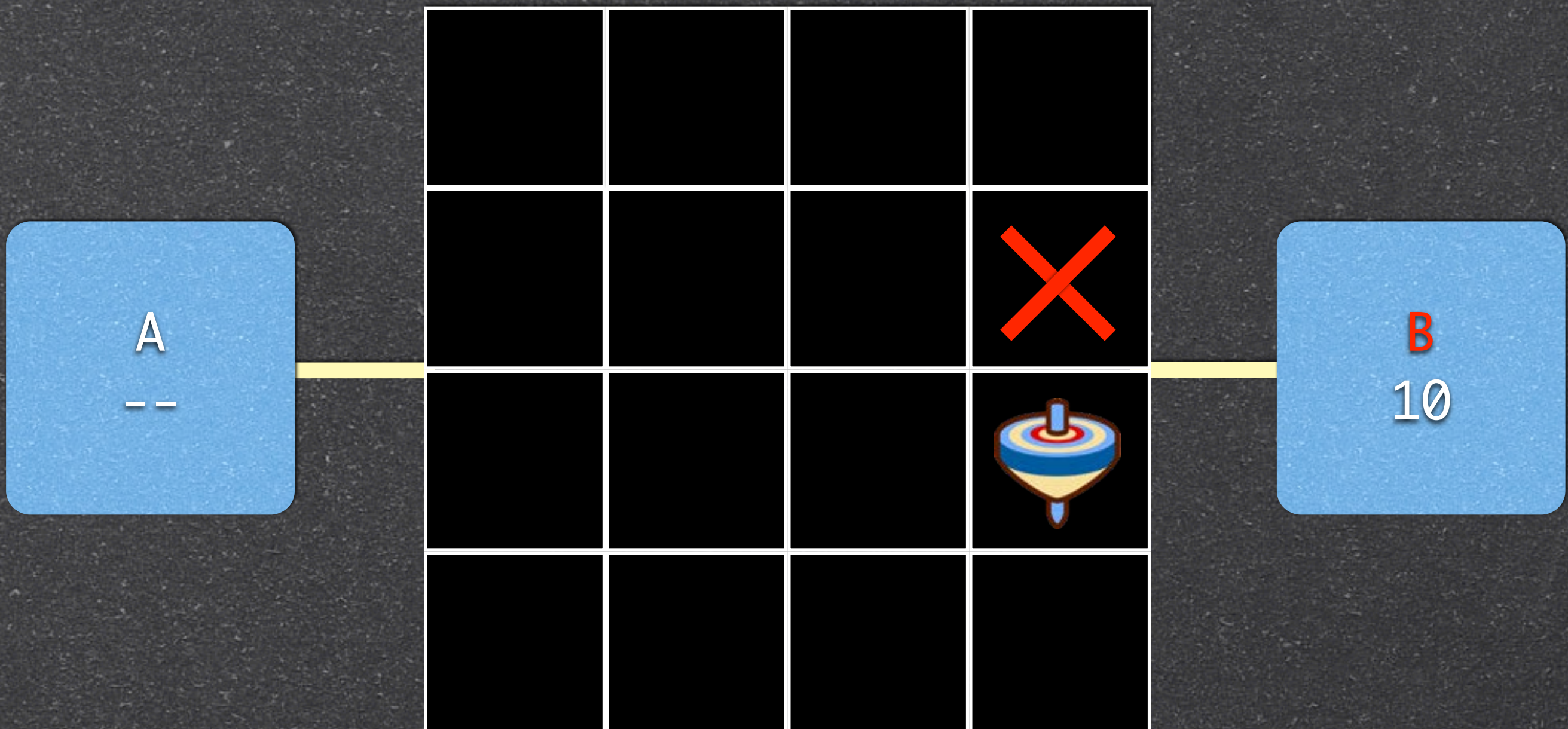
Aが1を伝えるために、駒を横に動かした

K理事長の部屋



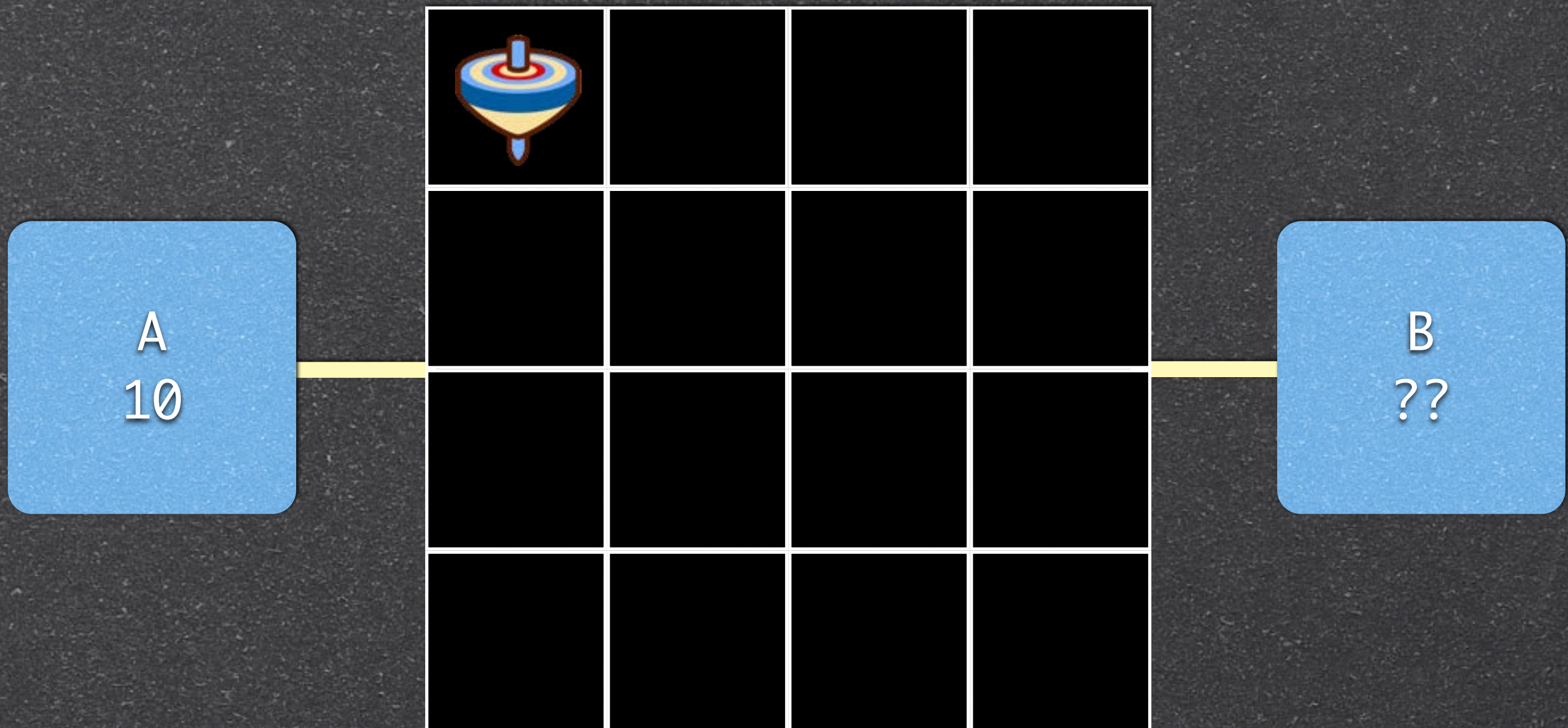
Aが1を伝えるために、駒を横に動かした

K理事長の部屋



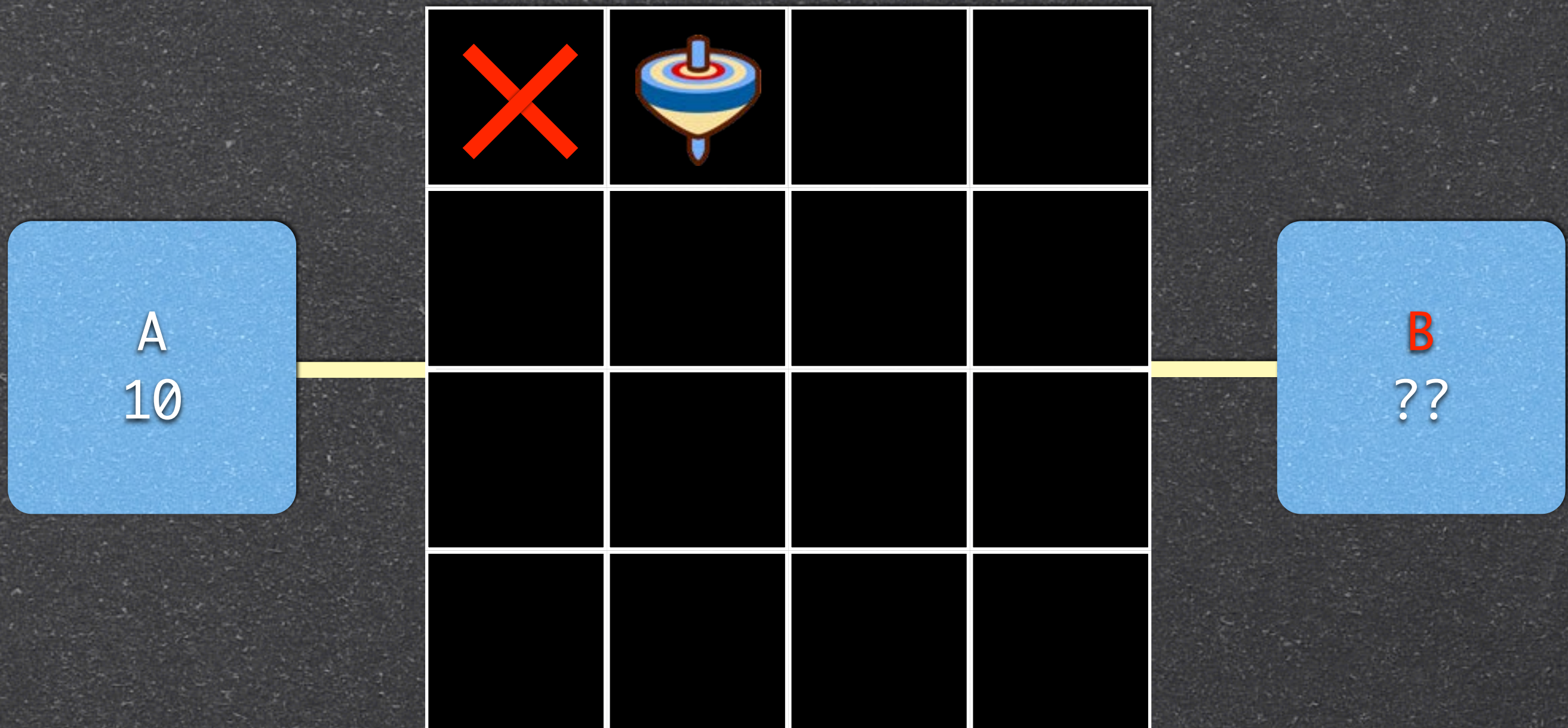
Bが1を読み取り、駒を適当に移動させた

K理事長の部屋



Bから呼び出す場合

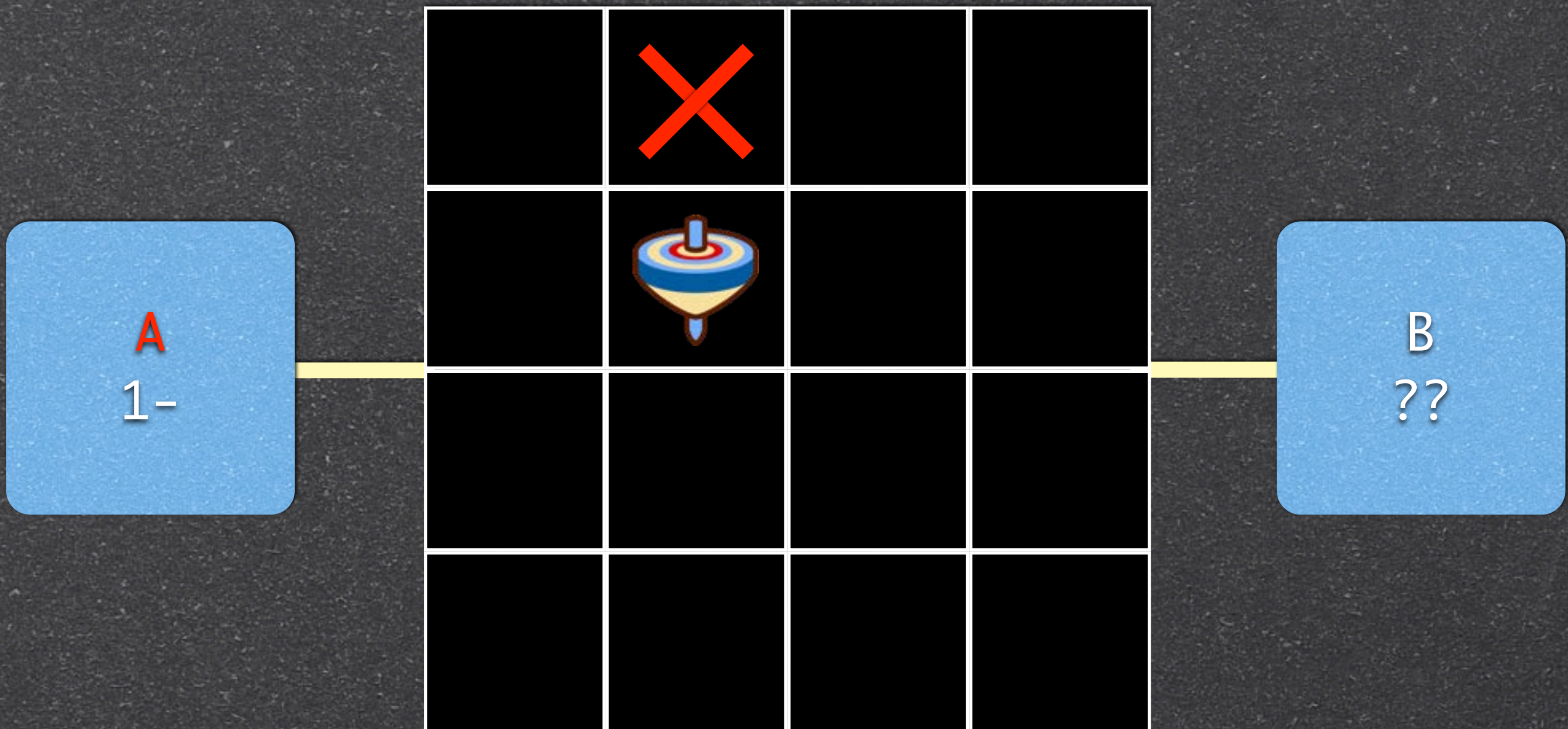
K理事長の部屋



(1,1)に駒があったので、Bは自分が最初に呼ばれたことが分かる

Bは駒を適当に動かした

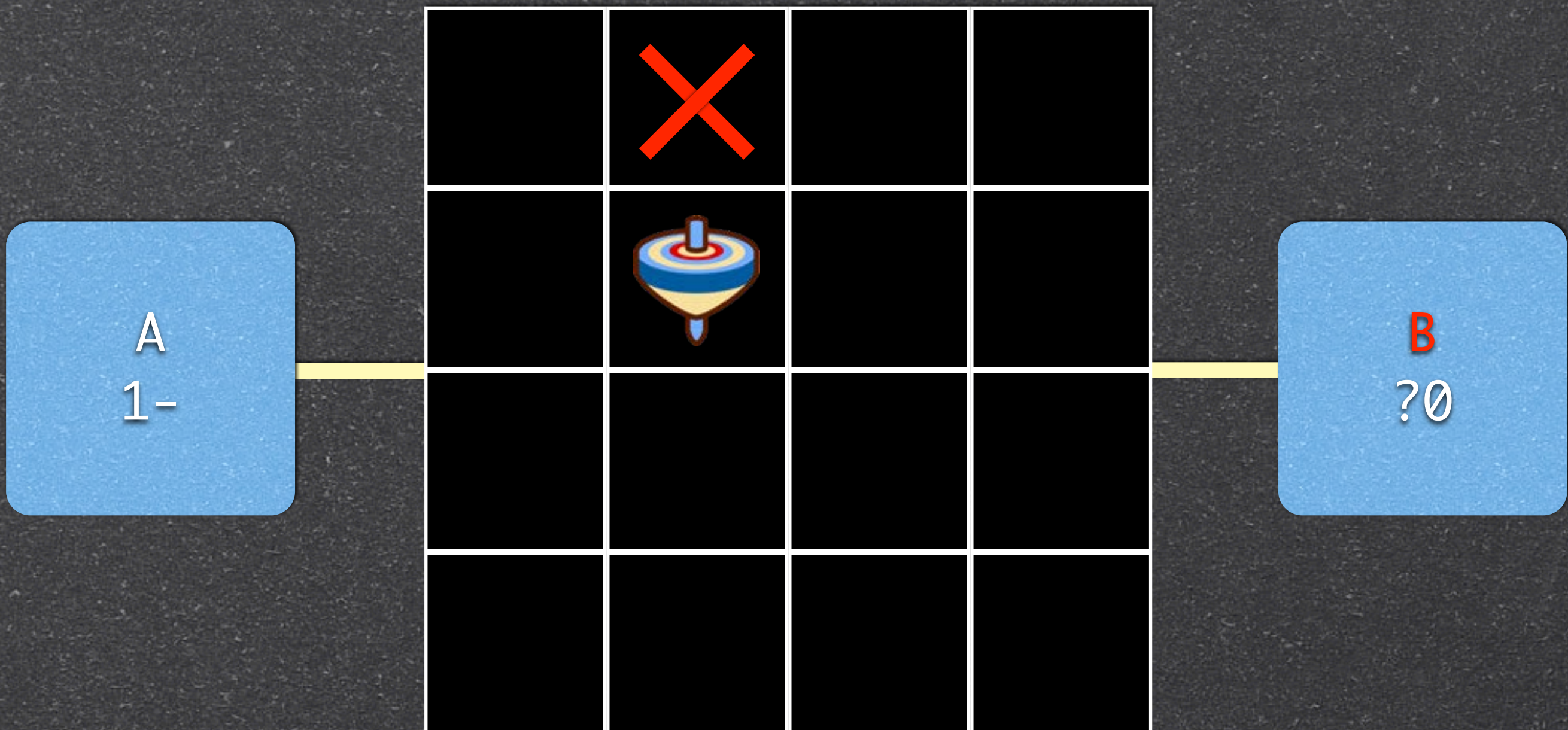
K理事長の部屋



(1,1)に駒がなかったなので、AはBが最初に呼ばれたことが分かる

Aが0を伝えるために、駒を縦に動かした

K理事長の部屋



Bが0を読み取り、駒を適当に移動させた
以下同様

subtask3 (60pts)

- 📌 開始場所すら分からない。

subtask3 (60pts)

- 📌 ならば無理矢理「開始位置」を作ってやるう！
- 📌 各プレイヤーの最初5ターン(連続して呼び出されるのを1ターンと数えることにします)くらいはなかったことにし、ひたすら $(1,1)$ に向かって移動する。

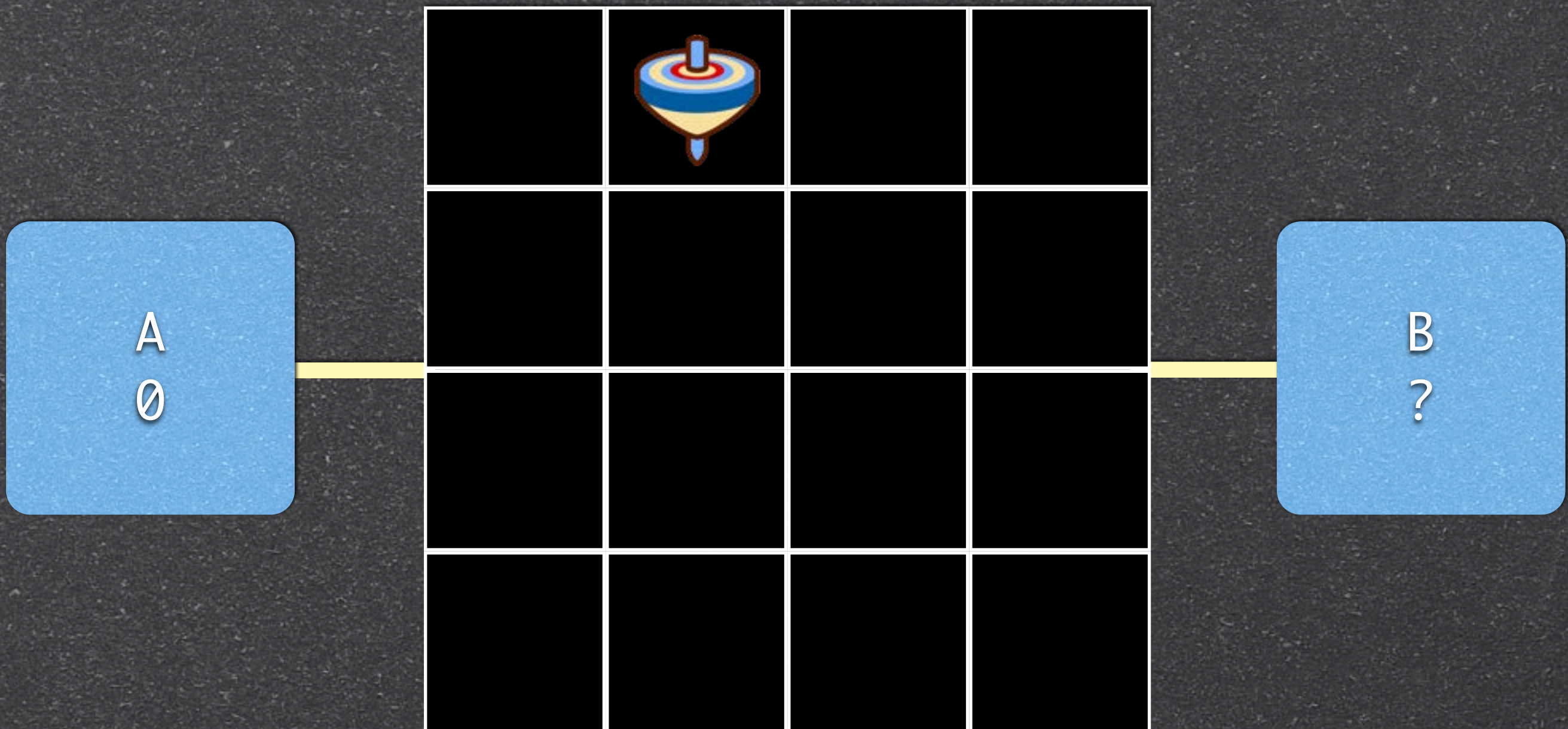
subtask3 (60pts)

- 📌 その後はsubtask2と全く同じ。
- 📌 ABABABABABABAB...
- 📌 BABABABABABABA...
- 📌 AAABBBBABAAABABBABBBABABBB...
- 📌 青い部分はなかったことにされる部分。
- 📌 なかったことにされる部分は必ず連続的。

subtask3 (60pts)

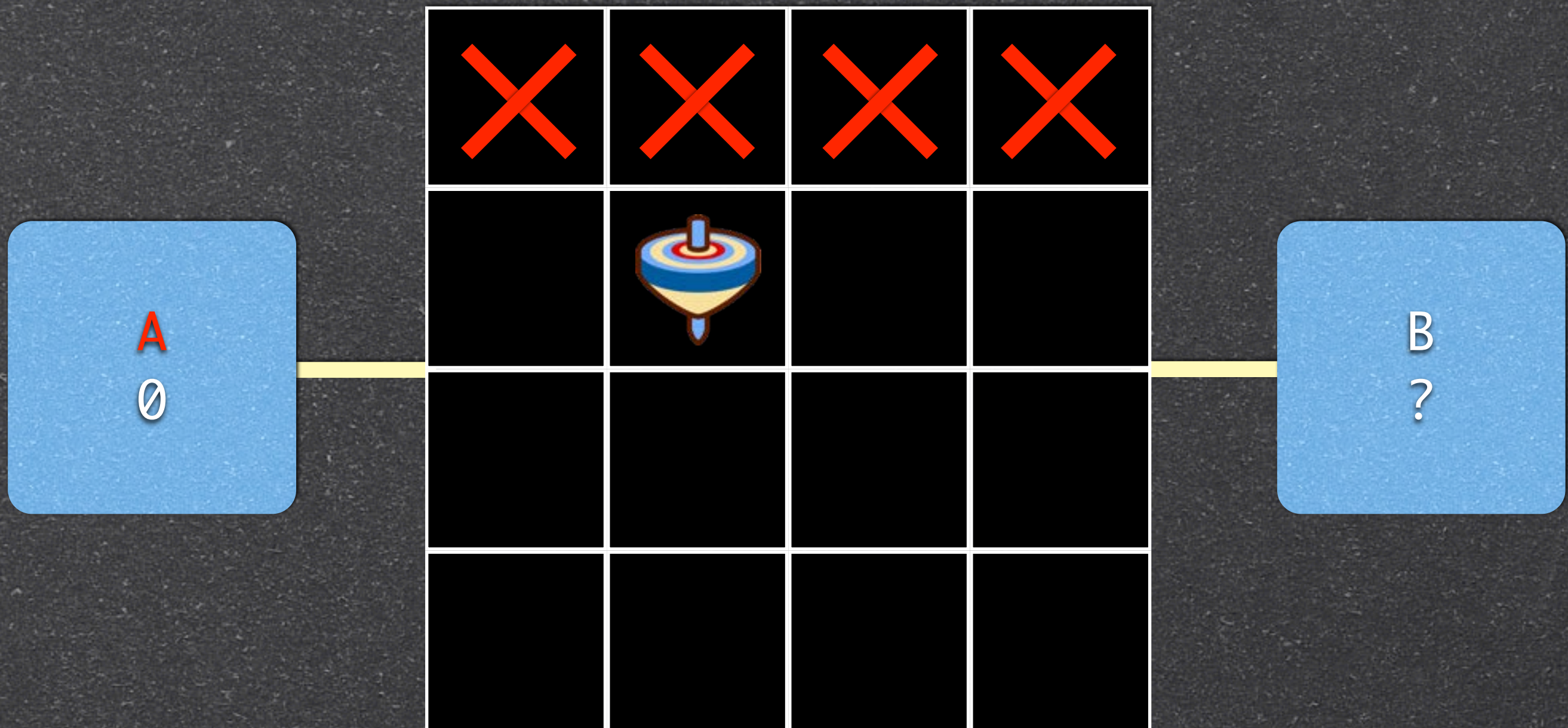
- 📌 なかったことにするターンが終わった後に $(1,1)$ に止まっていなくてもある。
- 📌 「 $(1,1)$ 」を「1行目」にするとうまくいく。

K理事長の部屋



5ターンずつくらいなかったことにして、
「初期位置」は(2,1)となった。

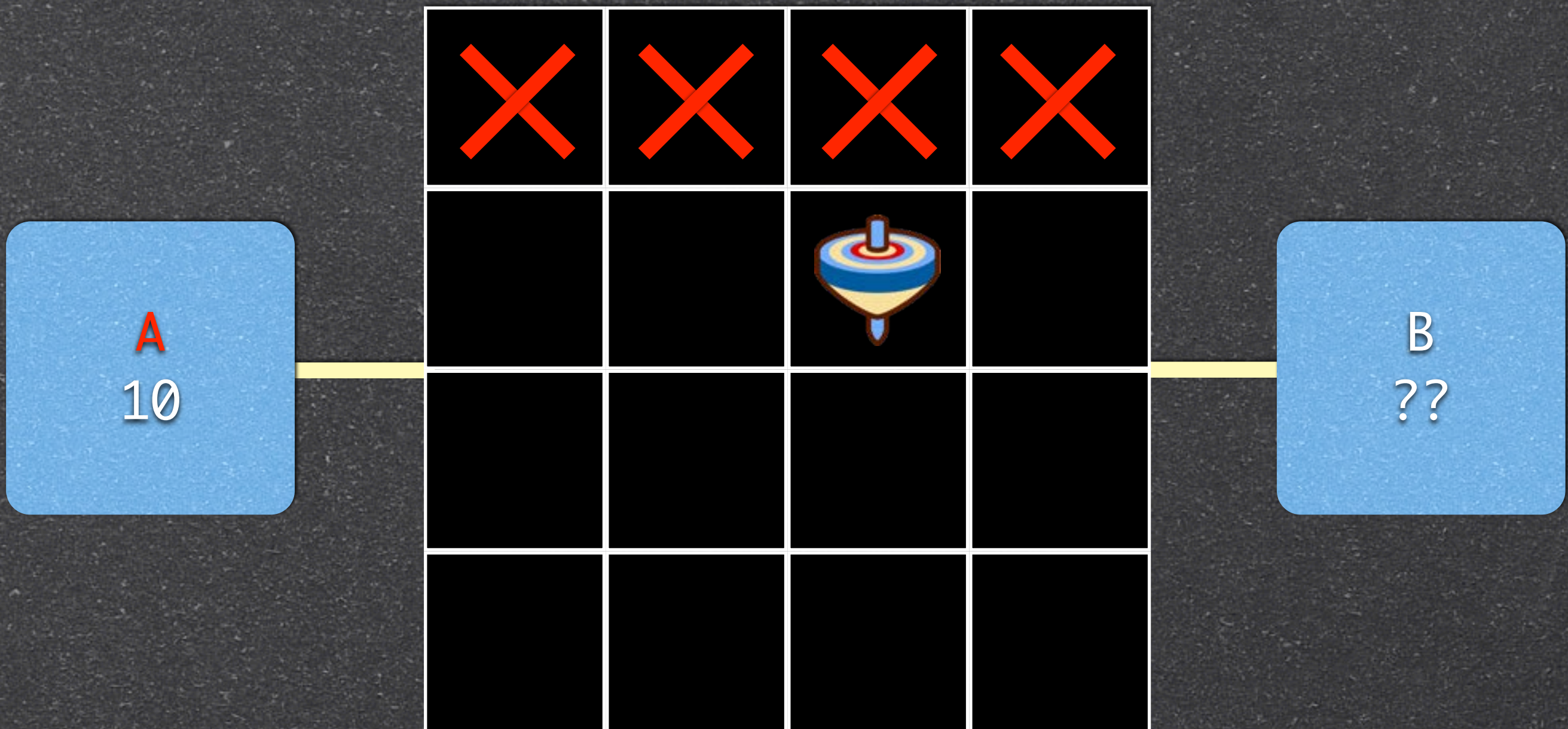
K理事長の部屋



1 行目に駒があったので、Aは自分が最初に呼ばれたことが分かる

Aは駒を適当に動かした

K理事長の部屋



Aは1行目以外のマスに駒を動かした
イカ略

subtask 4 (おまけ)

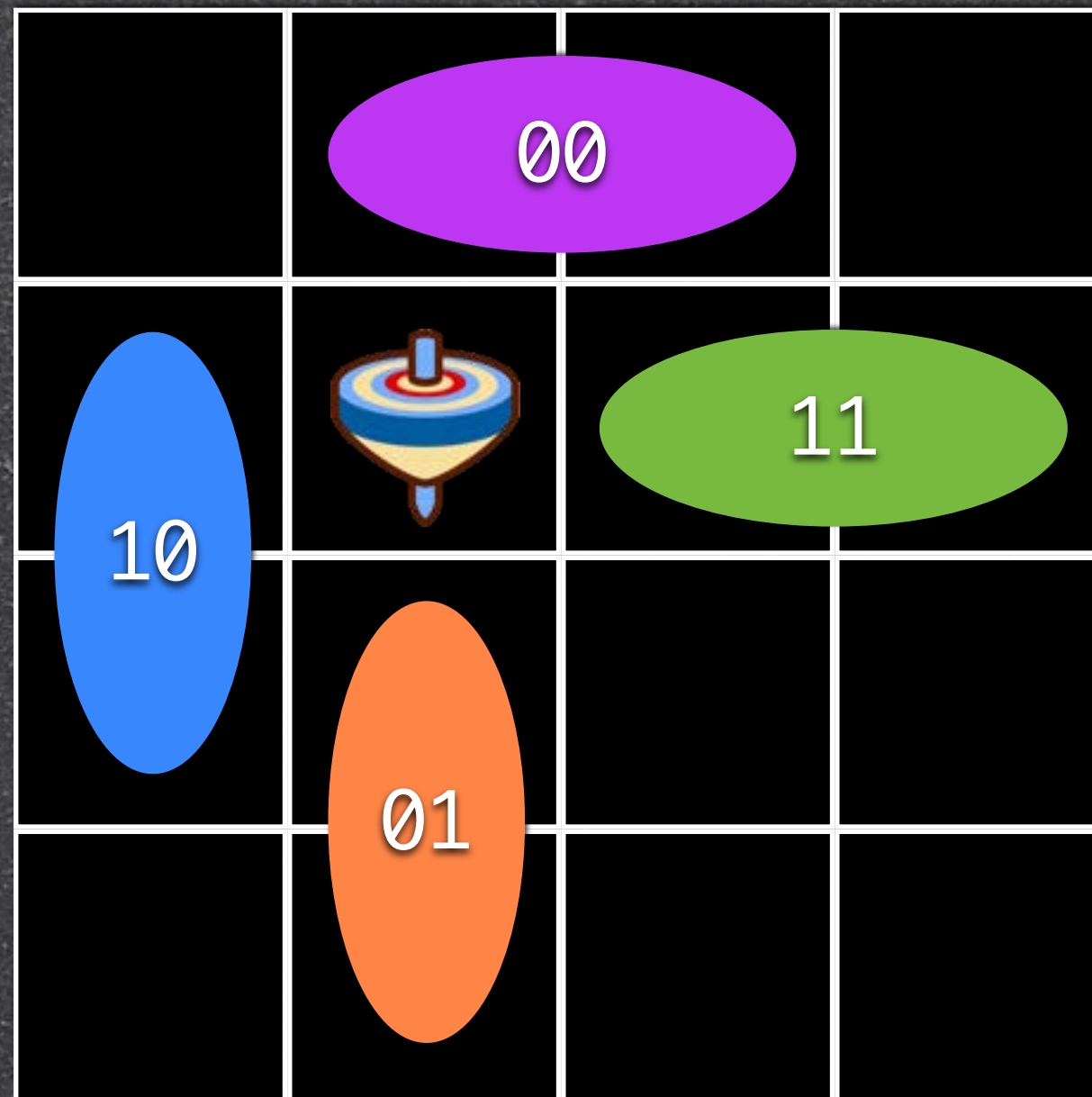
- 📌 関数呼び出しの合計を3301回に抑えられます。
- 📌 400回で初期処理、2900回で値の伝達、1回で値を答える

subtask 4 (おまけ)

📌 値伝達phase

- 📌 Aは1回に2bit(0~3)を伝える。
- 📌 Bは駒を必ず真ん中の4つのいずれかに動かす。

K理事長の部屋



伝えたい2bitに対応する場所をうろうろする。

subtask 4 (おまけ)

📌 初期処理phase

📌 Aの1ターン目：隅以外のマスに移動

📌 Aの2ターン目以降：

Bが「move1」をしたとき：隅以外のマスに移動

Bが「move2」をしたとき：値伝達phaseに移行

subtask 4 (おまけ)

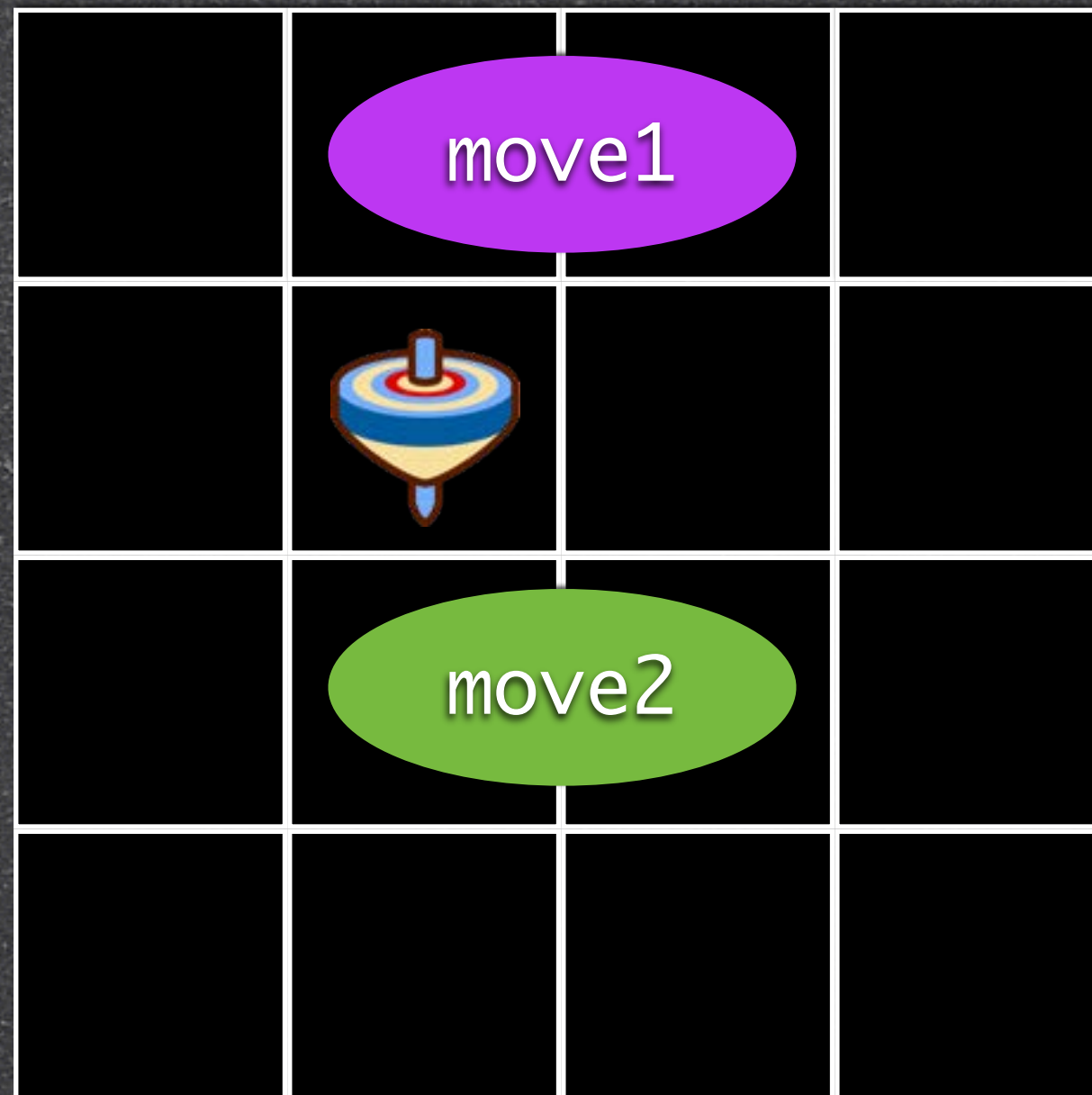
- 📌 初期処理phase

- 📌 Bの1ターン目：move1

- 📌 Bの2ターン目：move2

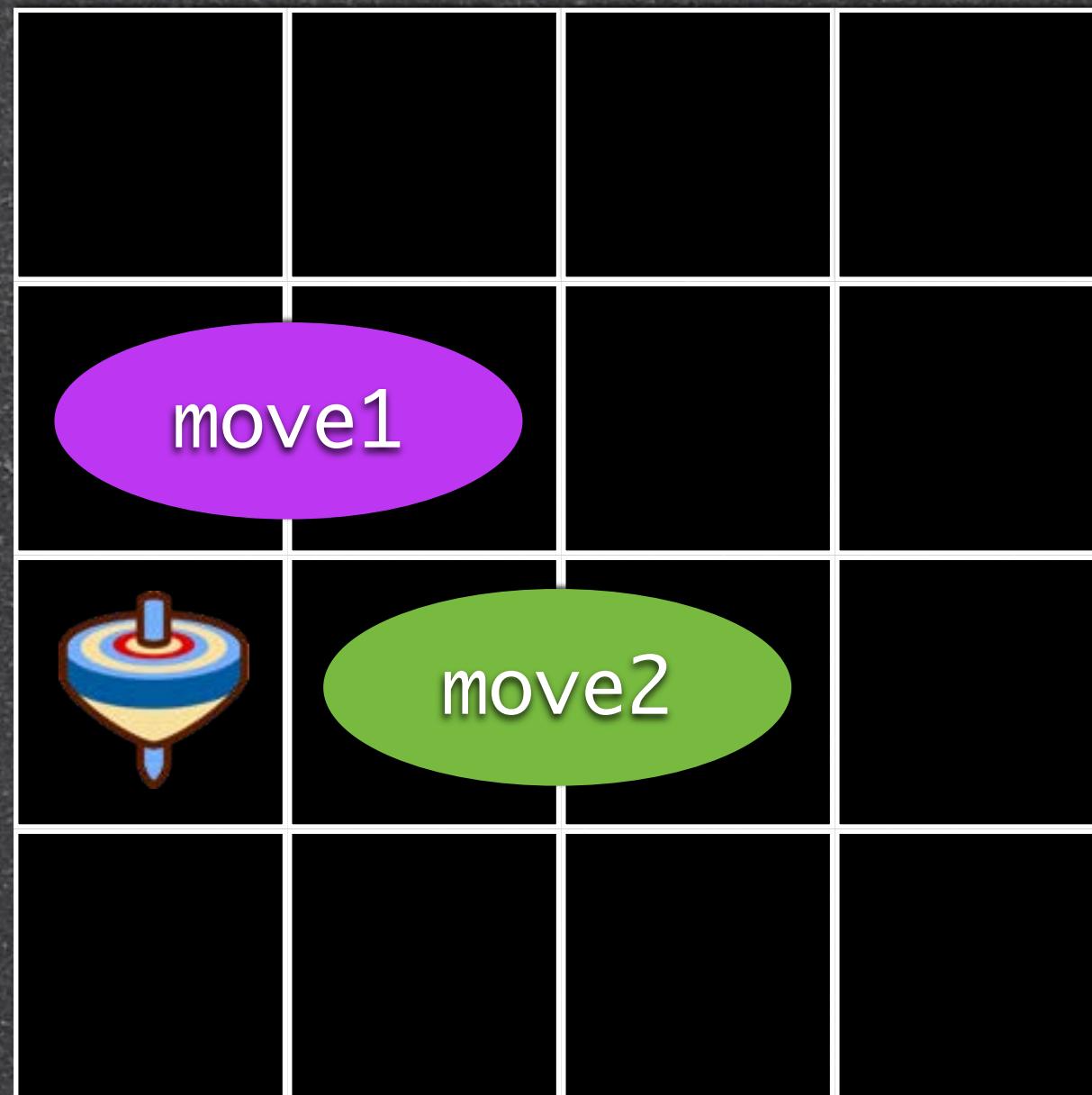
- 📌 move2の直後は必ず真ん中の4つのいずれかのマスに駒があるようにする

K理事長の部屋



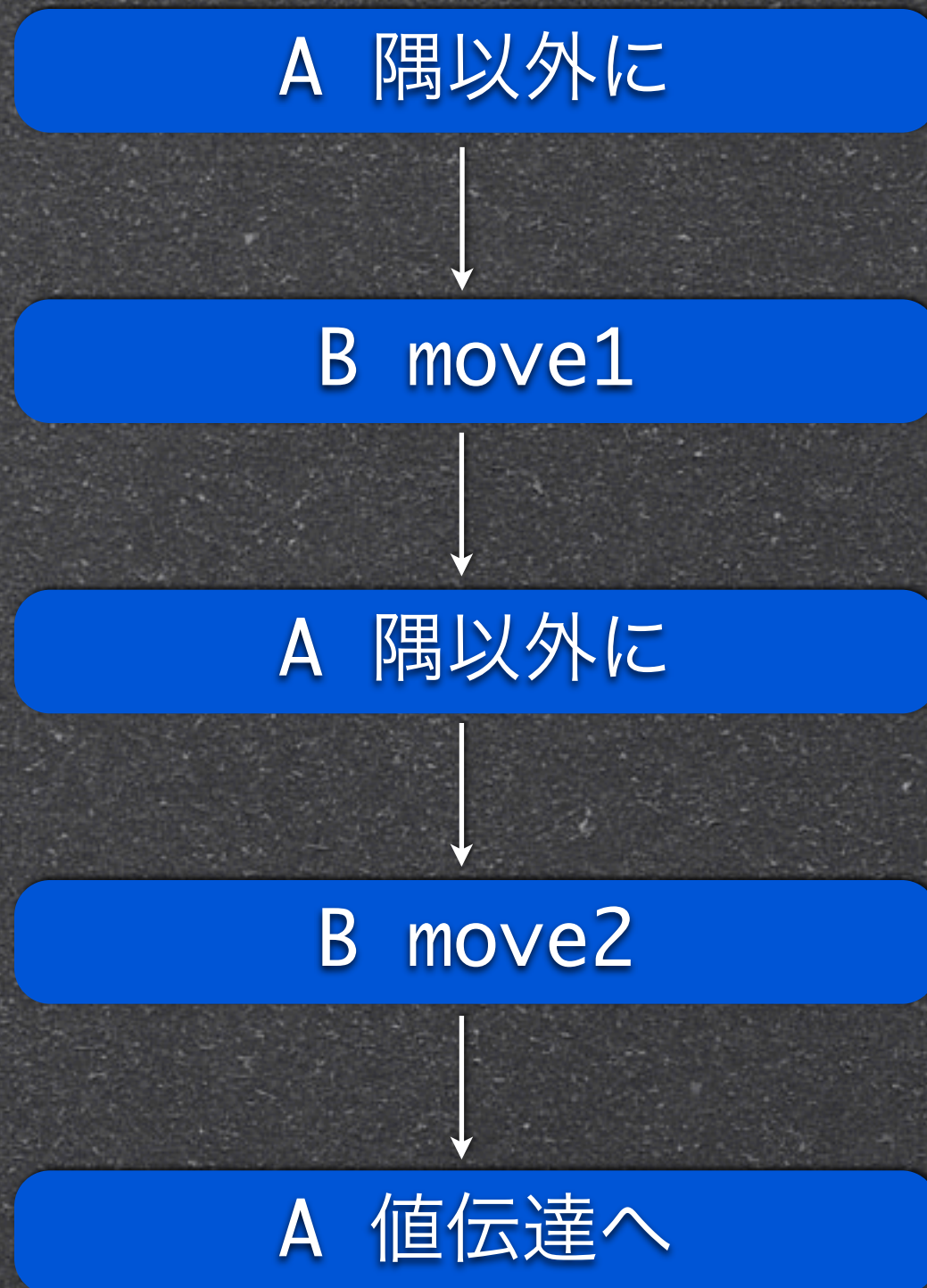
駒が中央にある時のmove1とmove2の例

K理事長の部屋

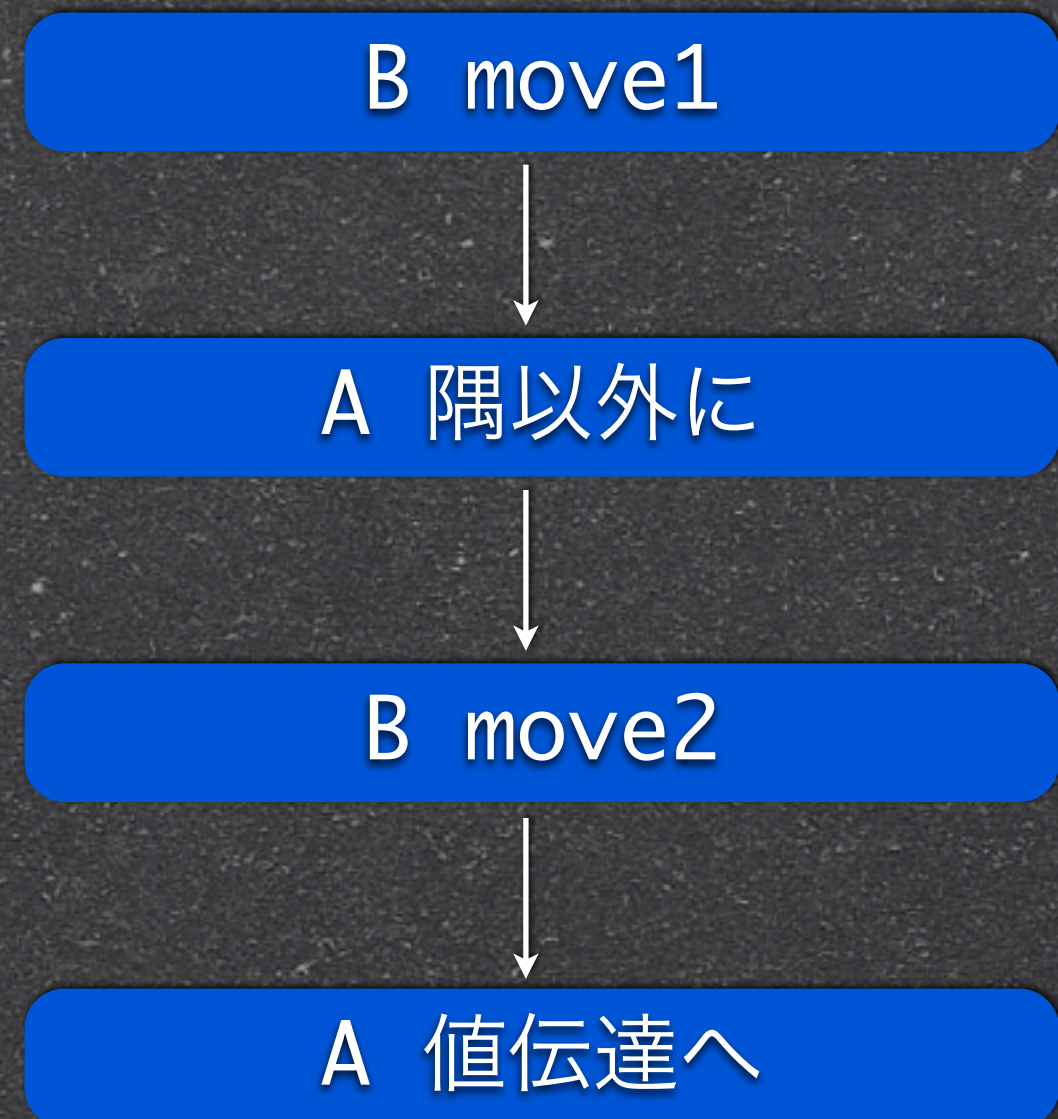


駒が辺にある時のmove1とmove2の例

Aからスタート



Bからスタート



Aからスタート

Bからスタート

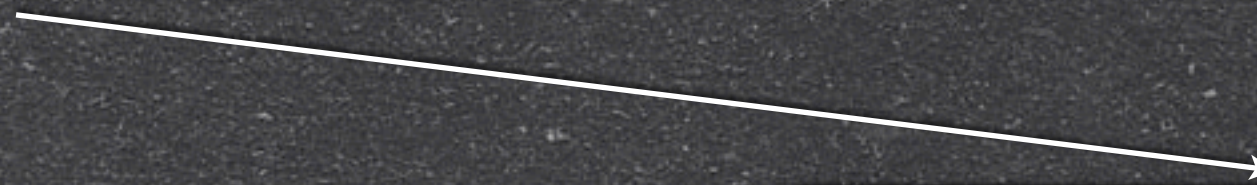
A 隅以外に

B move1

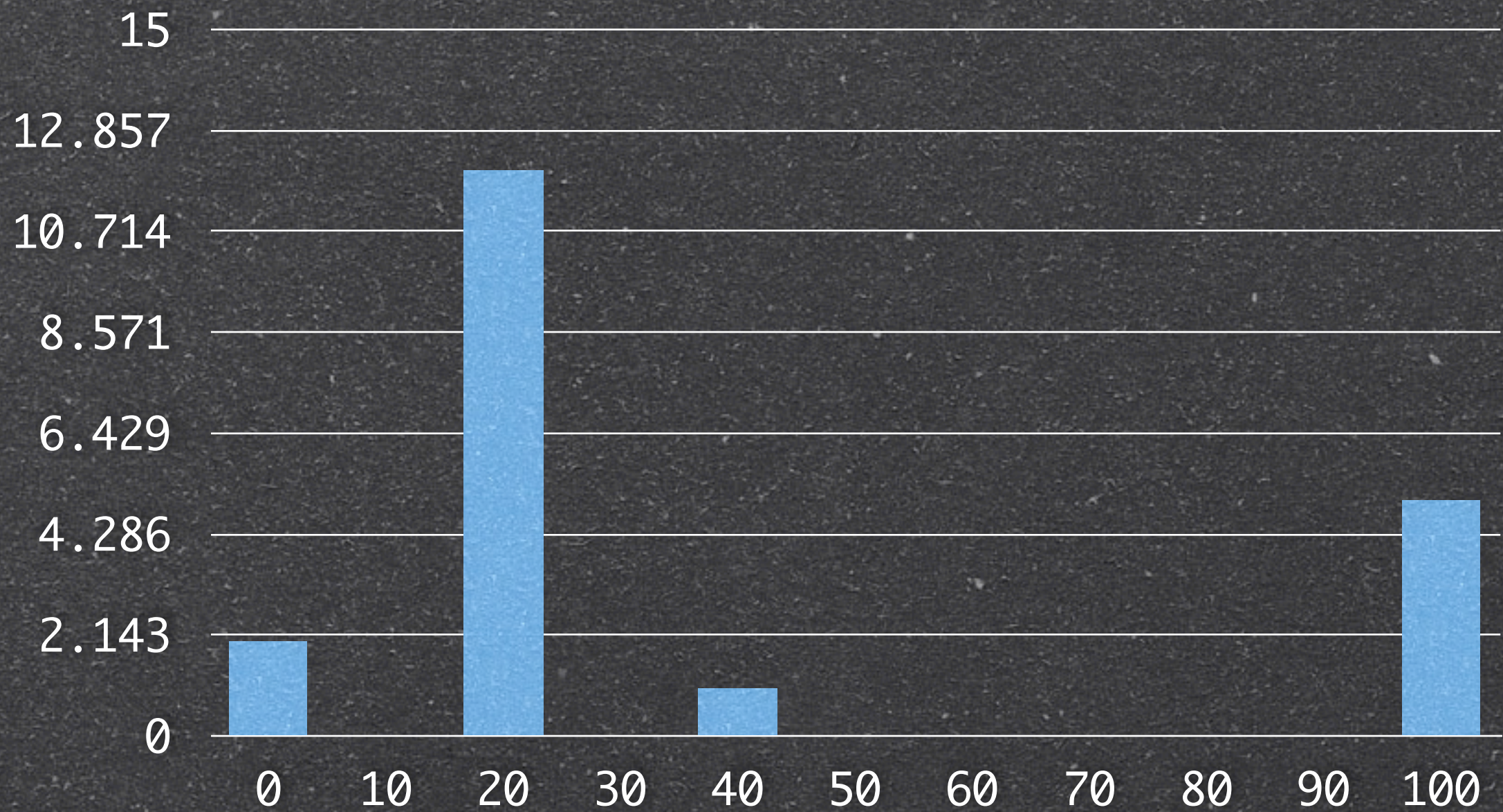
A 隅以外に

B move2

A 値伝達へ



統計



Day4 Messenger 別解

解説： 平野湧一郎 (nai)

想定解の問題点

- むずい
- マスの状態を考えるのが面倒
- マスを使う

想定解の問題点

- むずい
- マスの状態を考えるのが面倒
- マスを使う

マスを使わない解法で解ければ, K 理事長にも褒められるはず(確信)

人人人人人

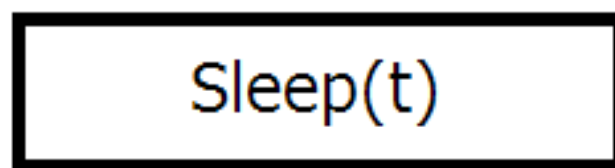
> Sleep通信 <

Y^Y^Y^Y^Y^Y

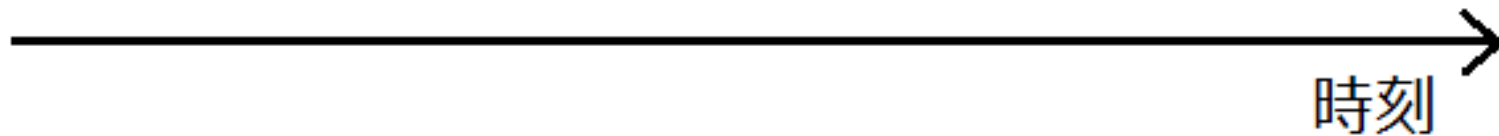
Sleep 通信とは

- Sleep することで時刻を介して情報を伝達する
⇒ マスがなくても通信可能！ すごい！

GameA



GameB



Sleep 通信の protocol

- 1 単位時間 (>> ノイズ) を定め, 何単位時間 Sleep するかで通信を行う
(ここでは 14ms とする)
- 直前の自分の呼び出し終了時からの経過時間を T ms とすると,
$$N = [T / 14]$$
が相手が送信した数字
 - $N = 0$ の場合は自分が連続して呼ばれた

注意点

- A が数字を送信した後，B は受信したことを示すために 1 を送信しなければならない
 - これを “Relay” と呼ぶことにする
 - （実はマスの状態を見ればこれはいらない）

部分点解法(1)

- A が与えられた数字 X をそのまま送信
 - $O(X)$
 - 最悪で 140 億 ms = 5 ヶ月半
 - 0 点の部分点

部分点解法 (2)

- 1bit ずつ送信
 - Aさんの i 回目の送信:
X を 2 進数表記したときの下から i 桁目が
 - 0 なら 1
 - 1 なら 2を送信することにする
- $O(\log X_{\max})$

部分点解法(2)

- 10 億は 2 進数表記で 30 桁 = 30 回の送受信
 - 1 回の送受信で,
 - A: 最大 2
 - B: 1 (Relay)
- で合計 3 の時間が必要
- 全体で 90 単位時間 = 1260ms 必要
 - TLE = 1 秒 (アカン)

問題点

- "1"を送信するのに"0"の倍の時間が必要
⇒ 最悪ケースが大きくなる
- できるだけ時間が均等になるように、あるいは、
時間がかかるほど得られる情報量が多くなるようにしたい

満点解法

- X のまだ通信を行っていない部分について,
下から N 個の 1 が連続している場合 $N + 1$ を送信
 - N 単位時間で N bits の情報が得られる
- ほとんど全ての bit が 0 の場合が最悪ケースで,
最大約 60 回の送受信が必要
 - $14 \times 60 = 840$ [ms] 間に合う！！

具体例

- $X = 01001011$ (2) の場合

0 1 0 0 1 0 1 1

- $3 \rightarrow 2 \rightarrow 1 \rightarrow 2$ の順に送信

応用例

コントラクトブリッジ



[http://ja.wikipedia.org/wiki/
%E3%83%95%E3%82%A1%E3%82%A4%E3%83%AB:Bridge_declarer.jpg](http://ja.wikipedia.org/wiki/%E3%83%95%E3%82%A1%E3%82%A4%E3%83%AB:Bridge_declarer.jpg)

コントラクトブリッジとは

- トランプを用いたトリックテイキングゲーム
- 競技人口 1 億人以上
- 4 人で行い, 2vs2 のペア同士の戦い

何の関係あるの？

- 手札が配られた後、**パートナーの手札がわからない状態**でオークションが開始
- ビッド＝「切り札を□にして 13 回中△回勝ちます」
 - □: クラブ, ダイヤモンド, ハート, スペード, 切り札無し
 - △: 7 ～ 13
- 順番にビッドまたはパスをし、最も高い△（同じなら□が右の方）を宣言した人の宣言に決定

やばい点

- パートナーの手札がわからないのに、合計何回勝つか宣言しないといけない（辛い）
- 言葉や仕草で伝えるのは（アカン）
- **ビッドそのものを利用して**、強さや形を伝える

Relay System

- 情報伝達の手段の一つ
- 強い人と弱い人に分かれる
- 強い人「俺ずっと一つ上（Relay）言うから、お前の情報よこせ」
- 弱い人「はいはい、ハートが4枚で、エースの枚数は1枚で……」

本問では

- Bさん「俺ずっと1単位時間の Sleep (Relay) するから, お前の情報よこせ」
- Aさん「はいはい. 一番下の桁は0で, 下から2～4桁目は順に110で……」

本問では

- Bさん「俺ずっと1単位時間の Sleep (Relay) するから, お前の情報よこせ」
- Aさん「はいはい. 一番下の桁は0で, 下から2～4桁目は順に110で……」

完全に一致

宣伝

- 私は東大コントラクトブリッジ同好会の会員です
- コントラクトブリッジは皆さんの情報系の力を存分に発揮できるととても楽しいマインドスポーツです
 - 実際、数学系や情報系の人が結構多い
- **みんな、コントラクトブリッジしようぜ！！**

注意点

注意点

- この作品はフィクションです.
- 本当にこのアルゴリズムで Submit した場合, 不正とみなされ無効となる場合があります.
 - 正直, これを書けるなら普通に想定解は書けるだろうけど.....
- ルールを守って楽しくメッセンジャー・ゲームしよう

解説

Day4 Presents

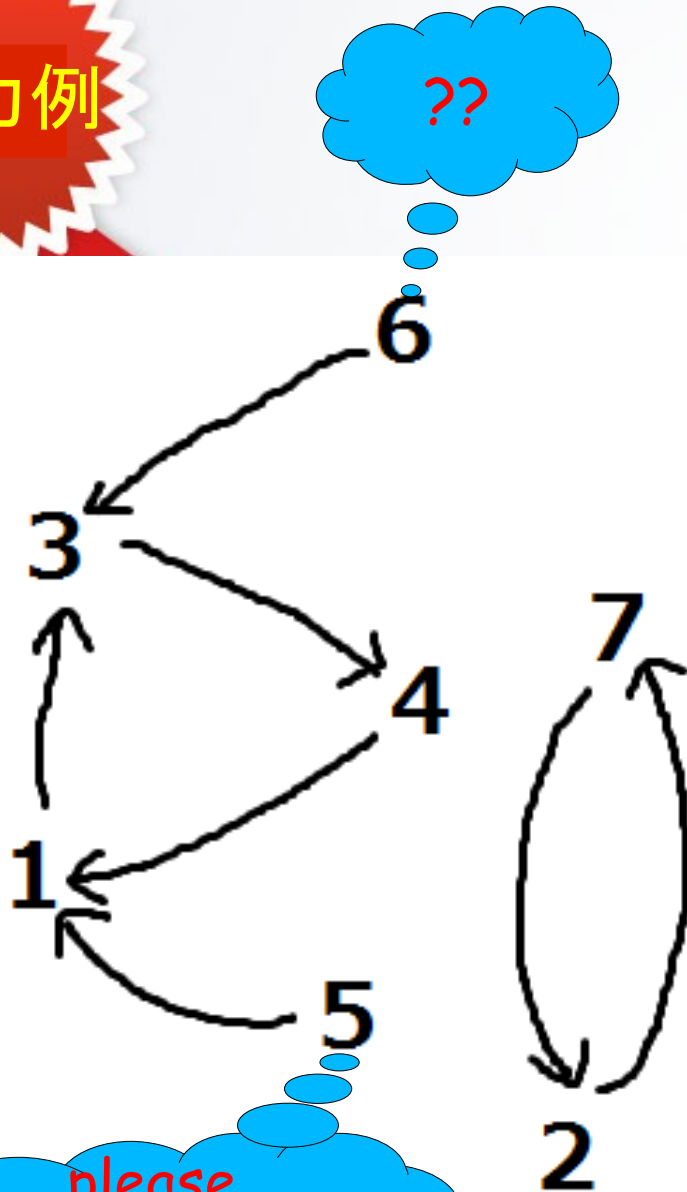


秀 郁未 (tozangezan)

概要

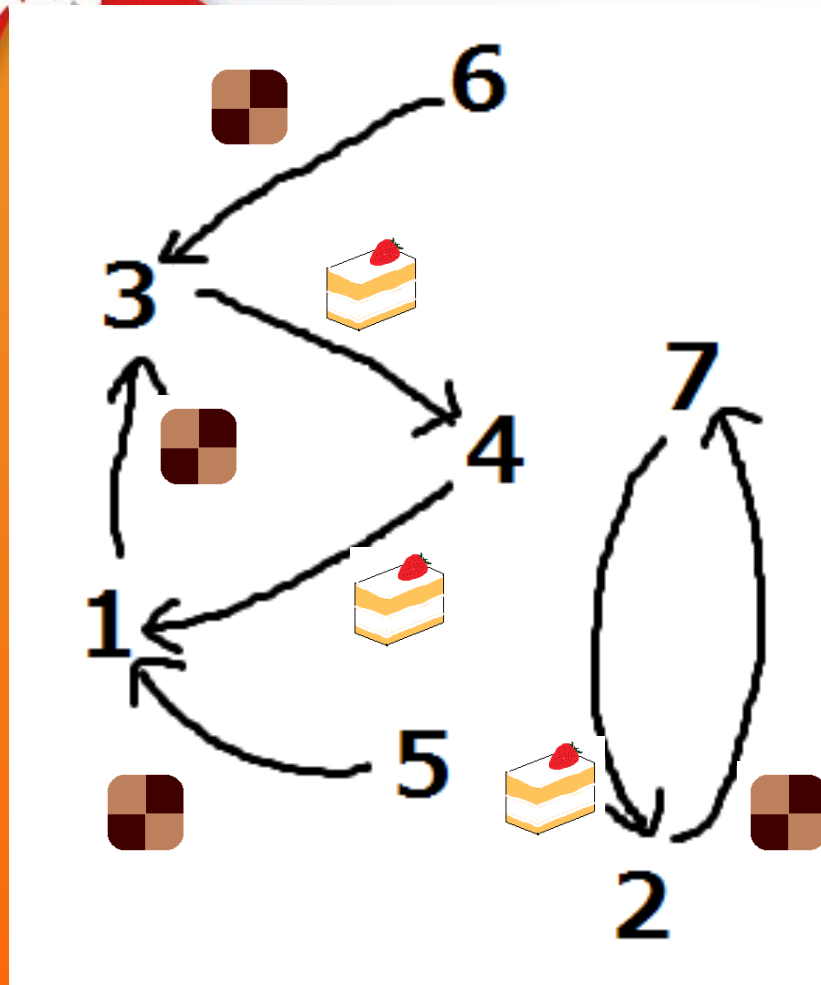
- N 人の人がいて、他のある人に 2 種類のお菓子（ケーキかクッキーのどちらか）のうち 1 種類を決めて何個か渡す
- 受け取る人は自分のあげたお菓子と同じお菓子 1 個をもらう、もしくは違うお菓子 1 個をもらうことで得られる嬉しさが決まっている
- それぞれの人がどちらのお菓子を作るか決めることができるので、 N 人の嬉しさの合計の最大値を求めよ。
- $N \leq 100,000$

入出力例



- こんな感じにお菓子を渡す
- 同じ種類のお菓子をもらおうと嬉しい人々
→ 1,4
- 別の種類のお菓子をもらおうと嬉しい人々
→ 3,7
- どっちでも良い人
→ 2
- どうせもらえない人
→ 5,6 (絶望)

入出力例



- 1,2,5,6 がクッキー、
3,4,7 がケーキを作ると、
- 嬉しさの合計は
 $88+40+90+35+0+0+4=257$
で最大

素朴
な
解法

- それぞれの人がケーキ、クッキーどちらかを渡すか決めてシミュレーション
- 決め方が $O(2^N)$ 、それぞれについてシミュレーションは $O(N)$ 。計 $O(2^N N) \rightarrow 10$ 点

考察

- 見た感じグラフの問題
- どんな感じの形になるのだろう??
- 各頂点からは辺が必ず1本だけ出ている
($\sim \ominus \sim$)。o(待てよ、こんな感じのグラフは過去にも出題されてたような…)

| JOI 合宿 過去問 | $\angle (\cup \setminus) \neg$ 三

報告 (Report)

情報オリンピック日本委員会は報告・連絡・相談が非常に徹底された組織である。委員会には N 人が所属し、すべての人に対しその報告を受ける人(「報告先」と呼ぶ)が 1 人ずつ定まっている。

今、情報オリンピック日本委員会の中で、あるプロジェクトを立ち上げることとなった。このプロジェクトは、仕事 1, 仕事 2, ..., 仕事 N という N 個の仕事に分けられており、 N 人には一つずつ異なる担当が割り当てられている。 N 個の仕事は仕事 1 から仕事 N まで番号順に処理される。ある仕事の担当者がその仕事を終えたとき、その人は自分の「報告先」に対して「作業報告」を行う。「作業報告」を受けた人は、それと同一の「作業報告」を自分の「報告先」に対して行う。ただし、同じ「作業報告」を既に自分の「報告先」にしたことがある場合は、同じ仕事に対する「作業報告」を再びすることはない。これが繰り返されることで、委員会中の何人かが「作業報告」を受けることとなる。次の仕事は、前の仕事の「作業報告」が全て行われてから処理される。

課題

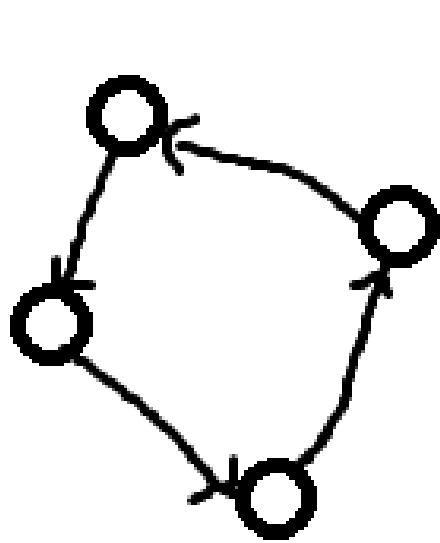
入力として各仕事の担当者の「報告先」が与えられたとき、それぞれの人が自分の仕事に取り組む段階において、受けた「作業報告」の種類数を出力するプログラムを作成せよ。

制限

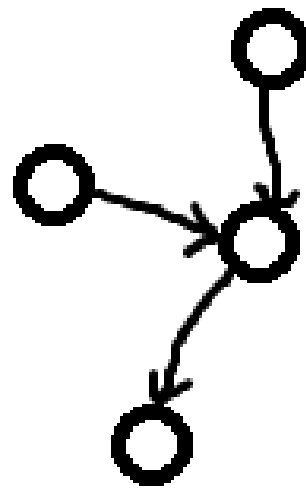
$2 \leq N \leq 100\,000$ 委員会に所属する人数

思考

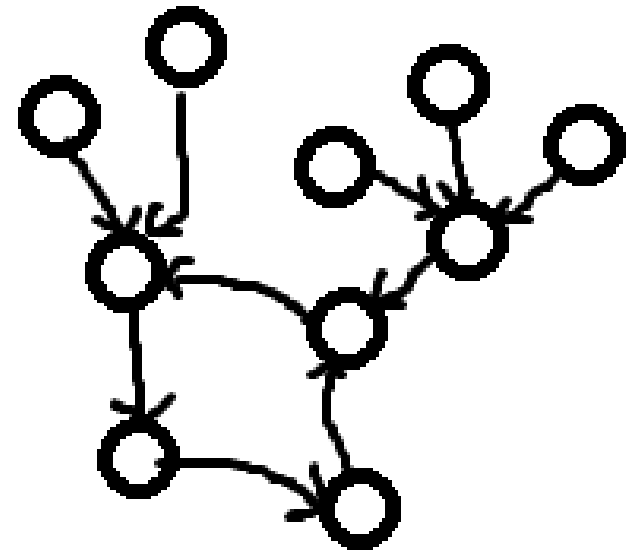
- どんな感じの形になるかいろいろ作って確かめてみる



こんなのや



こんなの



こんなのも!!

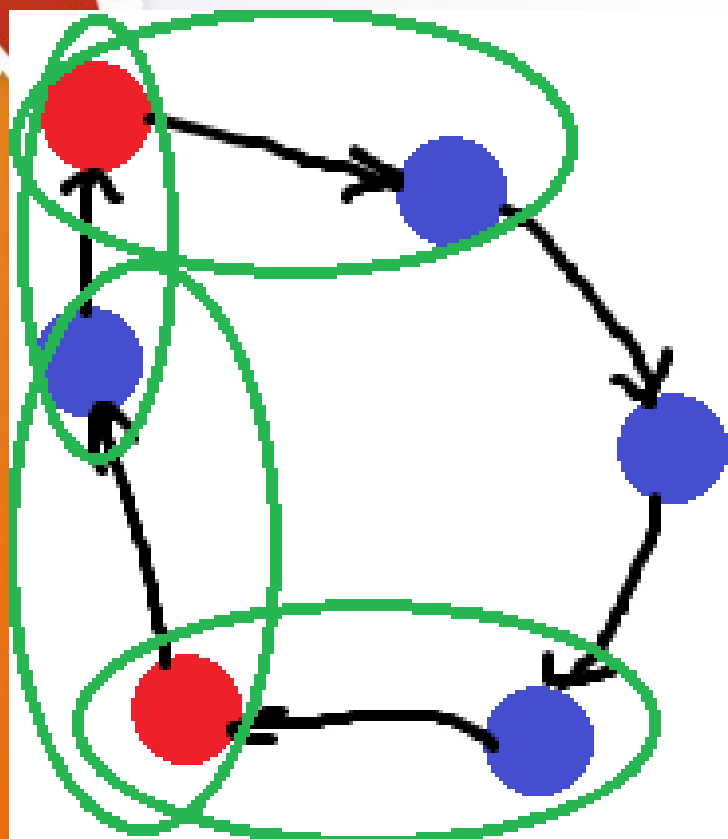
- 輪におまけの木がついたような形??
- これらが複数存在することも



満点 解法

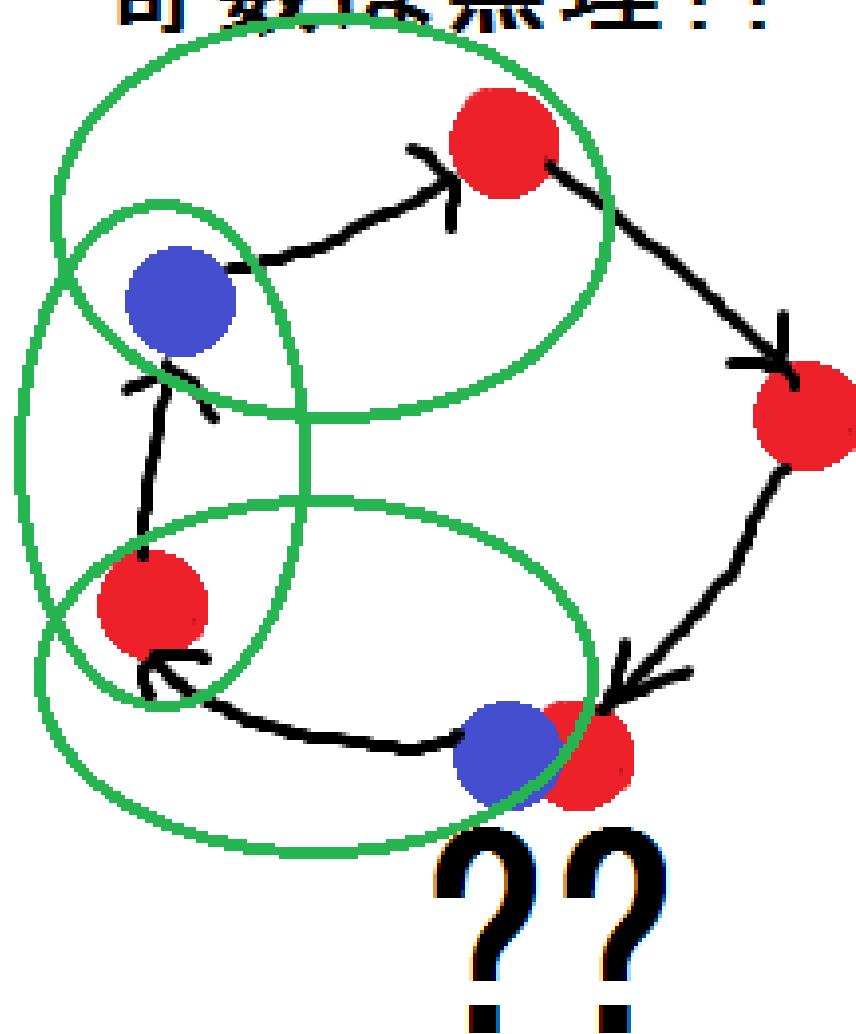
- まず、サイクル部分について考えることにする。
- 各人がクッキー、ケーキどちらかを渡すか、ということよりもらう物が自分があげた物と同じか違うか、という考え方のほうが考えやすい。
- サイクル内では「自分のあげた物と違う物を（サイクル内にいる人に）もらった」という人は偶数人でなければならない。

図説



この例だと4回
変わっている

奇数は無理!!





満点解法

- i 番目までで j 人「違うお菓子をもらう」ときの嬉しさの合計の最大値を $dp[i][j]$ とすると
→ $O(N^2)$
- そもそも DP しなくても Greedy で十分！
- $\{(\text{違う嬉しさ}) - (\text{同じ嬉しさ})\} * (\text{もらう個数})$ をソートして、大きい順に偶数個取ったときの合計の最大値を取ればよい
- すこし考えればソートしないで $O(N)$ でも出来る。(偶奇で DP)



満点解法

- サイクルでない部分→根（サイクルに含まれるところ）から順に Greedy するだけ
- ここではクッキー、ケーキのことを全く考えず「同じ」「違う」のみを考えるべき
- 「違うものがほしい」人には違うものを渡し、「同じものがほしい」人には同じものを渡す。複数人で競合しないのでこれで OK。



まとめ

- 全探索だと $O(2^N N)$ で 10 点
- サイクルの中で最大値を求めるのに DP を使うと $O(N^2)$ で 30 点
- Greedy なら $O(N \log N)$ または $O(N)$ で 100 点
- ということで、



実装

- この問題の実装でいちばんやりにくいところは、サイクルあたりの実装
- いろいろやり方あると思うので説明していきます



実装例

1

頂点にいくつか属性をつける

- まだ訪れていない点、訪れたがサイクルの内部の点か分らない点、サイクルであると分かった点、サイクルでないと分かった点などの4つ程度に種類を分ける。
- 最初各頂点から辺をたどりたいが、工夫せず全部たどると $O(N^2)$ となるので絶望
- 一度たどった点はたどらずにループを抜ける（メモ化風）なら $O(N)$



実装例 1

頂点にいくつか属性をつける

- サイクルが複数ある場合というものがあるので、すこし実装に戸惑う。
- いろいろ頂点に種類つけるのが好きな人はこういう実装よくしてそう。



実装例 2

強連結成分分解

- 概要は蟻本や Spaghetti Source で。
- この問題では1つのサイクルが1つの強連結成分で、それ以外の頂点は一つ一つが別々の強連結成分。それぞれのサイズを見ればサイクルの点か判別できそう。
- 実装無駄に長いが、慣れてたら一瞬で書ける、という人もいるかもしれない。



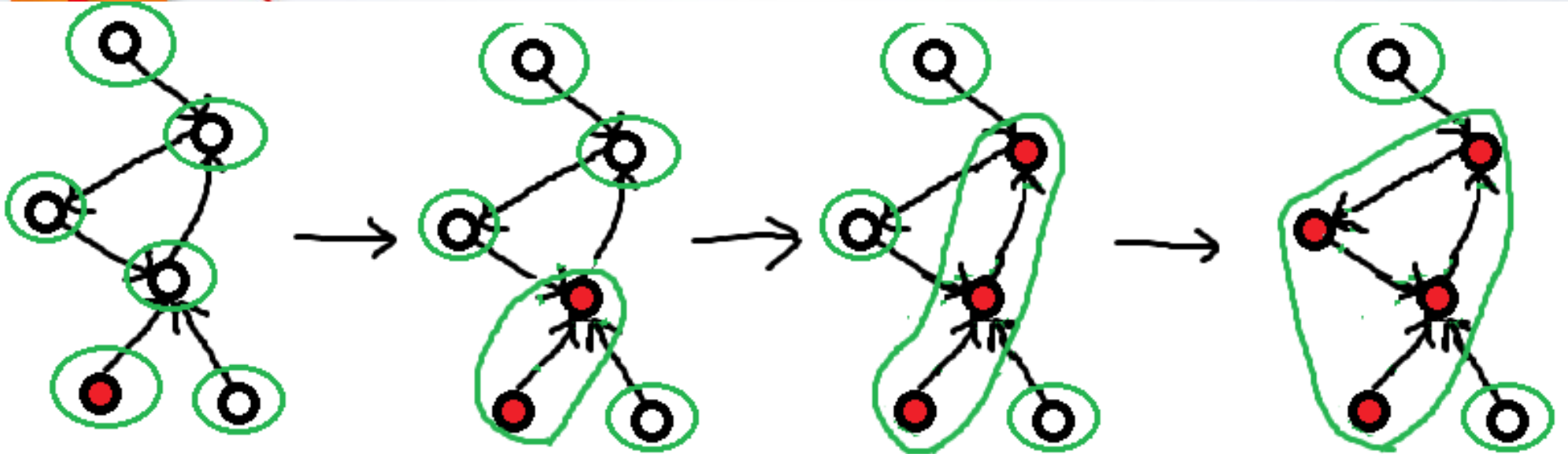
実装例 3

Union-Find

- 概要は蟻本や Spaghetti Source で。
- 辺をたどるごとに両方の端点が含まれる集合を統合する
- その前から端点と同じ集合に属していたらサイクルがある
- 結局辺を一つ一つたどるので実装があんまり楽ではなさそう。

実装例 3

Union-Find



- 同じ集合になったらそのときに見ている頂点からいける点は全部サイクルの中



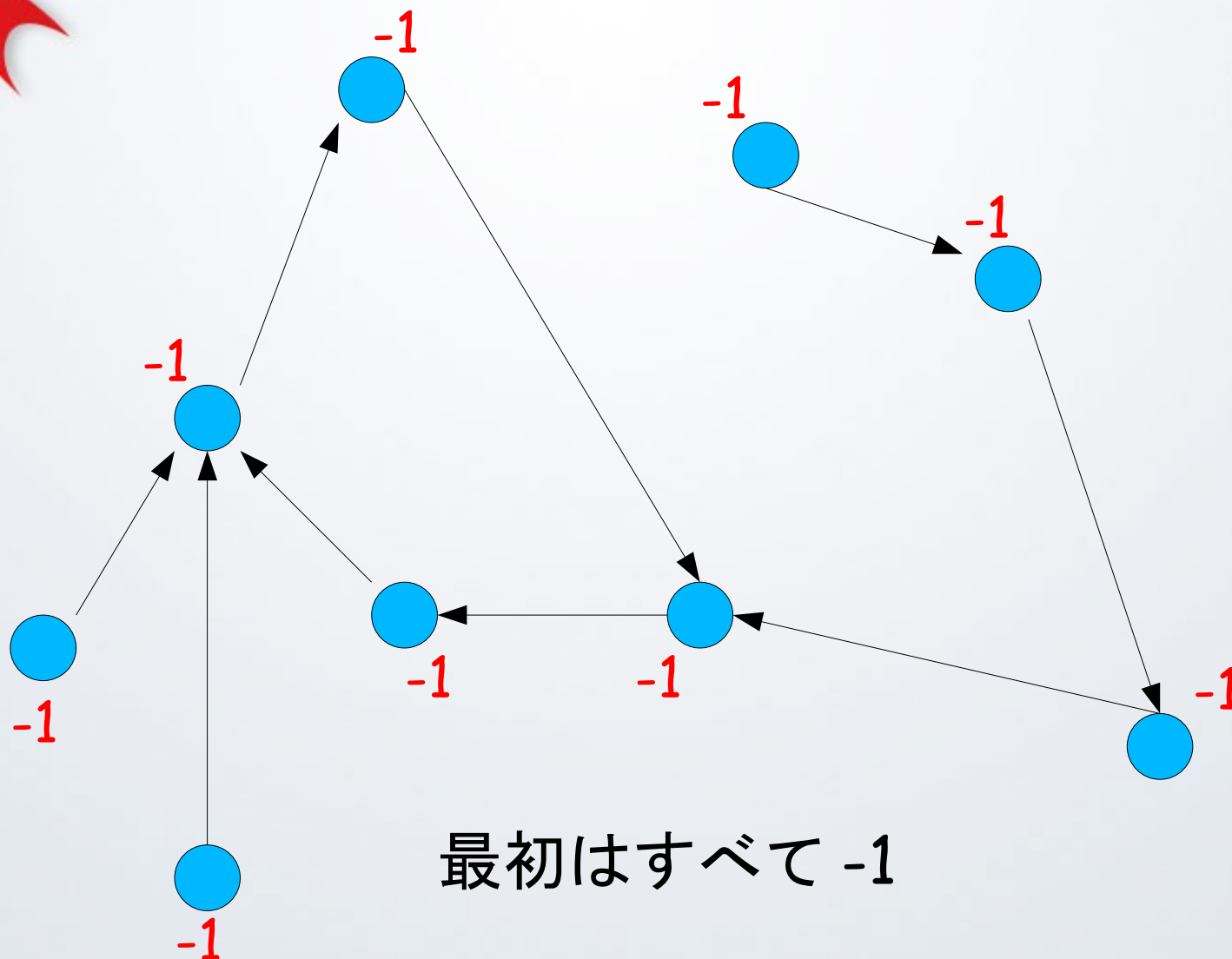
実装例 4

よさげな実装方法

- 実装例 1 (属性をつける) と同じような方法だが、属性ではなくたどり始めた頂点の番号をつける
- 最初この属性をあらわす数値はすべて -1 にしておく

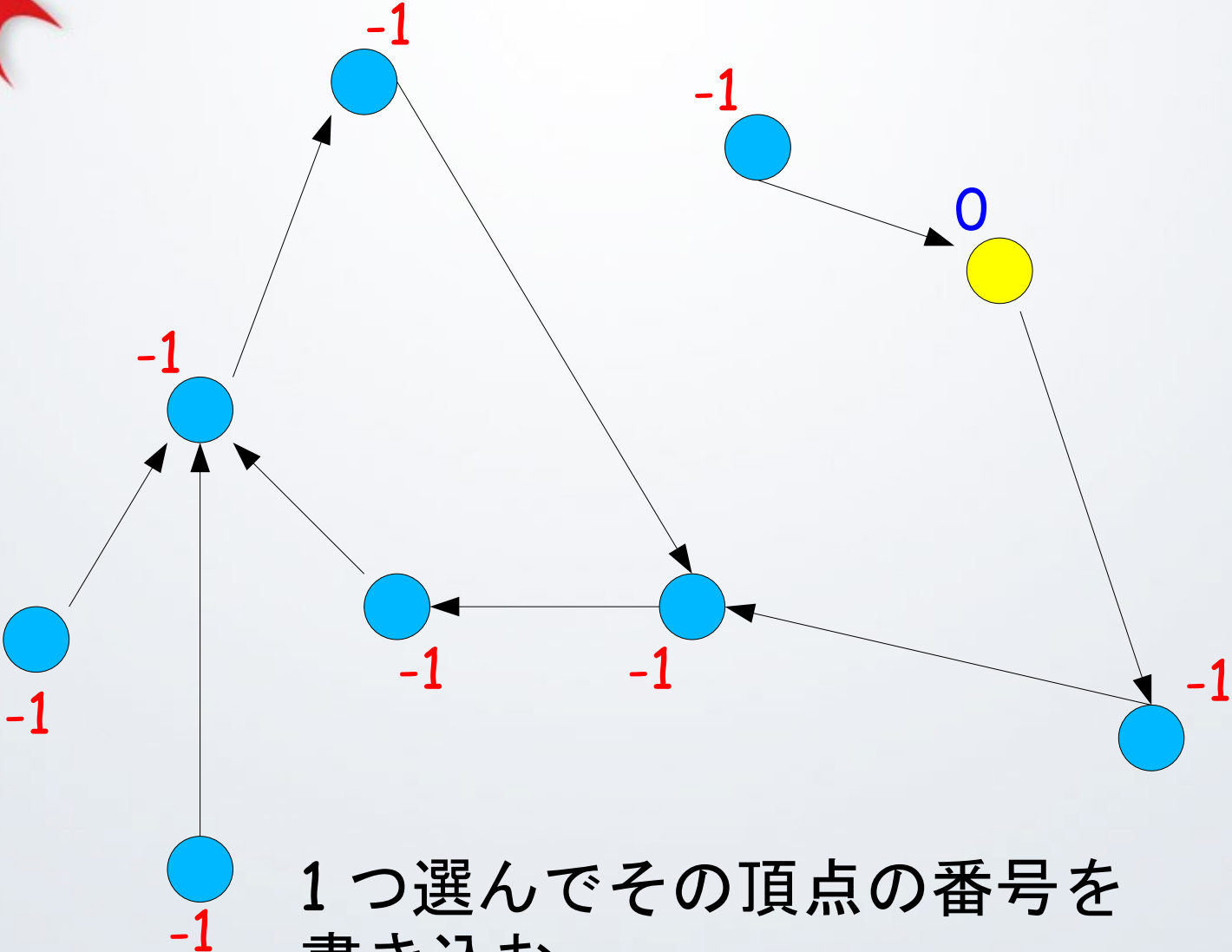
実装例 4

よさげな実装方法



実装例
4

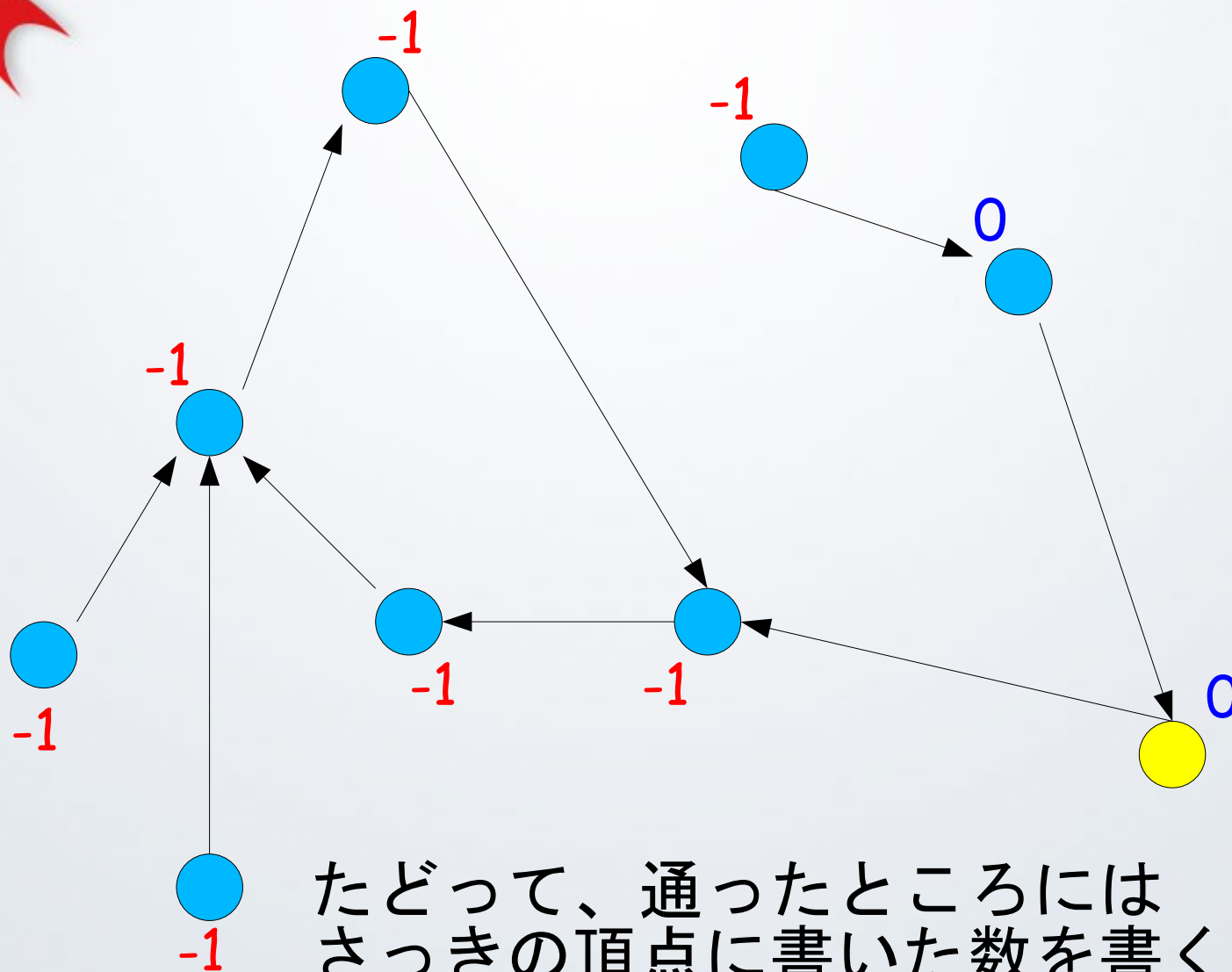
よさげな実装方法



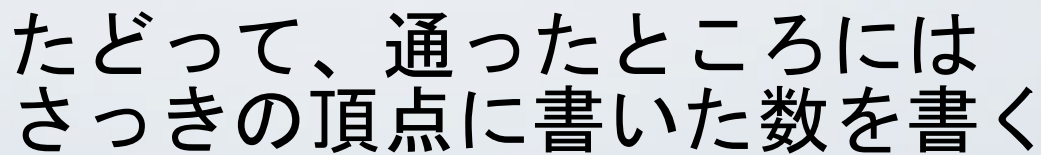
1つ選んでその頂点の番号を
書き込む

実装例 4

よさげな実装方法

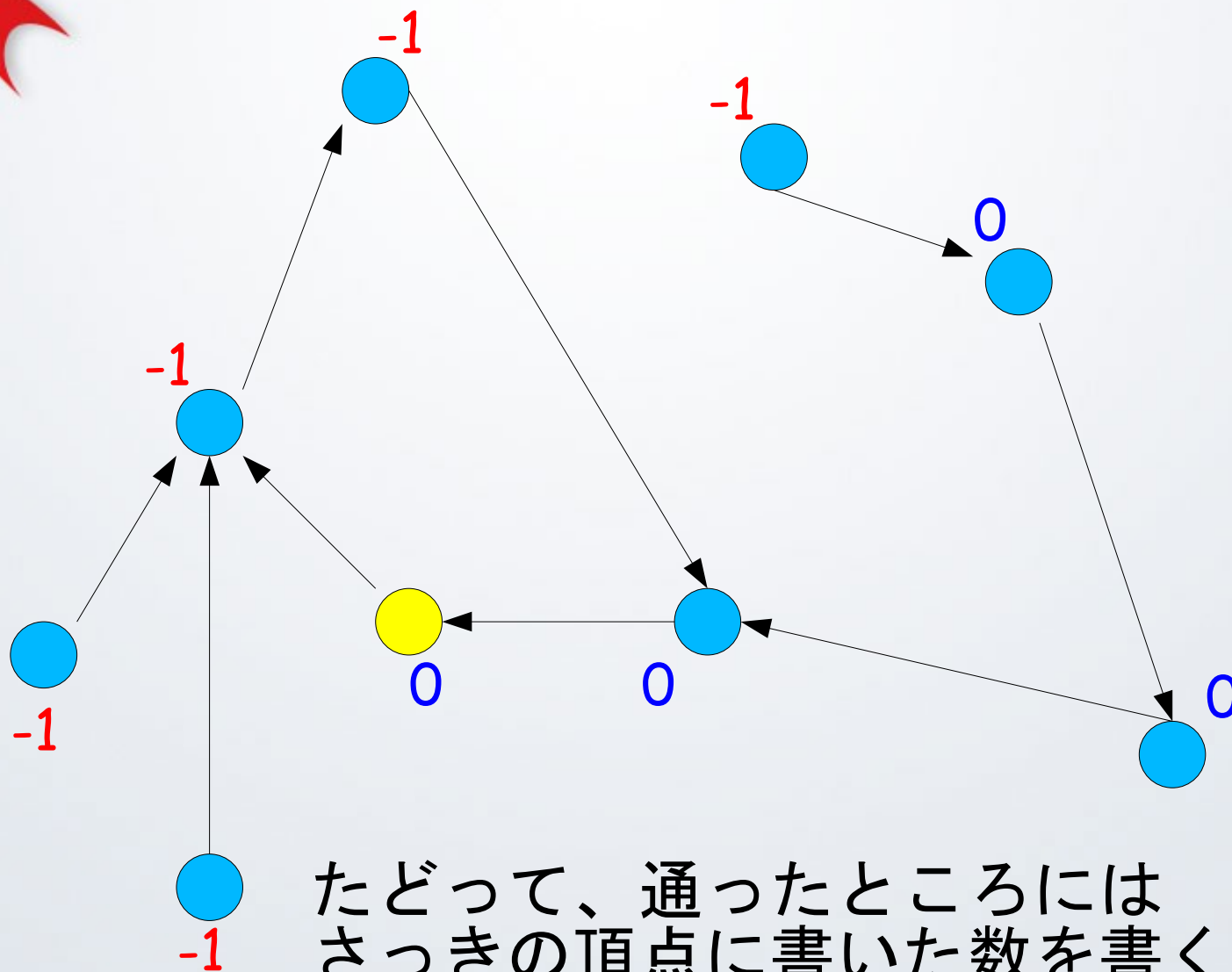


よさげな実装方法

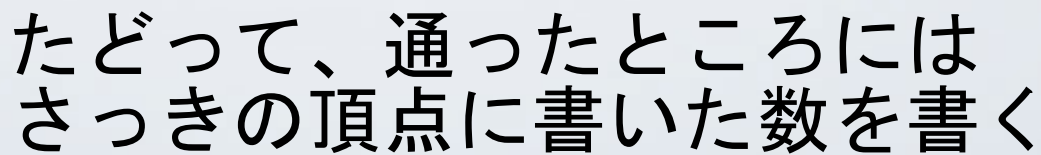


実装例 4

よさげな実装方法

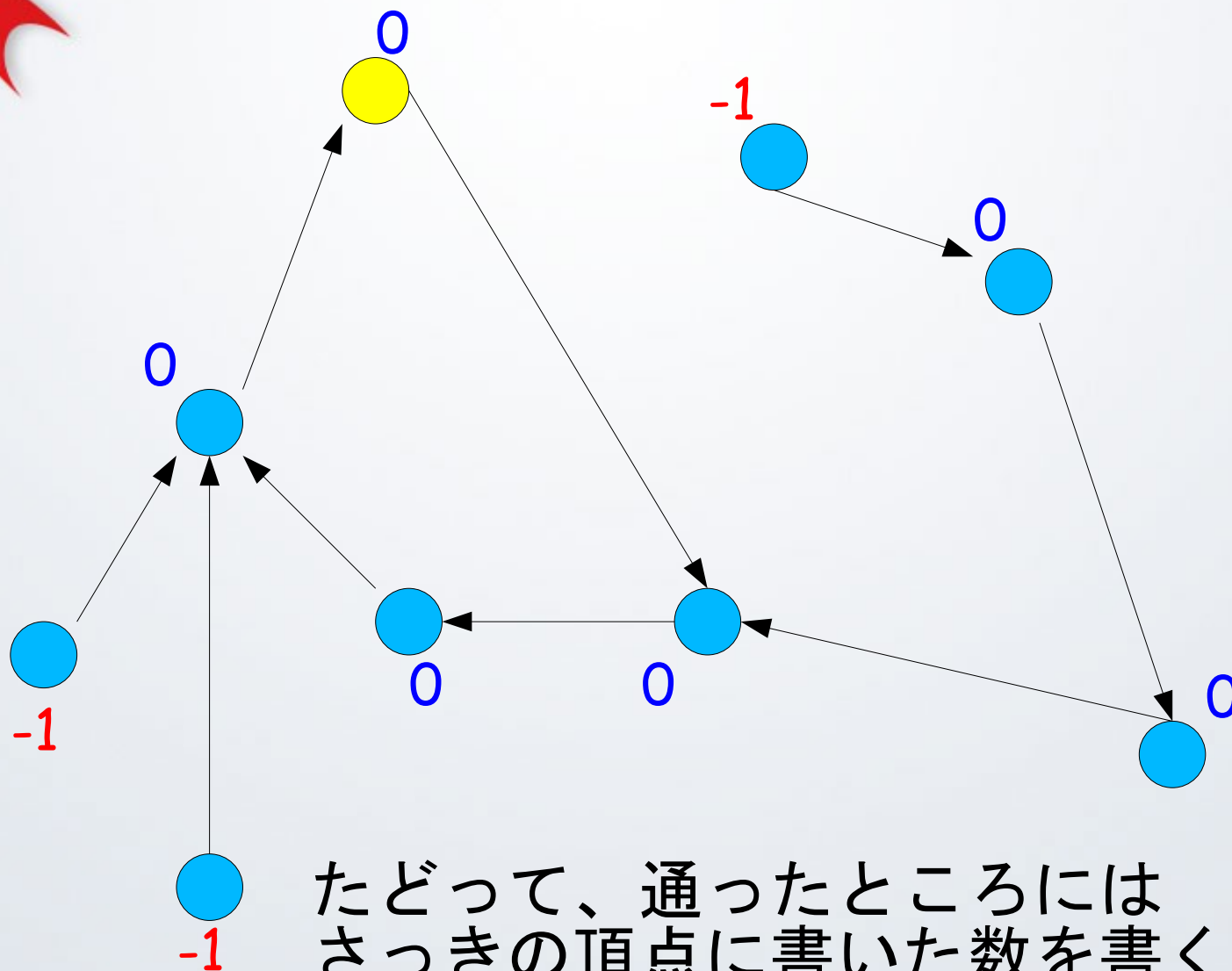


よさげな実装方法



実装例 4

よさげな実装方法

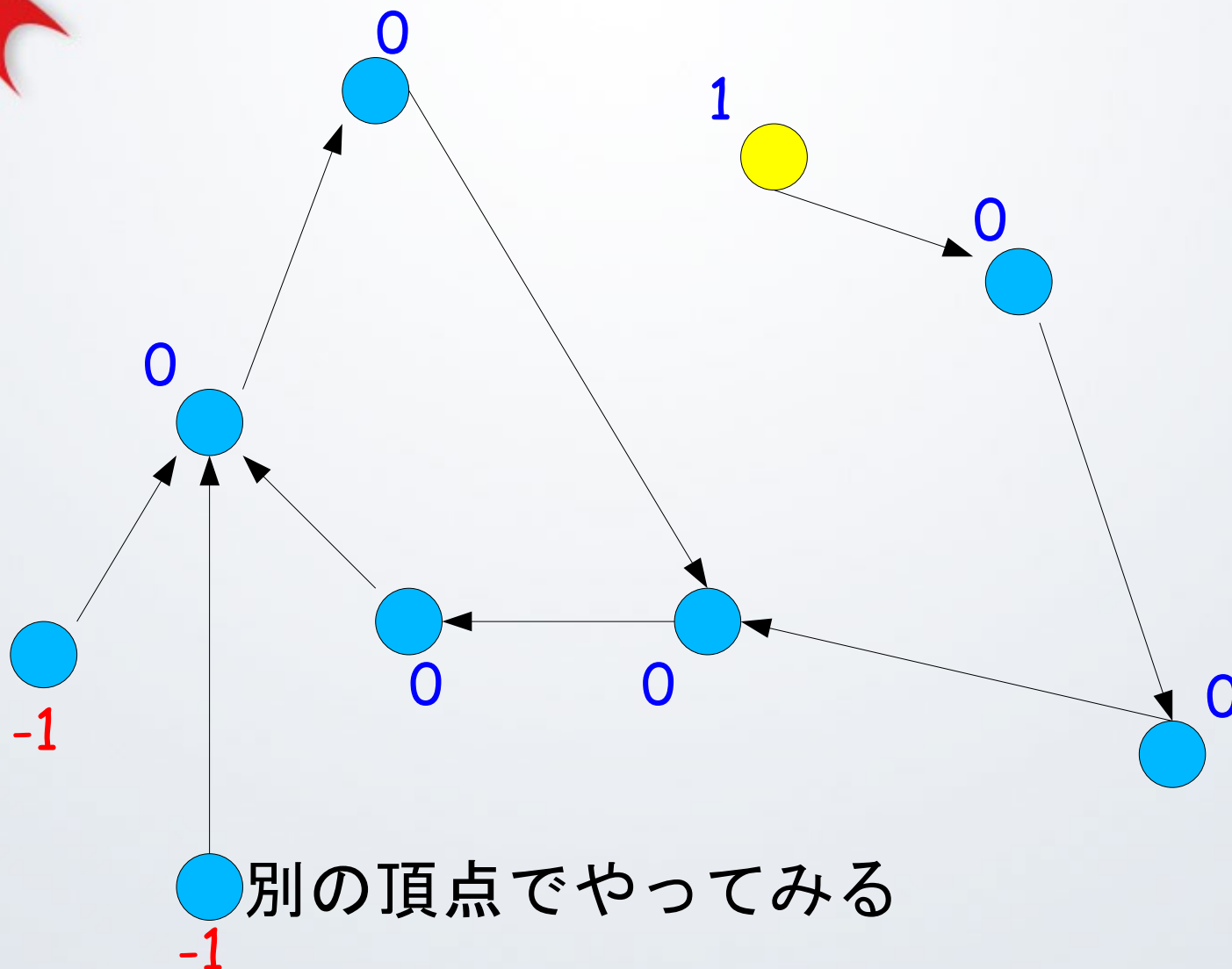


よさげな実装方法



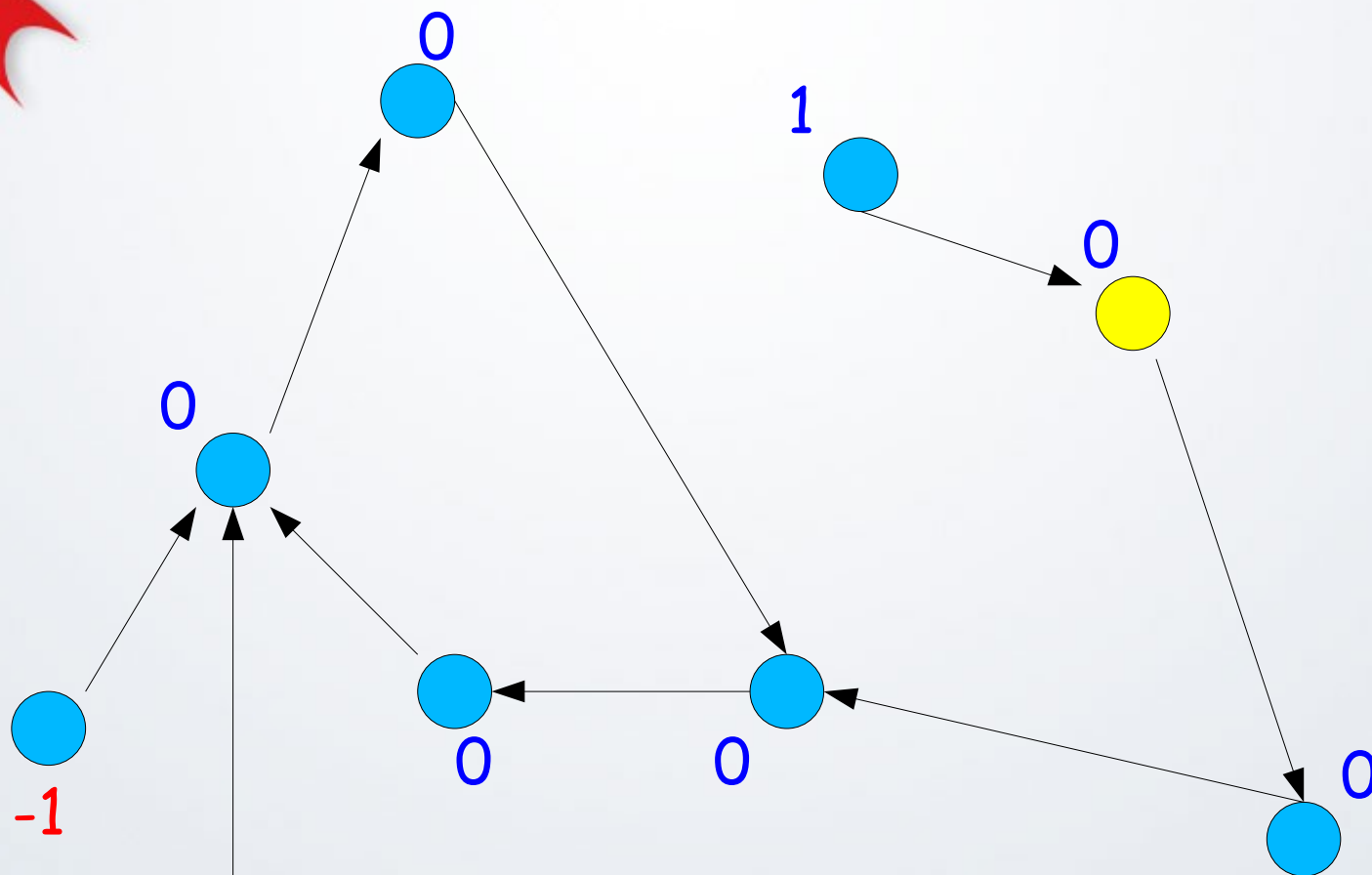
実装例 4

よさげな実装方法



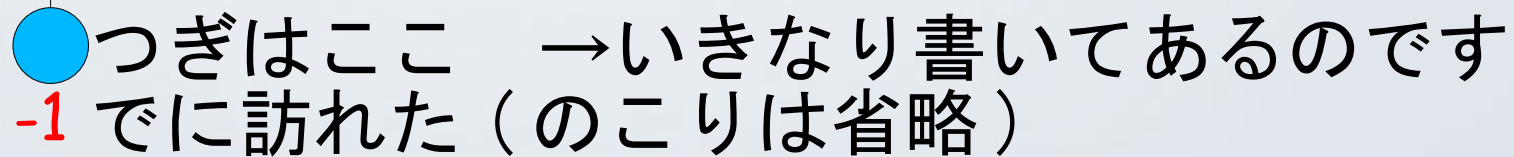
実装例 4

よさげな実装方法



-1 ではないが、最初書いたのと同じ数字に出会う → サイクルではなかった

よさげな実装方法



実装例
4

よさげな実装方法

- サイクルを検出する作業とそれぞれのサイクルに関する作業はまとめてしまったほうが楽（どのサイクルを処理したかわかりやすい）
- サイクル以外の辺に関する作業はどんな順番でも求められるのでその都度できる
 - サイクル検出と最大値を求めるのを同時に行うと実装が楽



練習問題

- 木 + 辺 1 本のグラフが出てくるもの
 - Report (JOI2011 春合宿)
 - ISLANDS(IOI2008)
 - Cthulhu (CF#80 Div2C & Div1B)
- 精神攻撃
 - パーティー (JOI2009 予選 3)

入力例 2 において, あなたには友達はいない. したがって, あ

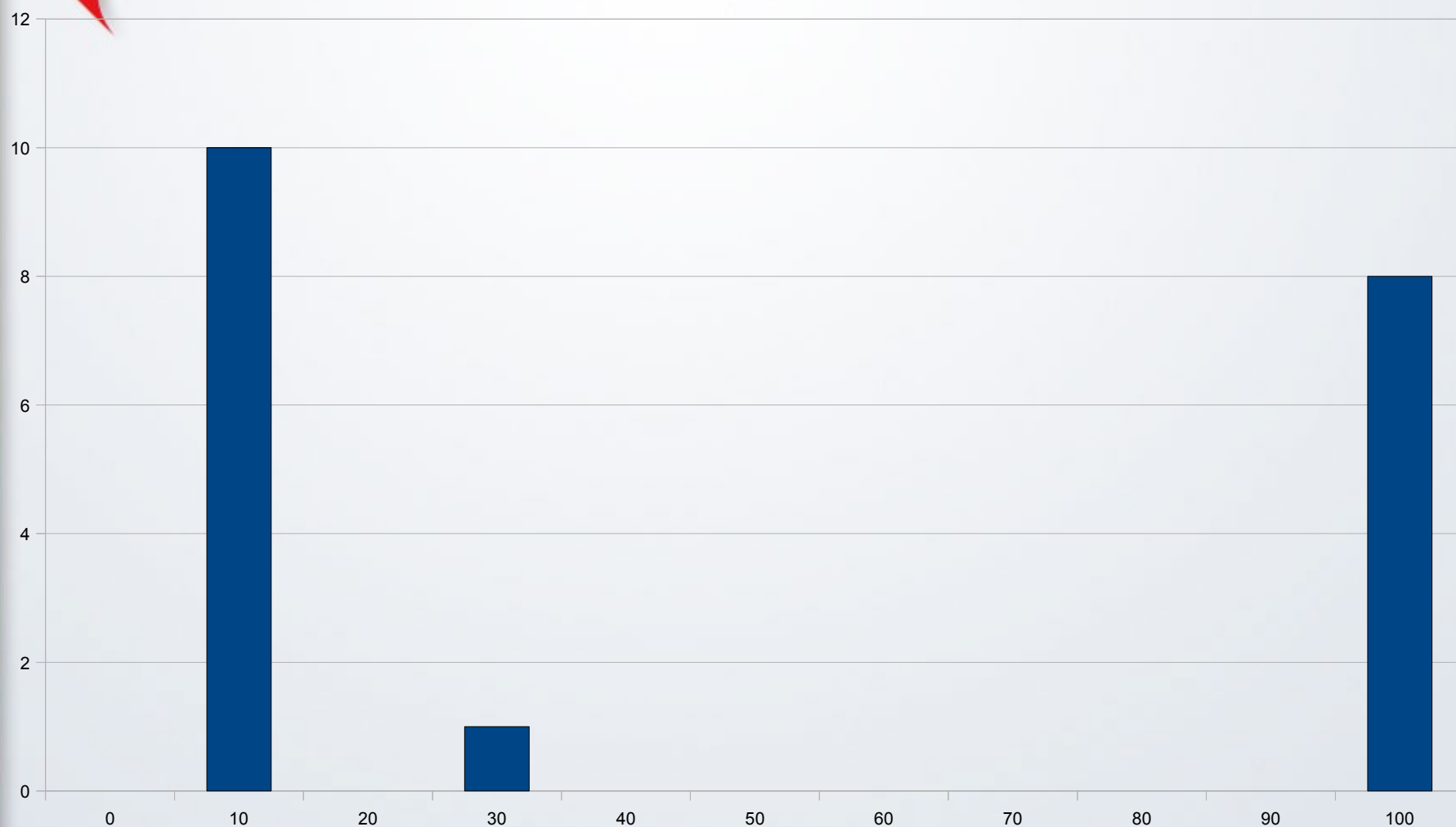


余談

- この問題を一般のグラフについて考えてみようとしたけど、負辺のあるフローが効率的に求められないらしいので冷えてます
- 何か分かったら教えてください……

得点分布

0点いない! Great!!



Spaceships 解説

Masaki Hara (qnighy)

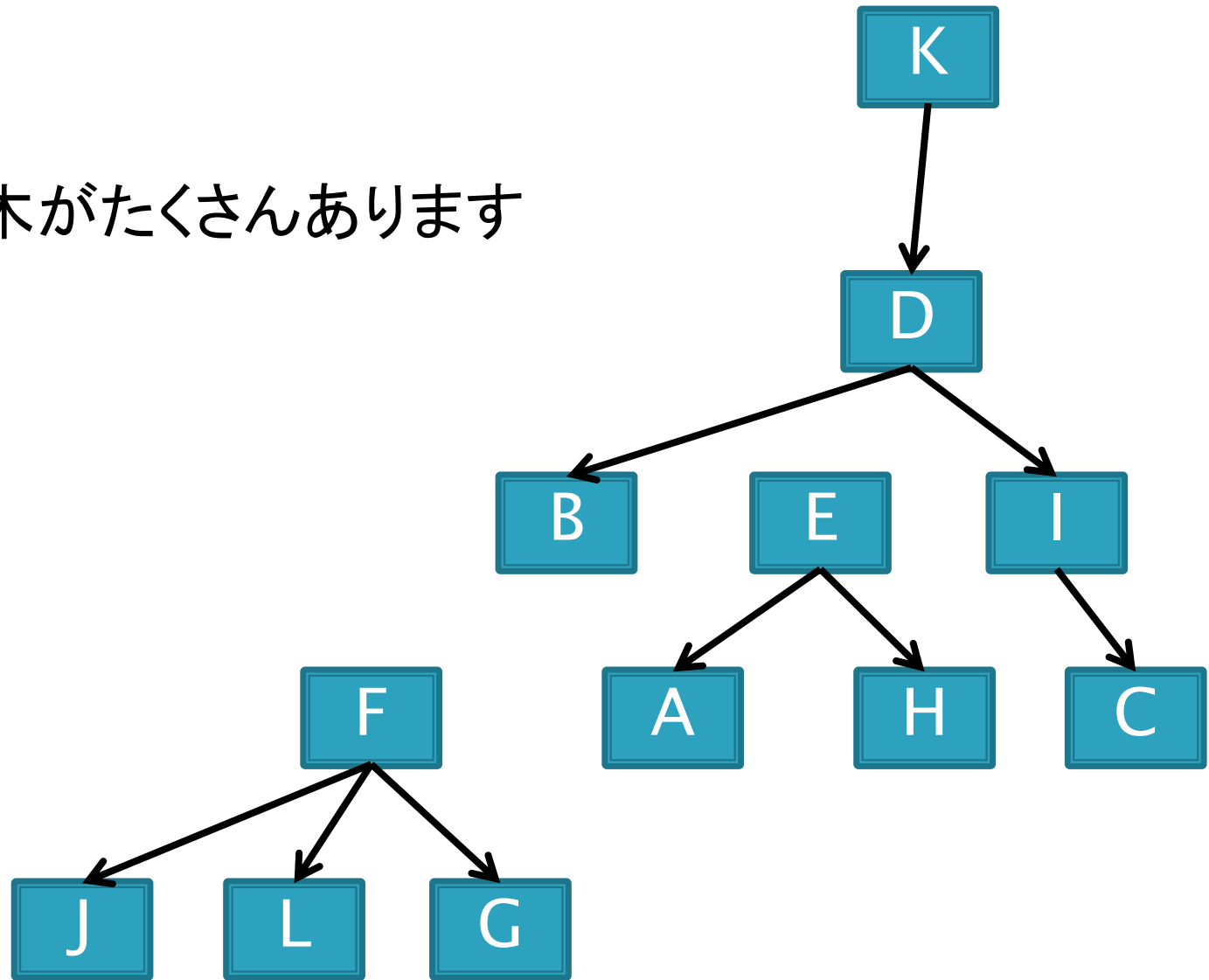
2013年 情報オリンピック春期トレーニング合宿にて

Task statement

- ▶ 有向木がたくさんあります
- ▶ 以下のクエリに高速で答えてください
 1. Aの親をBにする
 - Aは木の根で、BはAの子孫ではない
 2. Aを親から切り離して木の根にする
 3. A,Bが同じ木に属するか判定し、
同じ木に属する場合はLCAを求める

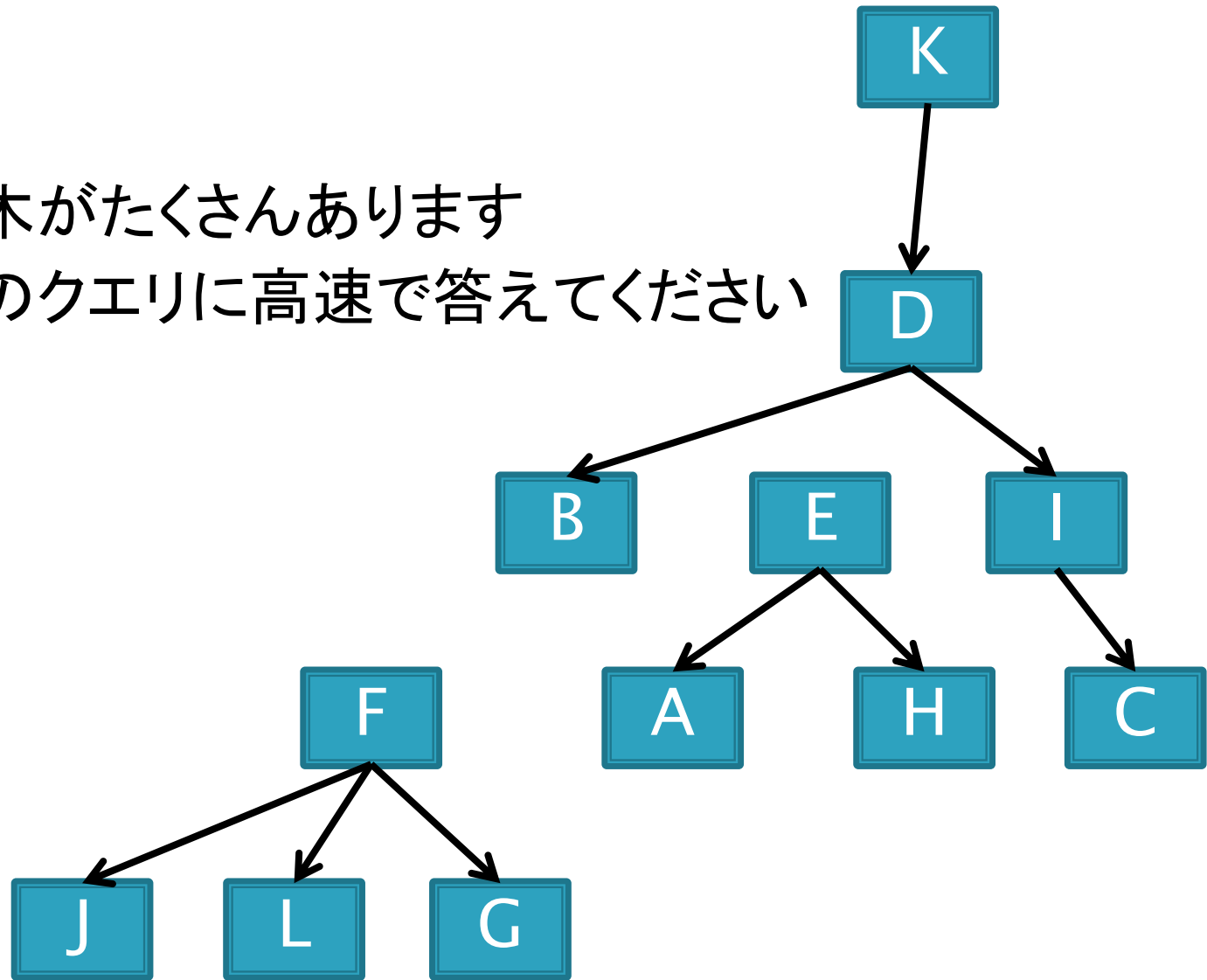
例

- ▶ 有向木がたくさんあります



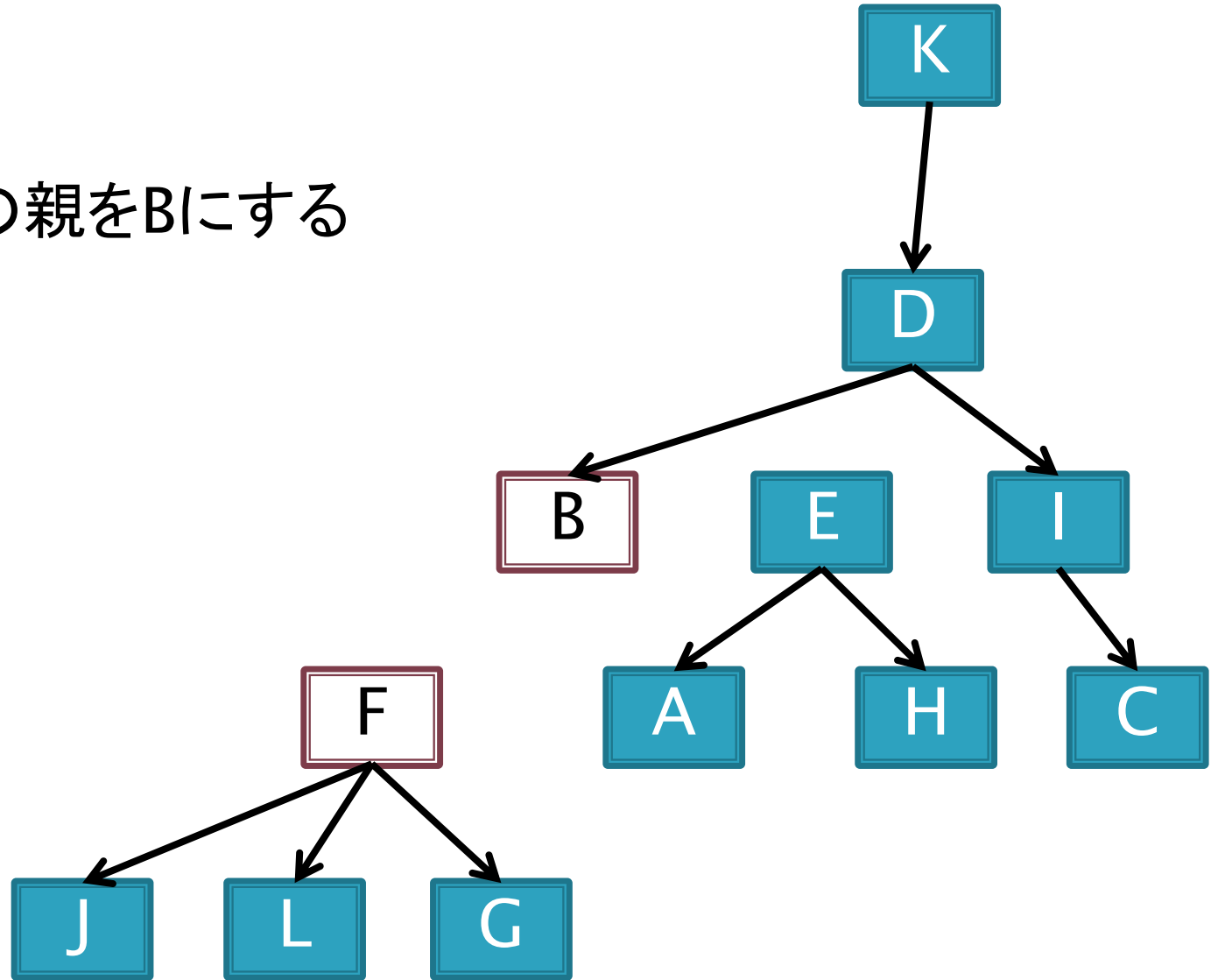
例

- ▶ 有向木がたくさんあります
- ▶ 以下のクエリに高速で答えてください



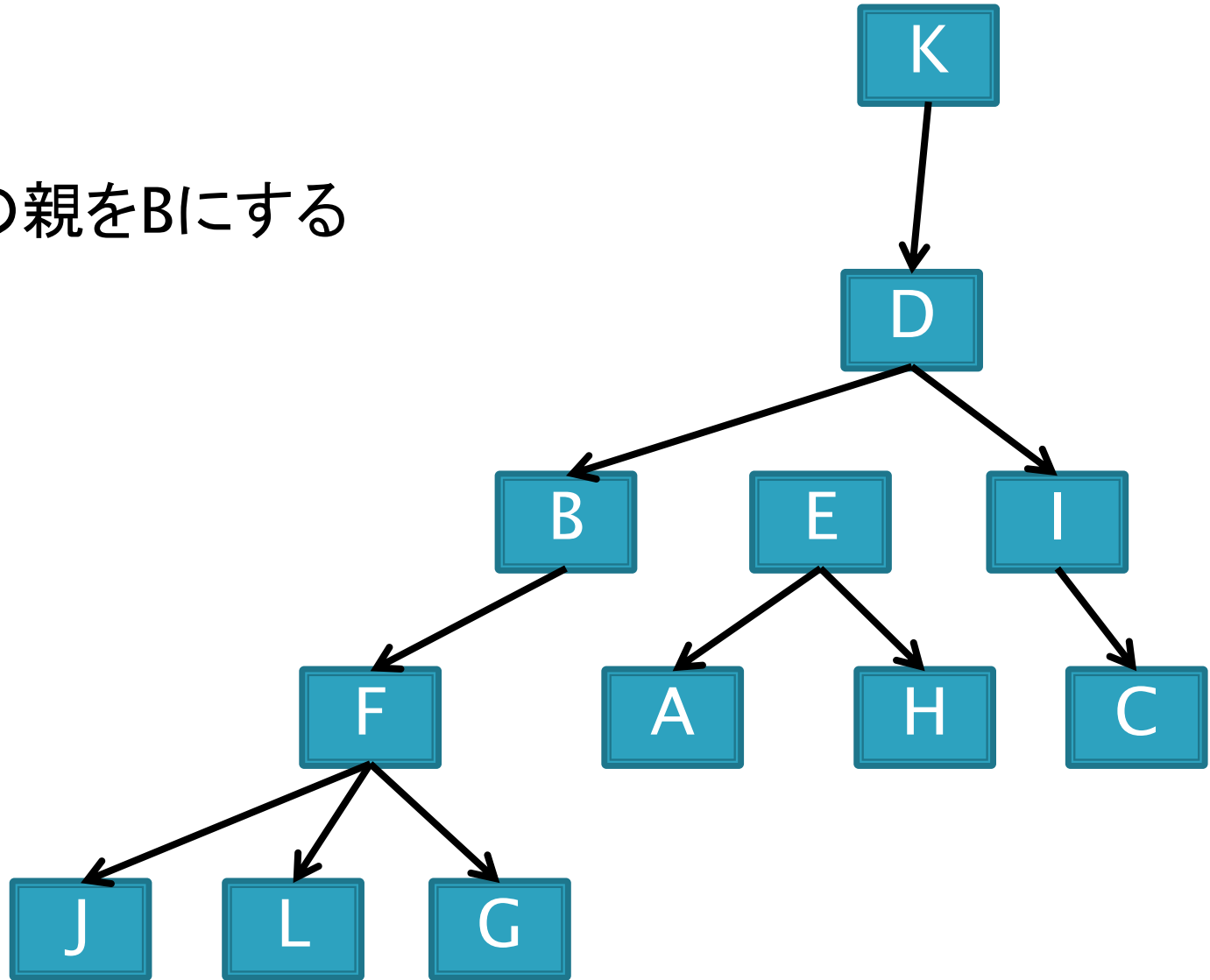
例

- ▶ 1. Fの親をBにする



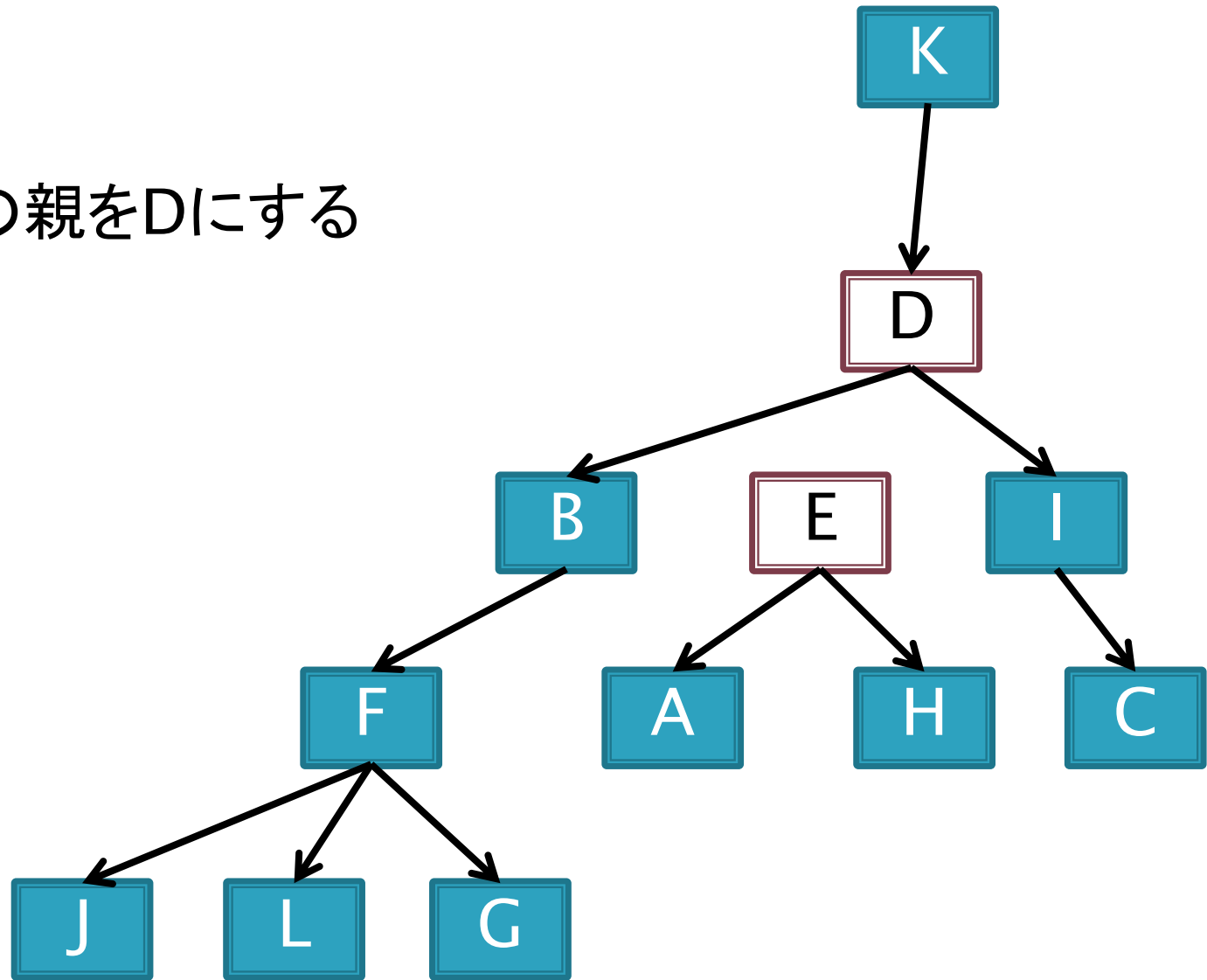
例

- ▶ 1. Fの親をBにする



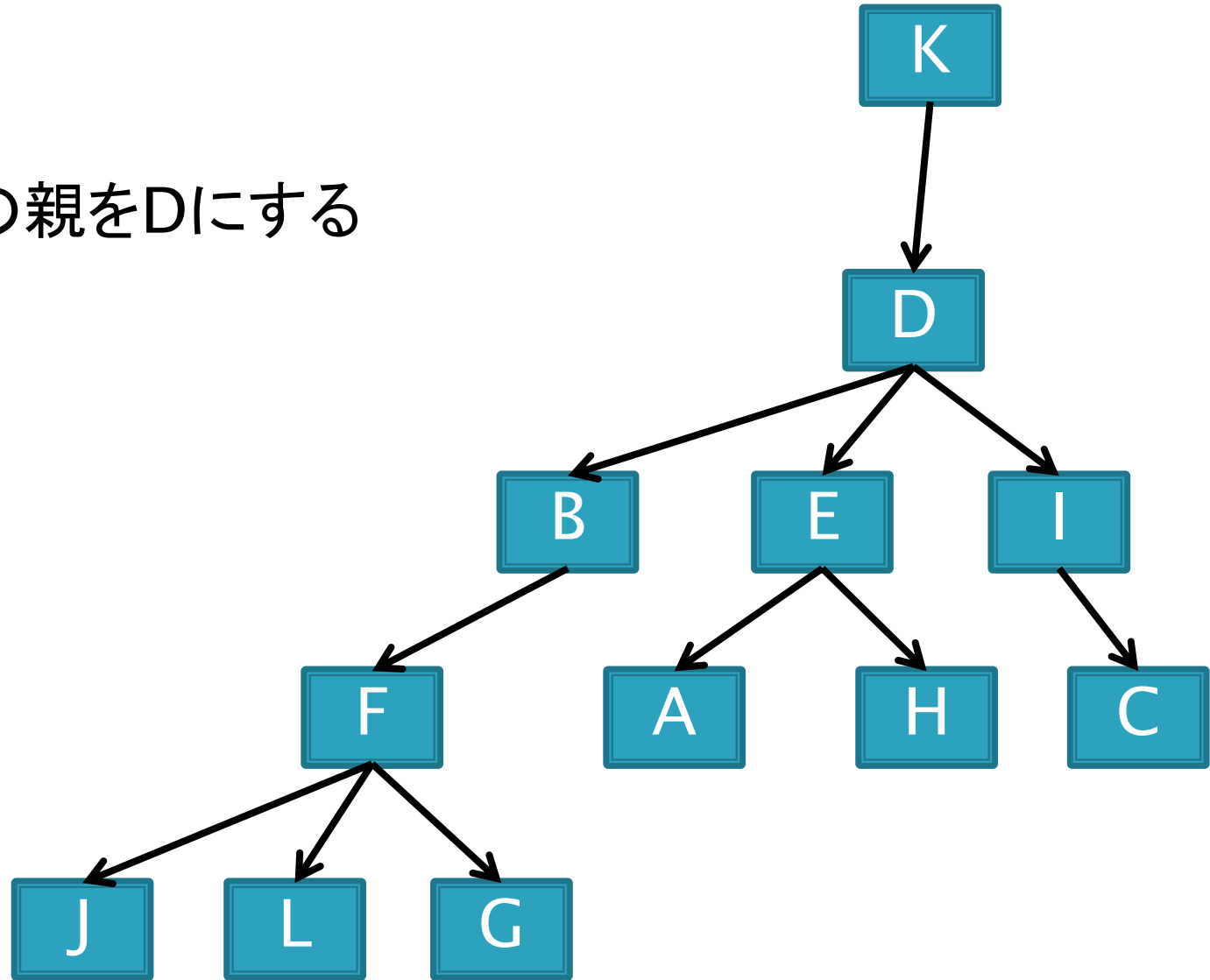
例

- ▶ 1. Eの親をDにする



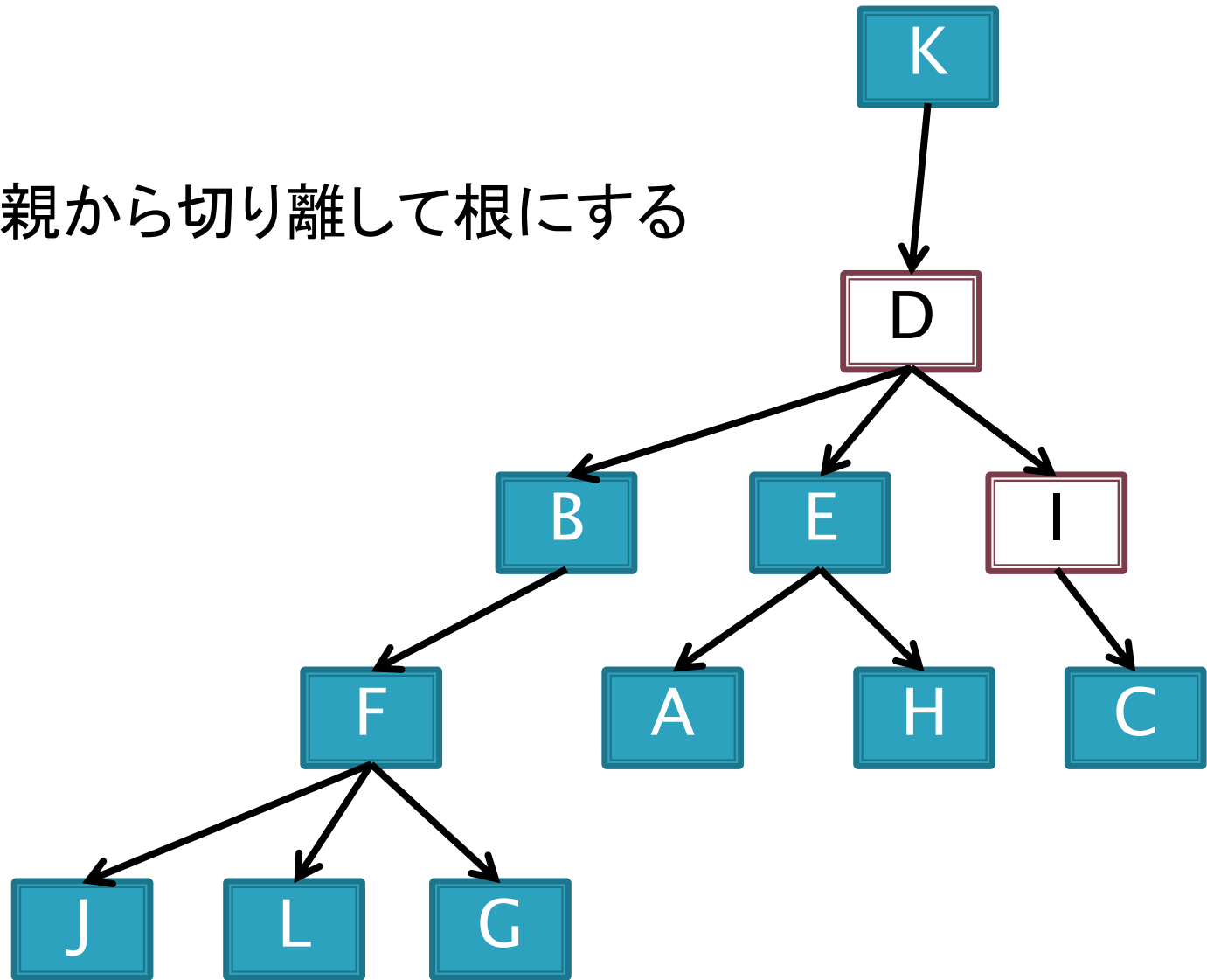
例

- ▶ 1. Eの親をDにする



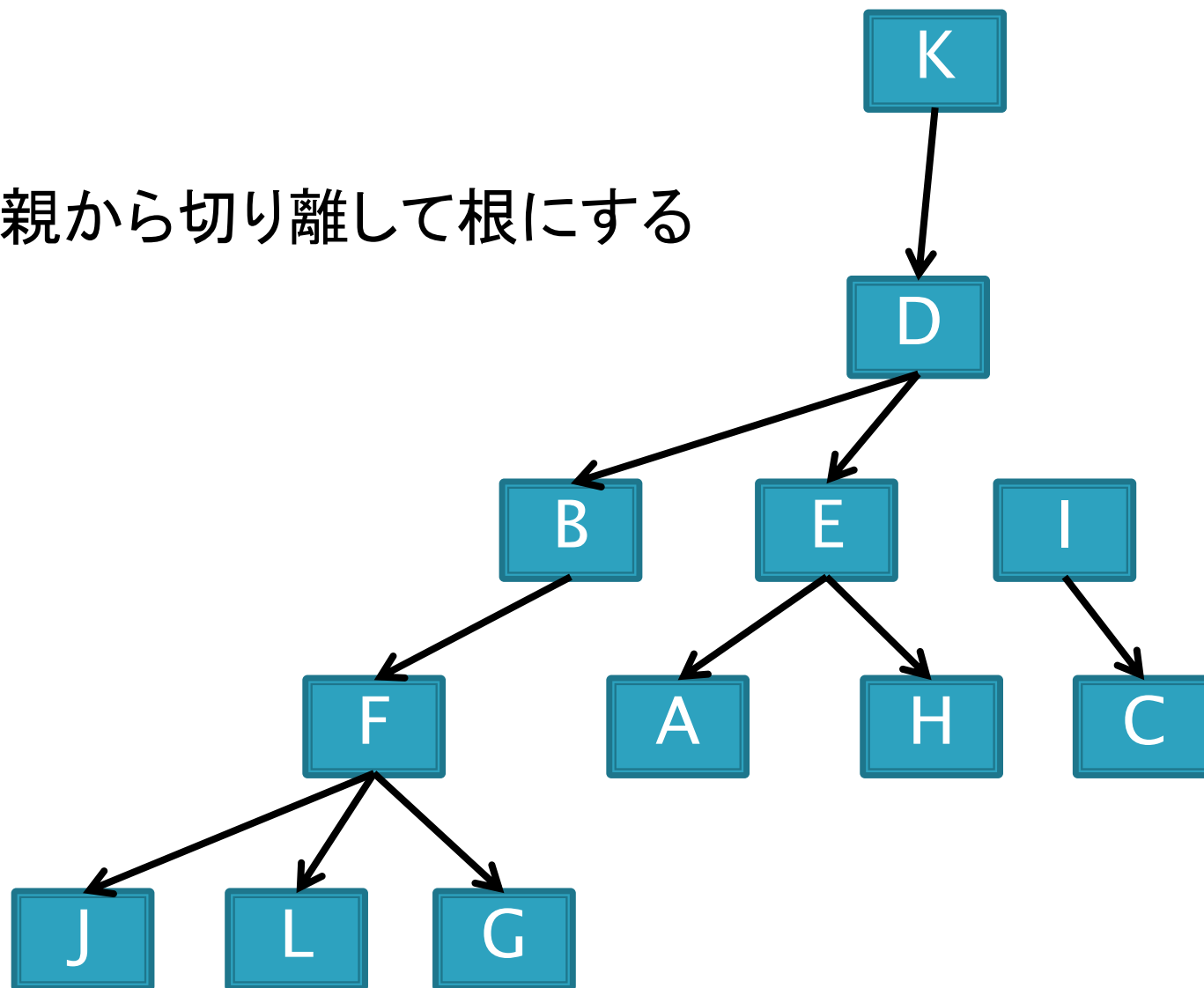
例

- ▶ 2. Iを親から切り離して根にする



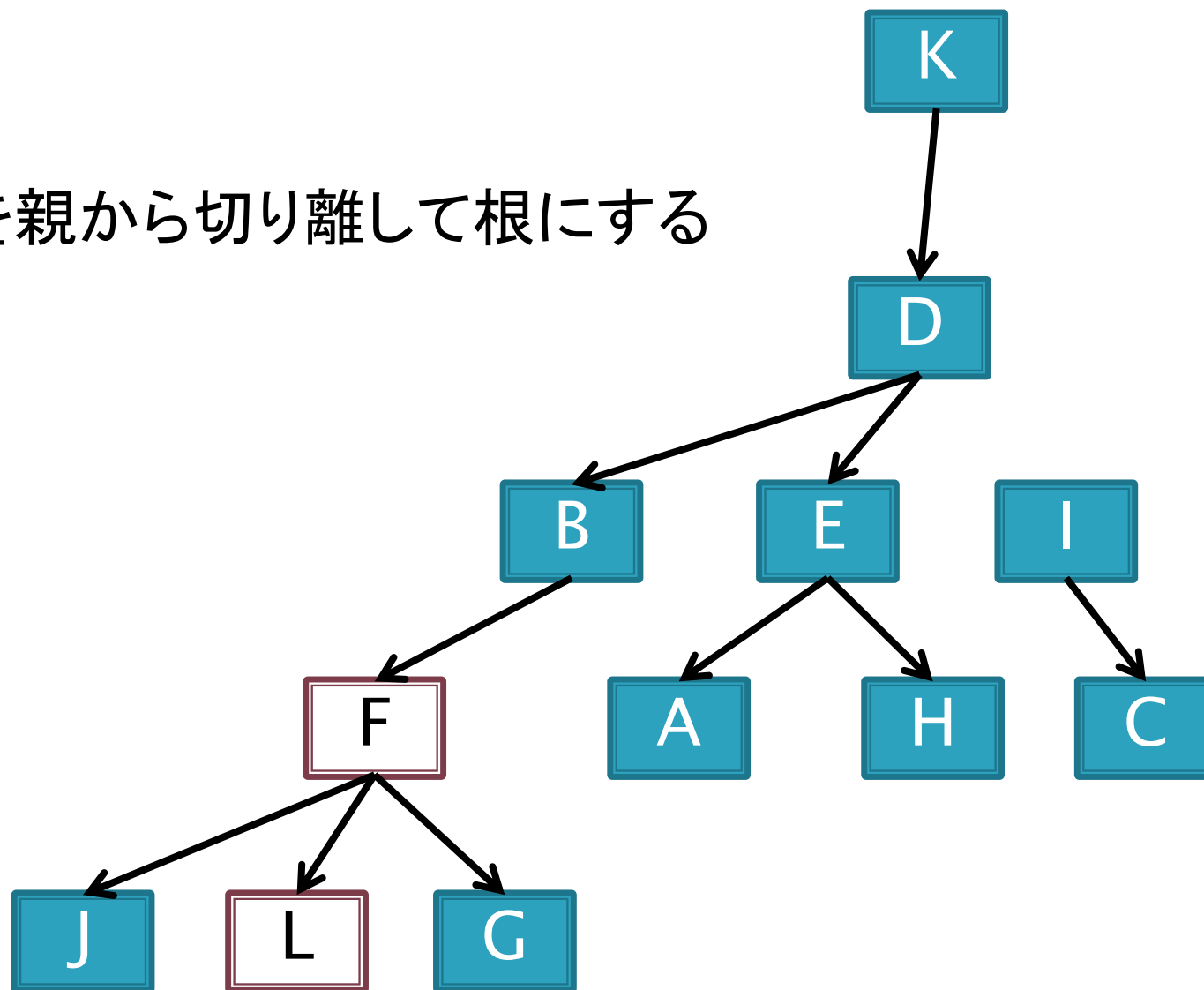
例

- ▶ 2. Iを親から切り離して根にする



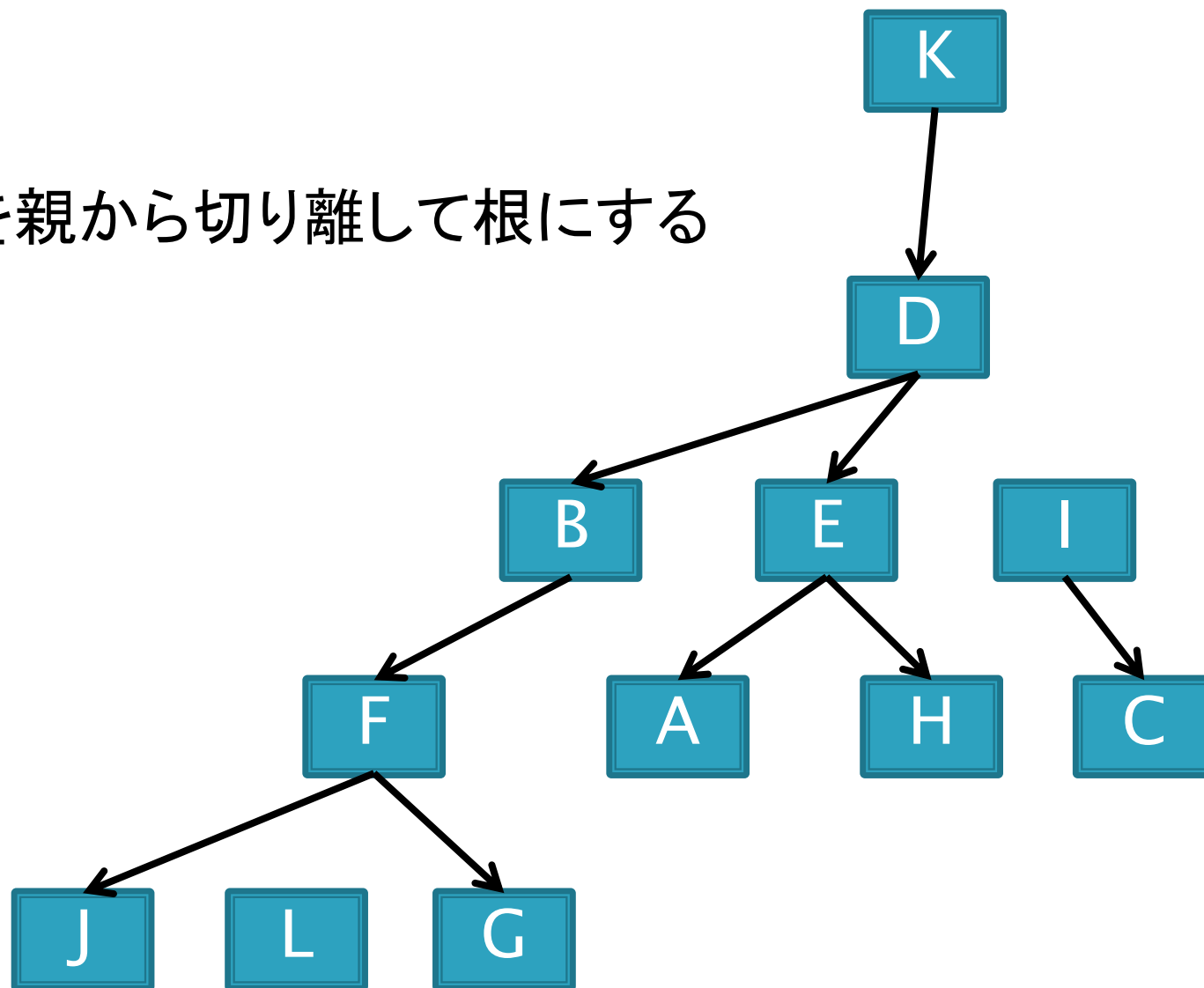
例

- ▶ 2. Lを親から切り離して根にする



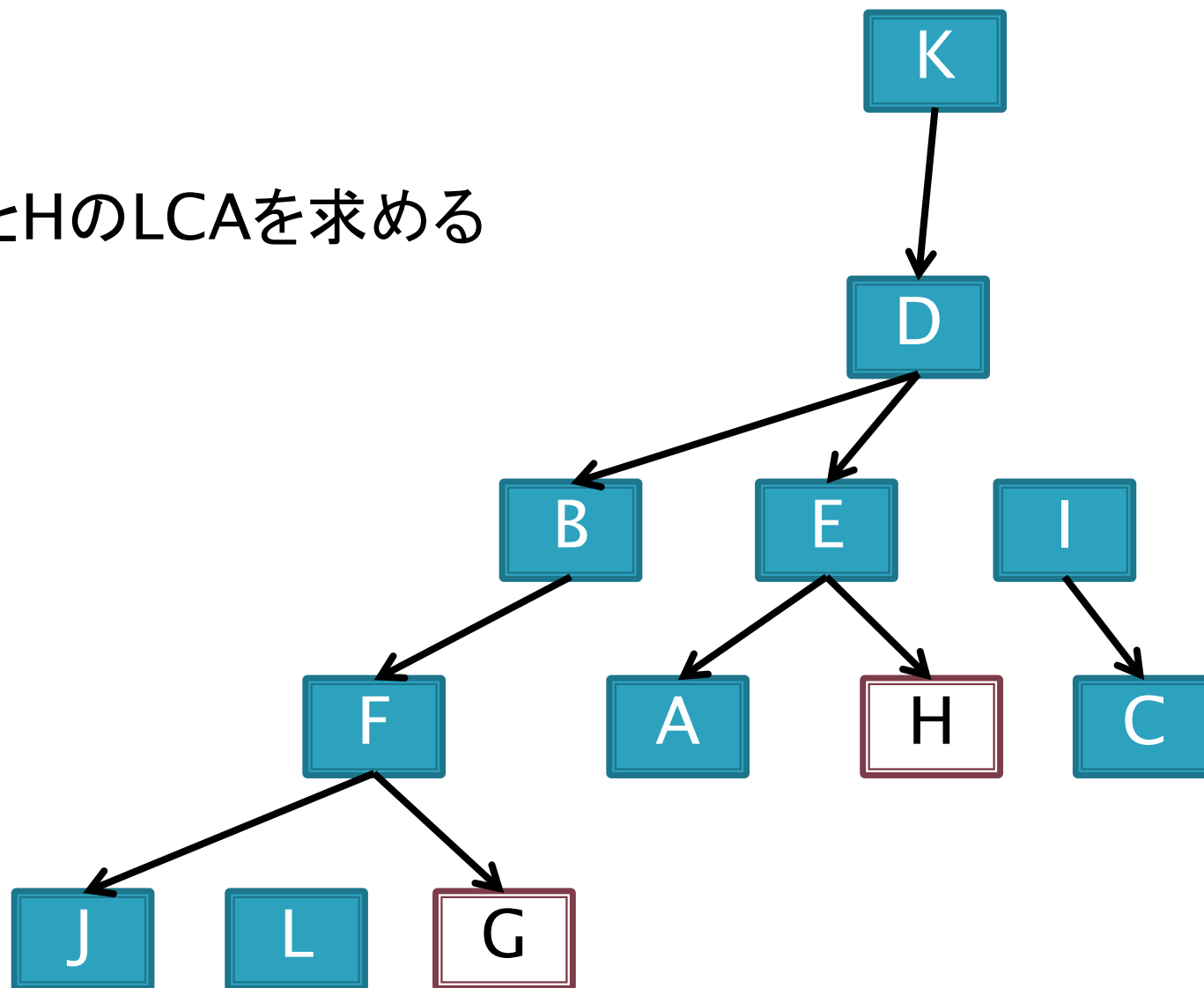
例

- ▶ 2. Lを親から切り離して根にする



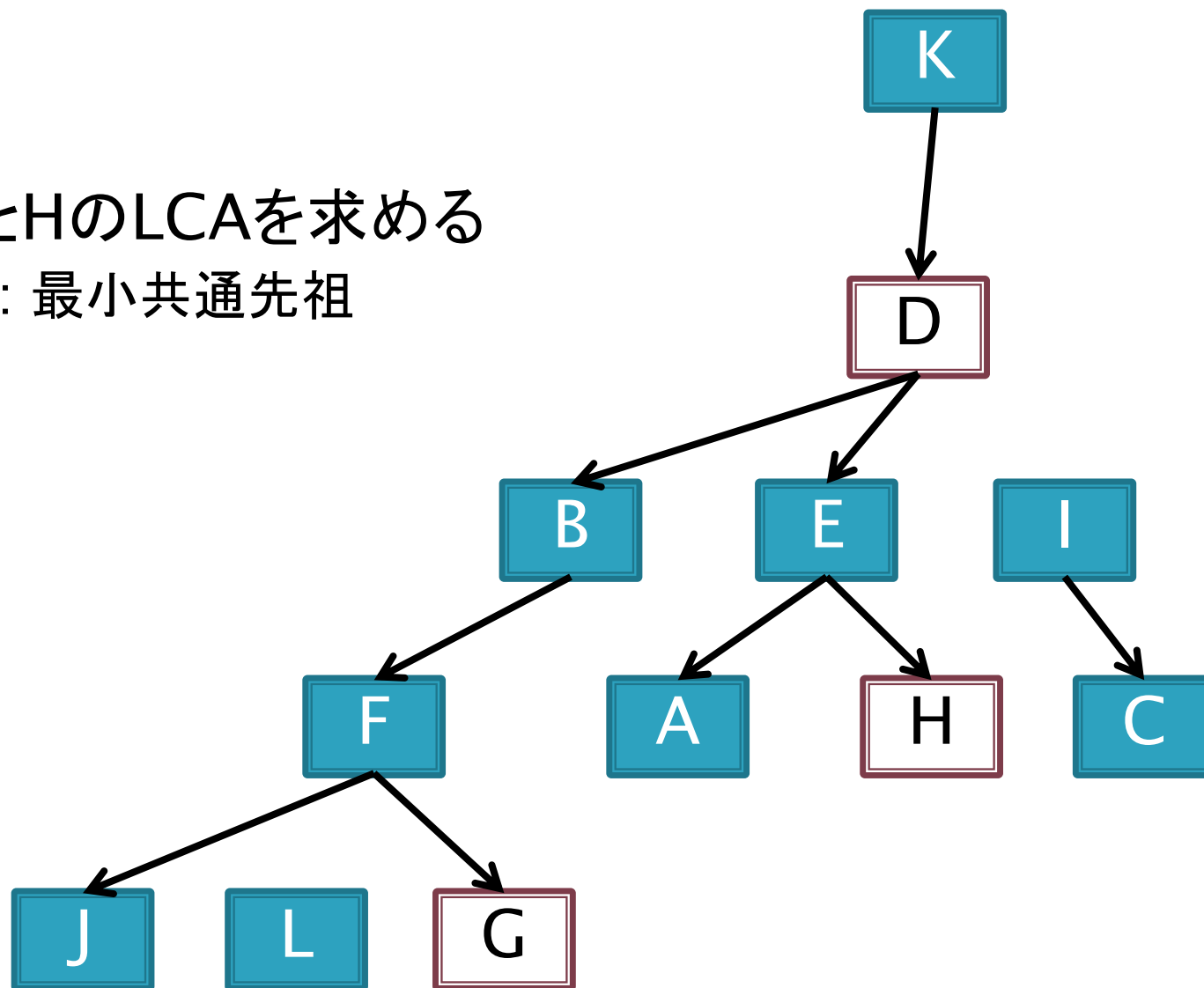
例

- ▶ 3. GとHのLCAを求める



例

- ▶ 3. GとHのLCAを求める
 - LCA: 最小共通先祖

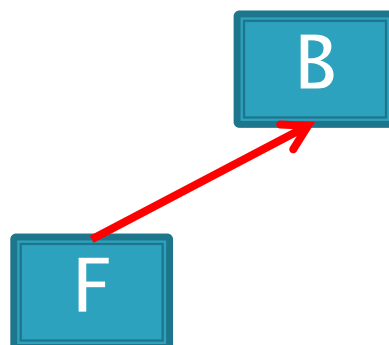


小課題1 (10点)

- ▶ 頂点数 $N \leq 5000$
- ▶ クエリ数 $Q \leq 5000$

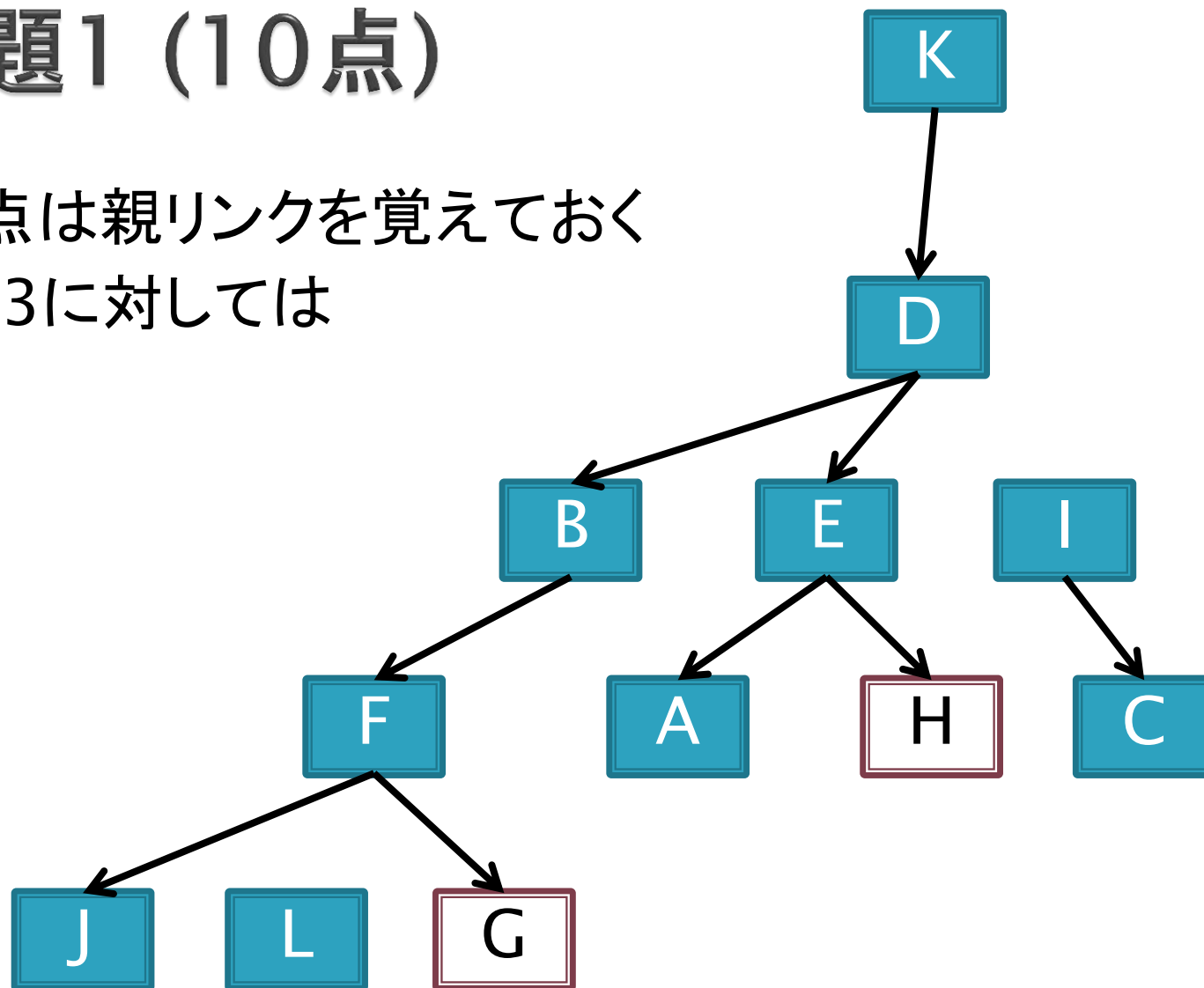
小課題1 (10点)

- ▶ 各頂点は親リンクを覚えておく
- ▶ クエリ1,2に対しては普通に答える



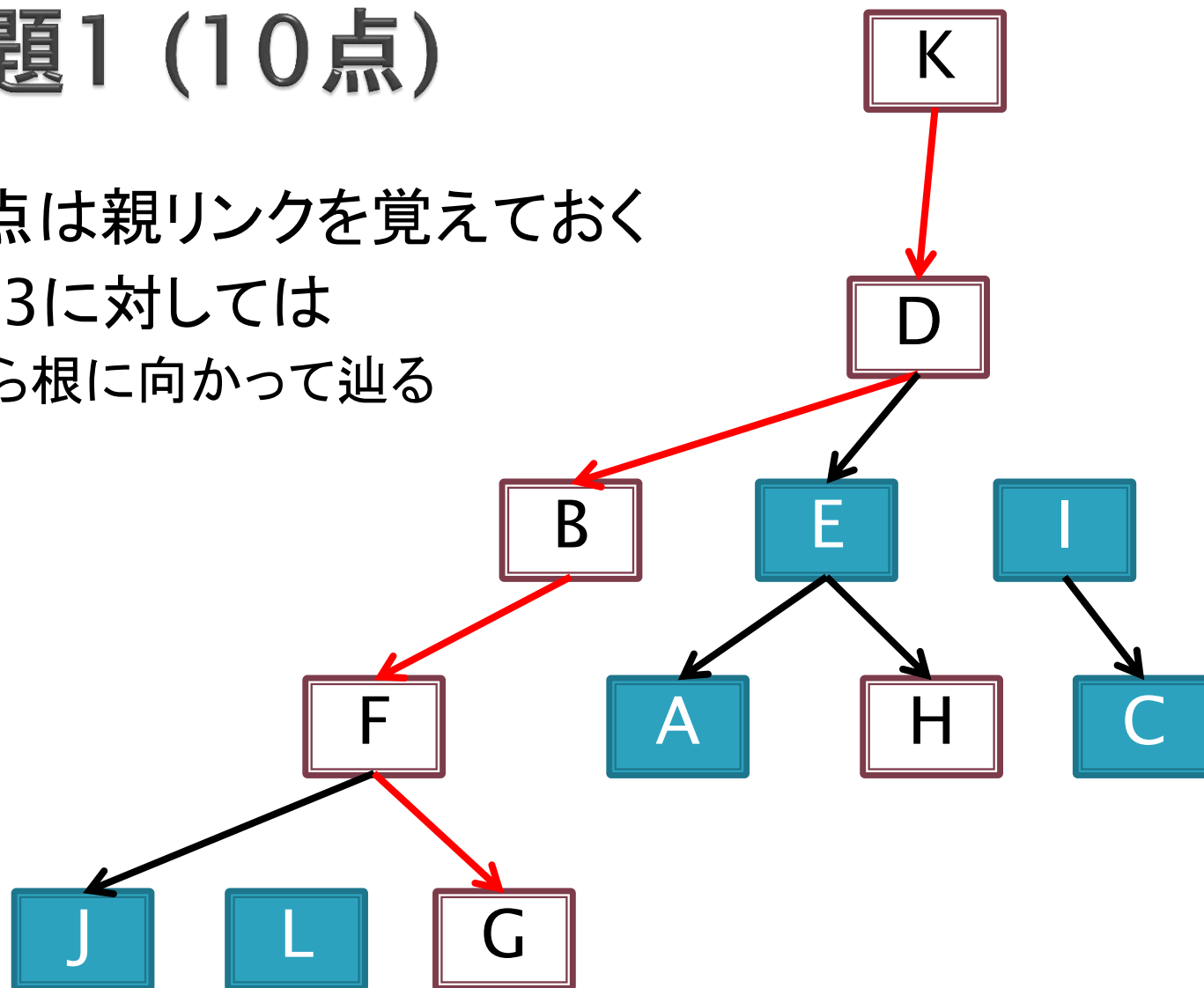
小課題1 (10点)

- ▶ 各頂点は親リンクを覚えておく
- ▶ クエリ3に対しては



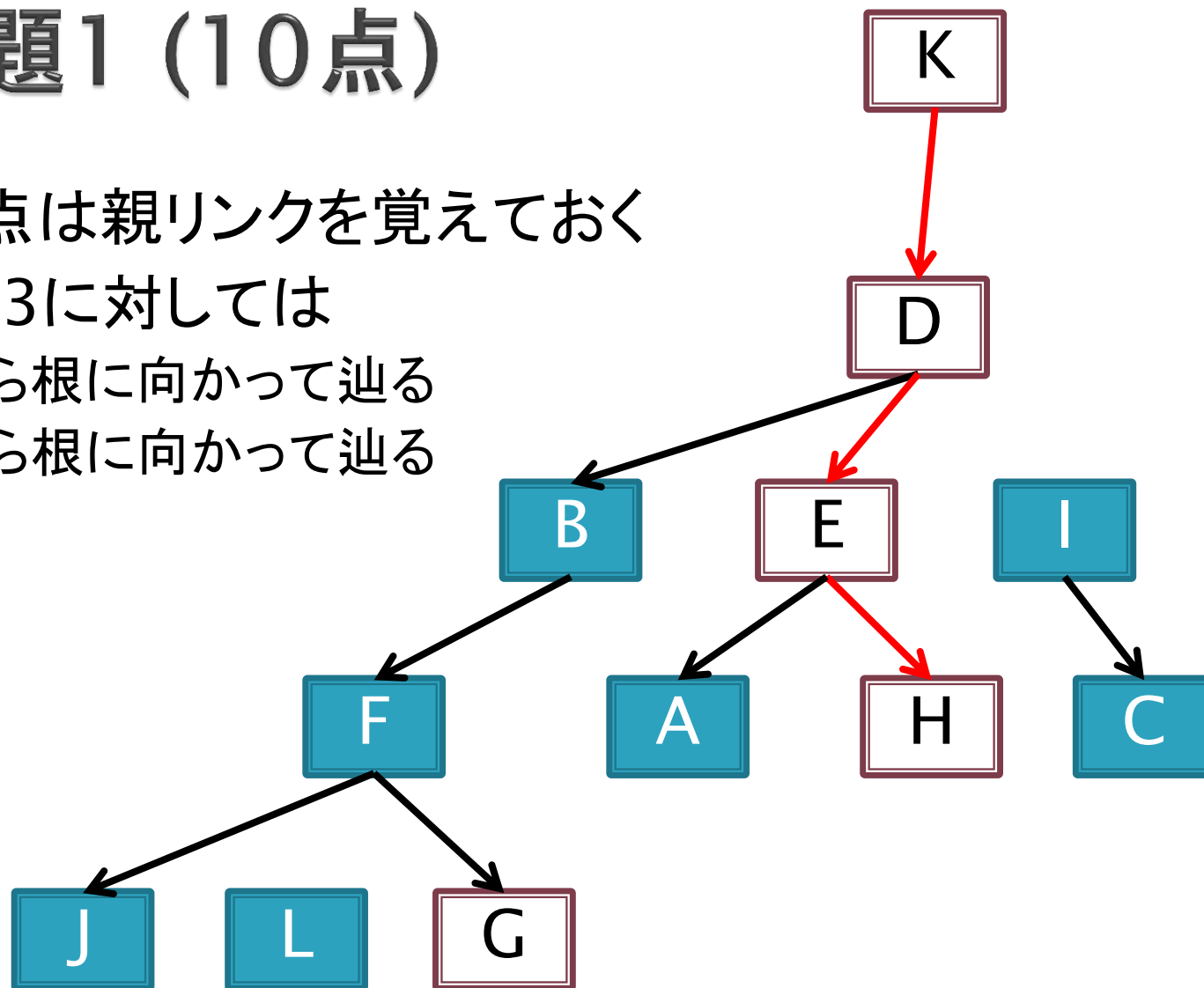
小課題1 (10点)

- ▶ 各頂点は親リンクを覚えておく
- ▶ クエリ3に対しては
 - Gから根に向かって辿る



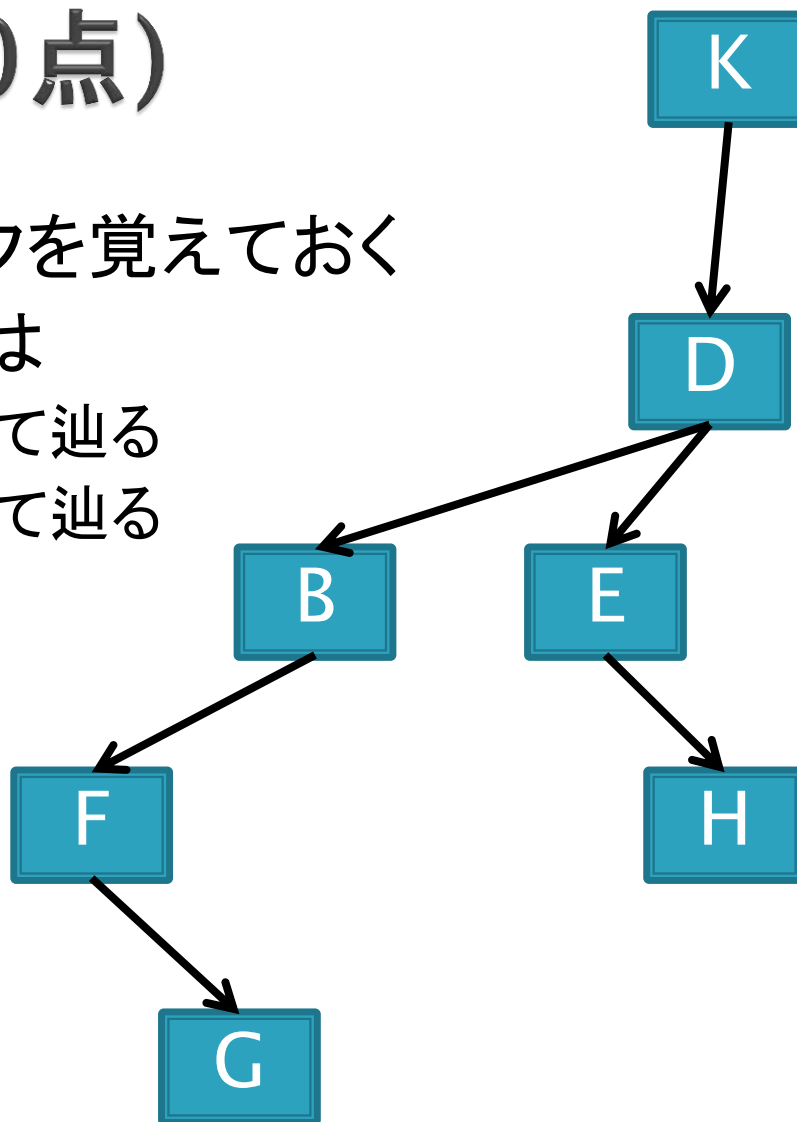
小課題1 (10点)

- ▶ 各頂点は親リンクを覚えておく
- ▶ クエリ3に対しては
 - Gから根に向かって辿る
 - Hから根に向かって辿る



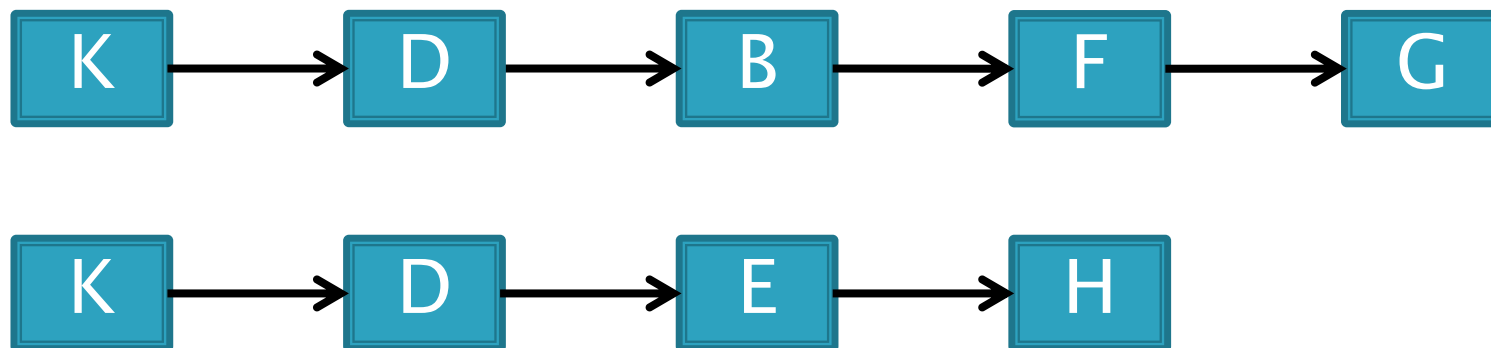
小課題1 (10点)

- ▶ 各頂点は親リンクを覚えておく
- ▶ クエリ3に対しては
 - Gから根に向かって辿る
 - Hから根に向かって辿る
 - 並べる



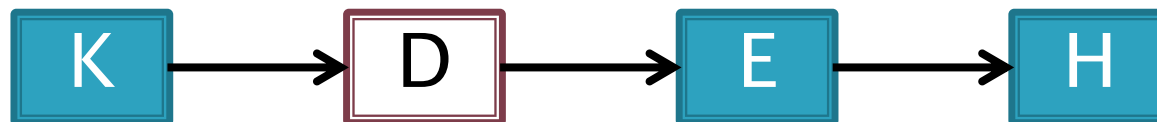
小課題1 (10点)

- ▶ 各頂点は親リンクを覚えておく
- ▶ クエリ3に対しては
 - Gから根に向かって辿る
 - Hから根に向かって辿る
 - 並べる



小課題1 (10点)

- ▶ 各頂点は親リンクを覚えておく
- ▶ クエリ3に対しては
 - Gから根に向かって辿る
 - Hから根に向かって辿る
 - 根からの順番で並べる
 - 一致する中で最も後ろのものを選ぶ



小課題1 (10点)

- ▶ 各頂点は親リンクを覚えておく
- ▶ クエリ3に対しては
 - Gから根に向かって辿る
 - Hから根に向かって辿る
 - 根からの順番で並べる
 - 根が一致しないときは-1

小課題1 (10点)

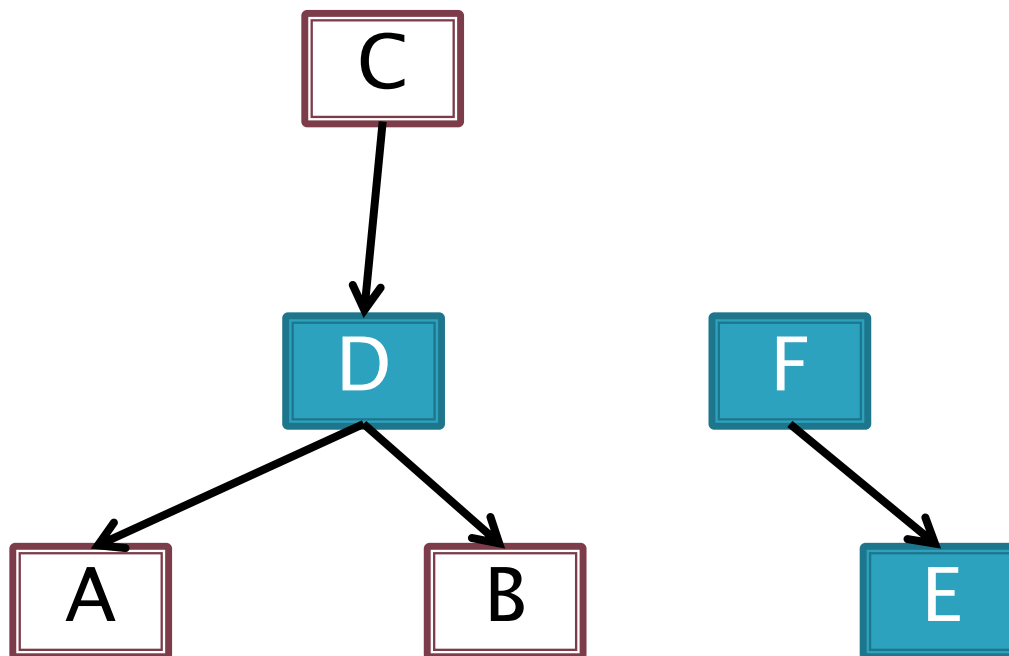
- ▶ 頂点数 $N \leq 5000$
- ▶ クエリ数 $Q \leq 5000$
- ▶ 計算量は $O(NQ)$ なので間に合う

小課題2 (30点)

- ▶ 頂点数 $N \leq 10^6$
- ▶ クエリ数 $Q \leq 10^6$
- ▶ 辺の削除は行われない

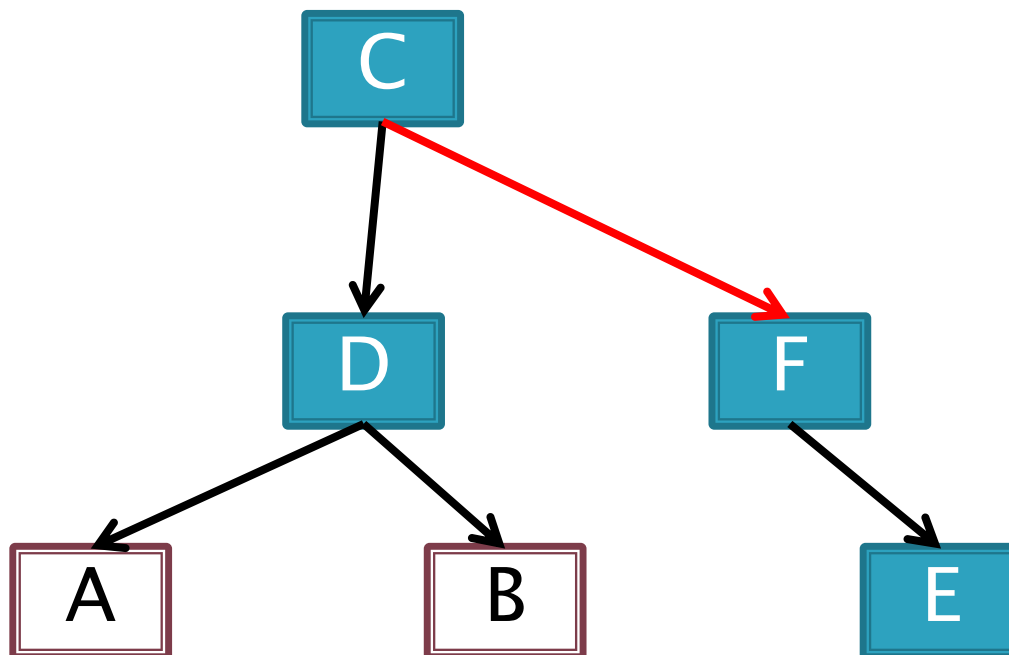
小課題2 (30点) – LCAならLCA

- ▶ CがAとBのLCA



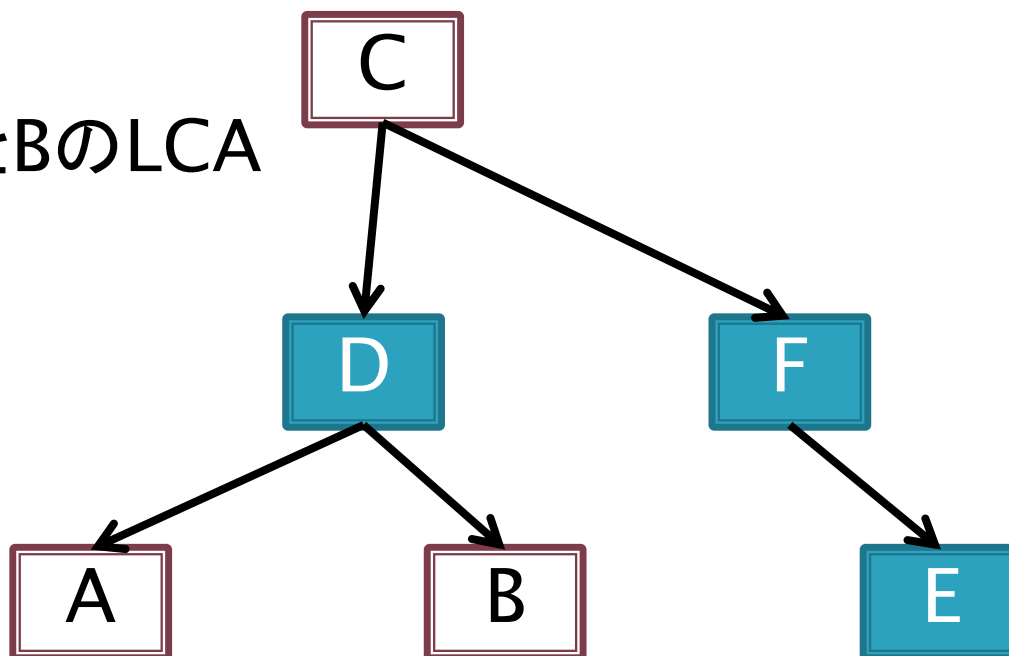
小課題2 (30点) – LCAならLCA

- ▶ CがAとBのLCA
↓辺を追加



小課題2 (30点) – LCAならLCA

- ▶ CがAとBのLCA
↓辺を追加
- ▶ Cは依然としてAとBのLCA



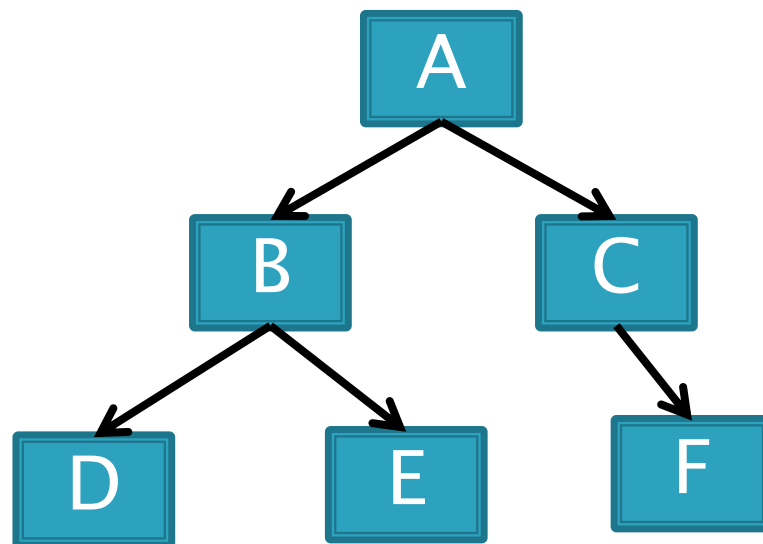
小課題2 (30点) – LCAならLCA

- ▶ CがAとBのLCA
↓辺を追加
- ▶ Cは依然としてAとBのLCA
- ▶ 最終的にできる木の上でLCAを計算できればよい

小課題2 (30点) – 1. LCAの計算

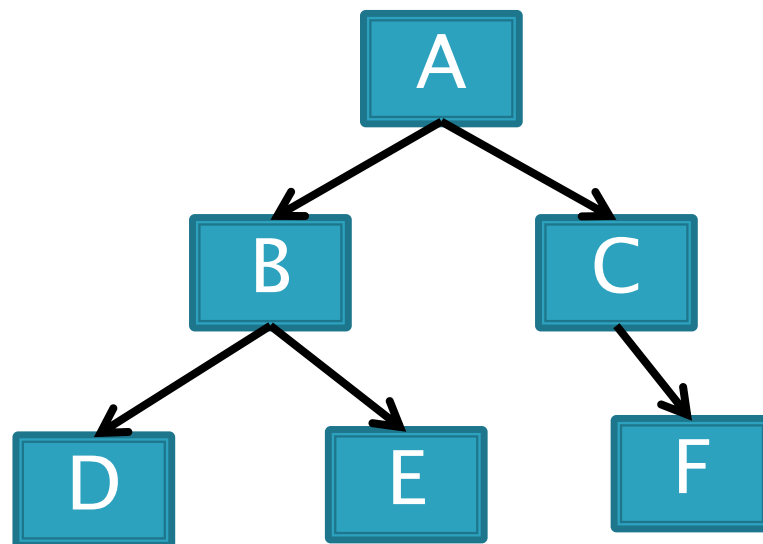
- ▶ 木の嬉しい順序(DFS順序)
 - Preorder –頂点に入るときに記録する順序

▶ A, B, D, E, C, F



小課題2 (30点) – 1. LCAの計算

- ▶ 木の嬉しい順序(DFS順序)
 - Postorder – 頂点から出るときに記録する順序
- ▶ D, E, B, F, C, A

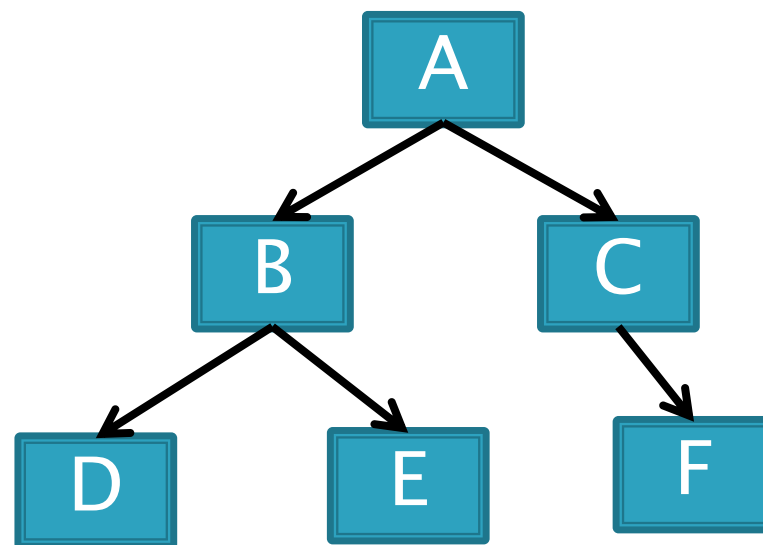


小課題2 (30点) – 1. LCAの計算

▶ 木の嬉しい順序(DFS順序)

◦ Euler Tour

- 頂点から入るときに記録し、
- 頂点から出るときにも
自分の親を記録する順序

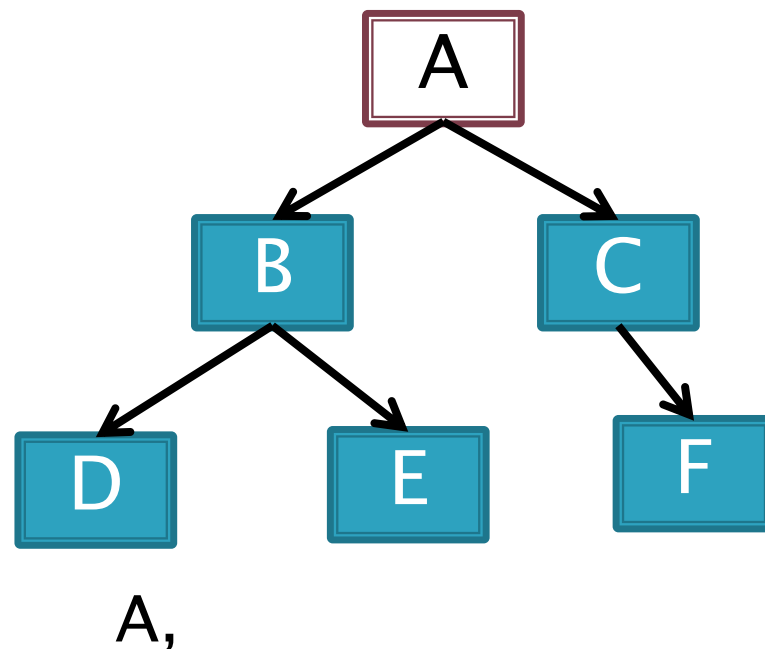


小課題2 (30点) – 1. LCAの計算

▶ 木の嬉しい順序(DFS順序)

◦ Euler Tour

- 頂点から入るときに記録し、
- 頂点から出るときにも
自分の親を記録する順序

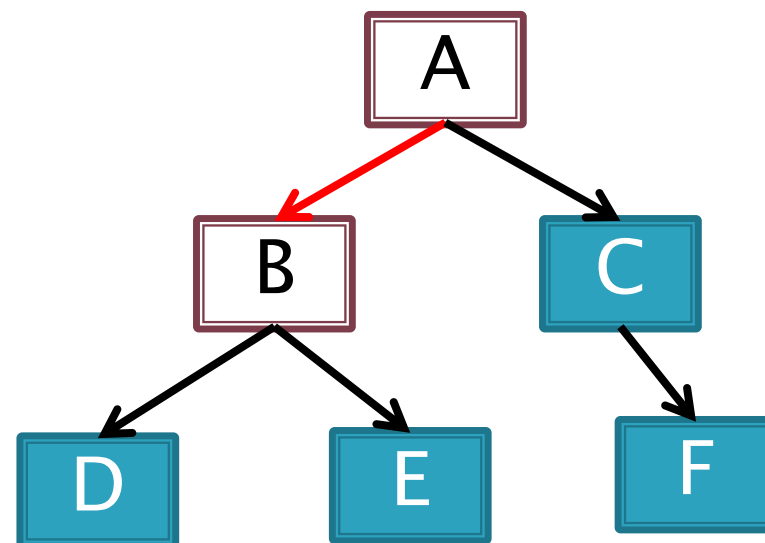


小課題2 (30点) – 1. LCAの計算

▶ 木の嬉しい順序(DFS順序)

◦ Euler Tour

- 頂点から入るときに記録し、
- 頂点から出るときにも
自分の親を記録する順序



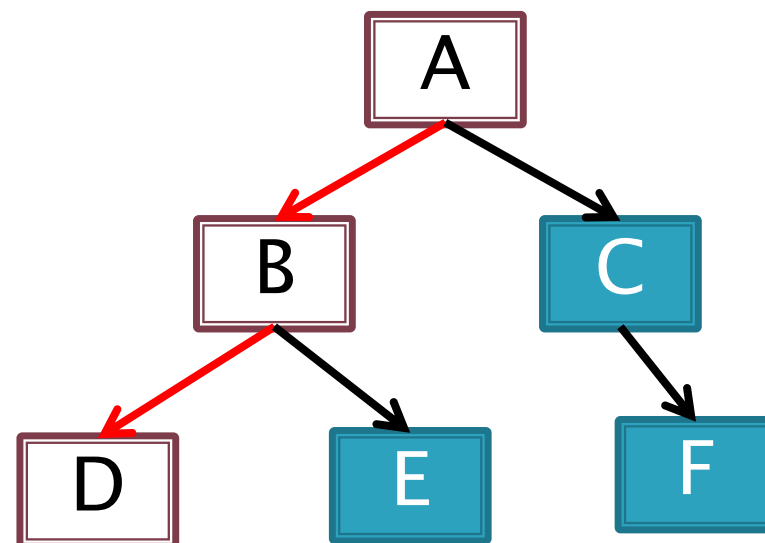
A, B,

小課題2 (30点) – 1. LCAの計算

▶ 木の嬉しい順序(DFS順序)

◦ Euler Tour

- 頂点から入るときに記録し、
- 頂点から出るときにも
自分の親を記録する順序



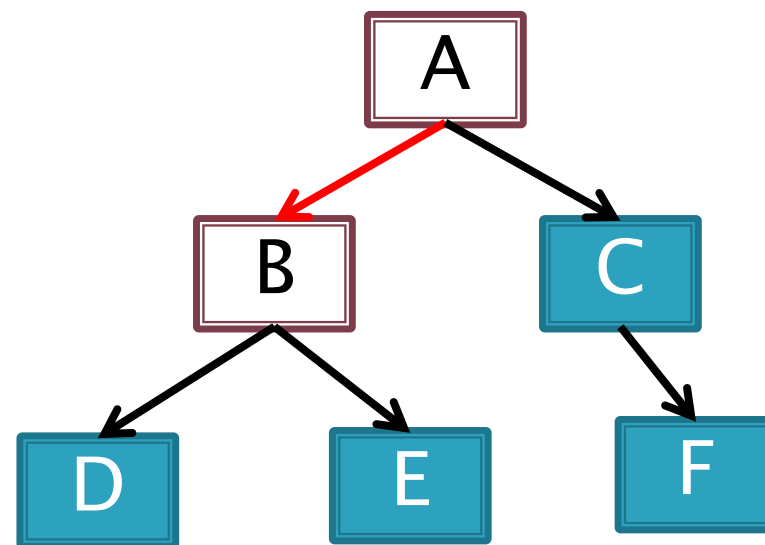
A, B, D,

小課題2 (30点) – 1. LCAの計算

▶ 木の嬉しい順序(DFS順序)

◦ Euler Tour

- 頂点から入るときに記録し、
- 頂点から出るときにも
自分の親を記録する順序



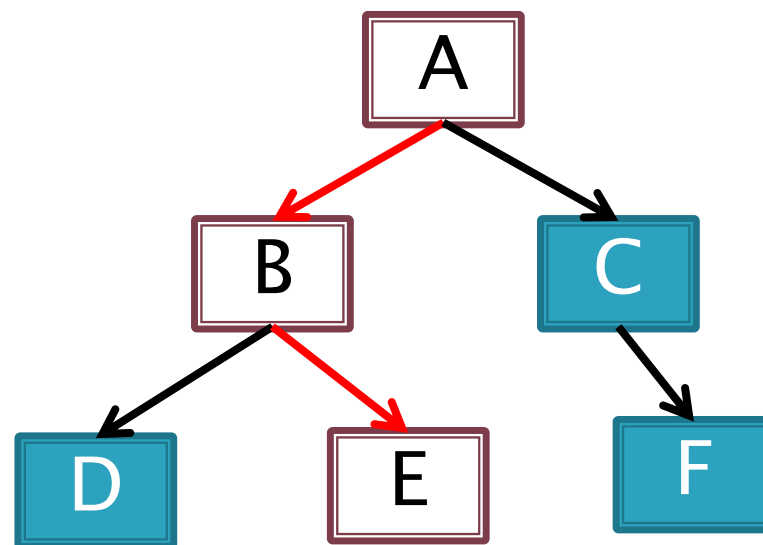
A, B, D, B,

小課題2 (30点) – 1. LCAの計算

▶ 木の嬉しい順序(DFS順序)

◦ Euler Tour

- 頂点から入るときに記録し、
- 頂点から出るときにも
自分の親を記録する順序



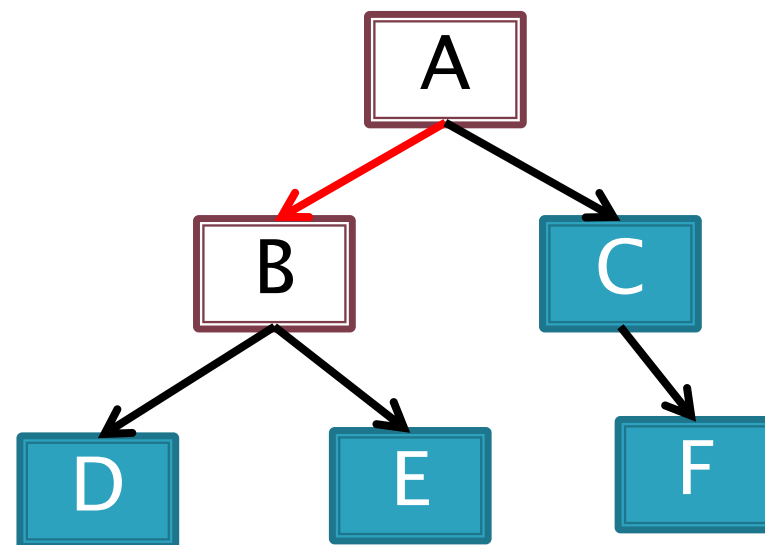
A, B, D, B, E

小課題2 (30点) – 1. LCAの計算

▶ 木の嬉しい順序(DFS順序)

◦ Euler Tour

- 頂点から入るときに記録し、
- 頂点から出るときにも
自分の親を記録する順序



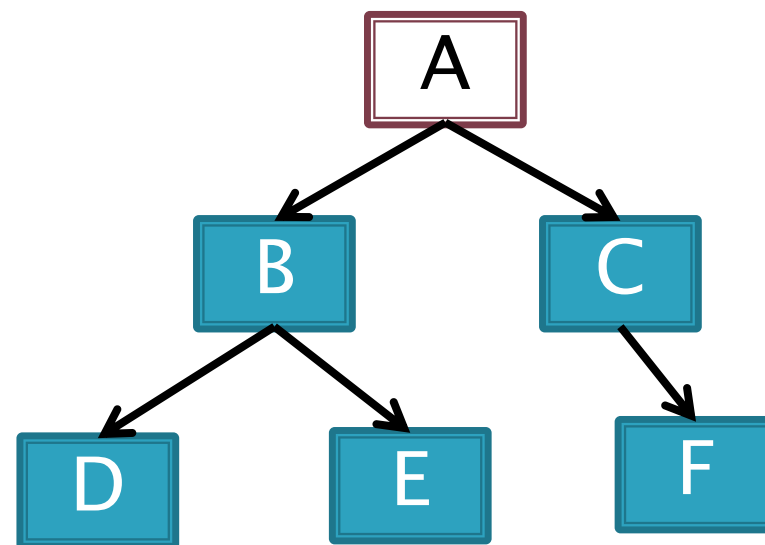
A, B, D, B, E, B,

小課題2 (30点) – 1. LCAの計算

▶ 木の嬉しい順序(DFS順序)

◦ Euler Tour

- 頂点から入るときに記録し、
- 頂点から出るときにも
自分の親を記録する順序



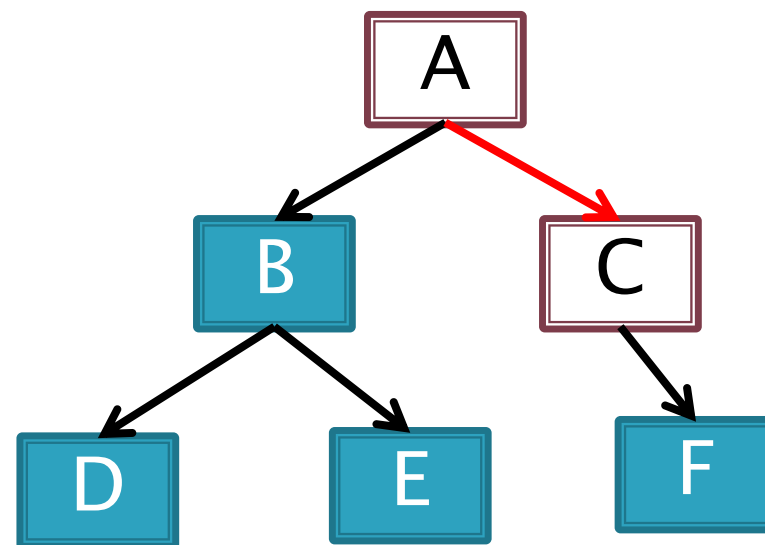
A, B, D, B, E, B, A,

小課題2 (30点) – 1. LCAの計算

▶ 木の嬉しい順序(DFS順序)

◦ Euler Tour

- 頂点から入るときに記録し、
- 頂点から出るときにも
自分の親を記録する順序



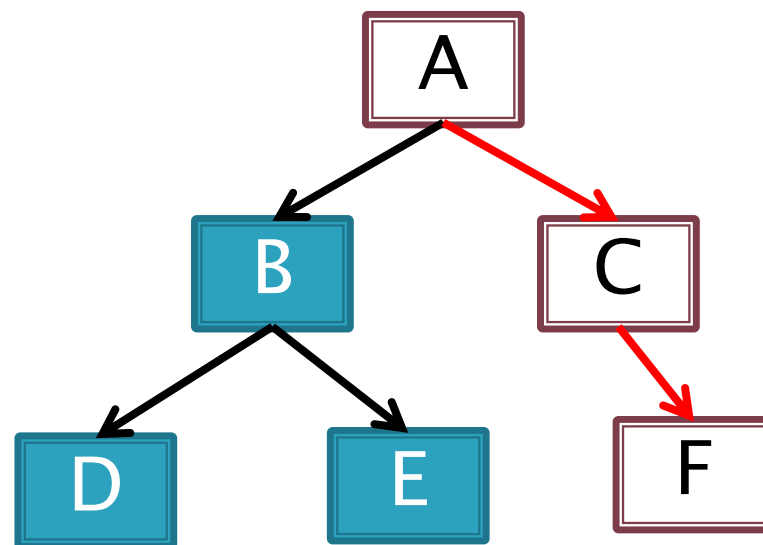
A, B, D, B, E, B, A,
C,

小課題2 (30点) – 1. LCAの計算

▶ 木の嬉しい順序(DFS順序)

◦ Euler Tour

- 頂点から入るときに記録し、
- 頂点から出るときにも
自分の親を記録する順序



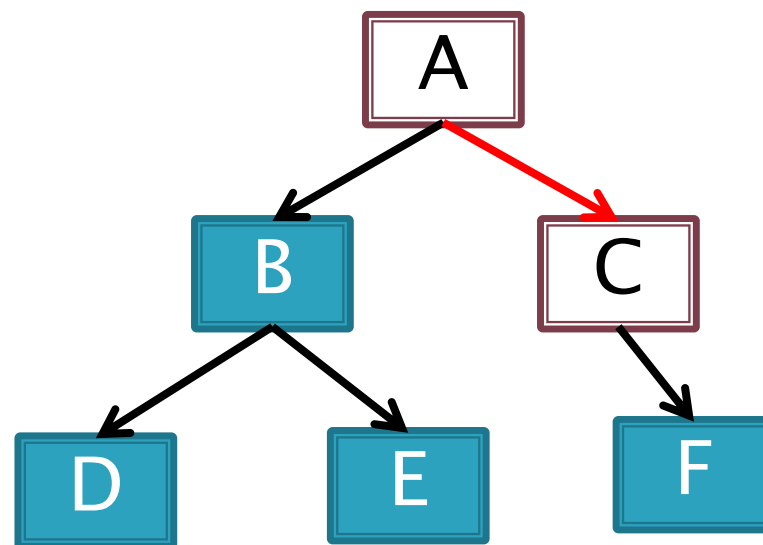
A, B, D, B, E, B, A,
C, F,

小課題2 (30点) – 1. LCAの計算

▶ 木の嬉しい順序(DFS順序)

◦ Euler Tour

- 頂点から入るときに記録し、
- 頂点から出るときにも
自分の親を記録する順序



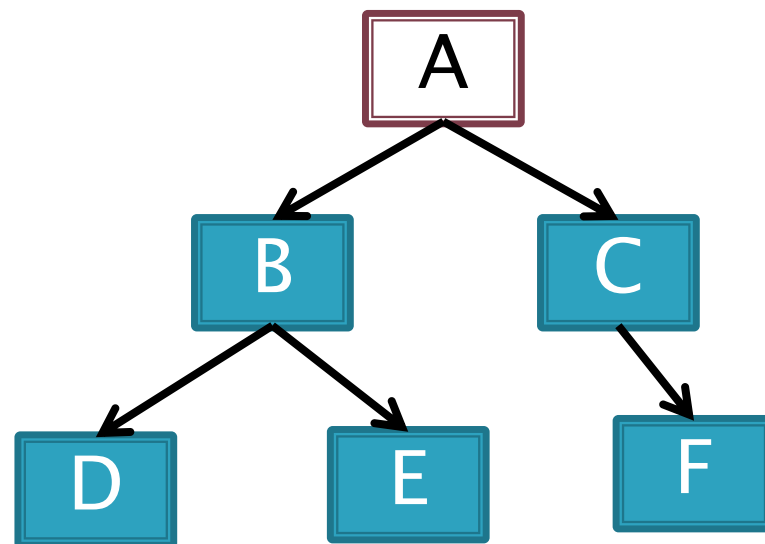
A, B, D, B, E, B, A,
C, F, C,

小課題2 (30点) – 1. LCAの計算

▶ 木の嬉しい順序(DFS順序)

◦ Euler Tour

- 頂点から入るときに記録し、
- 頂点から出るときにも
自分の親を記録する順序



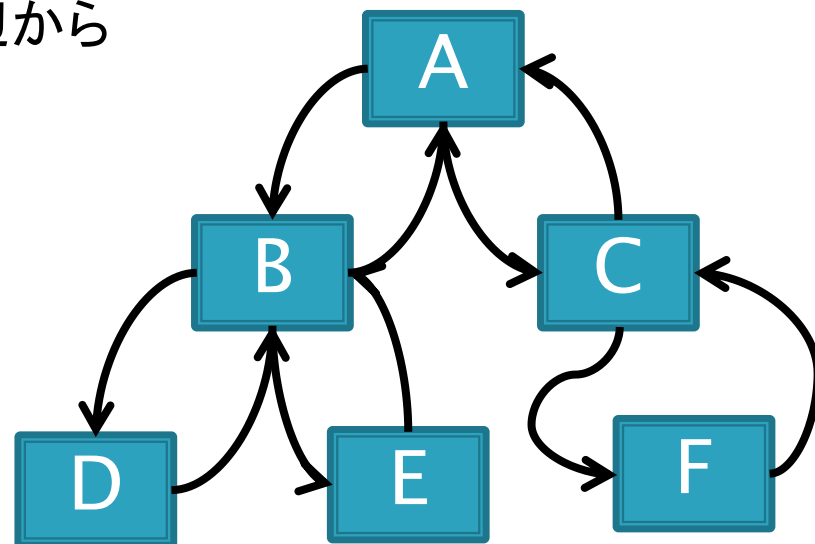
A, B, D, B, E, B, A,
C, F, C, A

小課題2 (30点) – 1. LCAの計算

▶ 木の嬉しい順序(DFS順序)

◦ Euler Tour

- 木の辺を、行きと帰りの2つの辺からなるとみなすときのオイラー閉路に対応する



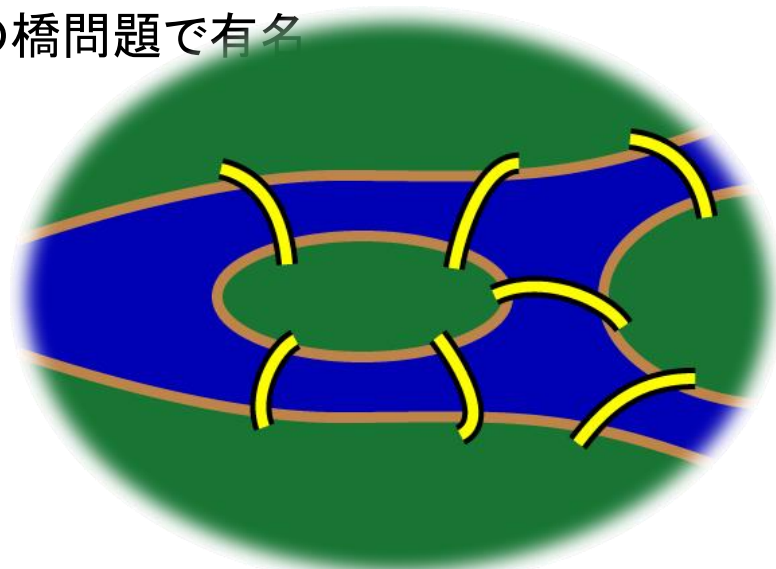
A, B, D, B, E, B, A,
C, F, C, A

小課題2 (30点) – 1. LCAの計算

▶ 木の嬉しい順序(DFS順序)

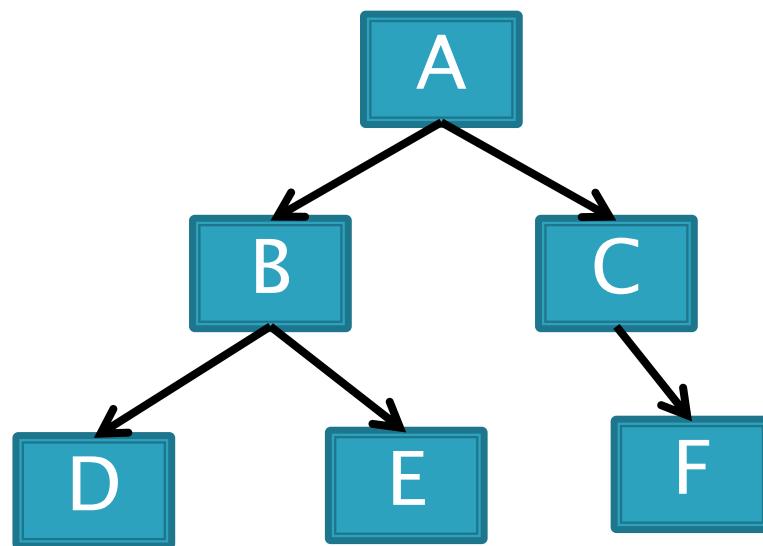
◦ Euler Tour

- 木の辺を、行きと帰りの2つの辺からなるとみなすときのオイラー閉路に対応する
- オイラー閉路 (Euler Tour) : 全ての辺を1度ずつ通る閉路
 - オイラー路はケーニヒスベルクの橋問題で有名



小課題2 (30点) – 1. LCAの計算

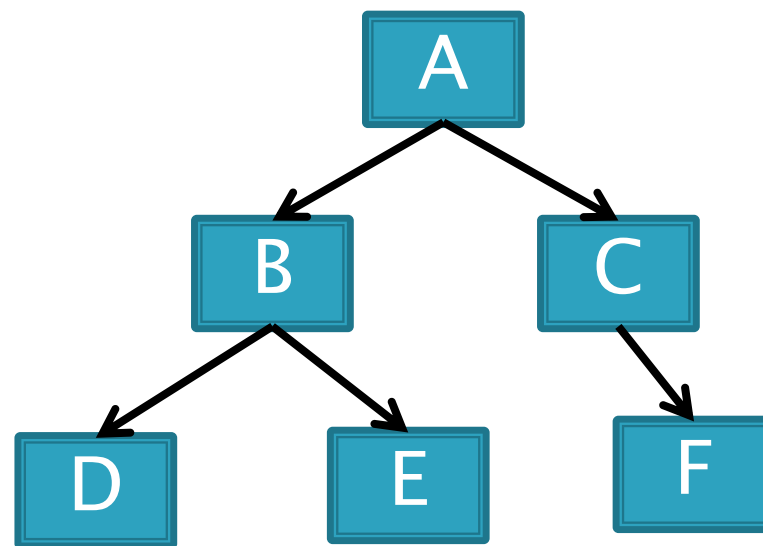
- ▶ Euler TourによるLCAの計算



小課題2 (30点) – 1. LCAの計算

▶ Euler TourによるLCAの計算

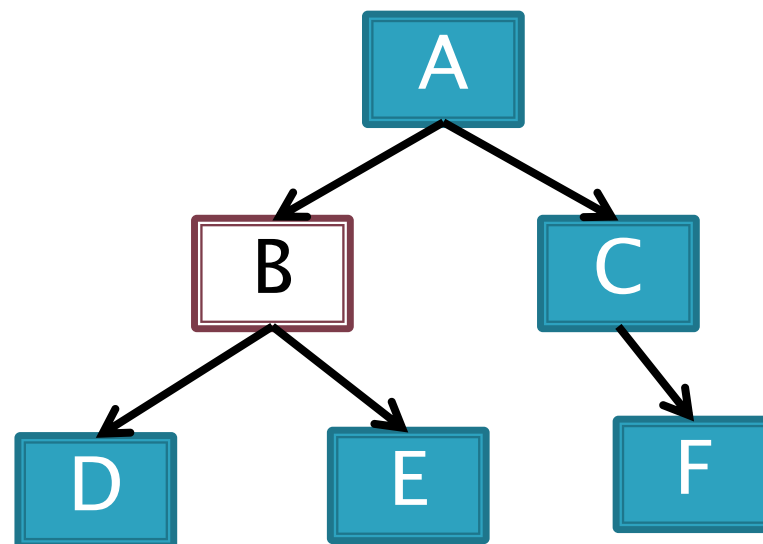
	A	B	D	B	E	B	A	C	F	C	A
	1	2	3	2	3	2	1	2	3	2	1



小課題2 (30点) – 1. LCAの計算

▶ Euler TourによるLCAの計算

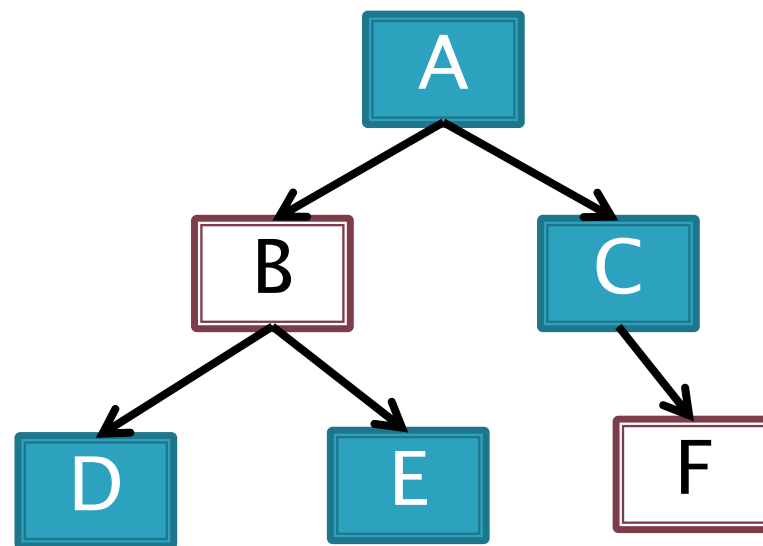
	A	B	D	B	E	B	A	C	F	C	A
	1	2	3	2	3	2	1	2	3	2	1



小課題2 (30点) – 1. LCAの計算

▶ Euler TourによるLCAの計算

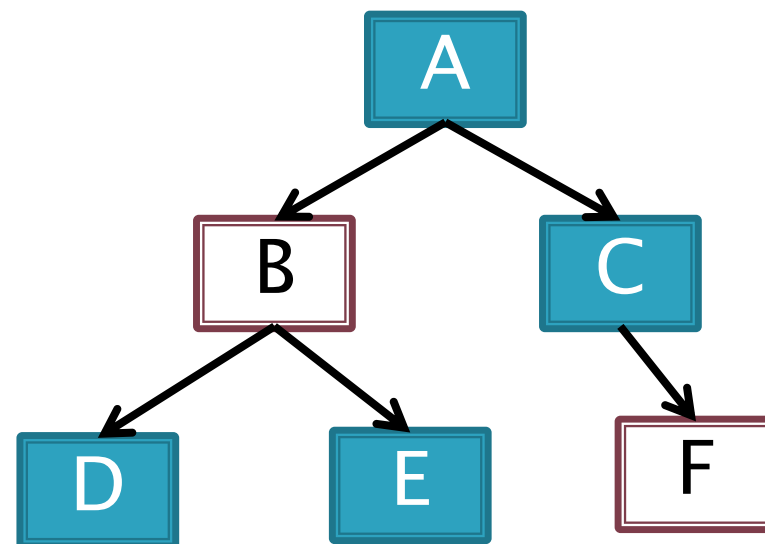
	A	B	D	B	E	B	A	C	F	C	A
	1	2	3	2	3	2	1	2	3	2	1



小課題2 (30点) – 1. LCAの計算

▶ Euler TourによるLCAの計算

	A	B	D	B	E	B	A	C	F	C	A
	1	2	3	2	3	2	1	2	3	2	1

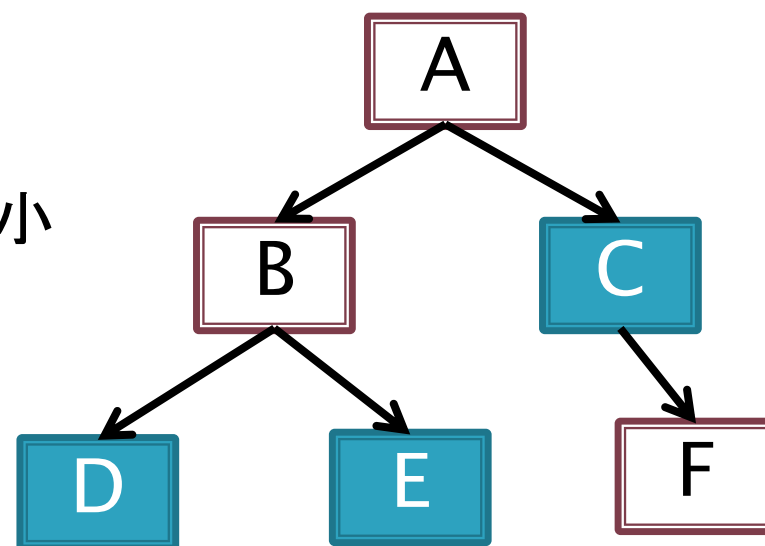


小課題2 (30点) – 1. LCAの計算

▶ Euler TourによるLCAの計算

	A	B	D	B	E	B	A	C	F	C	A
	1	2	3	2	3	2	1	2	3	2	1

↑
深さ最小



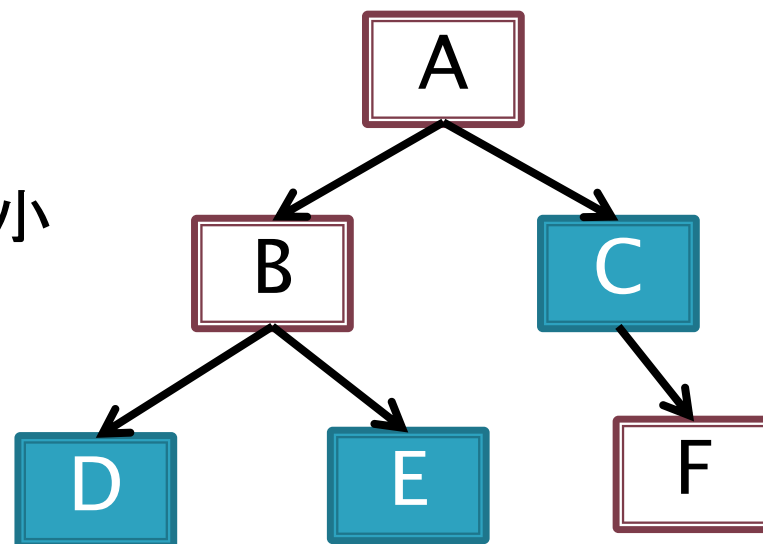
小課題2 (30点) – 1. LCAの計算

▶ Euler TourによるLCAの計算

	A	B	D	B	E	B	A	C	F	C	A
	1	2	3	2	3	2	1	2	3	2	1

▶ RMQを利用

↑
深さ最小



小課題2 (30点) – 1. LCAの計算

- ▶ 正当性

小課題2 (30点) – 1. LCAの計算

- ▶ 正当性: C が A と B のLCAのとき
 - Euler Tour上で C は $[A, B]$ に含まれる
 - Euler Tour上で $[A, B]$ に含まれるのは C の部分木
- ▶ を言えばよい

小課題2 (30点) – 1. LCAの計算

- ▶ 正当性(1): Euler Tour上でCは[A,B]に含まれる
 - AからBに行くにはCを経由しないといけない(LCAの性質より)
 - ので当たり前

小課題2 (30点) – 1. LCAの計算

- ▶ 正当性(2): Euler Tour上で[A,B]に含まれるのはCの部分木
 - Euler Tourにおいて部分木は連続した部分列として現れる
 - ので当たり前

小課題2 (30点)

- ▶ LCAを求めて終わり？

小課題2 (30点)

- ▶ LCAを求めて終わり？
 - あと少しだけやる必要があります

小課題2 (30点) – 2. 連結性

- ▶ 「削除がない場合の利点」の考察を思い出す

小課題2 (30点) – 2. 連結性

- ▶ 「削除がない場合の利点」の考察を思い出す
- ▶ LCAが存在するなら、最終的な木の上で計算すればよい

小課題2 (30点) – 2. 連結性

- ▶ 「削除がない場合の利点」の考察を思い出す
- ▶ LCAが存在するなら、最終的な木の上で計算すればよい

小課題2 (30点) – 2. 連結性

- ▶ A, Bが同じ木上にあるかどうかの判定が必要

小課題2 (30点) – 2. 連結性

- ▶ A, Bが同じ木上にあるかどうかの判定が必要ですが

小課題2 (30点) – 2. 連結性

- ▶ A, Bが同じ木上にあるかどうかの判定が必要
ですが

辺の追加クエリしかないのでUnionFindでよい
ということはすぐにわかると思います

小課題2 (30点)

- ▶ 頂点数 $N \leq 10^6$
- ▶ クエリ数 $Q \leq 10^6$
- ▶ 辺の削除は行われない
- ▶ $O(N + Q \log N)$ なので間に合う

小課題1, 2

- ▶ ここまでの両方を実装すれば40点

小課題1, 2

- ▶ ここまでの両方を実装すれば40点
- ▶ 複数のアルゴリズムを条件によって使い分けるテクはさすがに使っていると思います

```
if (N <= 5000) {<br>    solve1(N, Q, T, A, B);<br>} else {<br>    solve2(N, Q, T, A, B);<br>}<br>
```

小課題3 (60点)

- ▶ 頂点数 $N \leq 10^6$
- ▶ クエリ数 $Q \leq 10^6$

小課題3 (60点)

- ▶ 頂点数 $N \leq 10^6$
- ▶ クエリ数 $Q \leq 10^6$
- ▶ 削除クエリもある

小課題3 (60点)

- ▶ 追加も削除もある場合の頻出テク

小課題3 (60点)

- ▶ 追加も削除もある場合の頻出テク
 - クエリの(平方)分割
 - がんばって動的になんとかする

小課題3 (60点)

- ▶ 追加も削除もある場合の頻出テク
 - がんばって動的になんとかする

小課題3 (60点)

- ▶ 追加も削除もある場合の頻出テク
 - がんばって動的になんとかする

小課題3 (60点) – 動的Euler Tour

- ▶ 木が静的な場合のLCA (復習)

小課題3 (60点) – 動的Euler Tour

- ▶ 木が静的な場合のLCA (復習)
 - Euler Tour上で必要とされるクエリは以下の通り

小課題3 (60点) – 動的Euler Tour

- ▶ 木が静的な場合のLCA (復習)
 - Euler Tour上で必要とされるクエリは以下の通り
 1. 区間の最小値をとる

小課題3 (60点) – 動的Euler Tour

- ▶ 木が静的な場合のLCA (復習)
 - Euler Tour上で必要とされるクエリは以下の通り
 1. 区間の最小値をとる
 - RMQで実現可能

小課題3 (60点) – 動的Euler Tour

- ▶ 木が動的な場合のLCA (絶望)
 - Euler Tour上で必要とされるクエリは以下の通り
 1. 区間の最小値をとる
 - 2.
 - RMQで実現可能な気がする

小課題3 (60点) – 動的Euler Tour

- ▶ 木が動的な場合のLCA (絶望)
 - Euler Tour上で必要とされるクエリは以下の通り
 1. 区間の最小値をとる
 2. 列の連結をする
 - 3.
 - RMQで実現可能な気がする

小課題3 (60点) – 動的Euler Tour

- ▶ 木が動的な場合のLCA (絶望)
 - Euler Tour上で必要とされるクエリは以下の通り
 1. 区間の最小値をとる
 2. 列の連結をする
 3. 列の分割をする
 - 4.
 - RMQで実現可能な気がする

小課題3 (60点) – 動的Euler Tour

- ▶ 木が動的な場合のLCA (絶望)
 - Euler Tour上で必要とされるクエリは以下の通り
 1. 区間の最小値をとる
 2. 列の連結をする
 3. 列の分割をする
 4. それだけ?

小課題3 (60点) – 動的Euler Tour

- ▶ 木が動的な場合のLCA (絶望)
 - Euler Tour上で必要とされるクエリは以下の通り
 1. 区間の最小値をとる
 2. 列の連結をする
 3. 列の分割をする
 4. 区間に値を足す

小課題3 (60点) – 動的Euler Tour

- ▶ 木が動的な場合のLCA (絶望)
 - Euler Tour上で必要とされるクエリは以下の通り
 1. 区間の最小値をとる
 2. 列の連結をする
 3. 列の分割をする
 4. 区間に値を足す
 - この業界では「Starry Sky木」として知られているもの

小課題3 (60点) – 動的Euler Tour

- ▶ 木が動的な場合のLCA (絶望)
 - Euler Tour上で必要とされるクエリは以下の通り
 1. 区間の最小値をとる
 2. 列の連結をする
 3. 列の分割をする
 4. 区間に値を足す
 - この業界では「Starry Sky木」として知られているものを平衡二分木として実装する必要がある (絶望)

小課題3 (60点) – 動的Euler Tour

- ▶ 木が動的な場合のLCA (絶望)
 - Euler Tour上で必要とされるクエリは以下の通り
 1. 区間の最小値をとる
 2. 列の連結をする
 3. 列の分割をする
 4. 区間に値を足す
 - この業界では「Starry Sky木」として知られているものを平衡二分木として実装する必要がある (絶望)
 - しかもmerge/splitベースで

小課題3 (60点) – 動的Euler Tour

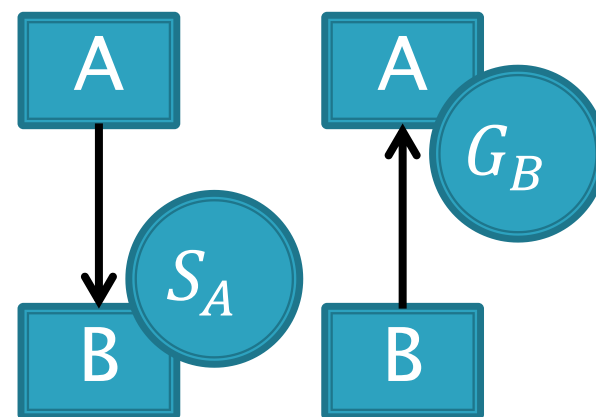
- ▶ 平衡二分木の中身は後回し

小課題3 (60点) – 動的Euler Tour

- ▶ 平衡二分木の中身は後回し
- ▶ 平衡二分木を使った具体的な実装方法

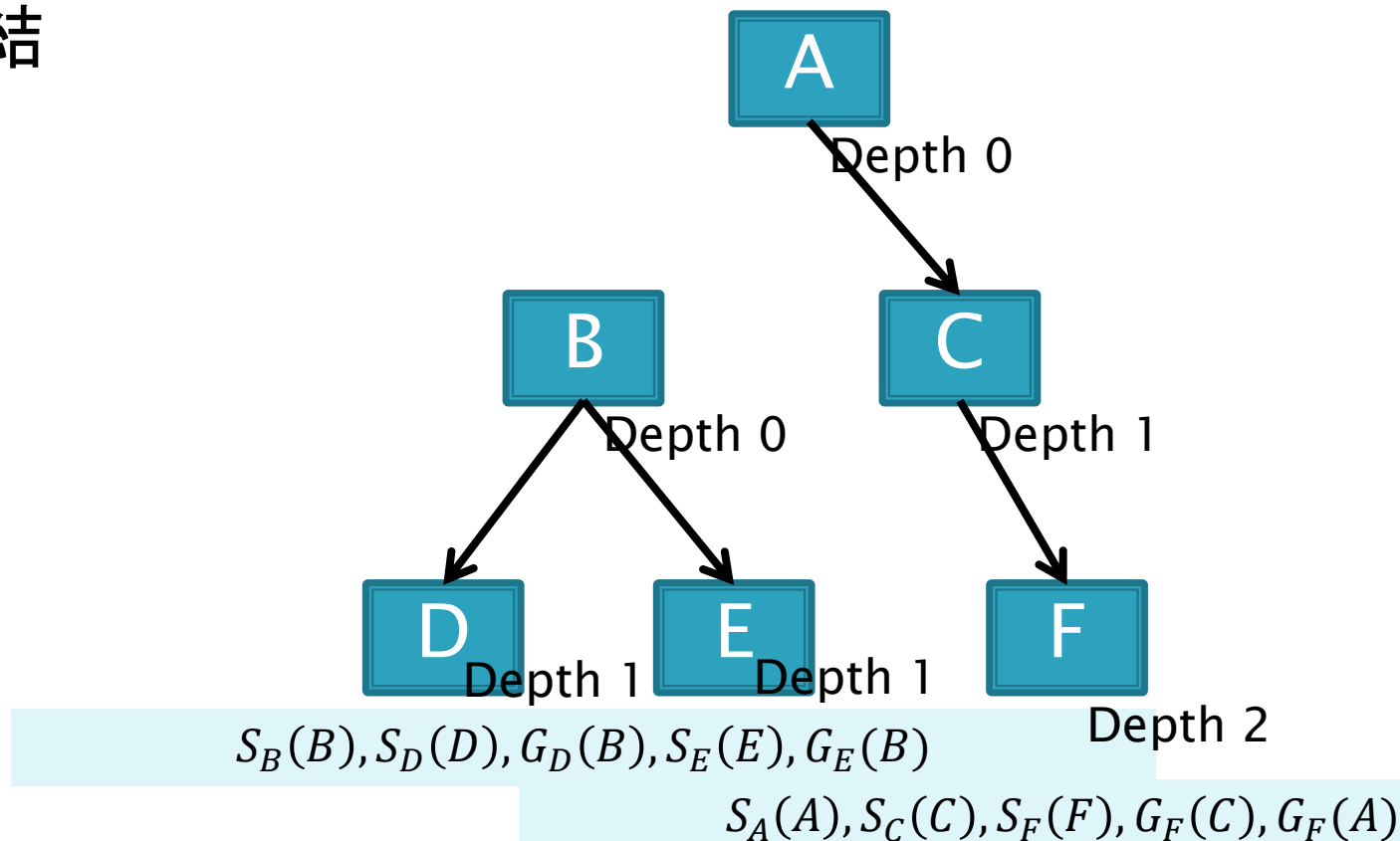
小課題3 (60点) – 動的Euler Tour

- ▶ 木の各頂点ごとに、Euler Tourのためのノードを2つ用意する(S_A, G_A)
 - S_A 上には頂点Aの番号と、その深さが記録されている
 - G_A 上には頂点Aの親Pの番号と、その深さが記録されている
 - Aが根のときは S_A のみ使う



小課題3 (60点) – 動的Euler Tour

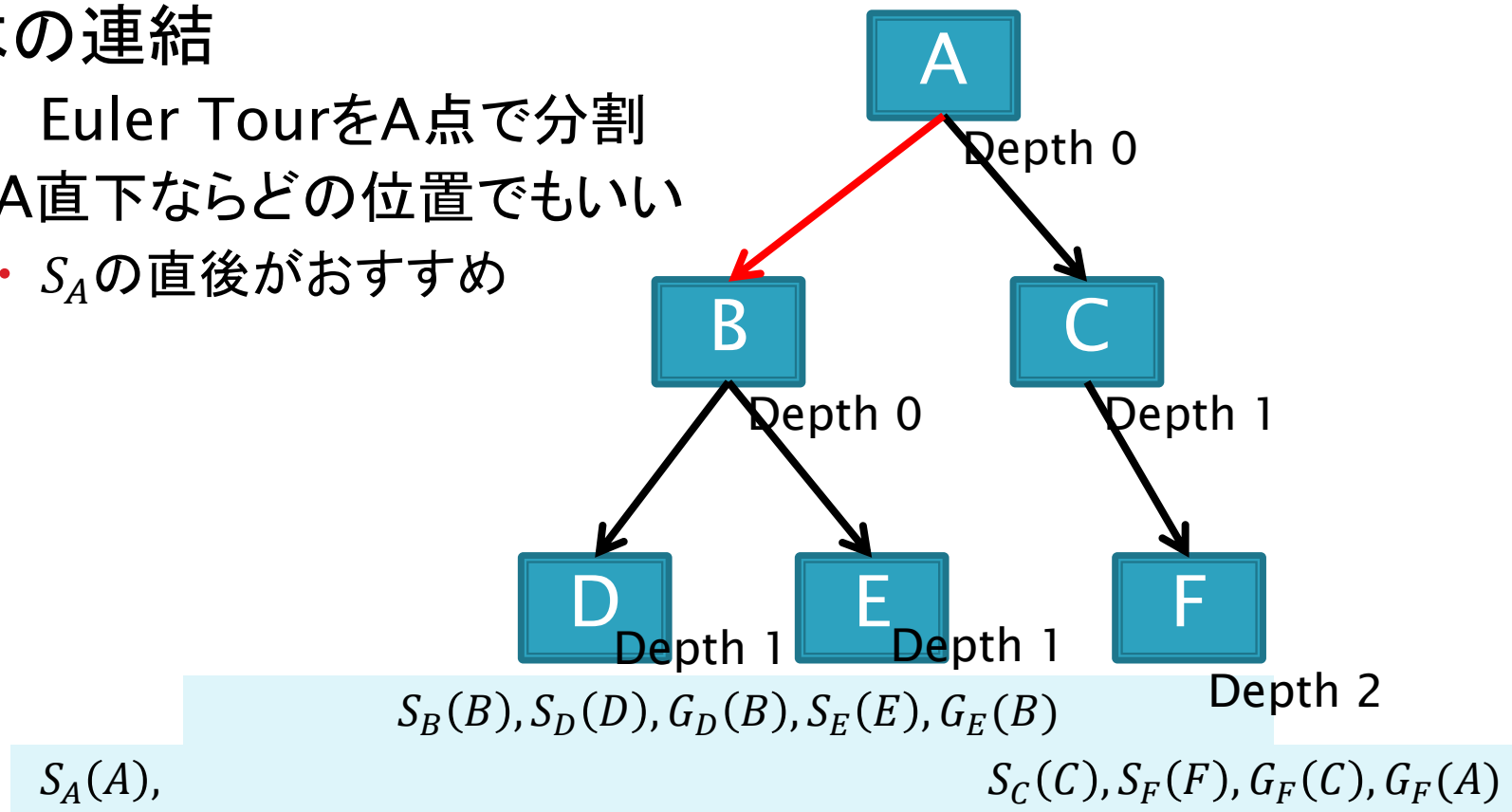
木の連結



小課題3 (60点) – 動的Euler Tour

木の連結

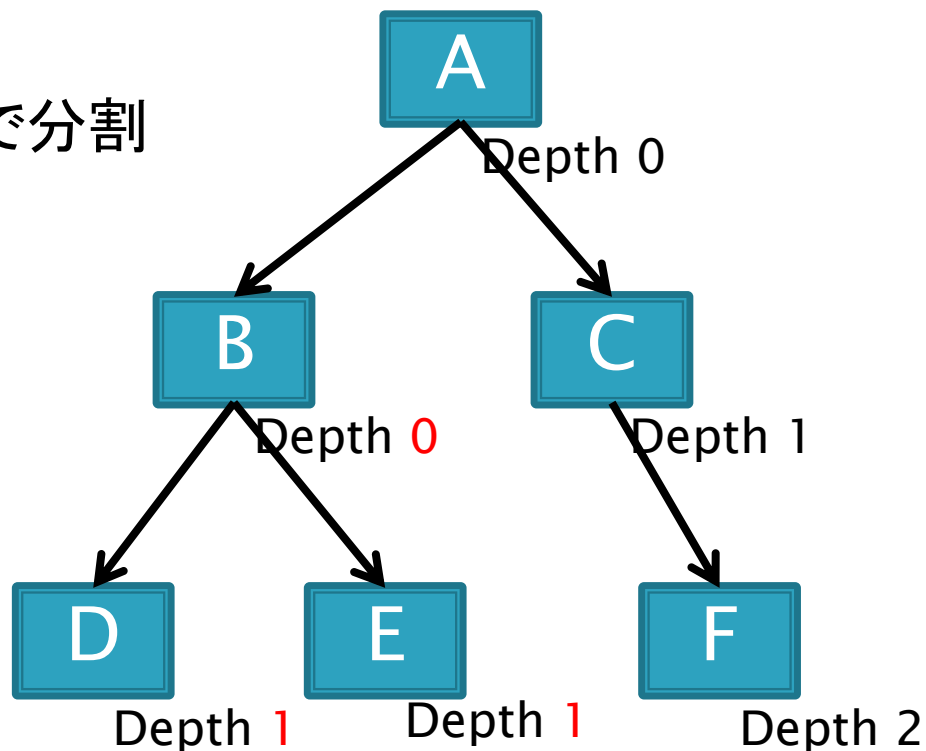
1. Euler TourをA点で分割
 - A直下ならどの位置でもいい
 - S_A の直後がおすすめ



小課題3 (60点) – 動的Euler Tour

木の連結

1. Euler TourをA点で分割
2. Euler Tourを挿入

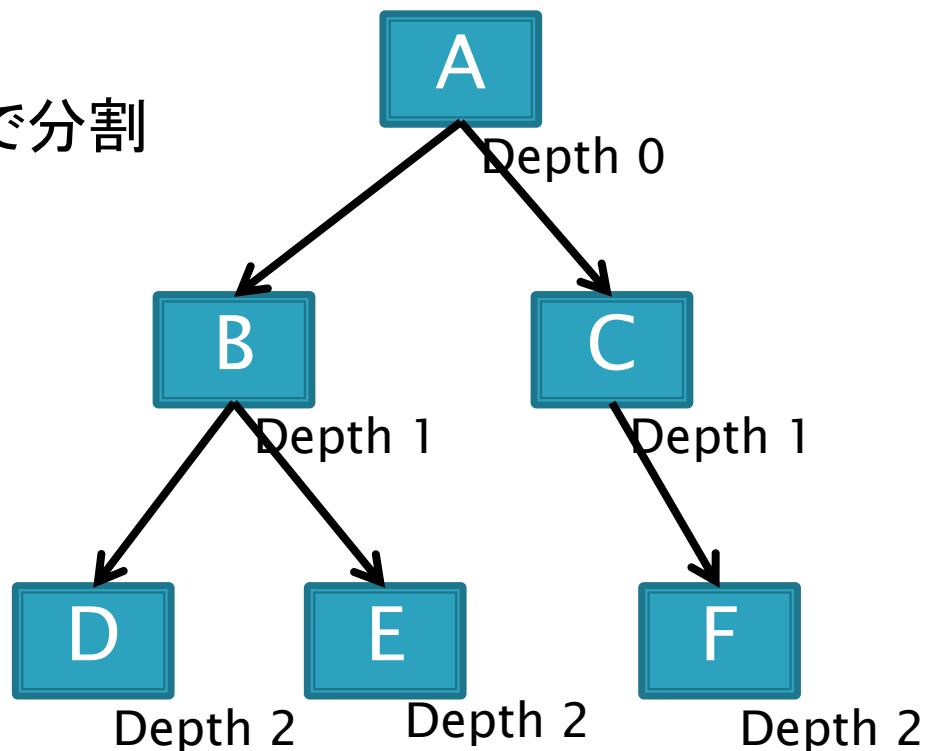


$S_A(A), S_B(B), S_D(D), G_D(B), S_E(E), G_E(B), G_B(E), S_C(C), S_F(F), G_F(C), G_F(A)$

小課題3 (60点) – 動的Euler Tour

木の連結

1. Euler TourをA点で分割
2. Euler Tourを挿入
3. 深さを調整



$S_A(A), S_B(B), S_D(D), G_D(B), S_E(E), G_E(B), G_B(E), S_C(C), S_F(F), G_F(C), G_F(A)$

小課題3 (60点) – 動的Euler Tour

- ▶ 木の削除: 追加のときと逆操作

小課題3 (60点) – 動的Euler Tour

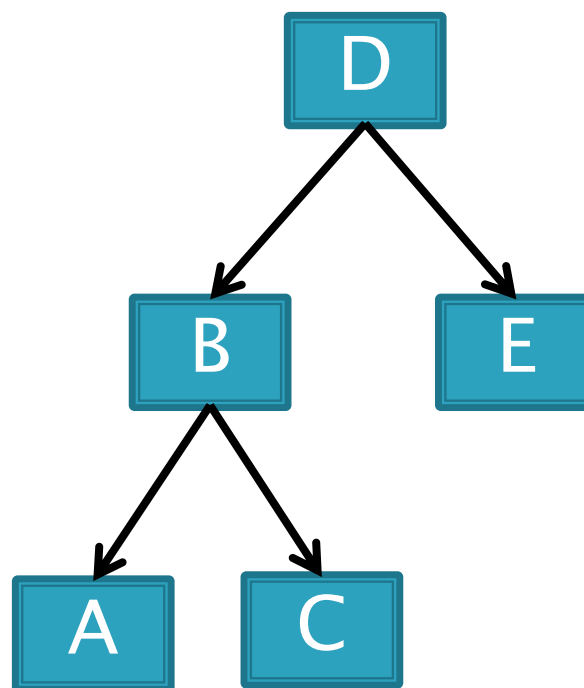
- ▶ 平衡Starry Sky Treeがあればよいことがわかった

小課題3 (60点) – SSTの回転

- ▶ 平衡Starry Sky Treeがあればよいことがわかった
- ▶ ではどのように実装するか？(実装例)

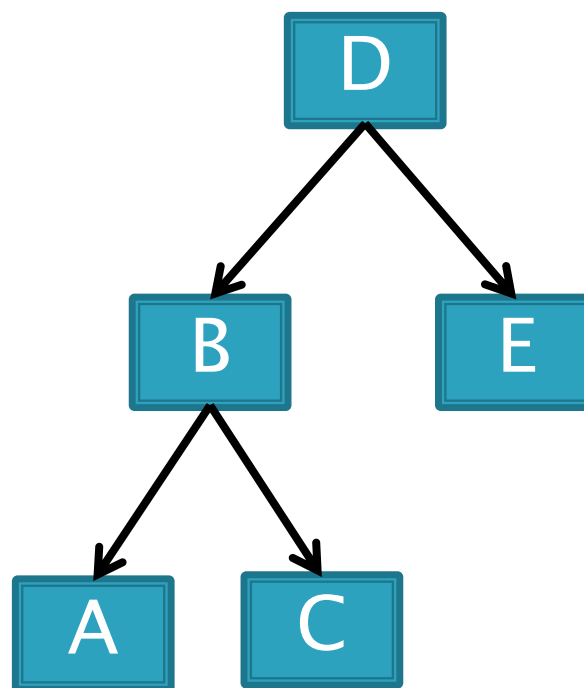
小課題3 (60点) – SSTの回転

- ▶ 今回は、葉ノードと内部ノードの区別をしない
 - A,B,C,D,Eはどれも列上の項目とする



小課題3 (60点) – SSTの回転

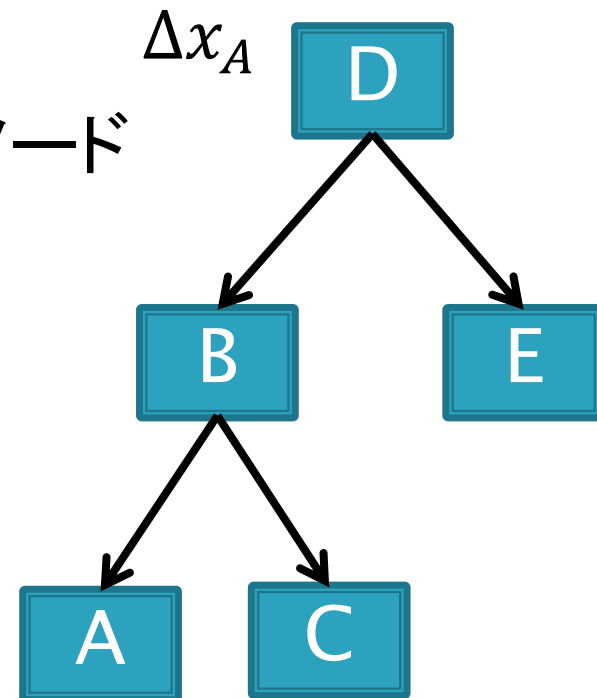
- ▶ 各ノードは、 Δx_A というフィールドを持つ



小課題3 (60点) – SSTの回転

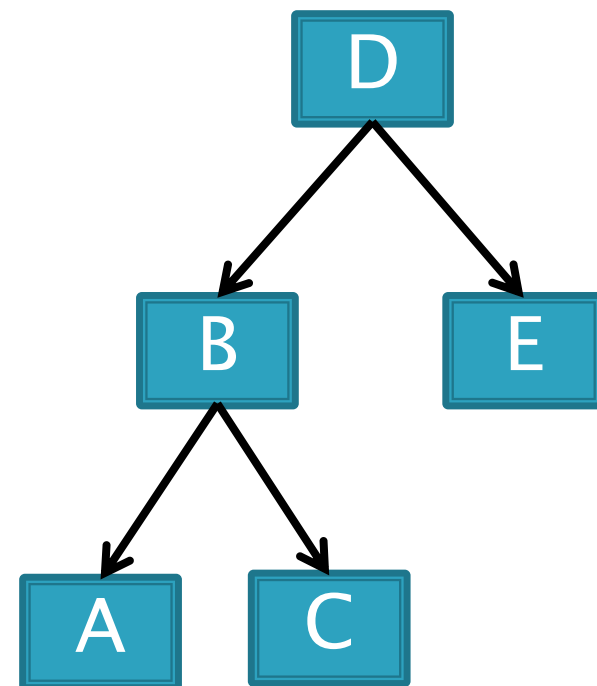
- ▶ 各ノードは、 Δx_A というフィールドを持つ
- ▶ 各ノードに定めたい値 x_A は、

$$x_A = \sum_{A \text{ から根までのパス上のノード}} \Delta x_A$$



小課題3 (60点) – SSTの回転

- ▶ $x_A = \Delta x_D + \Delta x_B + \Delta x_A$
- ▶ $x_B = \Delta x_D + \Delta x_B$
- ▶ $x_C = \Delta x_D + \Delta x_B + \Delta x_C$
- ▶ $x_D = \Delta x_D$
- ▶ $x_E = \Delta x_D + \Delta x_E$



小課題3 (60点) – SSTの回転

▶ 木の回転: x_A が保存されるように行う

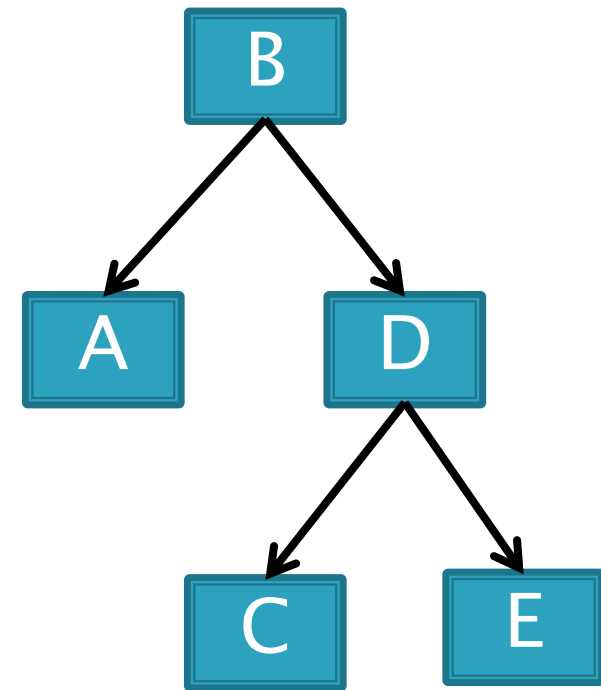
▶ $\Delta x'_A = x_A - x_B$

▶ $\Delta x'_B = x_B$

▶ $\Delta x'_C = x_C - x_D$

▶ $\Delta x'_D = x_D - x_B$

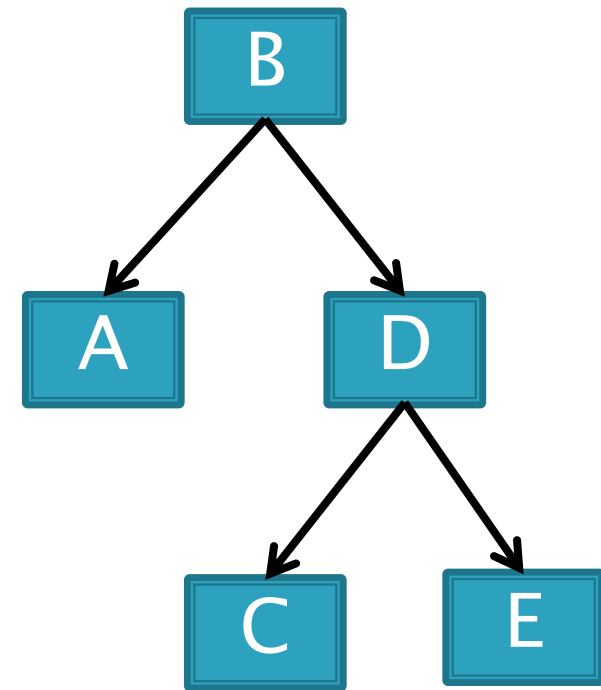
▶ $\Delta x'_E = x_E - x_D$



小課題3 (60点) – SSTの回転

▶ 木の回転: x_A が保存されるように行う

- ▶ $\Delta x'_A = \Delta x_A$
- ▶ $\Delta x'_B = \Delta x_D + \Delta x_B$
- ▶ $\Delta x'_C = \Delta x_B + \Delta x_C$
- ▶ $\Delta x'_D = -\Delta x_B$
- ▶ $\Delta x'_E = \Delta x_E$



小課題3 (60点) – SSTの回転

- ▶ 最後に、ノードに値 y_A を
- ▶ $y_A = \min_{A\text{の全ての子孫}B} x_B - x_A$
- ▶ となるように計算して保持しておく

小課題3 (60点) – SSTの回転

- ▶ 最後に、ノードに値 y_A を
- ▶ $y_A = \min_{A\text{の全ての子孫}B} x_B - x_A$
- ▶ となるように計算して保持しておく
- ▶ これでStarry Sky Tree相当の計算を行えるようになる

小課題3 (60点) – 平衡二分木

- ▶ 平衡二分木の基本

小課題3 (60点) – 平衡二分木

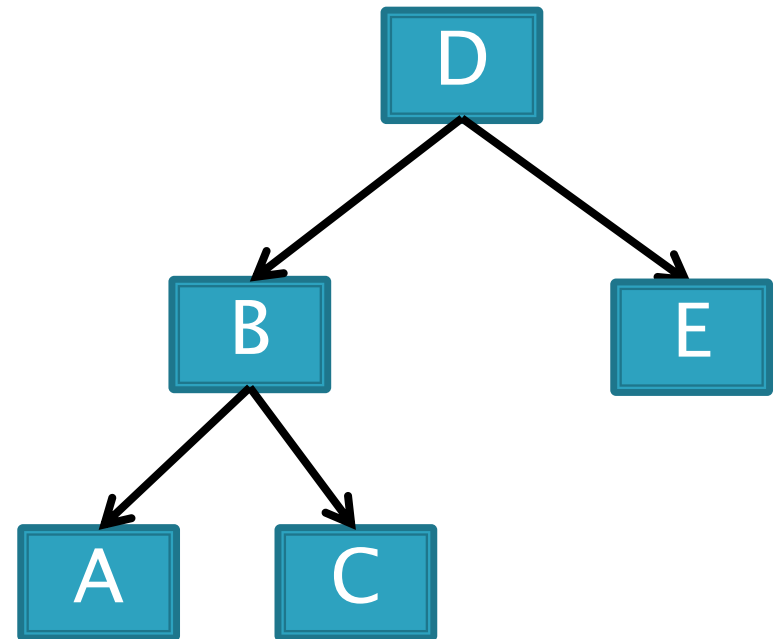
- ▶ 平衡二分木の基本: 回転操作

小課題3 (60点) – 平衡二分木

- ▶ 回転操作: 順序を保存したまま木構造を変形

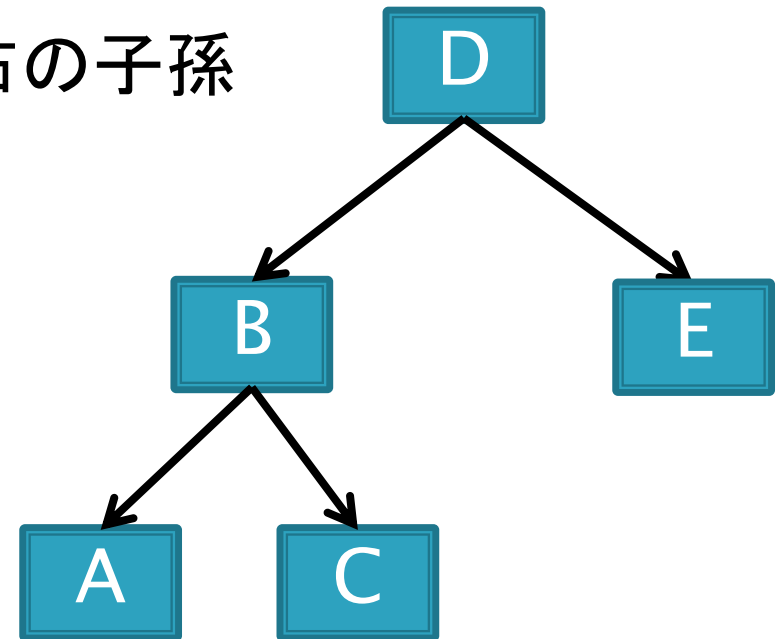
小課題3 (60点) – 平衡二分木

- ▶ 回転操作： 順序を保存したまま木構造を変形
- ▶ 次のような二分木を考える



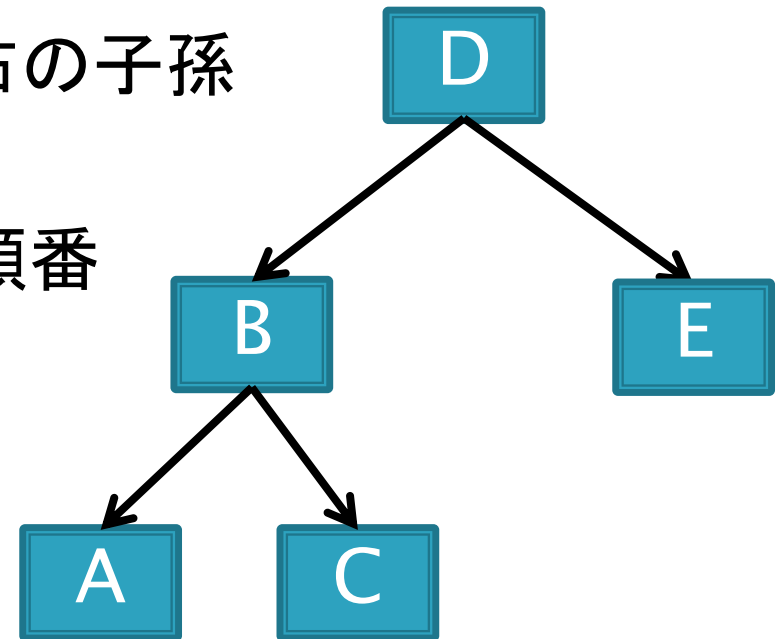
小課題3 (60点) – 平衡二分木

- ▶ 回転操作： 順序を保存したまま木構造を変形
- ▶ 次のような二分木を考える
- ▶ 順序： 左の子孫→自分→右の子孫



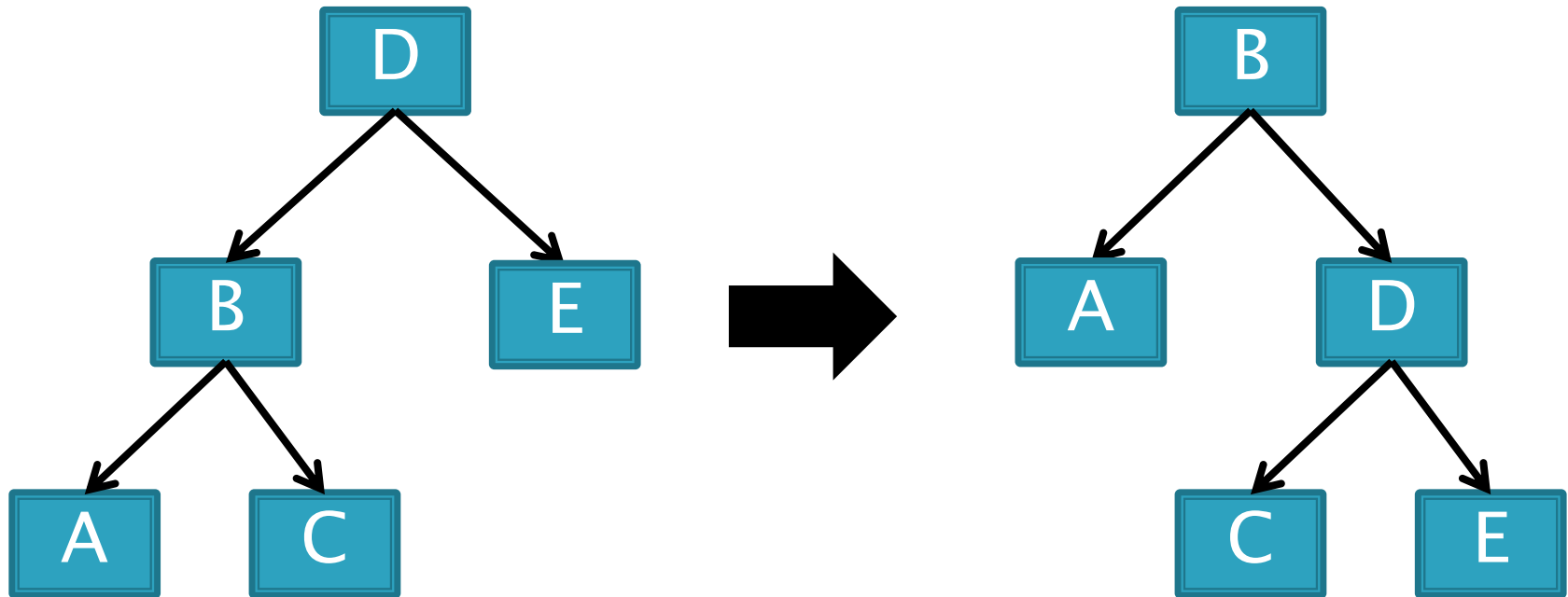
小課題3 (60点) – 平衡二分木

- ▶ 回転操作： 順序を保存したまま木構造を変形
- ▶ 次のような二分木を考える
- ▶ 順序： 左の子孫→自分→右の子孫
- ▶ この場合は A, B, C, D, E の順番



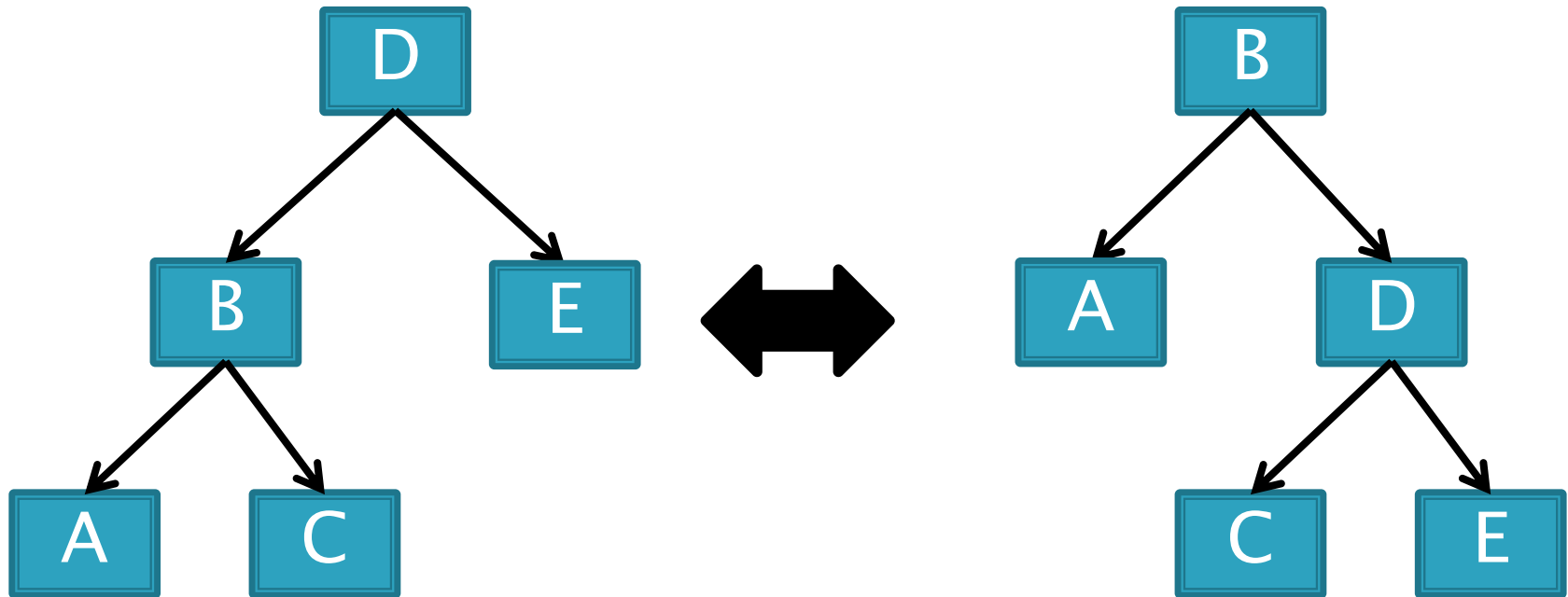
小課題3 (60点) - 平衡二分木

- 次のように変形しても順番はA,B,C,D,E



小課題3 (60点) - 平衡二分木

- ▶ 次のように変形しても順番はA,B,C,D,E
- ▶ これを「木の回転」と言う



小課題3 (60点) – 平衡二分木

- ▶ 次のように変形しても順番はA,B,C,D,E
- ▶ これを「木の回転」と言う
- ▶ うまく回転をすることで、偏りが起きないようにする二分木を「平衡二分木」と言う
 - 回転以外の方法で平衡を保つものもある

小課題3 (60点) – 平衡二分木

- ▶ 平衡二分木の実装方法

小課題3 (60点) – 平衡二分木

- ▶ 平衡二分木の実装方法
- ▶ 今回は何でもOK!
 - 赤黒木
 - RBST
 - Treap
 - Splay木
 - などなど...

小課題3 (60点) – Splay木

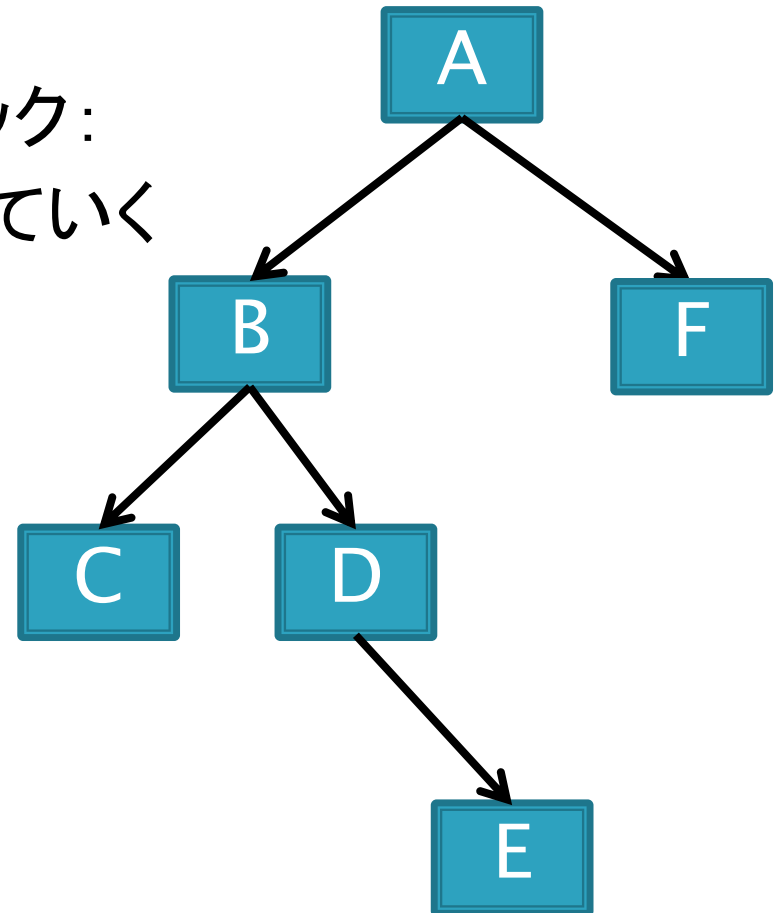
- ▶ この解説ではSplay木の説明をします

小課題3 (60点) – Splay木

- ▶ 突然ですが、Union Findの復習をします

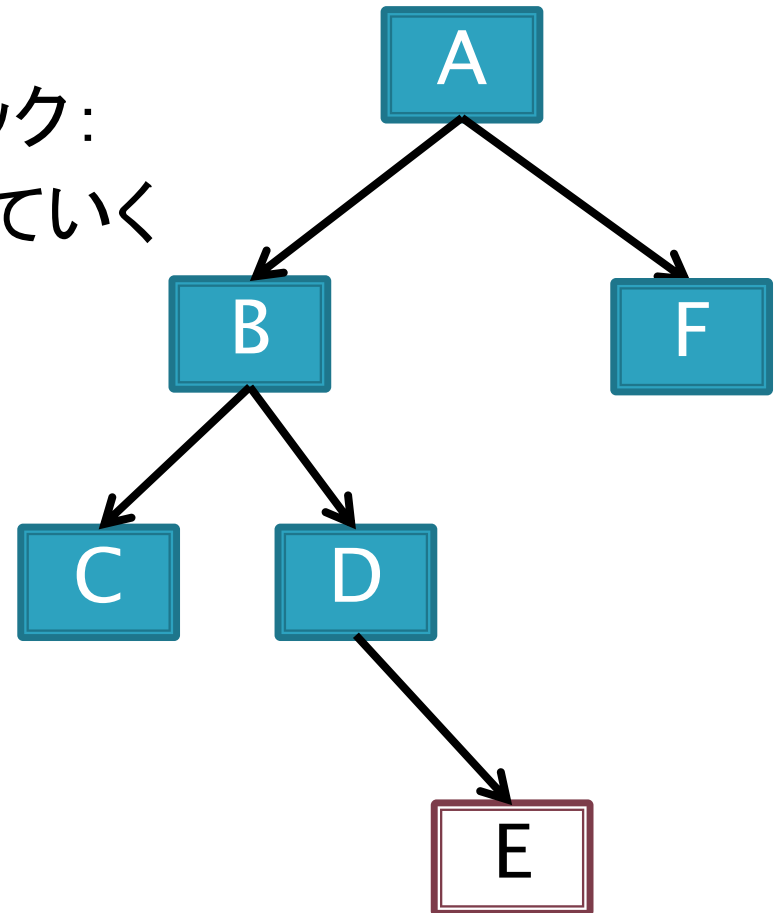
小課題3 (60点) – Splay木

- ▶ 突然ですが、Union Findの復習をします
- ▶ Union Find の効率化テクニック:
- ▶ アクセスした頂点を根へ持っていく



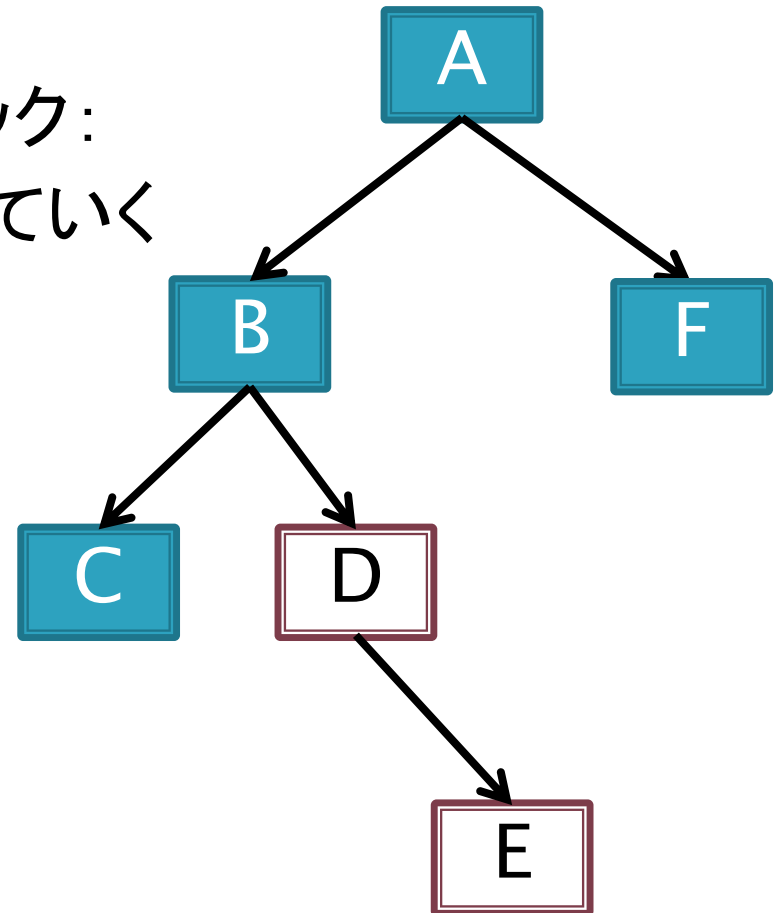
小課題3 (60点) – Splay木

- ▶ 突然ですが、Union Findの復習をします
- ▶ Union Find の効率化テクニック:
- ▶ アクセスした頂点を根へ持っていく



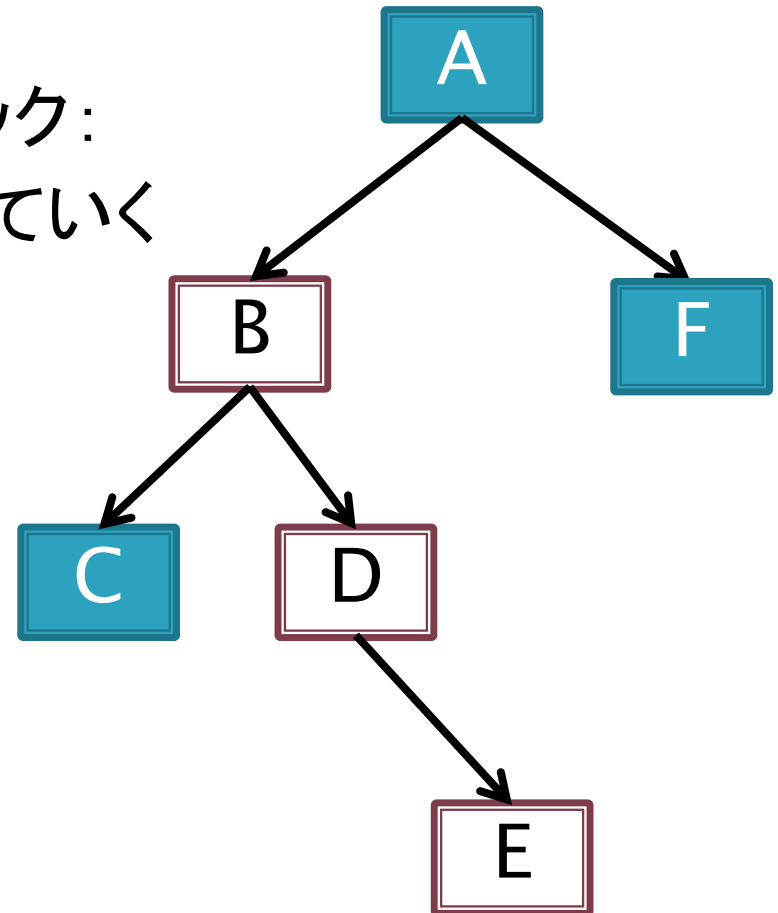
小課題3 (60点) – Splay木

- ▶ 突然ですが、Union Findの復習をします
- ▶ Union Find の効率化テクニック:
- ▶ アクセスした頂点を根へ持っていく



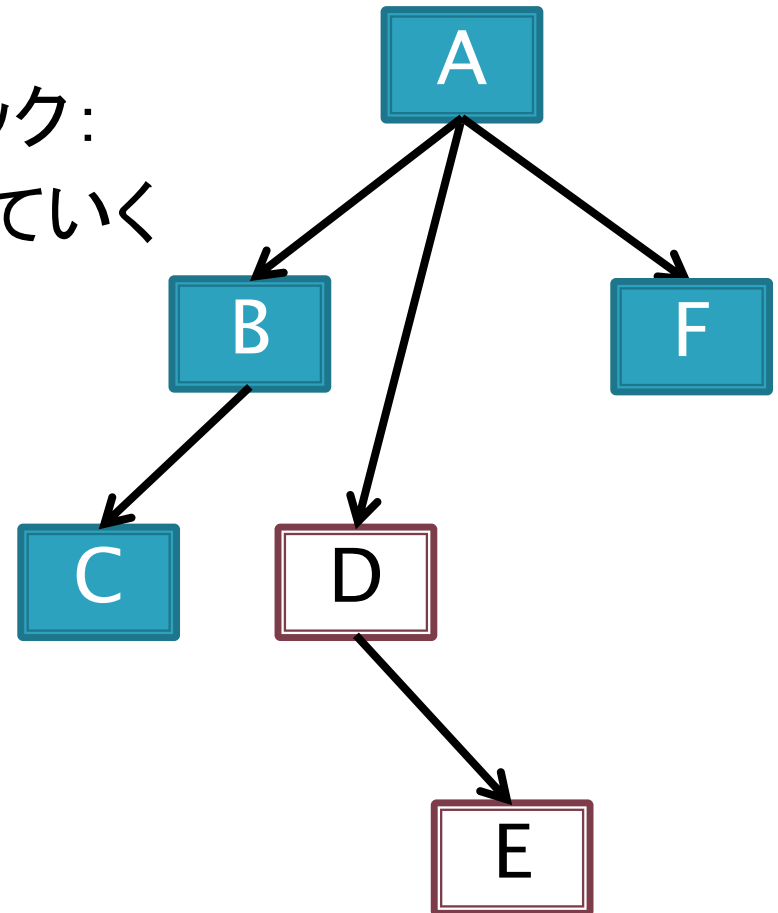
小課題3 (60点) – Splay木

- ▶ 突然ですが、Union Findの復習をします
- ▶ Union Find の効率化テクニック:
- ▶ アクセスした頂点を根へ持っていく



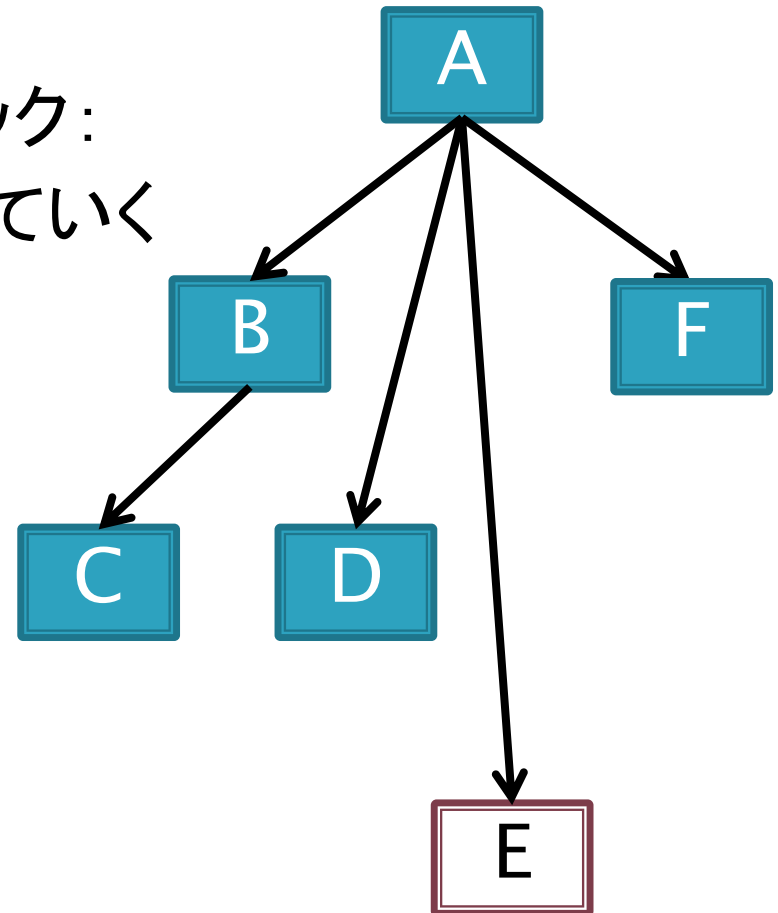
小課題3 (60点) – Splay木

- ▶ 突然ですが、Union Findの復習をします
- ▶ Union Find の効率化テクニック:
- ▶ アクセスした頂点を根へ持っていく



小課題3 (60点) – Splay木

- ▶ 突然ですが、Union Findの復習をします
- ▶ Union Find の効率化テクニック:
- ▶ アクセスした頂点を根へ持っていく



小課題3 (60点) – Splay木

- ▶ 同じようなことを、二分探索木でもできないか？

小課題3 (60点) – Splay木

- ▶ 同じようなことを、二分探索木でもできないか？
 - →Move-to-root heuristic

小課題3 (60点) – Splay木

- ▶ Move-to-root heuristic
 - 頂点にアクセスしたら、それが根に行くまで繰り返し回転する

小課題3 (60点) – Splay木

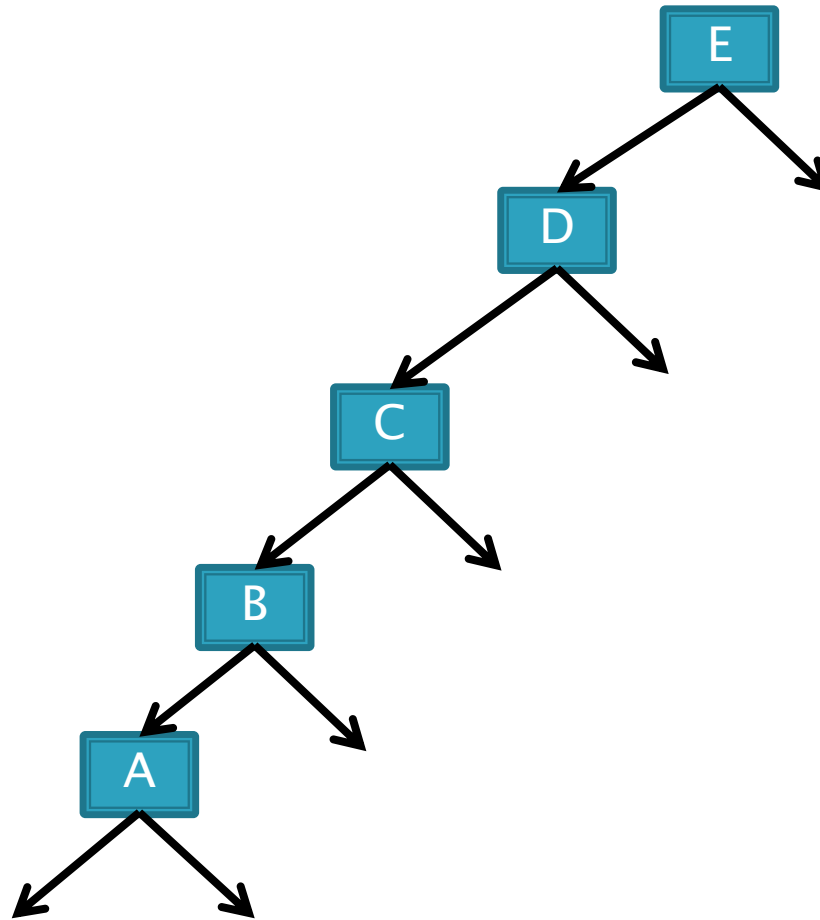
- ▶ Move-to-root heuristic
 - 頂点にアクセスしたら、それが根に行くまで繰り返し回転する
 - そんなので上手くいくわけないだろ！！

小課題3 (60点) – Splay木

- ▶ 実際ダメ

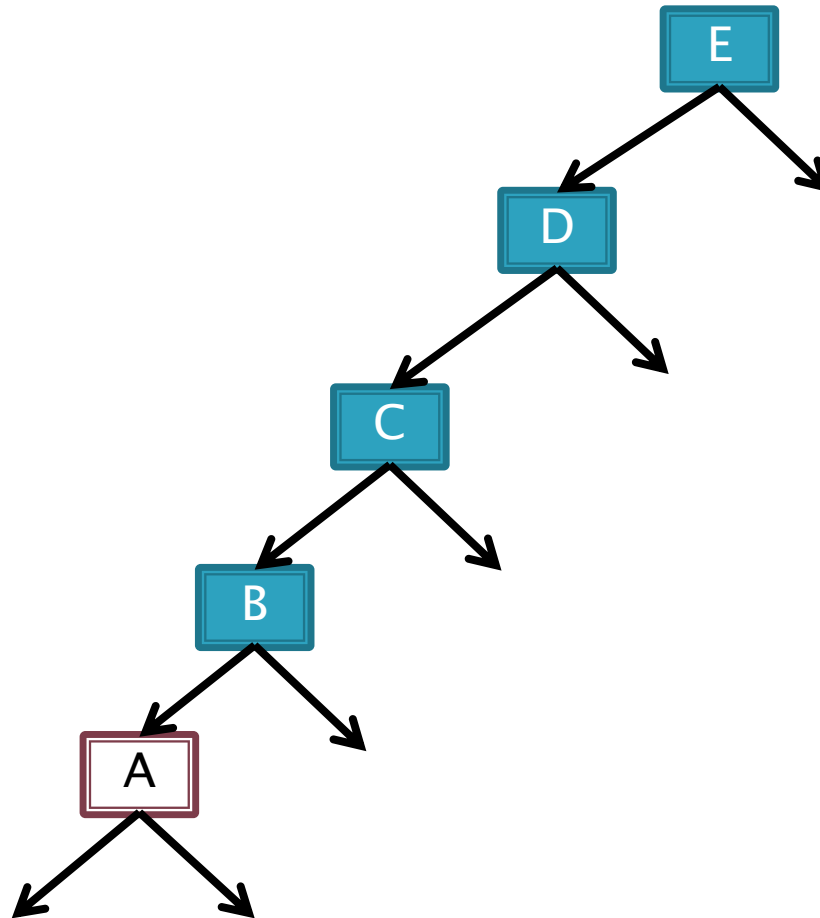
小課題3 (60点) – Splay木

▶ 実際ダメ



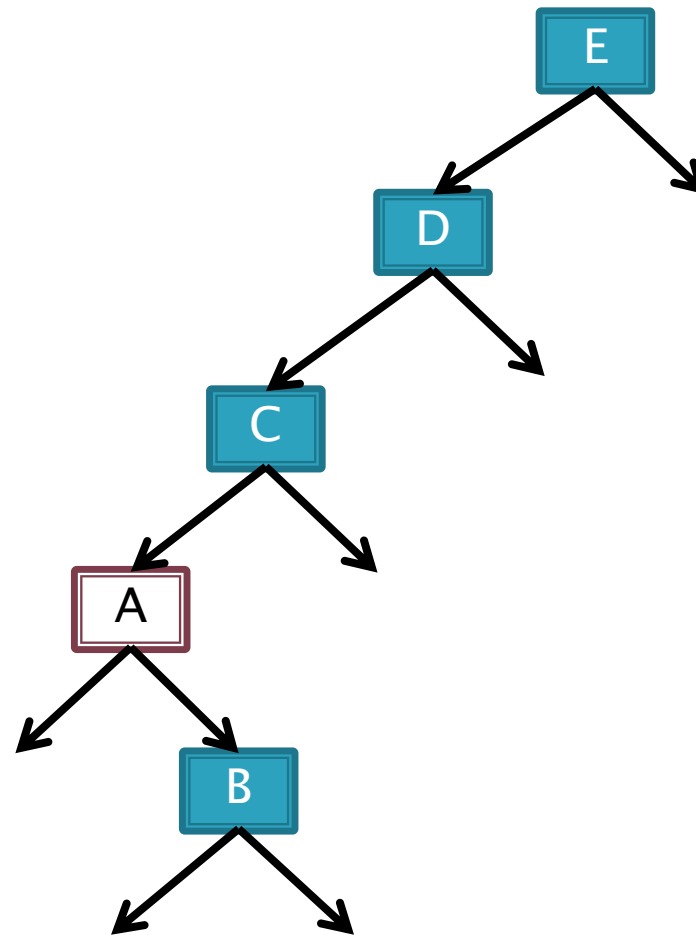
小課題3 (60点) – Splay木

▶ 実際ダメ



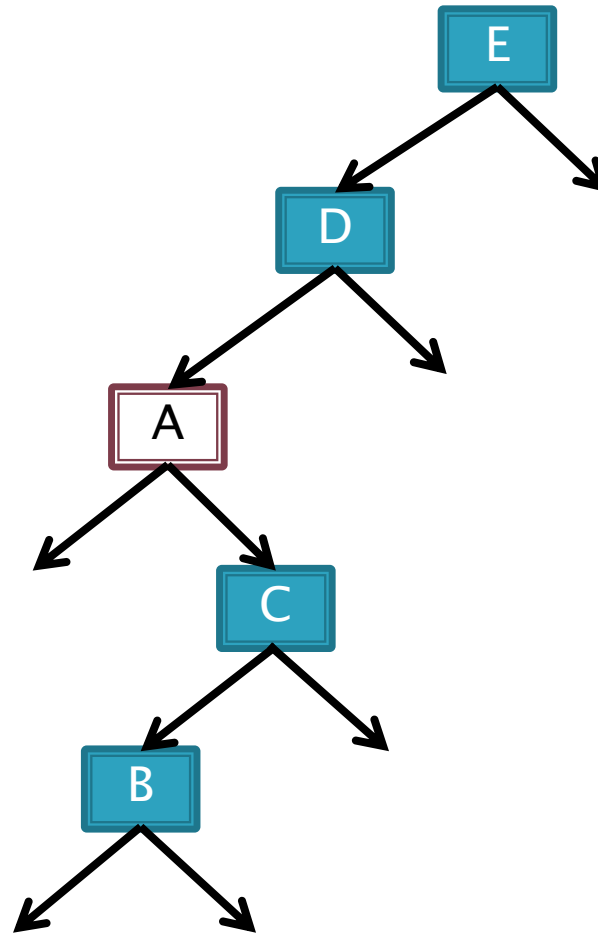
小課題3 (60点) – Splay木

▶ 実際ダメ



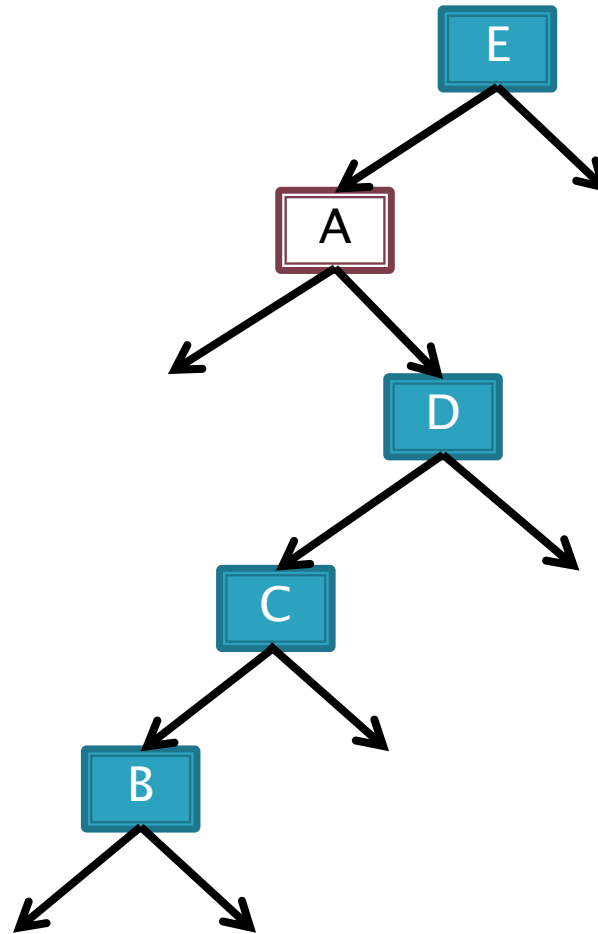
小課題3 (60点) – Splay木

▶ 実際ダメ



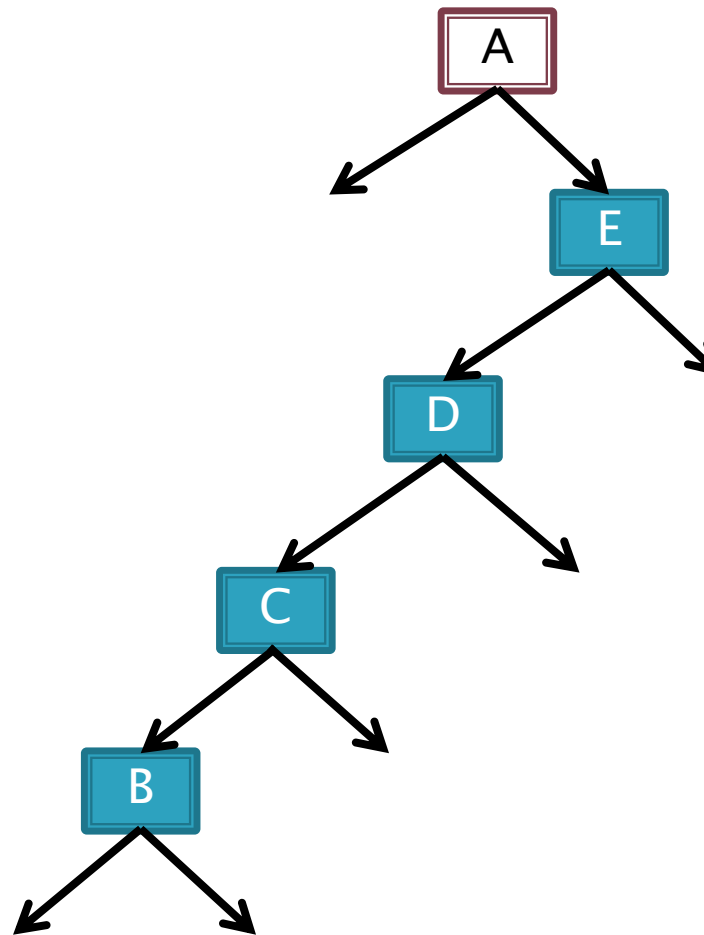
小課題3 (60点) – Splay木

▶ 実際ダメ



小課題3 (60点) – Splay木

▶ 実際ダメ



小課題3 (60点) – Splay木

- ▶ 実際ダメ
- ▶ この後A, B, C, D, Eの順にアクセスしたら
$$n + (n - 1) + \dots + 1 = O(n^2)$$
- ▶ のコストがかかってしまう

小課題3 (60点) – Splay木

- ▶ 実際ダメ
- ▶ この後A, B, C, D, Eの順にアクセスしたら
$$n + (n - 1) + \dots + 1 = O(n^2)$$
- ▶ のコストがかかってしまう
- ▶ どうする？

小課題3 (60点) – Splay木

- ▶ 解決策: 木の回転を3つに分ける

小課題3 (60点) – Splay木

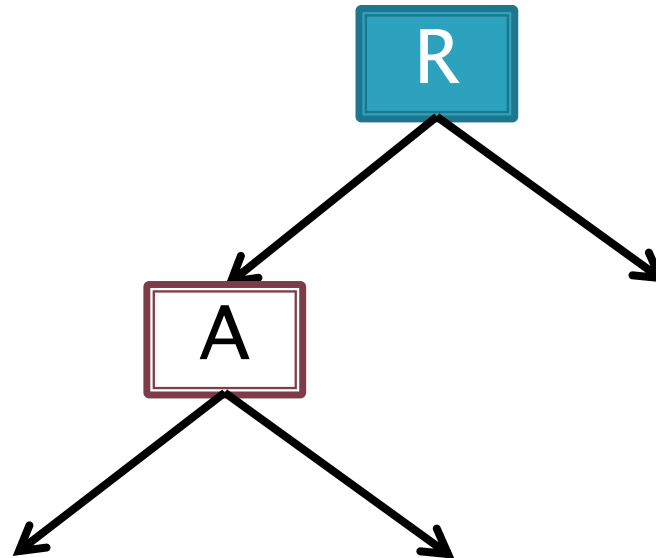
- ▶ 解決策：木の回転を3つに分ける
 - “zig” step
 - “zig-zag” step
 - “zig-zig” step

小課題3 (60点) – Splay木

- ▶ (1) “zig”-step

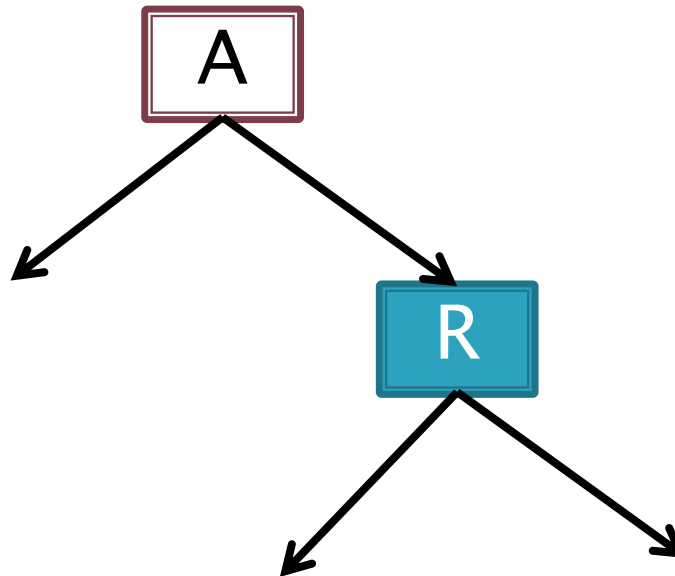
小課題3 (60点) – Splay木

- ▶ (1) “zig”-step
- ▶ すぐ上が根の場合



小課題3 (60点) – Splay木

- ▶ (1) “zig”-step
- ▶ すぐ上が根の場合
- ▶ 普通に回転する

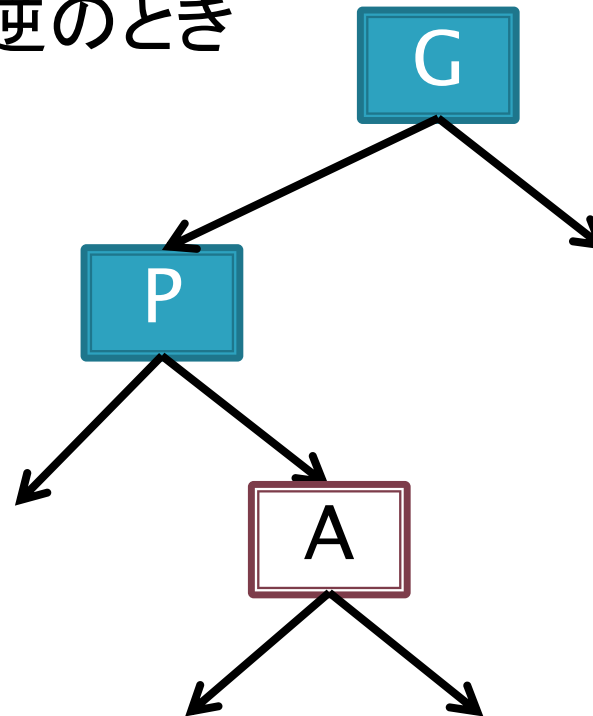


小課題3 (60点) – Splay木

- ▶ (2) “zig-zag”-step

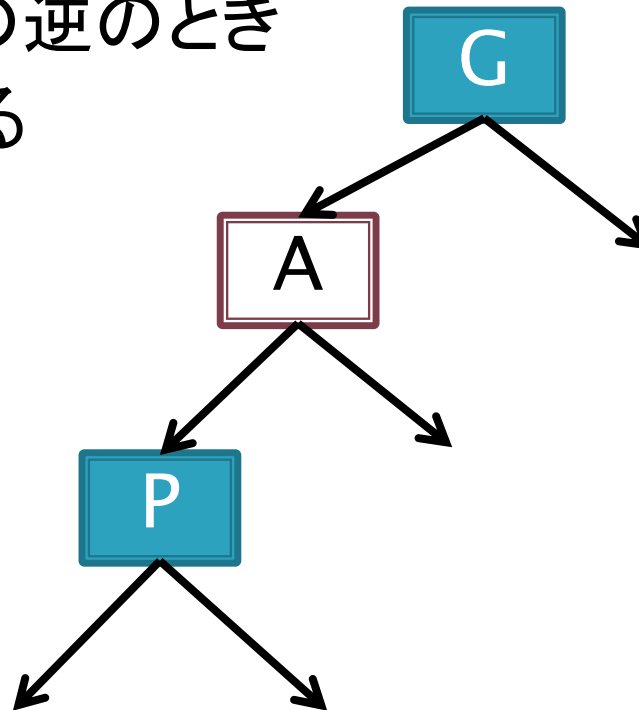
小課題3 (60点) – Splay木

- ▶ (2) “zig-zag”-step
- ▶ 左→右、またはその逆のとき



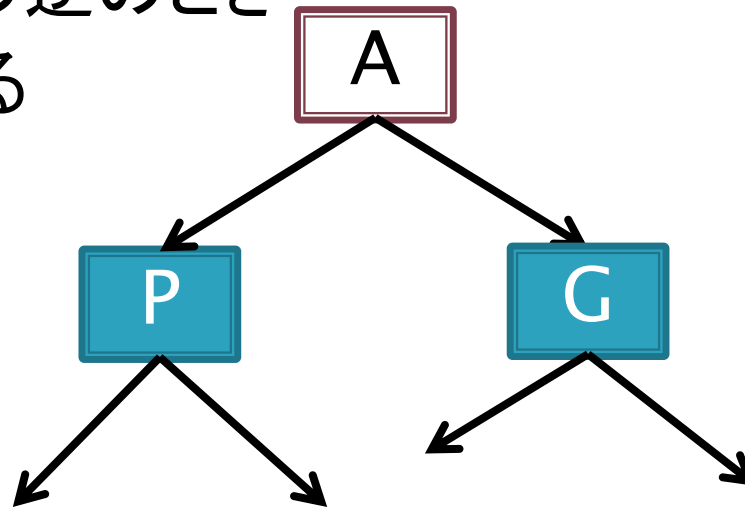
小課題3 (60点) – Splay木

- ▶ (2) “zig-zag”-step
- ▶ 左→右、またはその逆のとき
- ▶ 普通に2回回転する



小課題3 (60点) – Splay木

- ▶ (2) “zig-zag”-step
- ▶ 左→右、またはその逆のとき
- ▶ 普通に2回回転する



小課題3 (60点) – Splay木

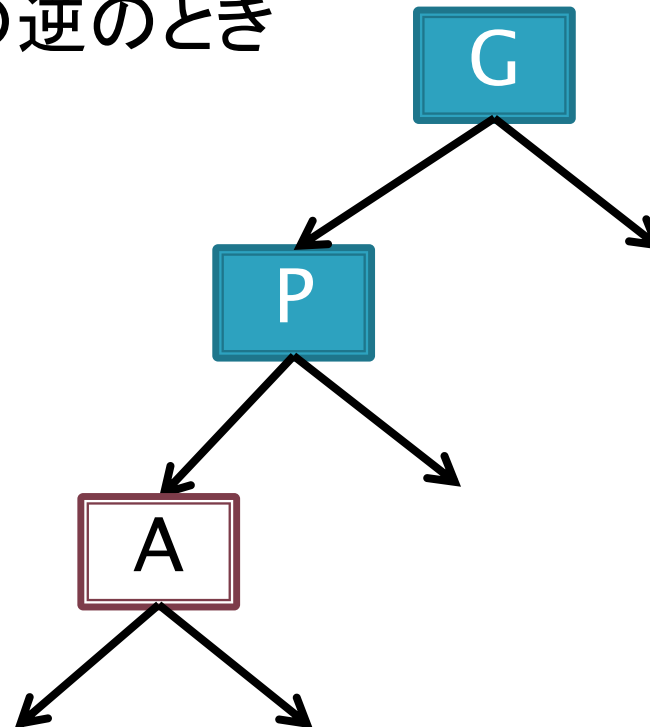
- ▶ (2) “zig-zag”-step
- ▶ 左→右、またはその逆のとき
- ▶ 普通に2回回転する
- ▶ ここまでは先ほどと同じ

小課題3 (60点) – Splay木

- ▶ (3) “zig-zig”-step

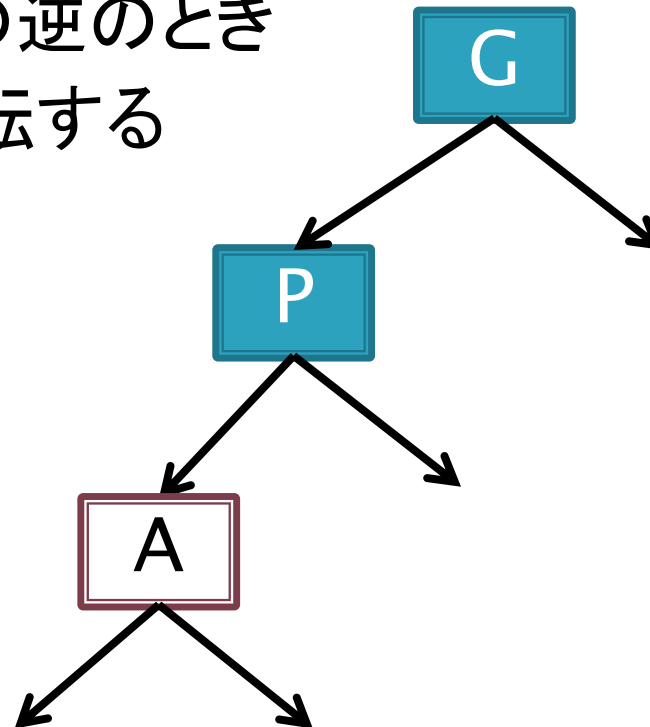
小課題3 (60点) – Splay木

- ▶ (3) “zig-zig”-step
- ▶ 左→左、またはその逆のとき



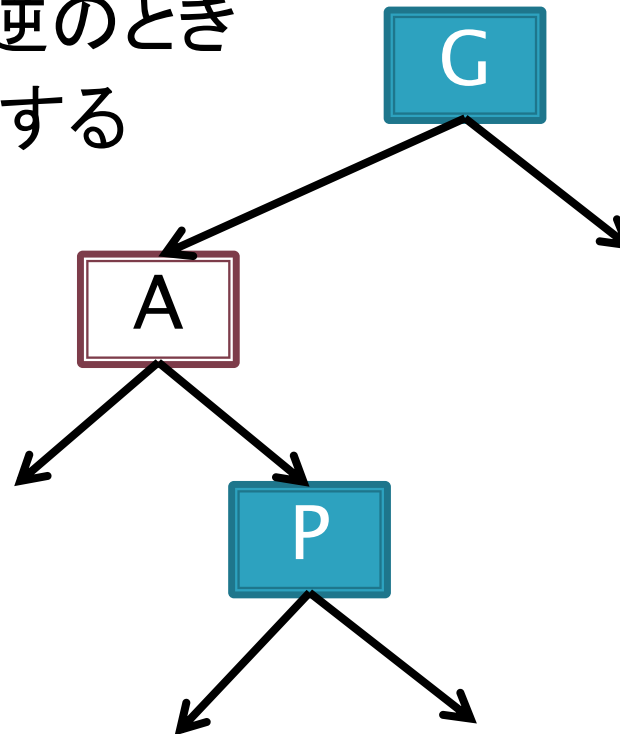
小課題3 (60点) – Splay木

- ▶ (3) “zig-zig”-step
- ▶ 左→左、またはその逆のとき
- ▶ 2回ではなく3回回転する



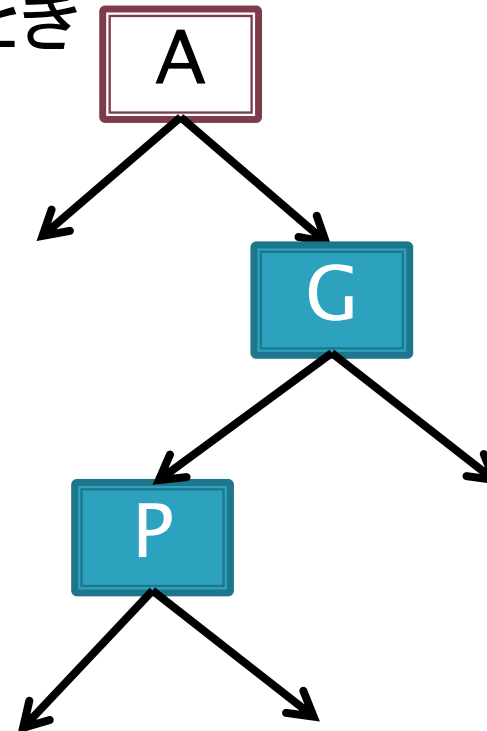
小課題3 (60点) – Splay木

- ▶ (3) “zig-zig”-step
- ▶ 左→左、またはその逆のとき
- ▶ 2回ではなく3回回転する



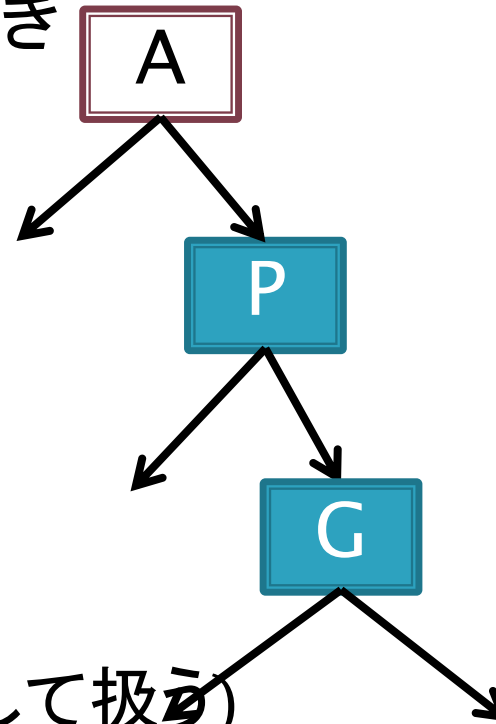
小課題3 (60点) – Splay木

- ▶ (3) “zig-zig”-step
- ▶ 左→左、またはその逆のとき
- ▶ 2回ではなく3回回転する



小課題3 (60点) – Splay木

- ▶ (3) “zig-zig”-step
- ▶ 左→左、またはその逆のとき
- ▶ 2回ではなく3回回転する



- ▶ (ただし、後でこれを2回として扱う)

小課題3 (60点) – Splay木

- ▶ Splaying operation
 - 偏った位置にあるときだけ余計に回転する

小課題3 (60点) – Splay木

- ▶ Splaying operation
 - 偏った位置にあるときだけ余計に回転する
 - そんなので上手くいくわけないだろ！！

小課題3 (60点) – Splay木

- ▶ 実は上手くいく

小課題3 (60点) – Splay木

- ▶ 実は上手くいく
- ▶ 具体的には: $O(\log N)$ amortized

小課題3 (60点) – Splay木

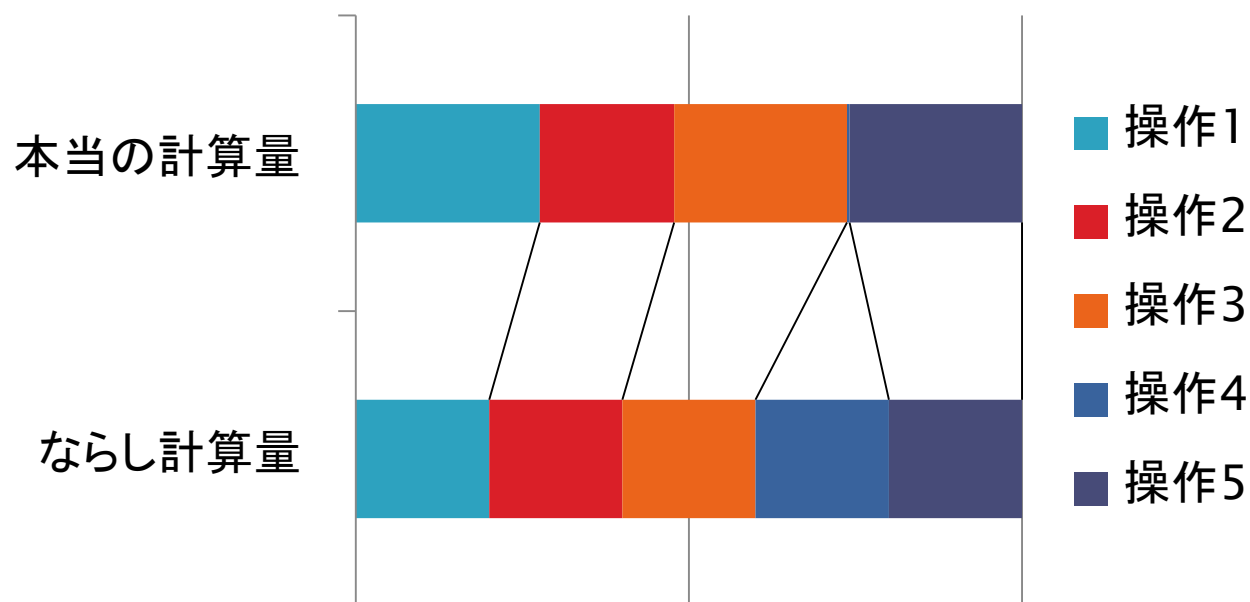
- ▶ 実は上手くいく
- ▶ 具体的には: $O(\log N)$ amortized

小課題3 (60点) – ならし計算量

- ▶ ならし計算量 (amortized time complexity)
- ▶ N 個の一連の操作が $O(f(N))$ で行えるとする
- ▶ 1つ1つの操作は、本当は $O(\frac{f(N)}{N})$ とは限らない
- ▶ これを $O(\frac{f(N)}{N})$ として扱うのが、ならし計算量

小課題3 (60点) – ならし計算量

▶ ならし計算量のイメージ



小課題3 (60点) – ならし計算量

- ▶ ならし計算量の向き/不向き

小課題3 (60点) – ならし計算量

- ▶ ならし計算量の向き/不向き
- ▶ 向いているもの
 - 全体での処理効率が重視されるバッチ型の処理
 - 例: プログラミングコンテスト
- ▶ 向いていないもの
 - リアルタイム性能が重視される処理
 - 例: 信号処理

小課題3 (60点) – ならし計算量

- ▶ ならし計算量と平均計算量

小課題3 (60点) – ならし計算量

- ▶ ならし計算量と平均計算量
 - この2つは別物！！
 - ならし計算量：時系列上での平均
 - 平均計算量：確率変数上での平均

小課題3 (60点) – ならし計算量

- ▶ ならし計算量と平均計算量
 - この2つは別物！！
 - ならし計算量：時系列上での平均
 - 平均計算量：確率変数上での平均
 - ならし計算量：不確定要素は無い！

小課題3 (60点) – Splay木の解析

- ▶ Splayのならし計算量の評価

小課題3 (60点) – Splay木の解析

- ▶ Splayのならし計算量の評価
- ▶ 「ポテンシャル関数」の概念を導入

小課題3 (60点) – Splay木の解析

- ▶ Splayのならし計算量の評価
- ▶ 「ポテンシャル関数」の概念を導入
 - 借金みたいなもの

小課題3 (60点) – Splay木の解析

- ▶ ポテンシャル関数を用いた計算量の均し(ならし)
- ▶ $a_j = t_j + \Phi_{j+1} - \Phi_j$
 - t_j : その操作の実際の計算量
 - $\Phi_{j+1} - \Phi_j$: ポテンシャルの増加量
 - a_j : その操作のならし計算量

小課題3 (60点) – Splay木の解析

- ▶ ポテンシャル関数を用いた計算量の均し(ならし)
- ▶ $a_j = t_j + \Phi_{j+1} - \Phi_j$
 - t_j : その操作の実際の計算量
 - $\Phi_{j+1} - \Phi_j$: ポテンシャルの増加量
 - a_j : その操作のならし計算量
- ▶ ポテンシャルの意味
 - より大きい: 木はより偏っている
 - より小さい: 木はより平坦になっている

小課題3 (60点) – Splay木の解析

- ▶ ならし計算量の総和をとる
- ▶ $\sum_j a_j = \sum_j t_j + \Phi_m - \Phi_0$
 - $\sum_j t_j$: 実際の計算量の総和
 - $\Phi_m - \Phi_0$: ポテンシャルの総変化量
 - $\sum_j a_j$: ならし計算量の総和

小課題3 (60点) – Splay木の解析

- ▶ ならし計算量の総和をとる
- ▶ $\sum_j a_j = \sum_j t_j + \Phi_m - \Phi_0$
 - $\sum_j t_j$: 実際の計算量の総和
 - $\Phi_m - \Phi_0$: ポテンシャルの総変化量
 - $\sum_j a_j$: ならし計算量の総和
- ▶ ポテンシャルの総変化量が小さければうまく評価できる

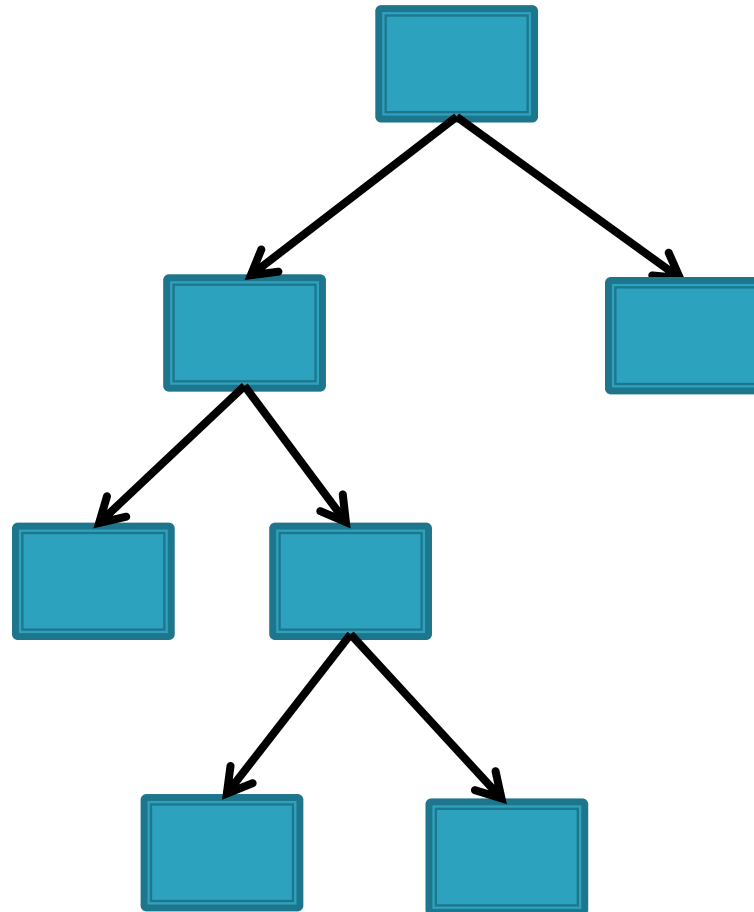
小課題3 (60点) – Splay木の解析

▶ Splay木のポテンシャル

- Splay木の各頂点の重さを $w(x)$ とする
 - 計算量の見積もり方にあわせて自由に決めてよい
- Splay木の頂点のサイズ $s(x) = \sum_x \text{の全ての子孫}_y w(y)$
- Splay木の頂点のランク $r(x) = \log_2 s(x)$
- Splay木のポテンシャル $\Phi = \sum \text{全ての頂点}_x r(x)$

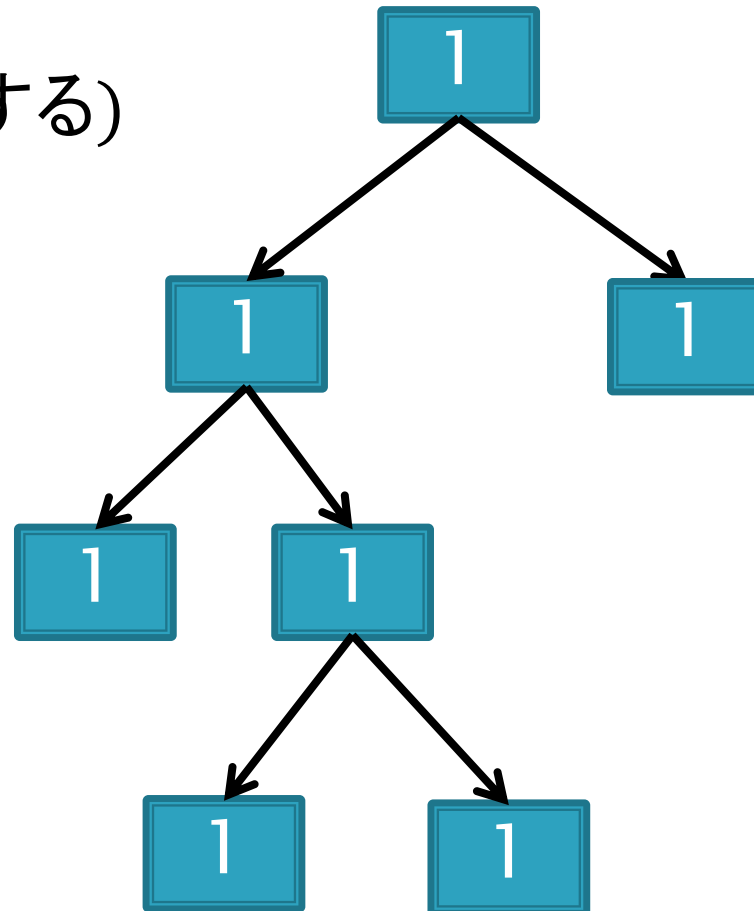
小課題3 (60点) – Splay木の解析

- ▶ Splay木のポテンシャル: 例



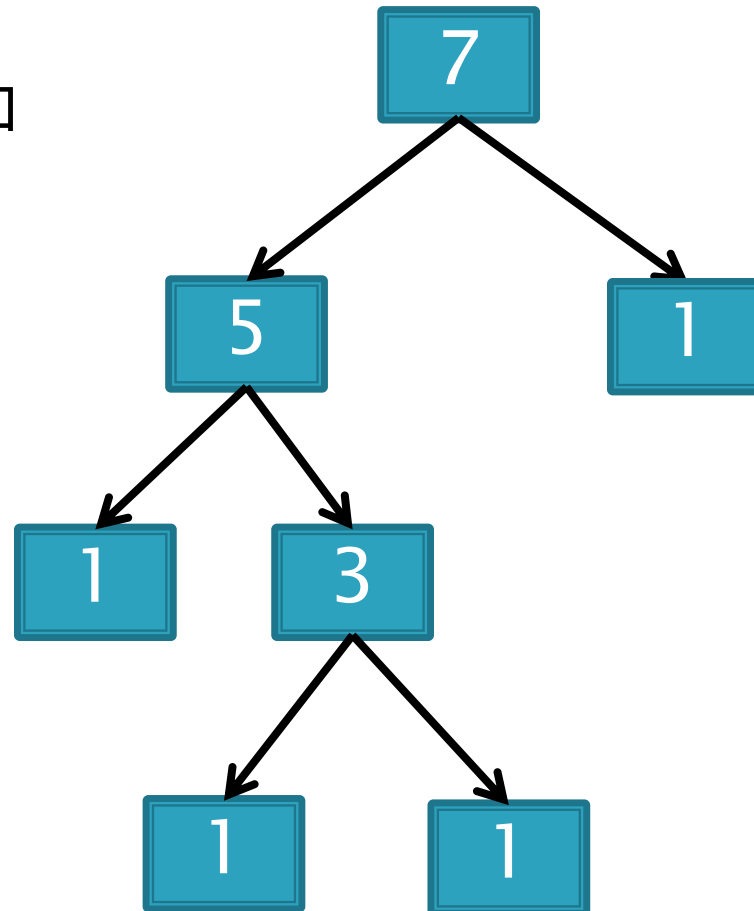
小課題3 (60点) – Splay木の解析

- ▶ Splay木のポテンシャル: 例
- ▶ 重さ $w(x)$
(今回は全て1とする)



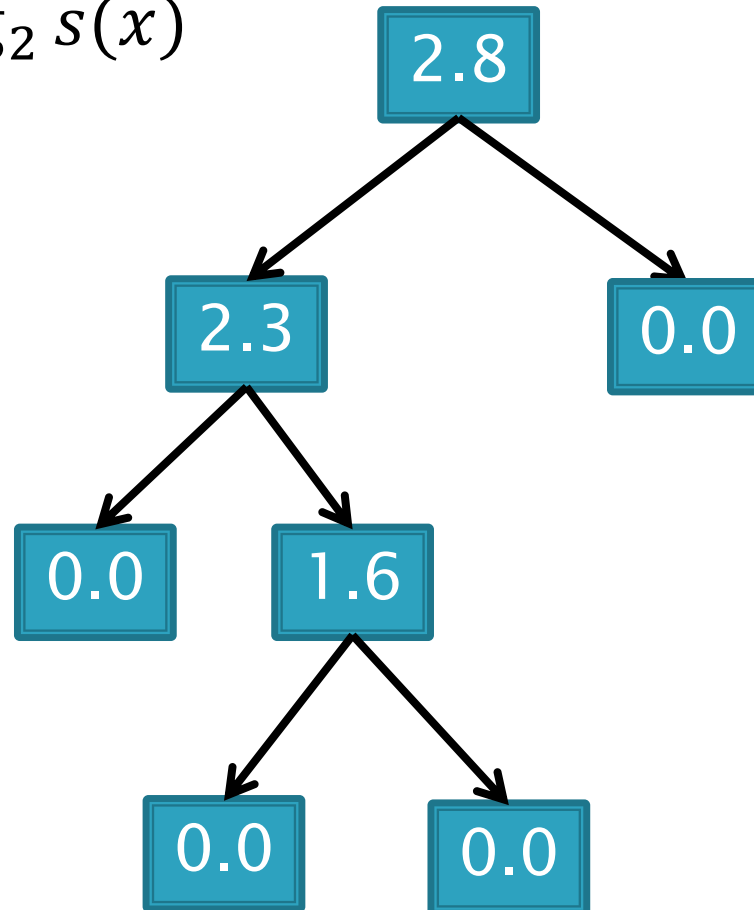
小課題3 (60点) – Splay木の解析

- ▶ Splay木のポテンシャル: 例
- ▶ サイズ $s(x)$
 - 部分木の重さの和



小課題3 (60点) – Splay木の解析

- ▶ Splay木のポテンシャル: 例
- ▶ ランク $r(x) = \log_2 s(x)$



小課題3 (60点) – Splay木の解析

- ▶ Splay木のポテンシャル: 例
- ▶ ポテンシャル: ランクの総和
- ▶

$$\Phi = 2.8 + 2.3 + 1.6 = 6.7$$

小課題3 (60点) – Splay木の解析

- ▶ Splay木のポテンシャルの良い性質

小課題3 (60点) – Splay木の解析

- ▶ Splay木のポテンシャルの良い性質 ...
- ▶ 回転の影響を受ける頂点が少ない
 - 解析が簡単になる

小課題3 (60点) – Splay木の解析

- ▶ アクセス補題 (Access Lemma)

小課題3 (60点) – Splay木の解析

- ▶ アクセス補題 (Access Lemma)
- ▶ x : 木のノード
- ▶ t : 木の根 とするとき
- ▶ 木をsplayする操作一回にかかる時間(回転の回数)は、なりし計算量で
$$3r(t) - 3r(x) + 1$$
- ▶ 以下である。

小課題3 (60点) – Splay木の解析

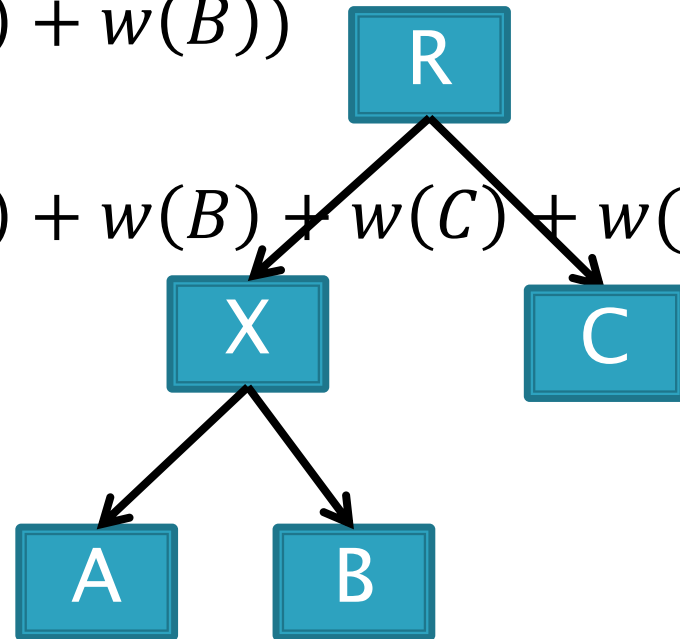
- ▶ アクセス補題の証明

小課題3 (60点) – Splay木の解析

- ▶ アクセス補題の証明
- ▶ 各回転ステップのならし計算量が
 1. “zig”-stepでは $3r'(x) - 3r(x) + 1$ 以下
 2. それ以外では $3r'(x) - 3r(x)$ 以下
- ▶ (ただし、 $r'(x)$: 操作後のランク)
- ▶ であることを示す。
- ▶ そうすると、1のケースに登場する $r'(x)$ は初期の $r(t)$ と等しい(木全体のサイズの対数)ので、合計すると $3r(t) - 3r(x) + 1$ になる。

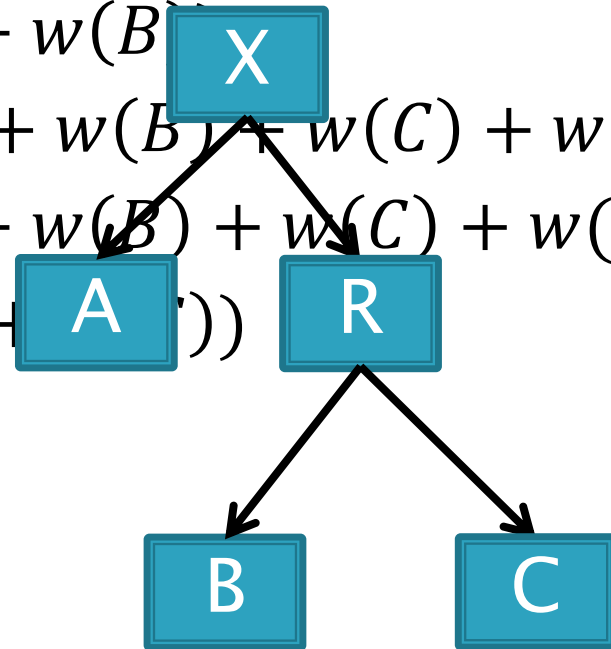
小課題3 (60点) – Splay木の解析

- ▶ アクセス補題の証明 (1) “zig”-step の場合
- ▶ $r(X) = \log_2(w(X) + w(A) + w(B))$
- ▶ $r(R) = \log_2(w(X) + w(A) + w(B) + w(C) + w(R))$



小課題3 (60点) – Splay木の解析

- ▶ アクセス補題の証明 (1) “zig”-step の場合
- ▶ $r(X) = \log_2(w(X) + w(A) + w(B))$
- ▶ $r'(X) = \log_2(w(X) + w(A) + w(B) + w(C) + w(R))$
- ▶ $r(R) = \log_2(w(X) + w(A) + w(B) + w(C) + w(R))$
- ▶ $r'(R) = \log_2(w(R) + w(B) + w(C))$



小課題3 (60点) – Splay木の解析

- ▶ アクセス補題の証明 (1) “zig”-step の場合
- ▶ $r(X) = \log_2(w(X) + w(A) + w(B))$
- ▶ $r'(X) = \log_2(w(X) + w(A) + w(B) + w(C) + w(R))$
- ▶ $r(R) = \log_2(w(X) + w(A) + w(B) + w(C) + w(R))$
- ▶ $r'(R) = \log_2(w(R) + w(B) + w(C))$

小課題3 (60点) – Splay木の解析

- ▶ アクセス補題の証明 (1) “zig”-step の場合
- ▶ $r(X) = \log_2(w(X) + w(A) + w(B))$
- ▶ $r'(X) = \log_2(w(X) + w(A) + w(B) + w(C) + w(R))$
- ▶ $r(R) = \log_2(w(X) + w(A) + w(B) + w(C) + w(R))$
- ▶ $r'(R) = \log_2(w(R) + w(B) + w(C))$
- ▶ $r(X) \leq r'(X), r'(R) \leq r(R)$

小課題3 (60点) – Splay木の解析

- ▶ アクセス補題の証明 (1) “zig”-step の場合
- ▶ $r(X) \leq r'(X), r'(R) \leq r(R)$
- ▶ ならし計算量

$$\begin{aligned} a &= t + \Phi' - \Phi \\ &= 1 + r'(X) + r'(R) - r(X) - r(R) \\ &\leq 1 + r'(X) - r(X) \\ &\leq 1 + 3r'(X) - 3r(X) \end{aligned}$$

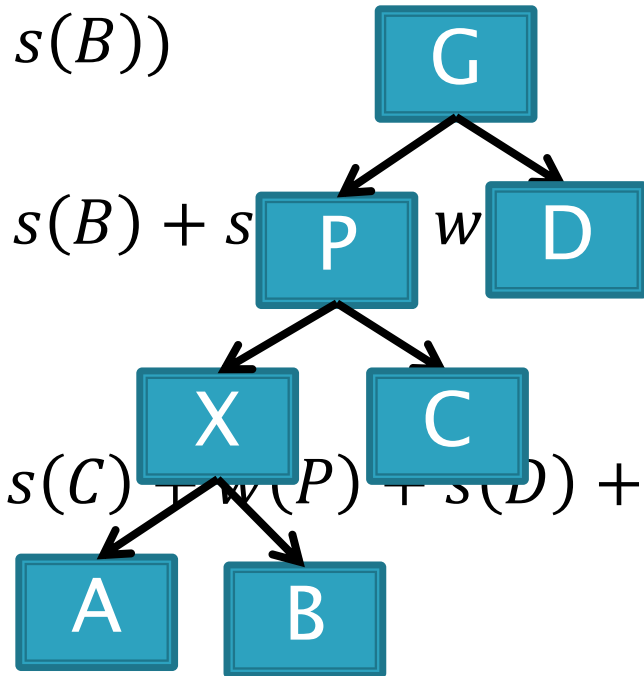
小課題3 (60点) – Splay木の解析

- ▶ アクセス補題の証明 (2) “zigzig”-step の場合

- ▶ $r(X) = \log_2(w(X) + s(A) + s(B))$

- ▶ $r(P) = \log_2(w(X) + s(A) + s(B) + s(C))$

- ▶ $r(G) = \log_2(w(X) + s(A) + s(B) + s(C) + w(P) + s(D) + w(G))$



小課題3 (60点) – Splay木の解析

- ▶ アクセス補題の証明 (2) “zigzig”-step の場合

- ▶ $r(X) = \log_2(w(X) + s(A) + s(B))$

- ▶ $r'(X) =$

$$\log_2(w(X) + s(A) + s(B) + s(C) + w(P) + s(P) + w(G))$$

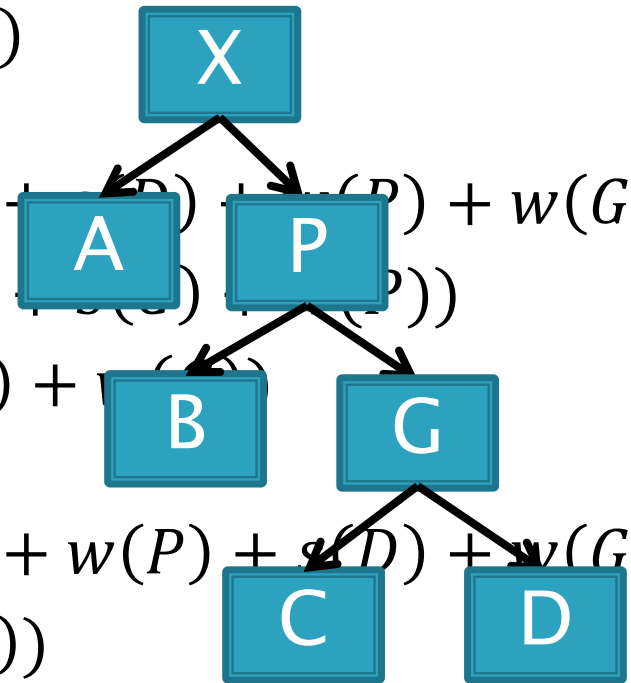
- ▶ $r(P) = \log_2(w(X) + s(A) + s(B) + s(C) + w(P) + s(P))$

- ▶ $r'(P) = \log_2(s(B) + s(C) + w(P) + s(P))$

- ▶ $r(G) =$

$$\log_2(w(X) + s(A) + s(B) + s(C) + w(P) + s(P) + w(G))$$

- ▶ $r'(G) = \log_2(s(C) + s(D) + w(G))$



小課題3 (60点) – Splay木の解析

- ▶ アクセス補題の証明 (2) “zigzig”-step の場合
- ▶ $r(X) = \log_2(w(X) + s(A) + s(B))$
- ▶ $r'(X) =$
 $\log_2(w(X) + s(A) + s(B) + s(C) + s(D) + w(P) + w(G))$
- ▶ $r(P) = \log_2(w(X) + s(A) + s(B) + s(C) + w(P))$
- ▶ $r'(P) = \log_2(s(B) + s(C) + w(P) + w(G))$
- ▶ $r(G) =$
 $\log_2(w(X) + s(A) + s(B) + s(C) + w(P) + s(D) + w(G))$
- ▶ $r'(G) = \log_2(s(C) + s(D) + w(G))$

小課題3 (60点) – Splay木の解析

- ▶ アクセス補題の証明 (2) “zigzig”-step の場合
- ▶ $r'(X) = r(G), r'(X) \leq r'(P), r(P) \leq r(X)$

小課題3 (60点) – Splay木の解析

- ▶ アクセス補題の証明 (2) “zigzig”-step の場合
- ▶ $r'(X) = r(G), r'(P) \leq r'(X), r(X) \leq r(P)$

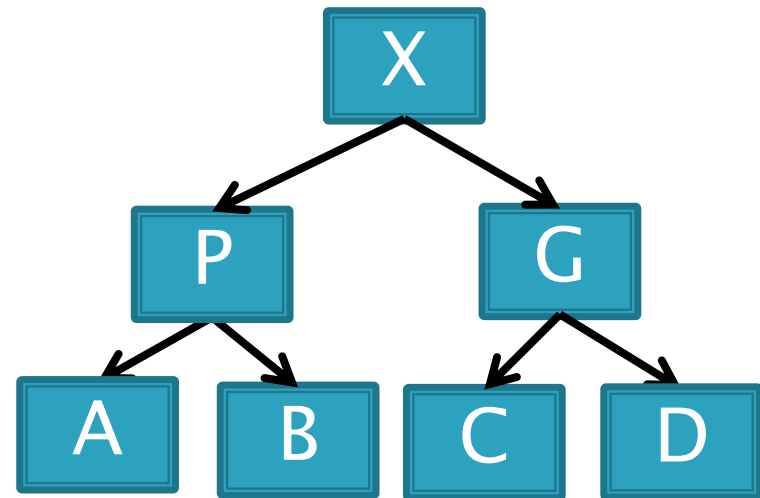
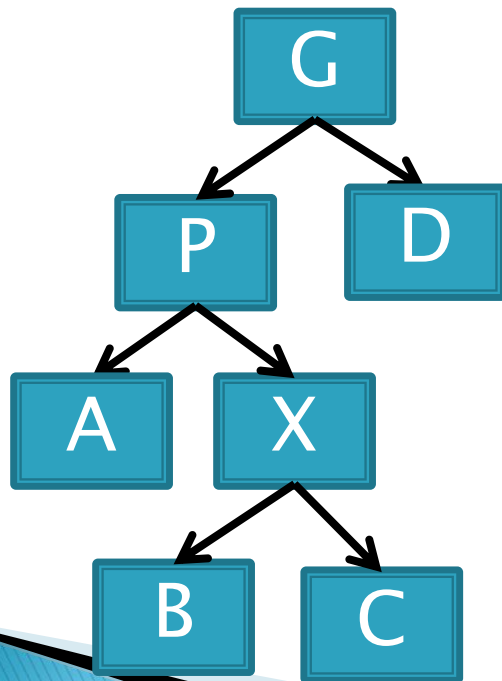
$$\begin{aligned} a &= t + \Phi' - \Phi \\ &= 2 + r'(X) + r'(P) + r'(G) - r(X) - r(P) \\ &\quad - r(G) \\ &\leq 2 + r'(X) + r'(G) - 2r(X) \end{aligned}$$

小課題3 (60点) – Splay木の解析

- ▶ アクセス補題の証明 (2) “zigzig”-step の場合
- ▶ $2 + r'(X) + r'(G) - 2r(X) \leq 3r'(X) - 3r(X)$
- ▶ 理由: $r'(G) + r(X) - 2r'(X) \leq -2$ を示したい。
- ▶ ところで左辺は $\log_2 \left(\frac{s'(G)}{s'(X)} \right) + \log_2 \left(\frac{s(X)}{s'(X)} \right)$ であり、
- ▶ $\log_2 x$ が上に凸で、 $s'(G) + s(X) \leq s'(X)$ なのでこの値は高々 -2
- ▶ よって不等式は示された。

小課題3 (60点) – Splay木の解析

- ▶ アクセス補題の証明 (3) “zigzag”-step の場合
- ▶ $r'(X) = r(G), r(X) \leq r(P)$
- ▶ $s'(P) + s'(G) \leq s'(X)$



小課題3 (60点) – Splay木の解析

- ▶ アクセス補題の証明 (3) “zigzag”-step の場合
- ▶ 以下(2)と同様

小課題3 (60点) – Splay木の解析

- ▶ 以上より、Splay操作がならし計算量で $O(\log N)$ であることがわかった。
- ▶ ところで、Splay操作のポテンシャルは高々 $O(N \log N)$ なので、全体で $O((Q + N) \log N)$ でクエリを処理できることがわかった。

小課題3 (60点) – Splay木の解析

- ▶ 以上より、Splay操作がならし計算量で $O(\log N)$ であることがわかった。
- ▶ ところで、Splay操作のポテンシャルは高々 $O(N \log N)$ なので、全体で $O((Q + N) \log N)$ でクエリを処理できることがわかった。
- ▶ 以上、満点解法その1

小課題3 (60点) - L/C木

- ▶ 満点解法その2 - Link/Cut木

小課題3 (60点) - L/C木

- ▶ 満点解法その2 - Link/Cut木
- ▶ Link/Cut木
 - 元々、フローアルゴリズムの高速化のためにSleatorとTarjanが考案したもの

小課題3 (60点) - L/C木

- ▶ 満点解法その2 - Link/Cut木
- ▶ Link/Cut木
 - 元々、フローアルゴリズムの高速化のためにSleatorとTarjanが考案したもの
 - この問題のために必要な実装は、それよりもはるかに容易

小課題3 (60点) - L/C木

- ▶ 満点解法その2 - Link/Cut木
- ▶ Link/Cut木
 - 元々、フローアルゴリズムの高速化のためにSleatorとTarjanが考案したもの
 - この問題のために必要な実装は、それよりもはるかに容易
→Link/Cut木の練習としても適している

小課題3 (60点) - L/C木

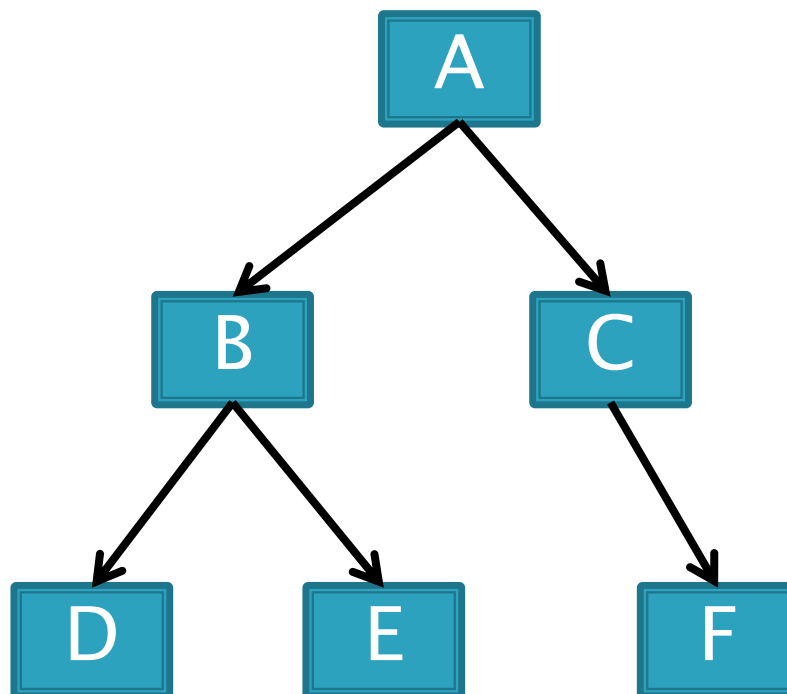
- ▶ 満点解法その2 - Link/Cut木
- ▶ Link/Cut木
 - 元々、フローアルゴリズムの高速化のためにSleatorとTarjanが考案したもの
 - この問題のために必要な実装は、それよりもはるかに容易
→Link/Cut木の練習としても適している
 - いろいろなバージョンがあるが、Splay木によるものが使いやすい

小課題3 (60点) – H/L分解

- ▶ 予備知識 – Heavy/Light decomposition

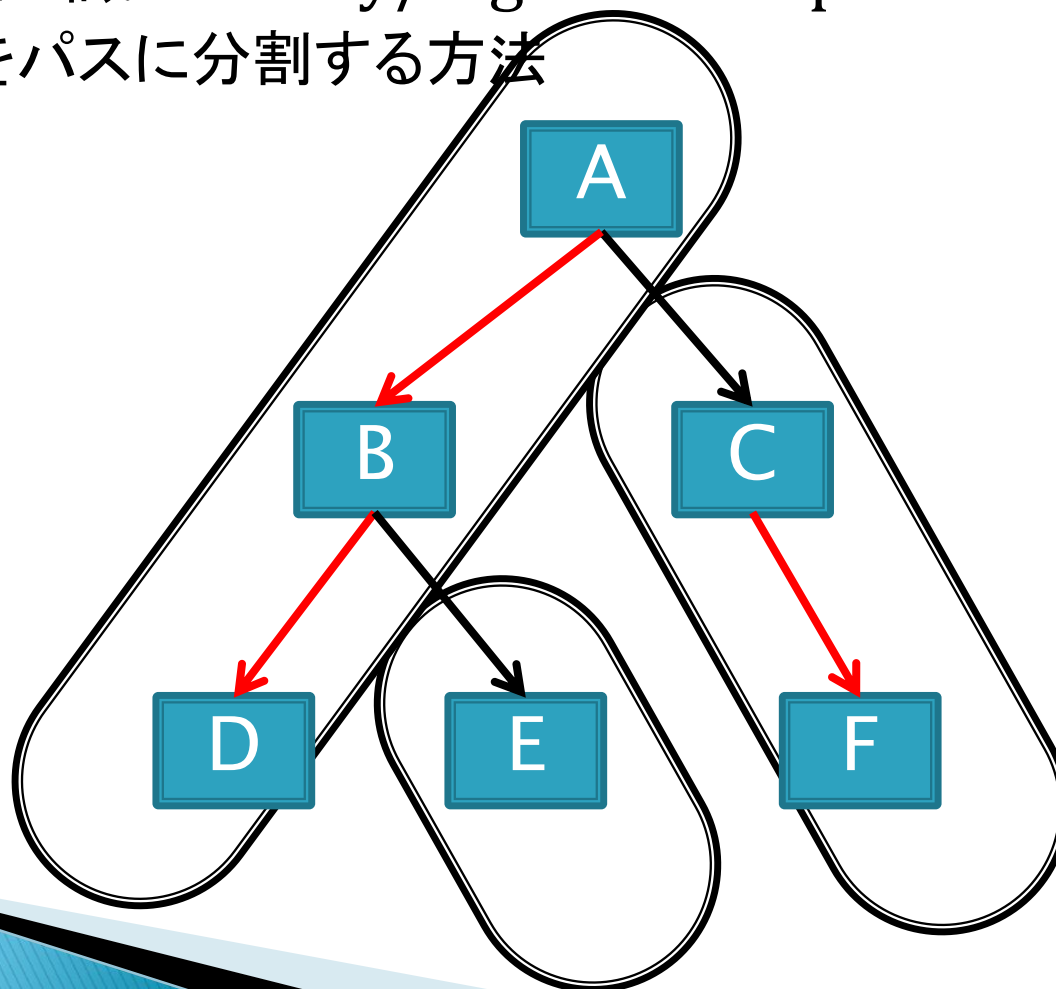
小課題3 (60点) – H/L分解

- ▶ 予備知識 – Heavy/Light decomposition
 - 木をパスに分割する方法



小課題3 (60点) – H/L分解

- ▶ 予備知識 – Heavy/Light decomposition
 - 木をパスに分割する方法



小課題3 (60点) – H/L分解

- ▶ 予備知識 – Heavy/Light decomposition
 - 木をパスに分割する方法
- ▶ 変な形の木でも、「パスの木」の形に潰すと安定する

小課題3 (60点) – L/C木

▶ Splay Treeの世界

列データ

Splay Tree

列の畳み込みを効率よく計算

小課題3 (60点) – L/C木

▶ Splay Treeの世界

列データ

Splay Tree

列の畳み込みを効率よく計算

小課題3 (60点) – L/C木

▶ Heavy/Light decompositionの世界

有向木



列データ

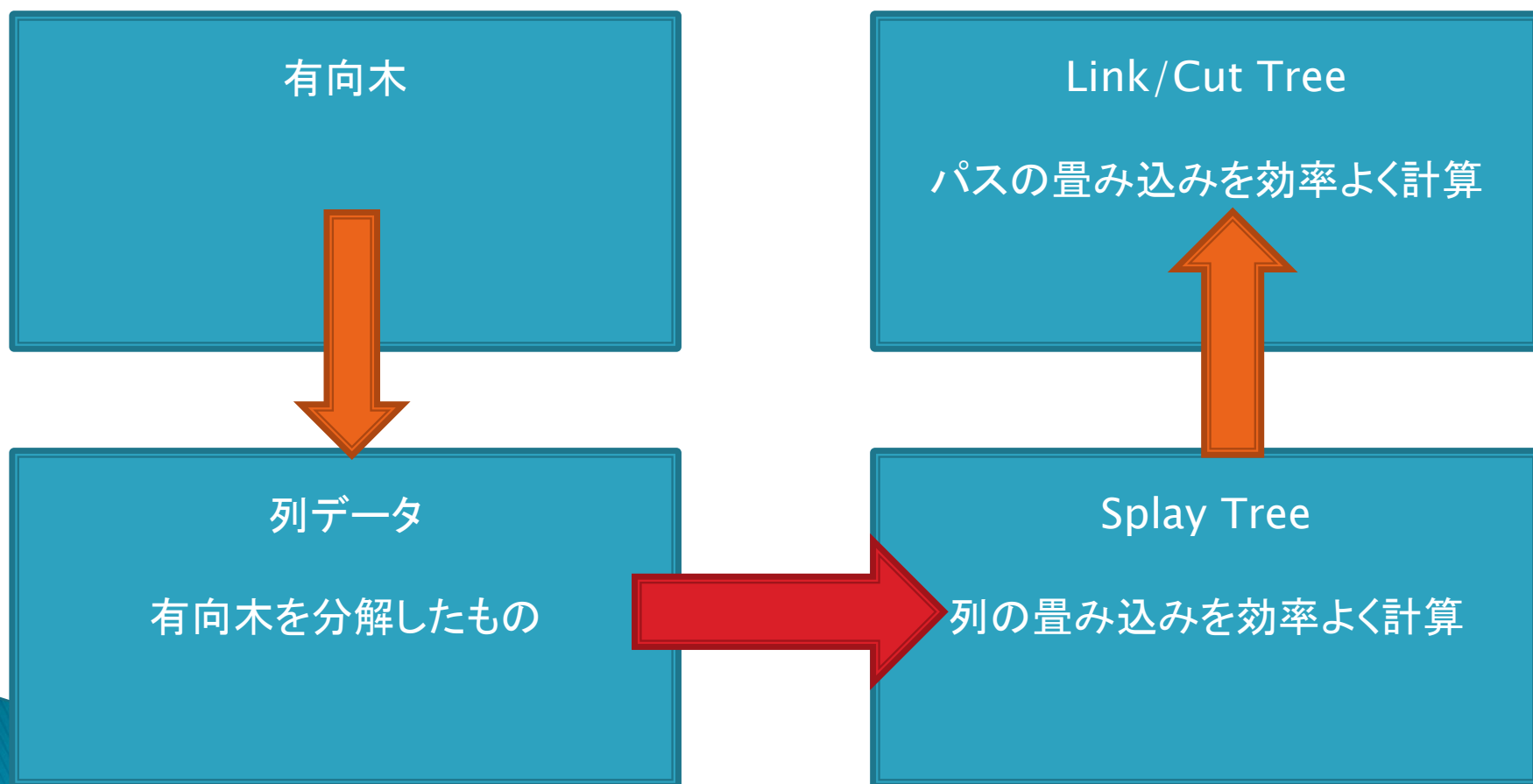
有向木を分解したもの

Splay Tree

列の畳み込みを効率よく計算

小課題3 (60点) – L/C木

▶ Link/Cut Treeの世界



小課題3 (60点) – L/C木

- ▶ Link/Cut Treeの世界
- ▶ H/L分解 = パスからなる木

小課題3 (60点) – L/C木

- ▶ Link/Cut Treeの世界
- ▶ H/L分解 = パスからなる木
- ▶ Link/Cut Tree = Splay木からなる木

小課題3 (60点) – L/C木の実装

- ▶ Link/Cut Tree の辺は二種類ある
 - Solid(Heavy) edge
 - Dashed(Light) edge

小課題3 (60点) – L/C木の実装

- ▶ Link/Cut Tree の辺は二種類ある

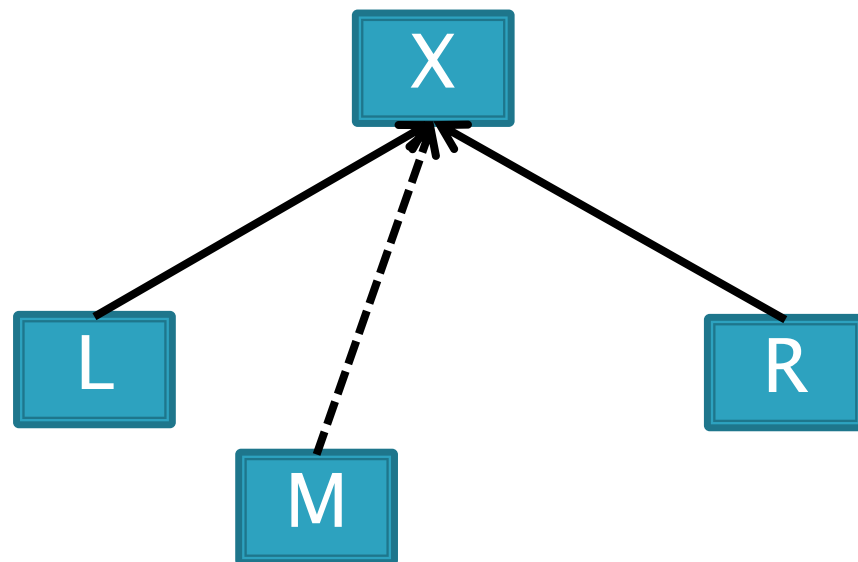
	Solid(Heavy)	Dashed(Light)
所属	Splay Tree	H/L分解の木
分類	二分木	多分木
左右の区別	左右の区別あり	なし
親	本当は祖先か子孫	本当の親
子供	本当は祖先か子孫	本当は子孫

小課題3 (60点) – L/C木の実装

- ▶ Solid, Dashedの区別

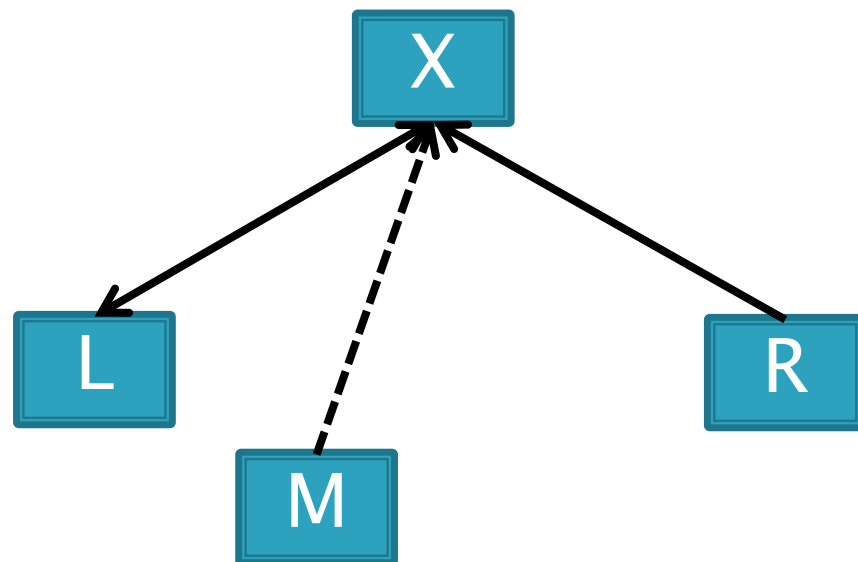
小課題3 (60点) – L/C木の実装

- ▶ Solid, Dashedの区別
- ▶ いずれも、親方向リンクを持つ



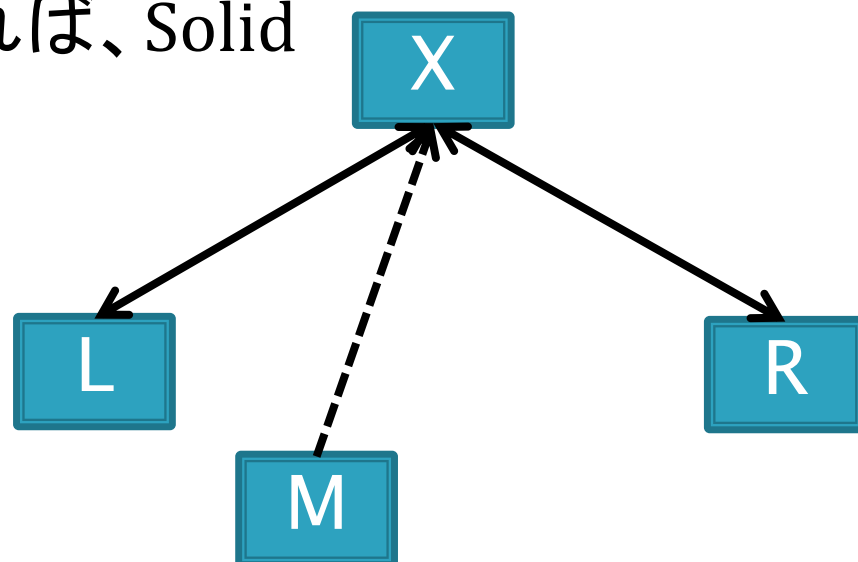
小課題3 (60点) – L/C木の実装

- ▶ Solid, Dashedの区別
- ▶ いずれも、親方向リンクを持つ
- ▶ 親から左方向リンクがあれば、Solid



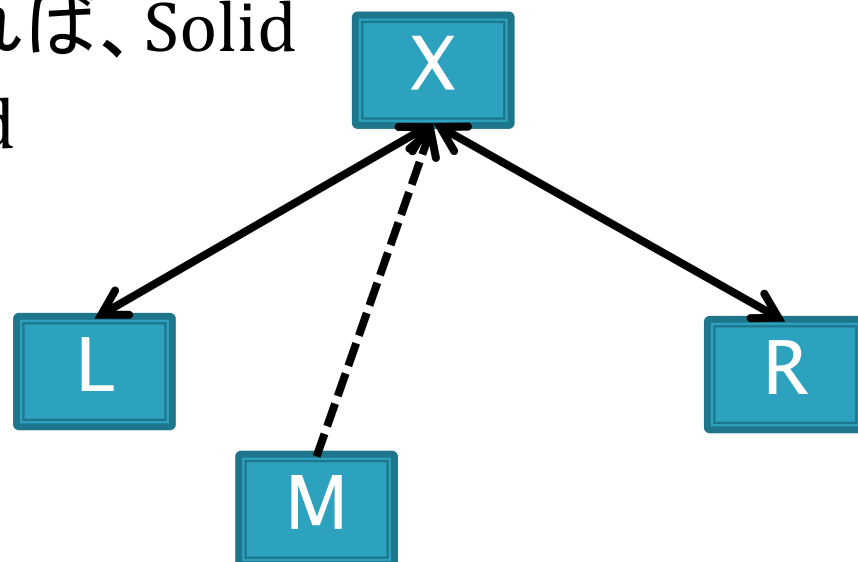
小課題3 (60点) – L/C木の実装

- ▶ Solid, Dashedの区別
- ▶ いずれも、親方向リンクを持つ
- ▶ 親から左方向リンクがあれば、Solid
- ▶ 親から右方向リンクがあれば、Solid



小課題3 (60点) – L/C木の実装

- ▶ Solid, Dashedの区別
- ▶ いずれも、親方向リンクを持つ
- ▶ 親から左方向リンクがあれば、Solid
- ▶ 親から右方向リンクがあれば、Solid
- ▶ どちらもなければ、Dashed

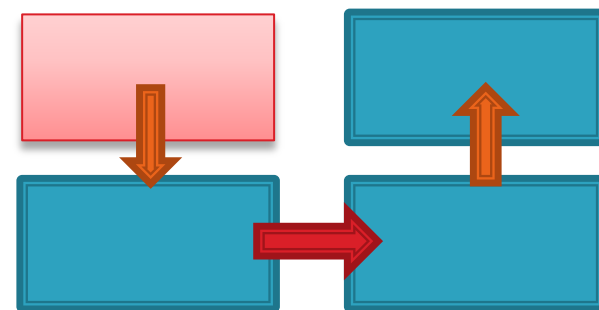
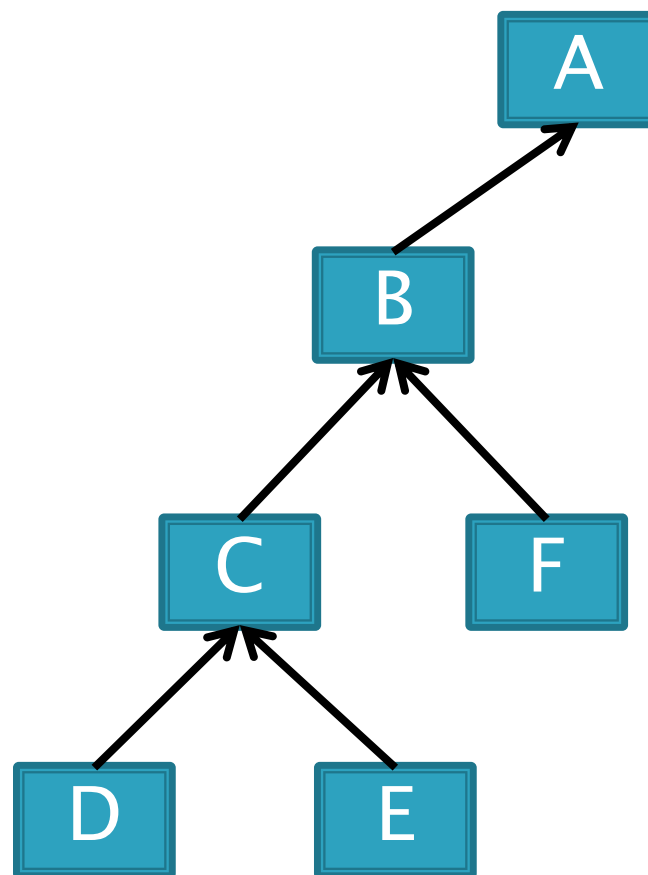


小課題3 (60点) – L/C木の実装

- ▶ Solid, Dashedの区別
- ▶ いずれも、親方向リンクを持つ
- ▶ 親から左方向リンクがあれば、Solid
- ▶ 親から右方向リンクがあれば、Solid
- ▶ どちらもなければ、Dashed
- ▶ 「右, 左, 親」の3つのリンクだけで構造を保持できる！

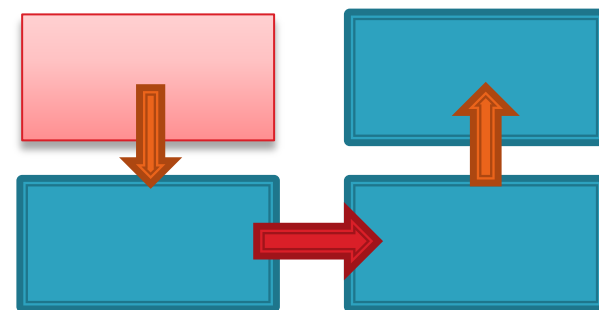
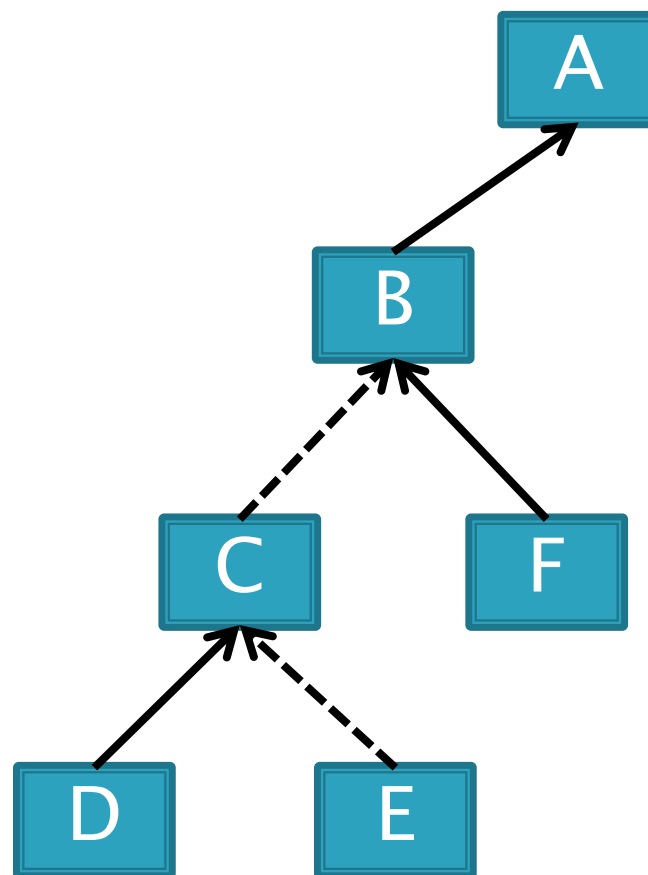
小課題3 (60点) - L/C木の実装

▶ 小さな例



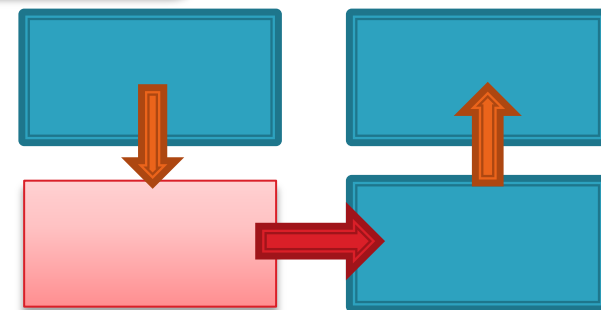
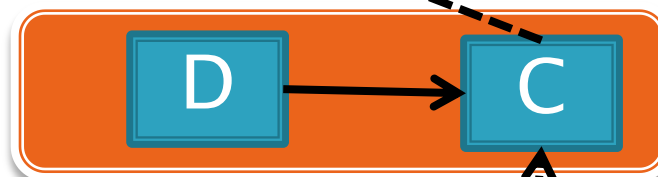
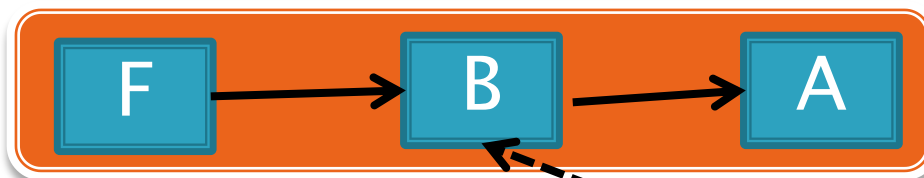
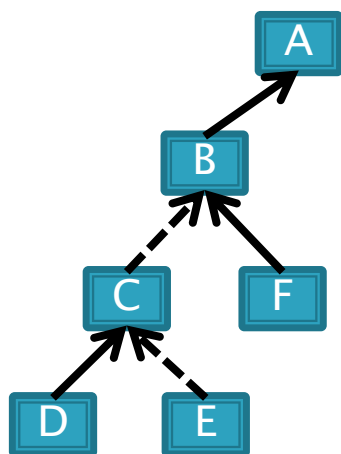
小課題3 (60点) - L/C木の実装

▶ 小さな例



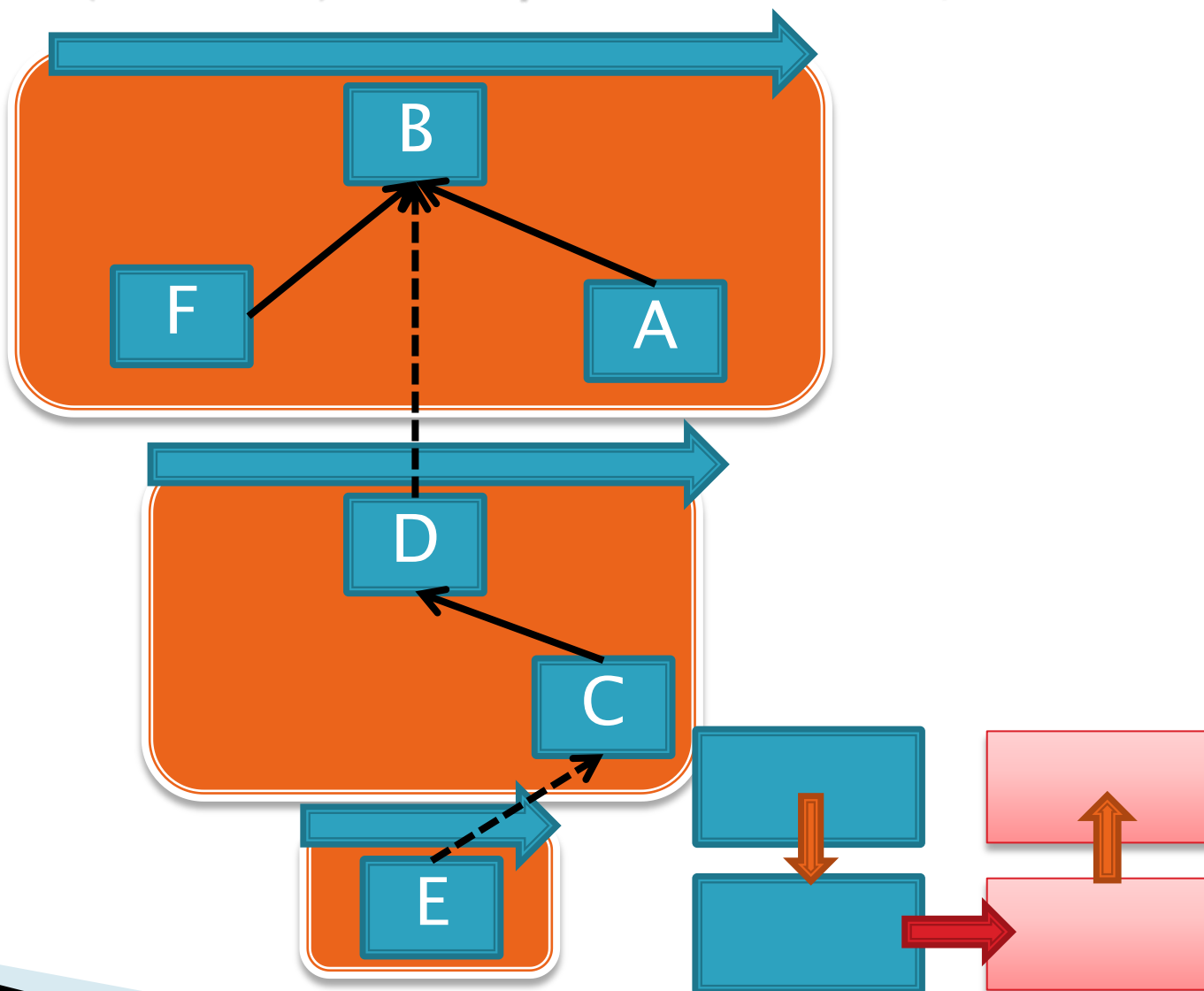
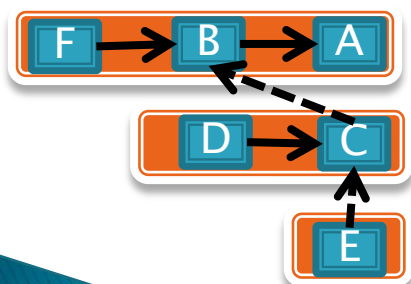
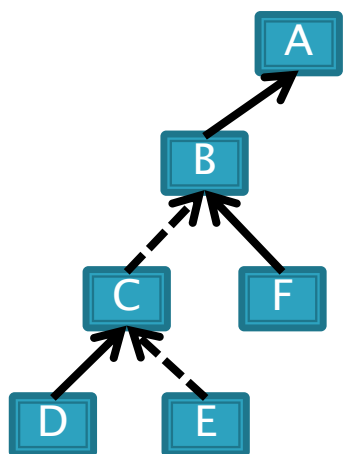
小課題3 (60点) - L/C木の実装

▶ 小さな例



小課題3 (60点) - L/C木の実装

▶ 小さな例



小課題3 (60点) – L/C木の実装

- ▶ Link/Cut Treeの操作: `splayLC()`
- ▶ Link/Cut Tree上では、任意の頂点Xを根に持っていくことができる
 - 元の木構造は変化しないことに注意

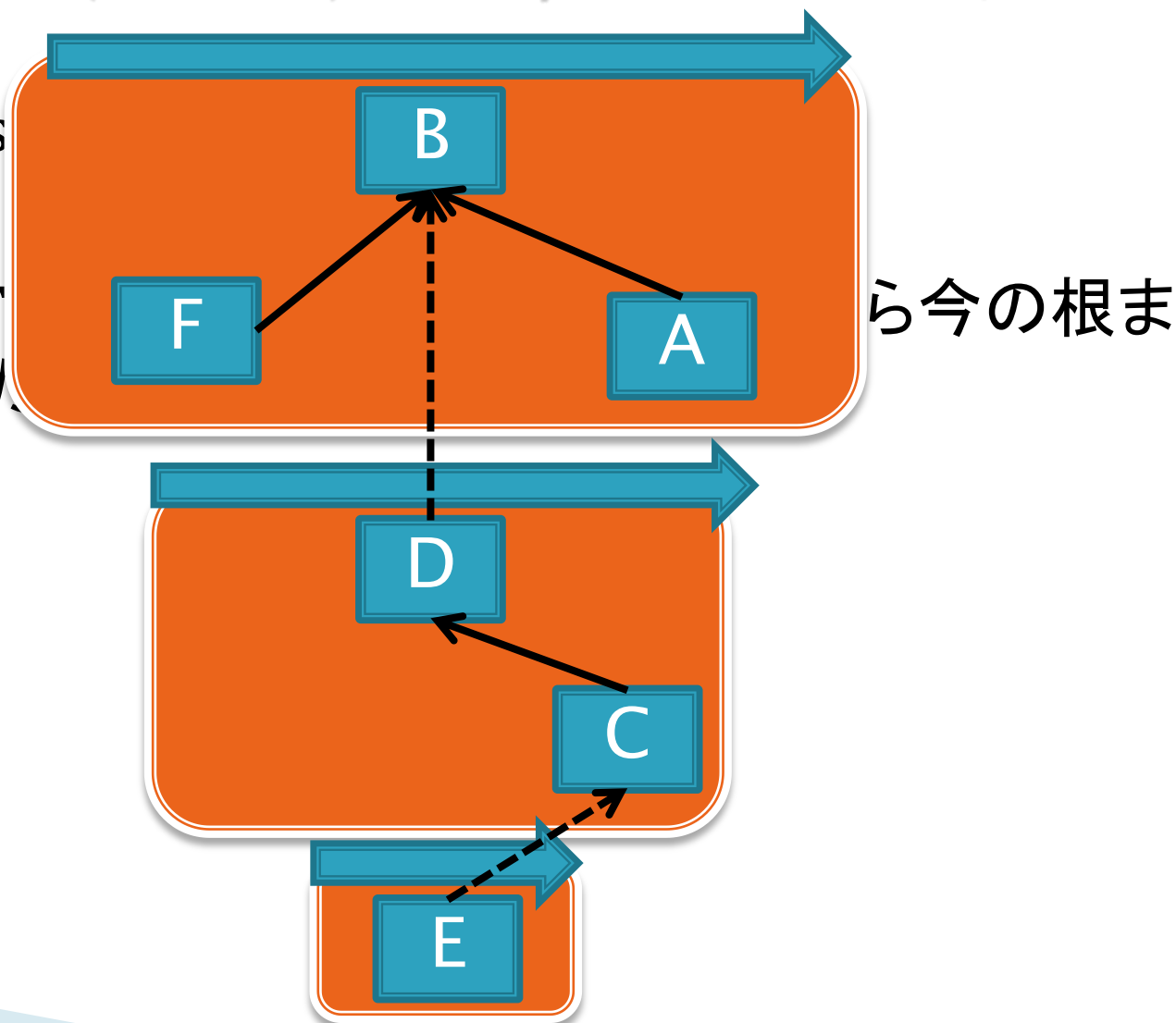
小課題3 (60点) – L/C木の実装

- ▶ 前準備: splay
- ▶ 各Splay Tree上でsplayをすることで、Xから今の根までを点線だけで行けるようにする

小課題3 (60点) - L/C木の実装

▶ 前準備: s

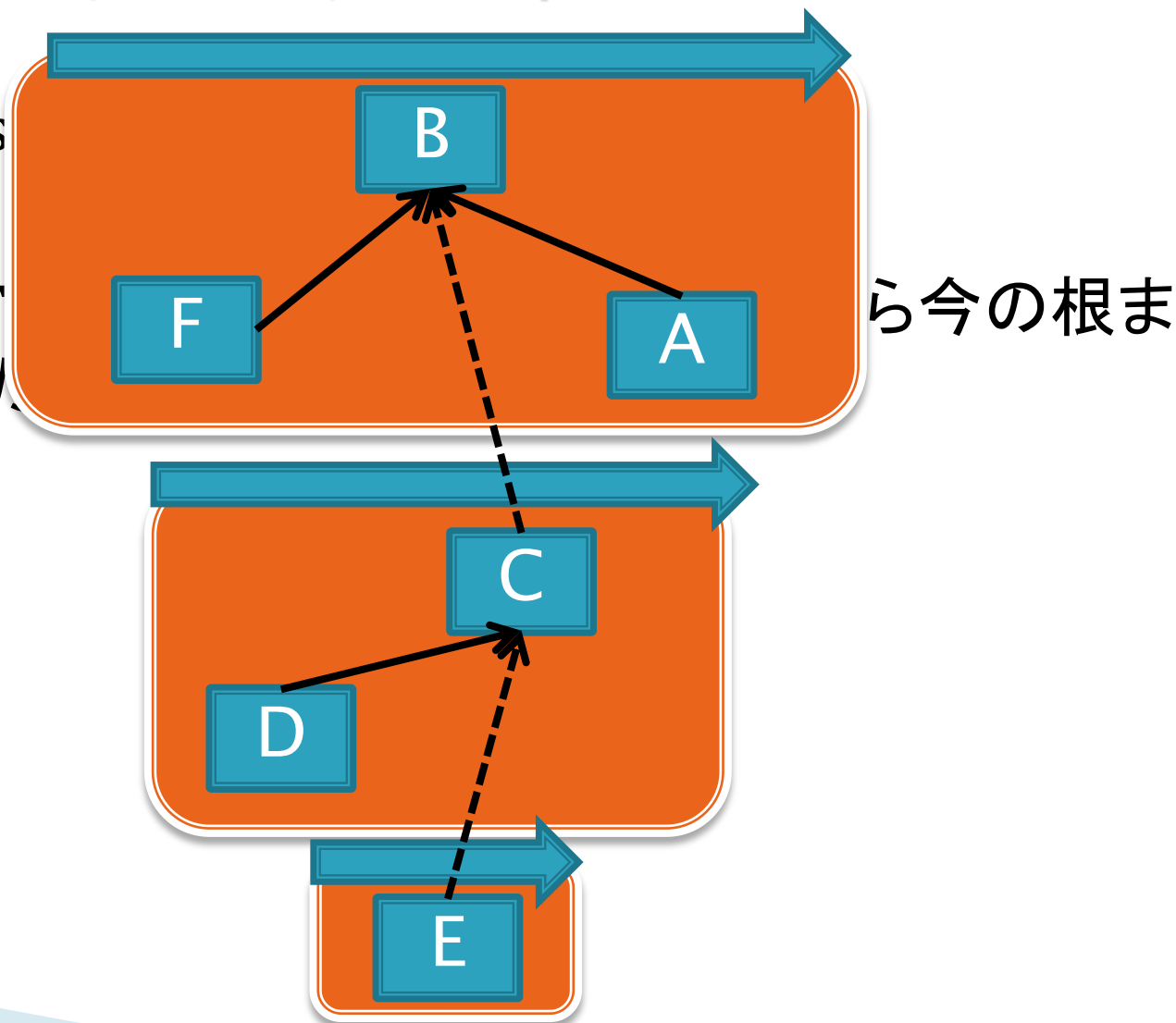
▶ 各Splay Tree
で点を点線



小課題3 (60点) - L/C木の実装

▶ 前準備: s

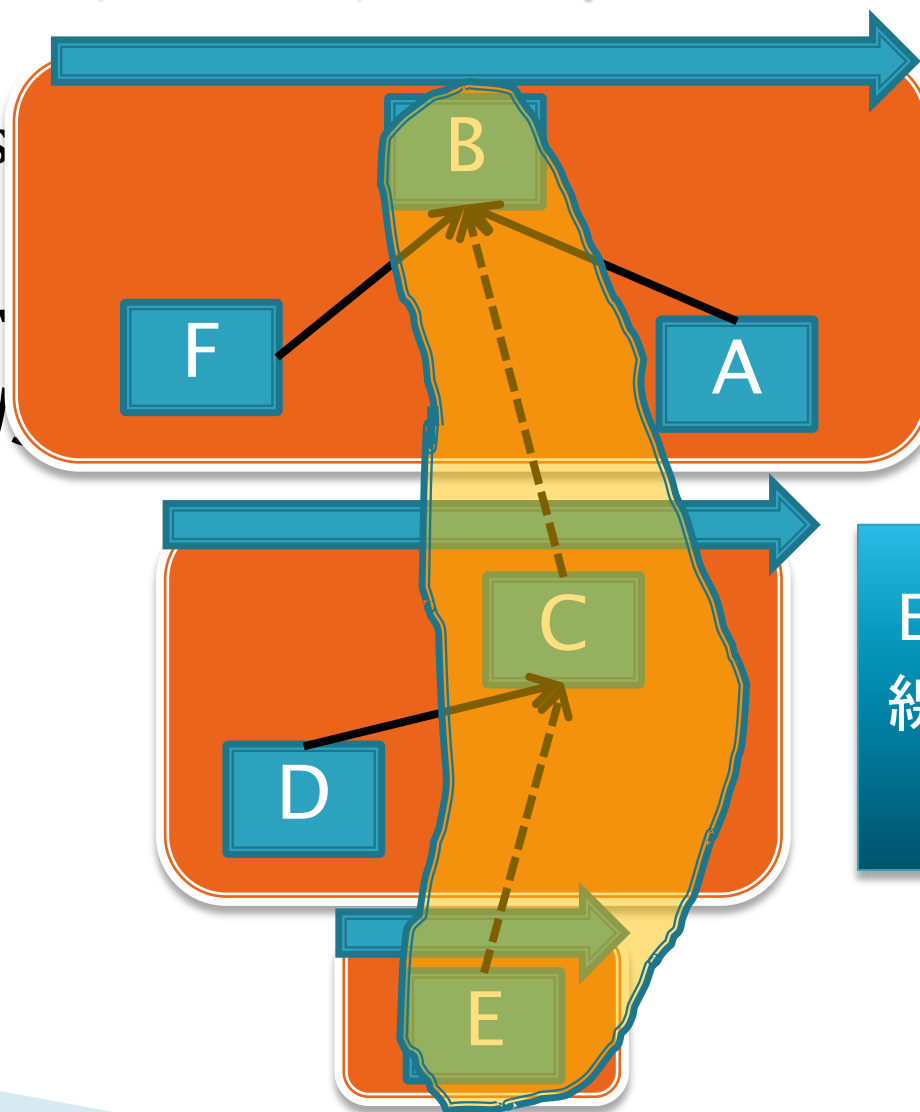
▶ 各Splay Tree
で点を点線



小課題3 (60点) – L/C木の実装

▶ 前準備: s

▶ 各Splay Tree
で点を点線



ら今の根ま

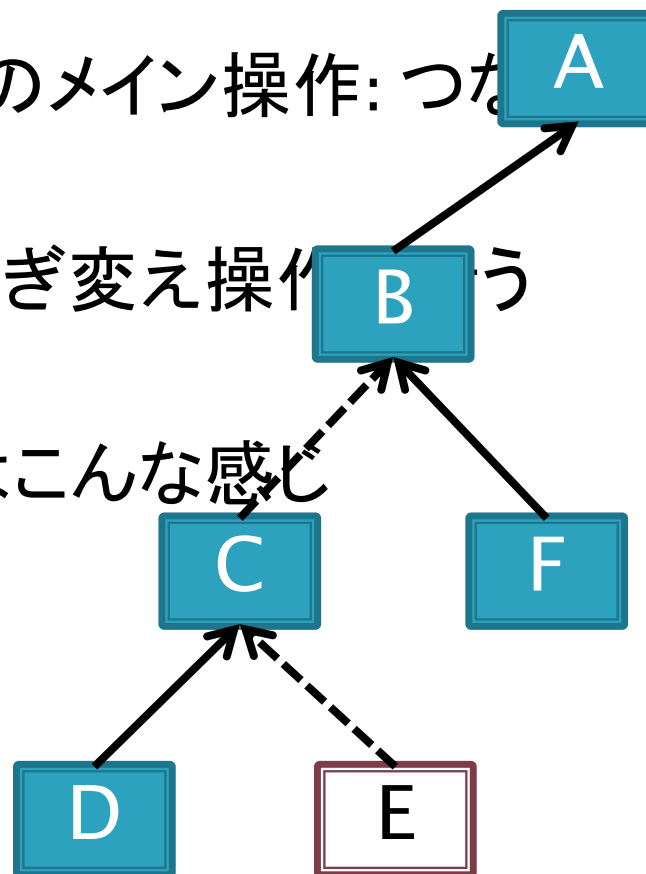
EからBまで、点
線のみで行ける
ようになる

小課題3 (60点) – L/C木の実装

- ▶ `splayLC()` のメイン操作: つなぎ変え(expose操作)
- ▶ パスのつなぎ変え操作を行う
- ▶ 元の木ではこんな感じ

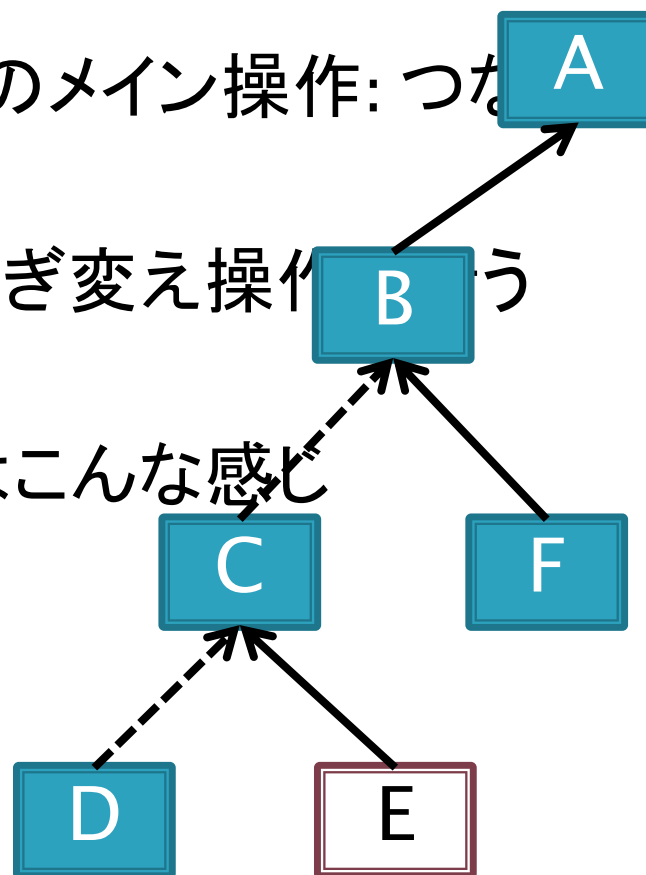
小課題3 (60点) – L/C木の実装

- ▶ `splayLC()` のメイン操作: つなぎ変え (expose操作)
- ▶ パスのつなぎ変え操作
- ▶ 元の木ではこんな感じ



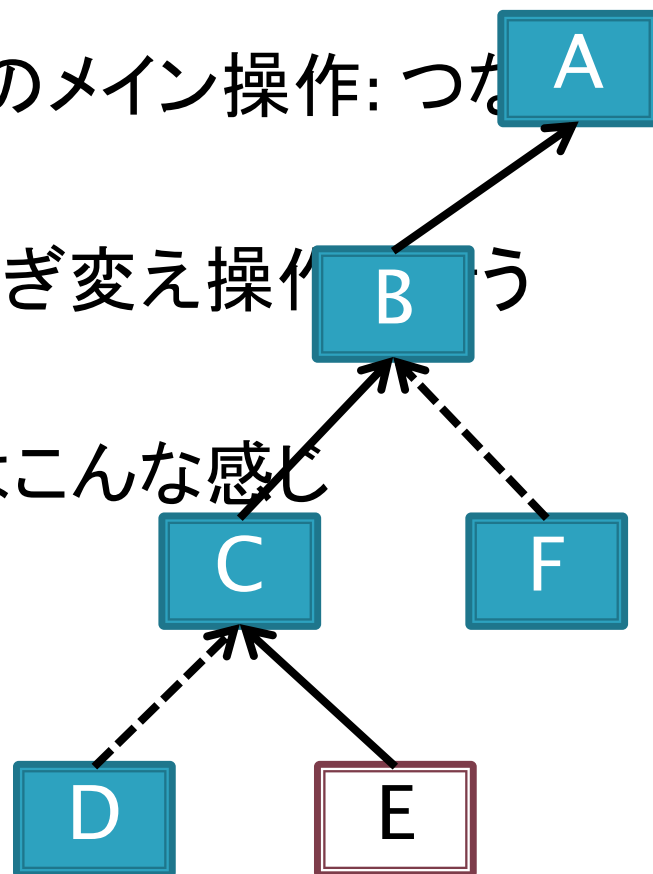
小課題3 (60点) – L/C木の実装

- ▶ `splayLC()` のメイン操作: つなぎ変え (expose操作)
- ▶ パスのつなぎ変え操作
- ▶ 元の木ではこんな感じ



小課題3 (60点) – L/C木の実装

- ▶ `splayLC()` のメイン操作: つなぎ変え (expose操作)
- ▶ パスのつなぎ変え操作
- ▶ 元の木ではこんな感じ



小課題3 (60点) – L/C木の実装

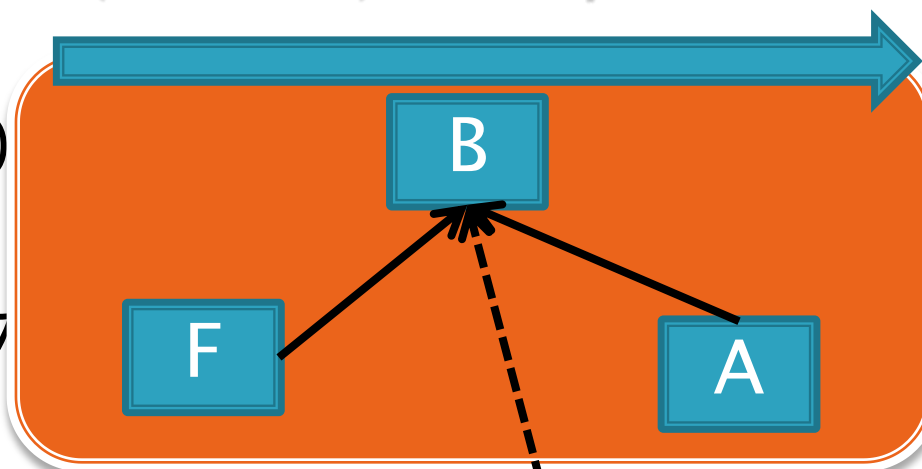
- ▶ `splayLC()` のメイン操作: つなぎ変え(expose操作)
- ▶ パスのつなぎ変え操作を行う
- ▶ Link/Cut 木でも、左向きのSolid辺を付け替えるだけ

小課題3 (60点) - L/C木の実装

▶ splayLC()

▶ パスのつ

▶ Link/Cut 木で



ose操作)

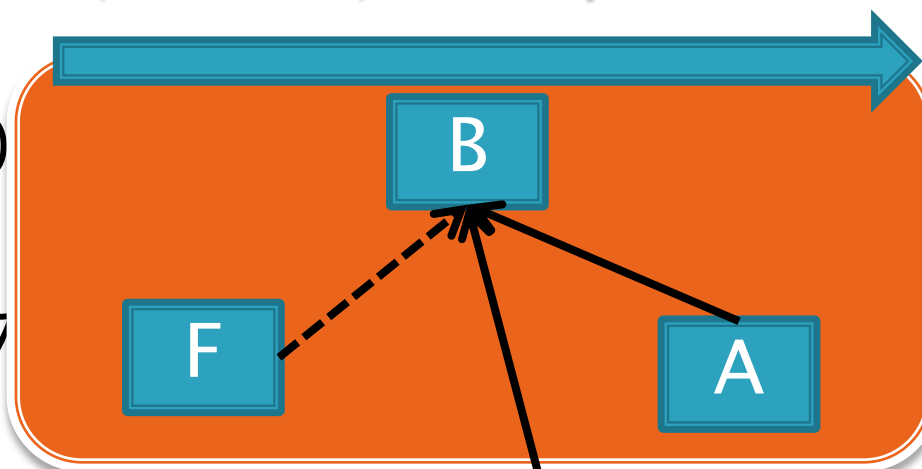
付け替えるだけ

小課題3 (60点) - L/C木の実装

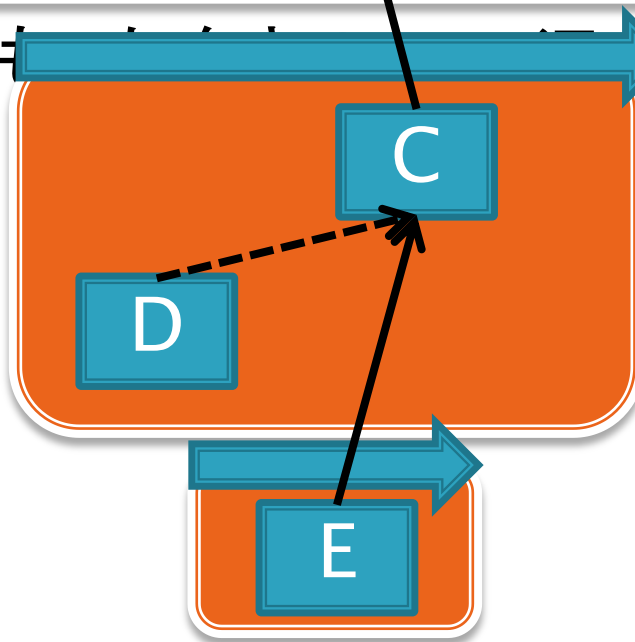
▶ splayLC()

▶ パスのつ

▶ Link/Cut 木で



ose操作)



付け替えるだけ

小課題3 (60点) - L/C木の実装

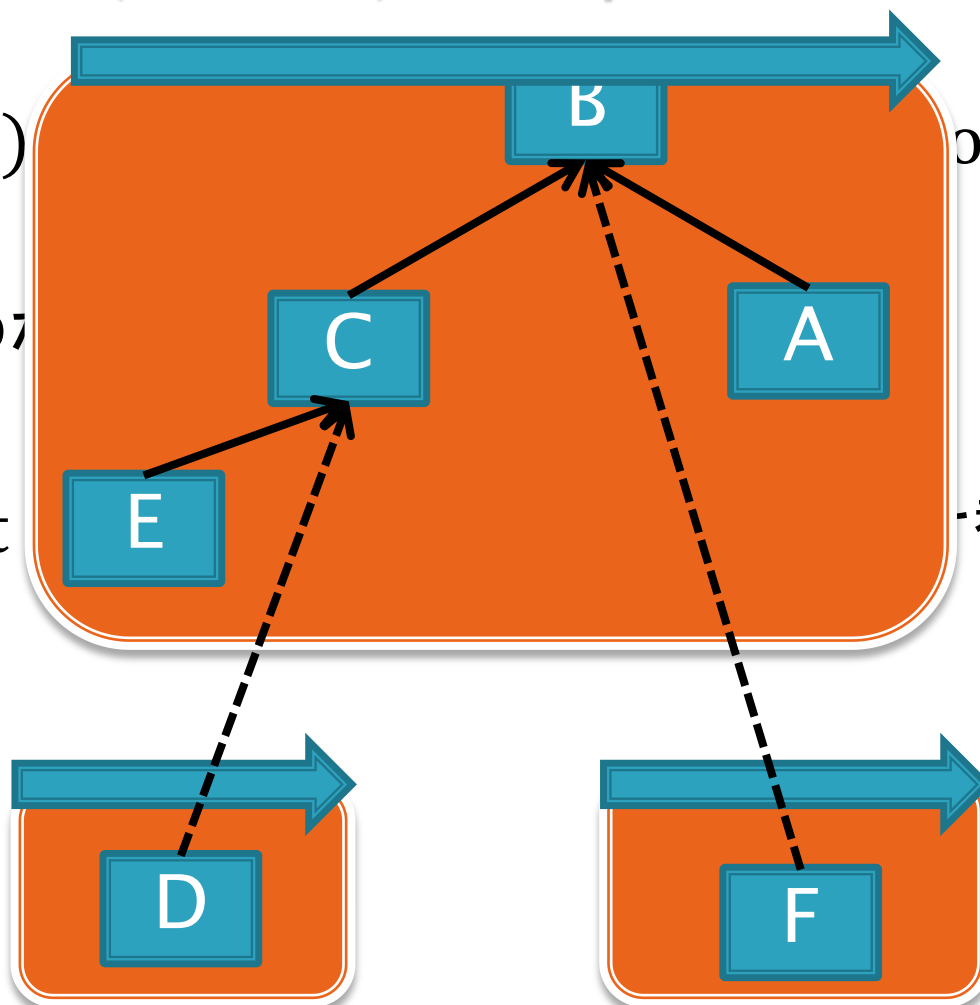
▶ splayLC()

▶ パスのつ

▶ Link/Cut

ose操作)

替えるだけ



小課題3 (60点) – L/C木の実装

- ▶ 左向きの辺をつなぎ替えるだけで、Eが一番上の木に所属するようになった

小課題3 (60点) – L/C木の実装

- ▶ 左向きの辺をつなぎ替えるだけで、Eが一番上の木に所属するようになった
- ▶ 最後にもう1度splay()操作を行うことで、Link/Cut Treeの一番上の根にEが来る

小課題3 (60点) – L/C木の解析

- ▶ L/C木の`splayLC()`はSplay Treeの解析を少し応用すると、対数時間であることが言える

小課題3 (60点) – L/C木の解析

- ▶ L/C木の`splayLC()`はSplay Treeの解析を少し応用すると、対数時間であると言える
- ▶ 1回ごとの`splay()`操作が対数時間であることは既にわかっている

小課題3 (60点) – L/C木の解析

- ▶ L/C木の`splayLC()`はSplay Treeの解析を少し応用すると、対数時間であると言える
- ▶ 1回ごとの`splay()`操作が対数時間であることは既にわかっている
- ▶ しかし実際には`splay()`操作が k 回呼ばれている
 - k は、パス分割された木の上での深さ

小課題3 (60点) – L/C木の解析

▶ ポテンシャルの定義

- サイズ = solid/dashedに関わらず、子孫になっている頂点の数
- ランク = その対数
- ポテンシャル = ランクの総和の2倍として定める

小課題3 (60点) – L/C木の解析

- ▶ Splay Treeのならし計算量は $1 + 3r(t) - 3r(x)$ だった
- ▶ 今回のならし計算量は $k + 6r(t) - 6r(x)$ になる
 - 「Splayが k 回呼ばれる」という認識を改めてみる
 - Splayは根に向かって順番に呼ばれるということを考慮すると、「Splayが1回呼ばれるが、途中で k 回、強制的にzigステップを使われるかもしれない」と考えることができる
 - 係数が2倍なのはポテンシャルの定義を変えたから

小課題3 (60点) – L/C木の解析

- ▶ 余った定数項 k の回収
- ▶ Expose操作のあとに1回行うsplay操作: k 回の回転を行う。
- ▶ ポテンシャルの定義を2倍にしたので、splayの回転操作1回につき1の追加コストを課しても問題ない

小課題3 (60点) – L/C木の解析

- ▶ ならしコスト $6 \log_2 N$ の splay 操作を2回呼んでいるので、splayLC() のならし計算量は $12 \log_2 N = O(\log N)$ であるとわかった。

小課題3 (60点) – LCAを求める

- ▶ AとBのLCAを求める。

小課題3 (60点) – LCAを求める

- ▶ AとBのLCAを求めるには、まず
 1. Bに対してsplayLC()を行う
 2. Aに対してsplayLC()を行う
- ▶ このとき、Bは浅い位置にいる。
 - Splay Treeに対するSplay操作1回で、他の頂点の深さは高々2段しか下がらないので、この時点でBは深さ高々4程度。

小課題3 (60点) – LCAを求める

- ▶ AとBの位置関係に基づいて条件分岐

小課題3 (60点) – LCAを求める

- ▶ AとBの位置関係に基づいて条件分岐
- ▶ (1) BがAの左側にある場合
 - この場合は、BはAの子孫ということになるので、AとBのLCAはAになる。
- ▶ (2) BがAの右側にある場合
 - 次のページへ

小課題3 (60点) – LCAを求める

- ▶ BがAの右側にある場合の条件分岐
- ▶ (1) BがAと同じSplay Treeに属する場合
 - この場合は、AはBの子孫ということになるので、AとBのLCAはBになる。

小課題3 (60点) – LCAを求める

- ▶ BがAの右側にある場合の条件分岐
- ▶ (1) BがAと同じSplay Treeに属する場合
 - この場合は、AはBの子孫ということになるので、AとBのLCAはBになる。
- ▶ (2) BがAと異なるSplay Treeに属する場合
 - 一番一般的な場合。
 - Bから上に辿り、Aと同じSplay Treeに到達したところの頂点が、AとBのLCAになる。

小課題3 (60点) – LCAを求める

- ▶ BがAの右側にある場合の条件分岐
- ▶ (1) BがAと同じSplay Treeに属する場合
 - この場合は、AはBの子孫ということになるので、AとBのLCAはBになる。
- ▶ (2) BがAと異なるSplay Treeに属する場合
 - 一番一般的な場合。
 - Bから上に辿り、Aと同じSplay Treeに到達したところの頂点が、AとBのLCAになる。
- ▶ これでLCAは求められた。

小課題3 (60点) – 木の操作

- ▶ クエリ1,2番に対応する「接続」「切断」は、Link/Cut Treeの”link”, “cut” に対応する。

小課題3 (60点) – 木の操作

- ▶ クエリ1,2番に対応する「接続」「切断」は、Link/Cut Treeの”link”, “cut” に対応する。
- ▶ (1) Link操作 – AをBの子にする
 - AとBをsplayLC()しておいてから、Aの親として(dashedで)Bを設定するだけ。
 - 計算量: AとBがLink/Cut Treeにおける根にあるので、Bのサイズが高々 N 増える程度。これによってポテンシャルは $O(\log N)$ しか増えない。

小課題3 (60点) – 木の操作

- ▶ クエリ1,2番に対応する「接続」「切断」は、Link/Cut Treeの”link”, “cut” に対応する。
- ▶ (2) Cut操作 – Aを親から切り離す
 - AをsplayLC()してからAの右の子を切り離す。
 - 計算量:ポテンシャルは明らかに減っている。

小課題3 (60点)

- ▶ 以上がLink/Cut Treeによる満点解法。

Euler Tour vs. Link/Cut

- ▶ Euler Tour Tree と Link/Cut Tree は動的木の筆頭

Euler Tour vs. Link/Cut

- ▶ Euler Tour Tree と Link/Cut Tree は動的木の筆頭
- ▶ 今回はどちらを選ぶべきだったか？

Euler Tour vs. Link/Cut

- ▶ Euler Tour Tree と Link/Cut Tree は動的木の筆頭
- ▶ 今回はどちらを選ぶべきだったか？
- ▶ (他の問題は解き終わっているとして)

Euler Tour vs. Link/Cut

▶ Euler Tour Tree

- 知識:
- 実装:

▶ Link/Cut Tree

- 知識:
- 実装:

Euler Tour vs. Link/Cut

▶ Euler Tour Tree

- 知識: 過去にも出題済みの知識の組合せ。
- 実装:

▶ Link/Cut Tree

- 知識:
- 実装:

Euler Tour vs. Link/Cut

▶ Euler Tour Tree

- 知識：過去にも出題済みの知識の組合せ。
- 実装：組み合わせてはいけないものを組み合わせってしまった感じ

▶ Link/Cut Tree

- 知識：
- 実装：

Euler Tour vs. Link/Cut

▶ Euler Tour Tree

- 知識：過去にも出題済みの知識の組合せ。
- 実装：組み合わせてはいけないものを組み合わせってしまった感じ

▶ Link/Cut Tree

- 知識：必須
- 実装：

Euler Tour vs. Link/Cut

▶ Euler Tour Tree

- 知識：過去にも出題済みの知識の組合せ。
- 実装：組み合わせてはいけないものを組み合わせってしまった感じ

▶ Link/Cut Tree

- 知識：必須
- 実装：頂点にデータを持たせなくてよいなど、この問題においては極めて有利

Euler Tour vs. Link/Cut

▶ Euler Tour Tree

- 知識：過去にも出題済みの知識の組合せ。
- 実装：組み合わせてはいけないものを組み合わせってしまった感じ

▶ Link/Cut Tree

- 知識：必須
 - 実装：頂点にデータを持たせなくてよいなど、この問題においては極めて有利
- ▶ 知っているならLink/Cut を書くべきだったかもしれない

Link/Cutを学ぶべきか？

- ▶ Link/Cutを学ぶべきか？

Link/Cutを学ぶべきか？

- ▶ Link/Cutを学ぶべきか？
 - Link/Cutでなければ出来ない、という問題は、恐らくない

Link/Cutを学ぶべきか？

- ▶ Link/Cutを学ぶべきか？
 - Link/Cutでなければ出来ない、という問題は、恐らくない
 - しかし、Link/Cutを使うと有利な問題は実際に存在している

Link/Cutを学ぶべきか？

- ▶ qnighyからの提案

Link/Cutを学ぶべきか？

- ▶ qnighyからの提案
 - 合宿参加者の大半にとっては、Link/Cut Treeを習得するコストが高くつく上に、他の学習をしたほうがずっと為になると思う。

Link/Cutを学ぶべきか？

▶ qnighyからの提案

- 合宿参加者の大半にとっては、Link/Cut Treeを習得するコストが高くつく上に、他の学習をしたほうがずっと為になると思う。
- より上位の人や、単純に興味があるという人に関しては、この限りではない。

Link/Cutを学ぶべきか？

▶ qnighyからの提案

- 合宿参加者の大半にとっては、Link/Cut Treeを習得するコストが高くつく上に、他の学習をしたほうがずっと為になると思う。
- より上位の人や、単純に興味があるという人に関しては、この限りではない。
- いずれにせよ、学習するつもりなら、身に付けるために問題を解くべきだろう。

参考問題

- ▶ JOI2010春合宿 Day4 “Highway”
- ▶ JOI2012本選 問題5 “Festivals in JOI Kingdom”
- ▶ IOI2011 Day2 “Elephants”
- ▶ IJPC2012 Day3 “Animals2”

参考資料

- ▶ 完全制覇・ツリー上でのクエリ処理技法 [iwiwi]
<http://topcoder.g.hatena.ne.jp/iwiwi/20111205/1323099376>
- ▶ プログラミングコンテストでのデータ構造 2 ～動的木編～ [iwiwi]
<http://www.slideshare.net/iwiwi/2-12188845>
- ▶ 蟻本 [iwiwi]

参考文献

- ▶ Daniel D. Sleator and Robert E. Tarjan, A Data Structure for Dynamic Trees, Journal of Computer and System Sciences, Volume 26 Issue 3, June 1983, pp. 362 – 391
- ▶ Daniel D. Sleator and Robert E. Tarjan, Self-adjusting binary search trees, Journal of the ACM, Volume 32 Issue 3, July 1985, pp. 652 – 686

得点分布

